

Exercício – Kanban – Passo a Passo

Nome(s): _____

PARTE 1 – CRIAÇÃO DAS TELAS DE AUTENTICAÇÃO

1. Abra o Android Studio e inicie um novo projeto selecionando o **template Empty Views Activity**. As configurações recomendadas estão apresentadas na Figura 1. Durante o processo, substitua o nome do pacote **marta** pelo seu próprio nome ou outro identificador adequado. Além disso, certifique-se de que o nome do projeto comece com letra maiúscula, seguindo as boas práticas de nomenclatura. Por fim, clique em **Finish** para concluir a criação do projeto.

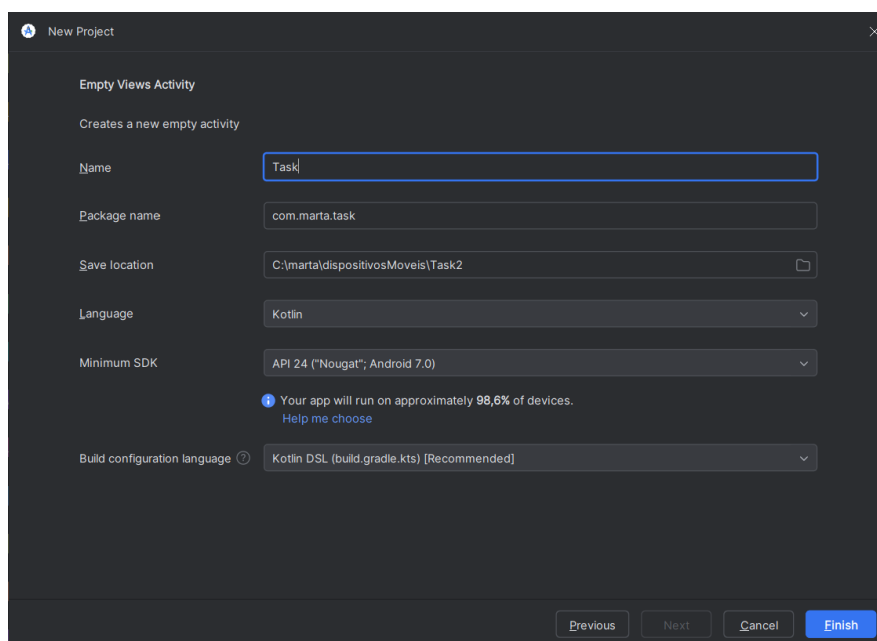


Figura 1

2. Clique com o botão direito do mouse sobre o pacote **com.marta.task**, selecione **New > Package** e crie um novo pacote com o nome **ui**. Esse pacote será utilizado para armazenar as telas da interface com o usuário. Na janela que será exibida, digite o nome **ui**, conforme mostrado na Figura 2, e pressione Enter para concluir a criação.

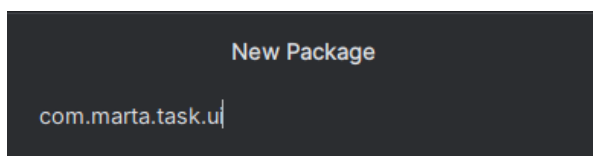


Figura 2

3. Verifique se as configurações do seu Android estão com **Flatten packages** desmarcado e **Compact Middle packages** marcado, clicando na barra de ferramentas do projeto conforme Figura 3. O “Flatten packages” significa achatar a estrutura de pacotes, ou seja, remover a hierarquia original dos diretórios e colocar todos os arquivos num nível só. Quando “Compact Middle Packages” está ativado, ele acha os pacotes intermediários, mostrando uma hierarquia mais curta.

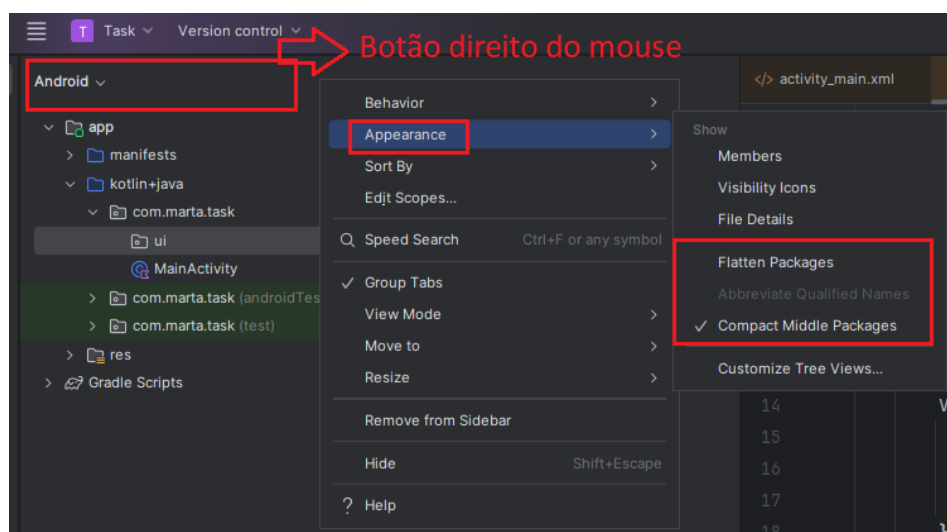


Figura 3

4. Dentro da pasta **ui**, clique com o botão direito do mouse e **selecione New>> Fragment>>Fragment (Blank)** para criar um novo fragmento em branco. Nomeie-o como **SplashFragment**, conforme ilustrado na Figura 4. Esse fragmento será utilizado como a tela de splash da aplicação.

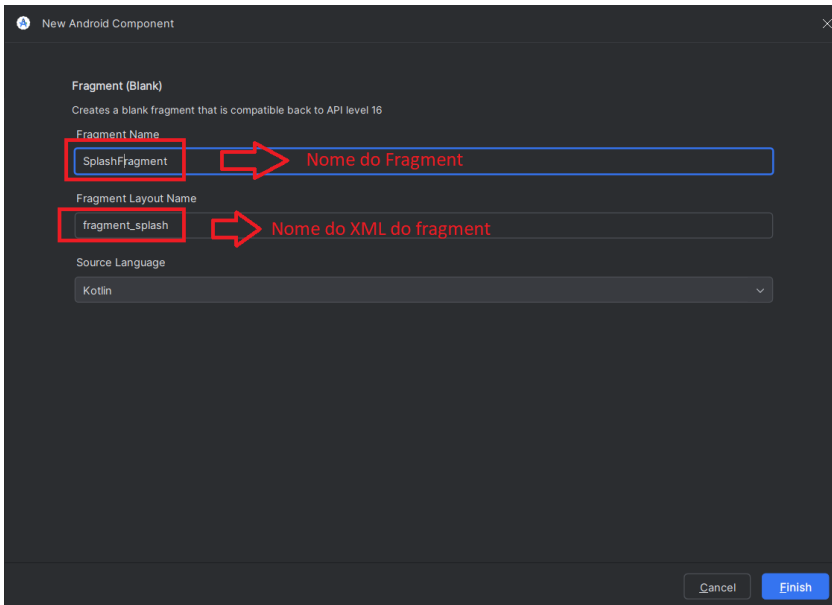


Figura 4

5. Remova o código Kotlin inútil de **SplashFragment.kt**, de forma que fique conforme Figura 5.



Figura 5

6. Agora será adicionado um ícone à tela de splash. Para isso, clique com o botão direito na pasta **drawable** e selecione a opção **New >>Vector Asset**. Em seguida, escolha o ícone desejado para compor a tela, conforme ilustrado na Figura 6. **Lembre-se que o nome do ícone não pode ter caracteres especiais.**

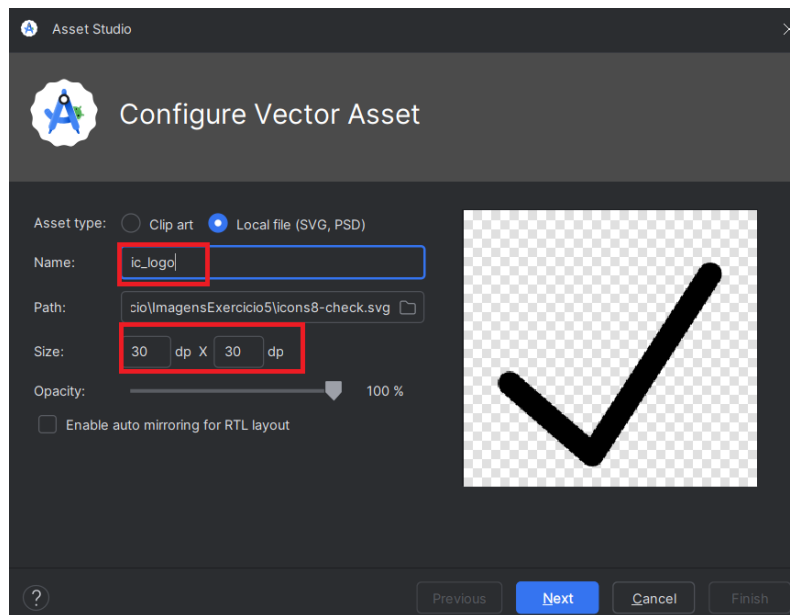


Figura 6

7. Após incluir o ícone, modifique sua cor para branco. Para isso, abra o arquivo `ic_logo.xml` correspondente ao ícone na pasta `drawable`, conforme ilustrado na Figura 7, e ajuste o atributo de cor **FillColor** conforme necessário.

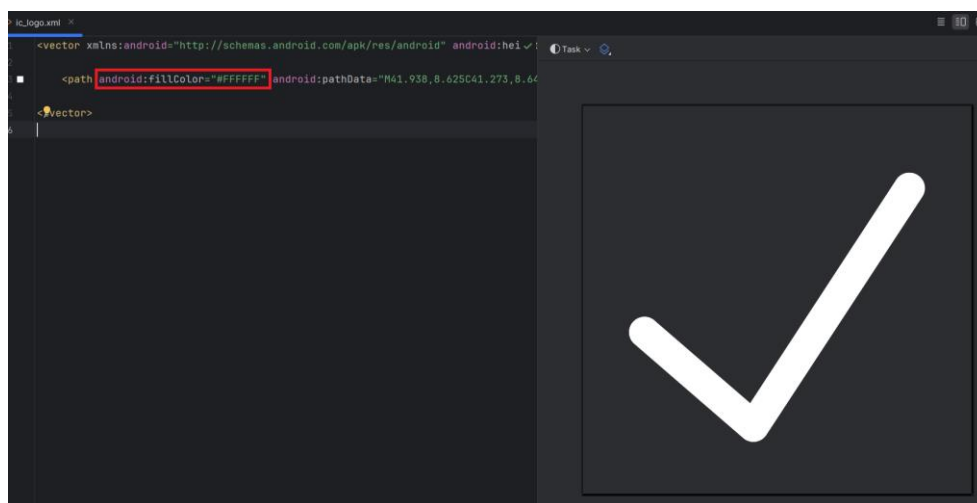


Figura 7

8. Vamos configurar a cor padrão do App. Abra o arquivo `colors.xml` e, dentro da tag `<resources>`, adicione a seguinte linha: `<color name="color_default">#0277BD</color>`. Você pode escolher outra cor de sua preferência.

9. Abra o arquivo XML do fragment correspondente à tela de splash e adicione a imagem conforme o código apresentado na Figura 8. Em seguida, substitua o contêiner `FrameLayout` por um `LinearLayout`, também conforme ilustrado na figura. Por fim, adicione os atributos **background** e **gravity** ao `LinearLayout`. Com isso, a tela splash estará pronta.

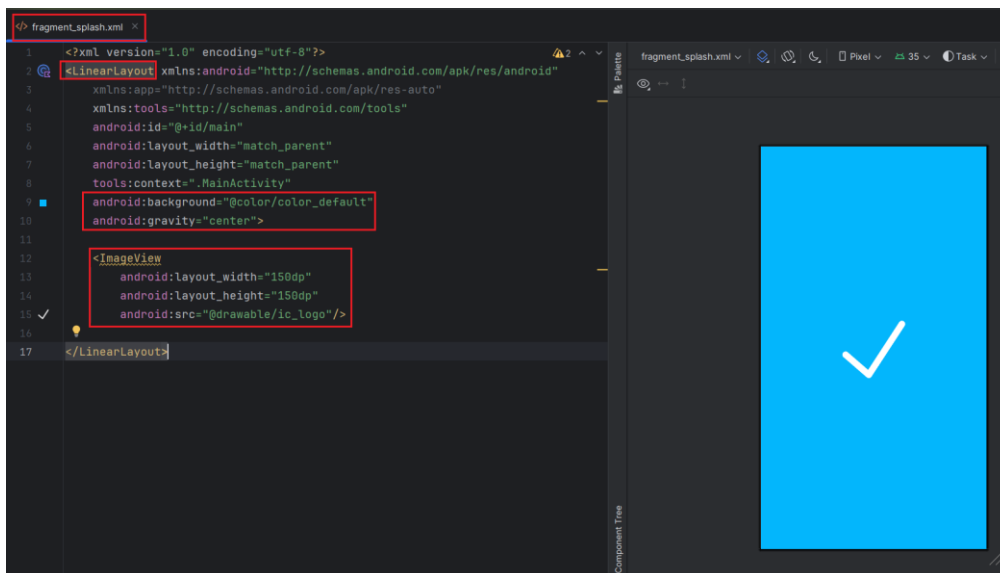


Figura 8

Funcionalidades e objetivos da Splash Screen:

- Dar tempo para carregamento: Enquanto o app carrega dados, configurações ou inicializa componentes, a splash screen ocupa visualmente o espaço, evitando que o usuário veja uma tela em branco ou travamentos.
- Fortalecer a identidade visual: Exibe o logotipo, nome, slogan ou animação da marca, reforçando a identidade da empresa ou do aplicativo.
- Melhorar a percepção de desempenho: Mesmo que o carregamento leve alguns segundos, uma splash screen bem desenhada faz o usuário sentir que o app está funcionando corretamente, gerando uma experiência mais fluida.
- Cumprir requisitos técnicos: Em alguns sistemas operacionais (como Android e iOS), a splash screen é usada também para carregar dependências, preparar banco de dados local, verificar autenticação, etc.
- Criar expectativa ou impacto visual: Pode ser usada como um momento para criar uma boa primeira impressão, com transições suaves ou animações agradáveis.

10. Habilite o `viewBinding` no arquivo `build.gradle` do Módulo, conforme Figura 9. Não esqueça de clicar no link **Sync Now**.

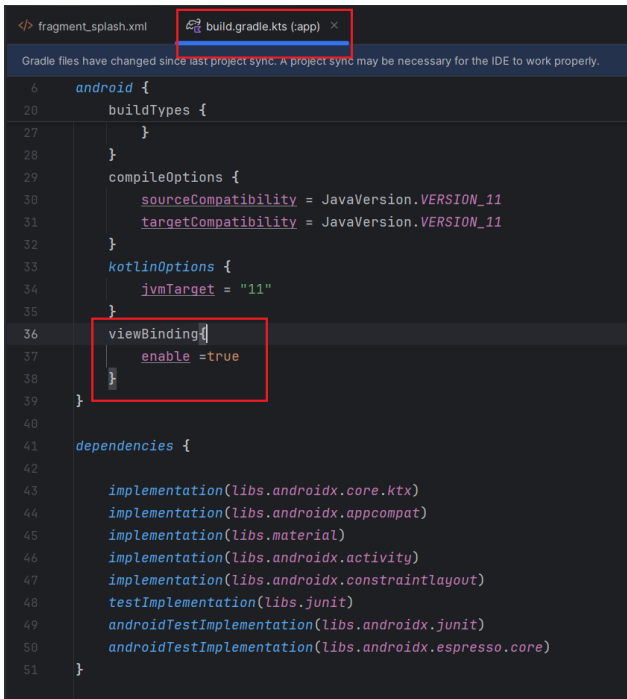


Figura 9

11. Primeiro, feche as janelas que não estiver utilizando, para evitar confusão com múltiplas abas abertas no ambiente de desenvolvimento. Depois, abra o arquivo **SplashFragment.kt** para criar a estrutura base do código que irá manipular os elementos da interface do fragmento, conforme mostrado na Figura 10.

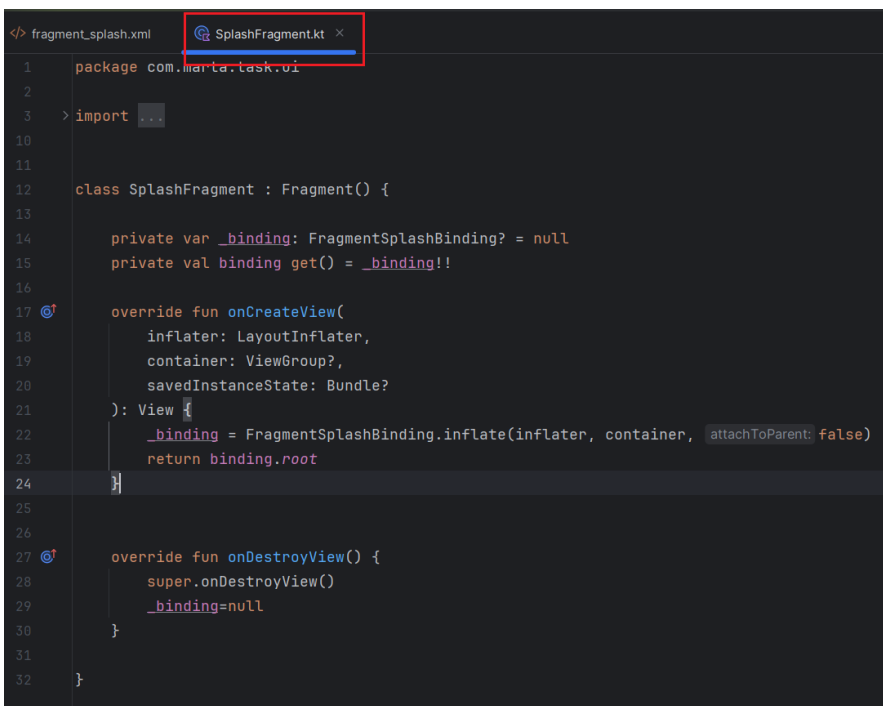


Figura 10

Vamos ao entendimento do código da Figura 10. A classe `SplashFragment` é declarada como uma subclasse de `Fragment`. Os `Fragments` são componentes reutilizáveis da interface do usuário que podem ser adicionados dentro de `Activity`s ou de outros `Fragments`.

A criação da propriedade somente leitura **binding** por meio do **get()** fornece uma forma segura de acessar a variável **_binding** e, conseqüentemente, **os elementos da interface**. Ao retornar o valor de `_binding` com a garantia de não-nulidade (!), evitamos a necessidade de lidar explicitamente com valores nulos. Como `_binding` é garantidamente inicializado entre `onCreateView()` e `onDestroyView()`, essa abordagem simplifica o código, eliminando a repetição de `if (_binding != null)` e o uso constante de `!!`, permitindo um acesso mais direto como em `binding.textViewTitulo.text = "Olá"`, ao invés de `_binding!!.textViewTitulo.text = "Olá"`.

O método **onCreateView(...)** é invocado durante a criação do fragmento. Nele, `FragmentSplashBinding.inflate(...)` realiza o processo de inflar o layout XML, gerando o `binding`. Em seguida, `return binding.root` retorna a view raiz do layout inflado para ser utilizada pelo fragmento.

Os parâmetros do `onCreateView` são:

- `inflater`: `LayoutInflater`: Um objeto que o Android fornece para inflar layouts XML em objetos `View`. "Inflar" significa pegar a descrição do seu layout em XML e transformá-la em objetos reais que podem ser exibidos na tela.
- `container`: `ViewGroup?`: O `ViewGroup` pai ao qual a view do `Fragment` eventualmente pertencerá. Pode ser nulo, especialmente durante a criação inicial. É importante notar que a view criada aqui não é imediatamente anexada a este container.
- `savedInstanceState`: `Bundle?`: Serve para restaurar o estado anterior da atividade (`Activity`) ou fragmento (`Fragment`) caso eles sejam recriados, como quando o usuário gira a tela ou o sistema operacional mata e recria a atividade por falta de memória. Exemplo de informações de estado: Fragmentos ativos (Qual fragmento está visível no momento), Valores numéricos (posição de um item), Textos da UI (Texto digitado num `EditText`) etc

O método `.inflate(inflater, container, false)`: O valor `false`: Um booleano que indica se a view inflada deve ser imediatamente anexada ao container. Aqui, passamos `false` porque o Android adicionará a view no momento apropriado no ciclo de vida do `Fragment`. A view retornada por `onCreateView` será adicionada ao container posteriormente.

O método `onDestroyView()` é chamado quando a view do fragmento está sendo destruída. Nele, a atribuição `_binding = null` libera a referência ao objeto de `binding`, prevenindo potenciais vazamentos de memória.

12. Clique com o botão direito do mouse sobre o pacote **ui**, selecione a opção **New>> Package** e crie um novo pacote com o nome **auth**, conforme ilustrado na Figura 11. Esse pacote será responsável por agrupar as telas de autenticação: login, cadastro de usuário e recuperação de senha.

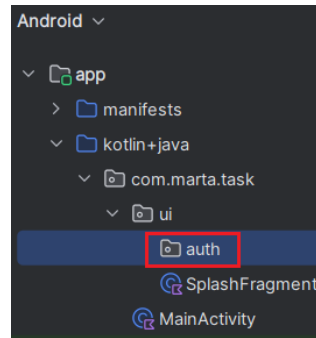


Figura 11

13. Dentro da pasta **auth**, crie um novo fragment selecionando a opção **New>>Fragment>>Fragment (Blank)**. Nomeie o fragment como **LoginFragment**, conforme mostrado na Figura 12.

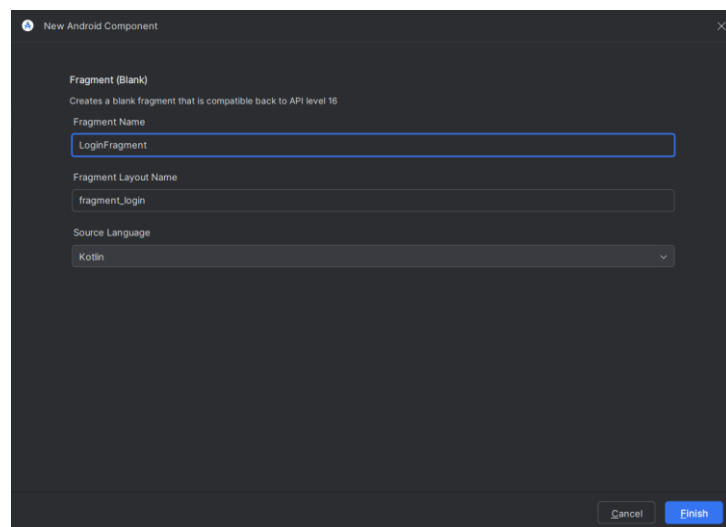


Figura 12

14. Remova o código Kotlin não utilizado do arquivo **LoginFragment.kt**, conforme ilustrado na Figura 13.

```
LoginFragment.kt
1 package com.marta.task.ui.auth
2
3 import android.os.Bundle
4 import androidx.fragment.app.Fragment
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import com.marta.task.R
9
10
11 class LoginFragment : Fragment() {
12
13
14     override fun onCreateView(
15         inflater: LayoutInflater, container: ViewGroup?,
16         savedInstanceState: Bundle?
17     ): View? {
18         // Inflate the layout for this fragment
19         return inflater.inflate(R.layout.fragment_login, container, attachToRoot: false)
20     }
21
22 }
```

Figura 13

15. Todas as telas do nosso aplicativo terão fundo azul. Em vez de definir a propriedade background individualmente em cada layout, como ilustrado na Figura 14, utilizaremos o arquivo **themes.xml** para aplicar essa configuração de forma global. Assim, a cor de fundo será automaticamente aplicada a todas as telas. Para isso, basta adicionar a linha **android:windowBackground** dentro do tema da aplicação, conforme mostrado na Figura 15. Não esqueça de remover o atributo background da tela `fragment_slash.xml`.

```
LoginFragment.kt  </> fragment_login.xml
1 <?xml version="1.0" encoding="utf-8"?>
2 <FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     tools:context=".ui.auth.LoginFragment"
7     android:background="@color/color_default">
```

Figura 14

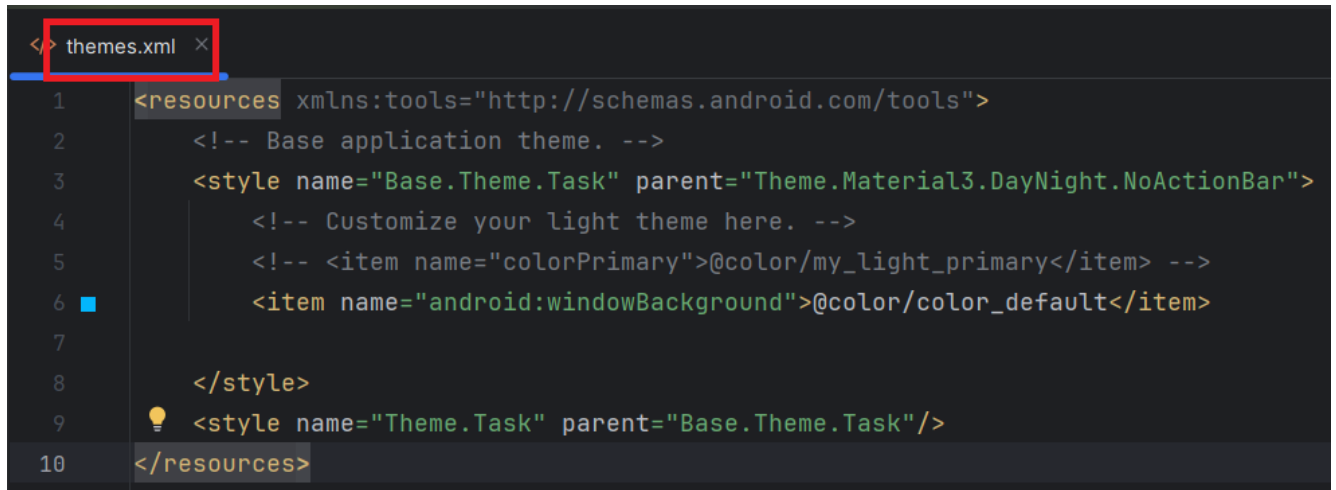


Figura 15

16. Abra o arquivo **fragment_login.xml**, e substitua a tag **FrameLayout** pela **ConstraintLayout**. Depois inclua a tag **LinearLayout** dentro da **ConstraintLayout**, conforme Figura 16.

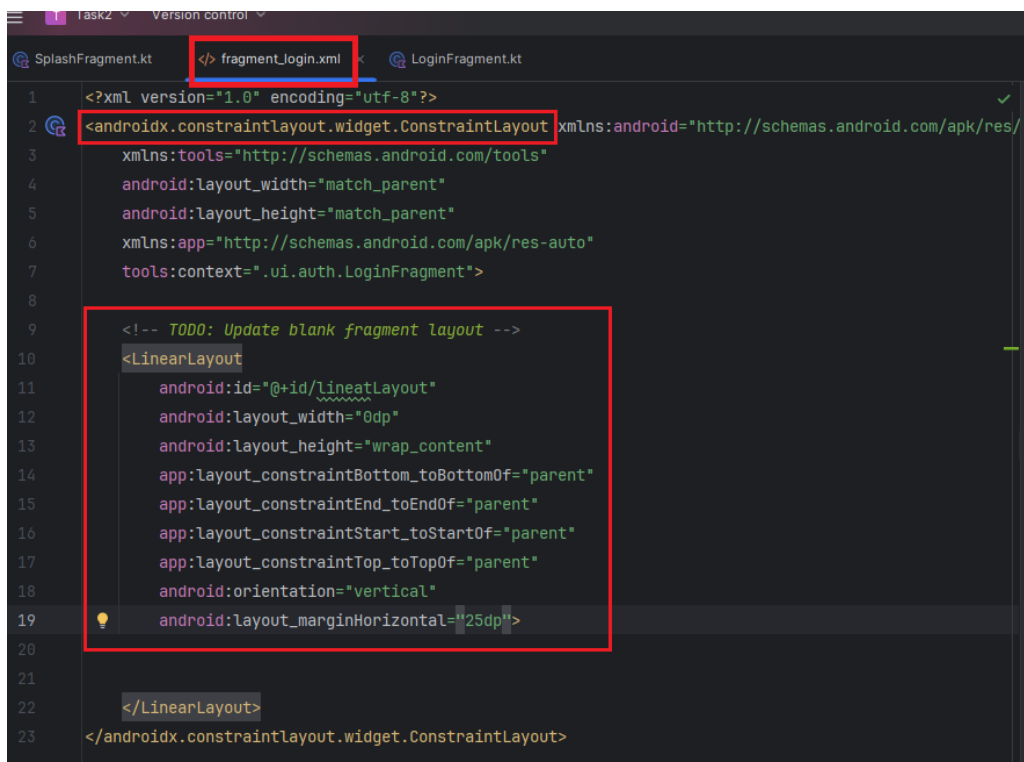


Figura 16

17. Abra o arquivo **fragment_login.xml** e insira o código dos componentes listados na Tabela 1 dentro do contêiner **LinearLayout**.

<TextView

android:layout_width="wrap_content"

```
android:layout_height="wrap_content"
```

```
android:text="E-mail"
```

```
android:textColor="@color/white"/>
```

```
<EditText
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="48dp"
```

```
    android:hint="Digite seu e-mail"
```

```
    android:inputType="textEmailAddress"
```

```
    android:padding="10dp"
```

```
    android:layout_marginTop="4dp"/>
```

```
<TextView
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Senha"
```

```
    android:textColor="@color/white"
```

```
    android:layout_marginTop="16dp"/>
```

```
<EditText
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="48dp"
```

```
    android:inputType="textPassword"
```

```
    android:hint="Digite sua senha"
```

```
    android:padding="10dp"
```

```
    android:layout_marginTop="4dp"/>
```

```
<com.google.android.material.button.MaterialButton
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Login"
```

```
    android:textColor="@color/color_default"
```

```
    app:backgroundTint="@null"
```

```
    android:layout_marginTop="16dp"/>
```

```
<LinearLayout
    android:layout_width="match_parent"

    android:layout_height="wrap_content"

    android:orientation="horizontal"
    android:layout_marginTop="4dp">

    <TextView

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Criar Conta"

        android:layout_weight="1"

        android:textAlignment="textStart"

        android:textColor="@color/white"/>

    <TextView

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Recuperar Conta"

        android:layout_weight="1"

        android:textAlignment="textEnd"

        android:textColor="@color/white"/>

</LinearLayout>

<ProgressBar

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:indeterminateTint="@color/white"

    android:layout_gravity="center"/>
```

Tabela 1

Vamos algumas explicações dos atributos dos componentes de tela da Tabela 1.

O atributo **layout_weight** só tem efeito dentro de um `LinearLayout`. Esse atributo define quanto espaço proporcional cada `View` deve ocupar dentro do `LinearLayout`, em relação às outras `Views`. Se duas `Views` têm `layout_weight="1"`, elas dividem igualmente o espaço restante. Se uma tem 1 e outra 2, a primeira pega 1/3 e a segunda 2/3 do espaço disponível.

Ao usar @null no backgroundTint a cor de fundo original do botão será exibida sem alterações. Em outras palavras, remove o **tint (desabilita o tint)**, deixando a cor de fundo (background) aparecer exatamente como você definiu.

O atributo Background: Além da cor, também exibe formas (shapes e icons). Já o backgroundTint aplica apenas uma cor de fundo. Se duas cores de fundo forem aplicadas tanto ao background quanto ao BackgroundTint, o background será substituído pela cor do backgroundTint.

Exemplo 1:

```
android:background="@drawable/custom_button_bg"
android:backgroundTint="@color/red"
```

O resultado será a imagem incluída ao botão pelo background com uma coloração vermelha modificada pelo backgroundTint.

Exemplo 2:

```
android:background="@drawable/rounded_shape"
android:backgroundTint="@color/blue" />
```

O botão usará o fundo definido no drawable rounded_shape, mas ele terá um fundo com uma coloração azul (backgroundTint).

Exemplo 3:

```
android:background="@drawable/rounded_shape"
android:backgroundTint="@null"
```

O fundo rounded_shape será exibido sem nenhuma modificação de cor.

Você pode criar efeitos de mistura de cores entre o background e o backgroundTint configurando o atributo backgroundTintMode com o valor "multiply". Por exemplo: se você definir android:background="@color/blue" e app:backgroundTint="@color/yellow", a combinação resultante será um tom de verde, pois o modo de mistura "multiply" multiplica os canais de cor (RGB) das duas camadas.

18. Agora vamos criar agora um shape que servirá neste caso para arredondamento das bordas das caixas de texto. No Android, um shape é um tipo especial de recurso XML usado para definir formas gráficas — como retângulos, ovais, linhas ou anéis — que podem ser aplicados como fundos (backgrounds), bordas, botões personalizados, entre outros elementos visuais. Clique com o botão direito na **pasta drawable**, e selecione a opção **New>>Drawable Resource File**, conforme Figura 17.

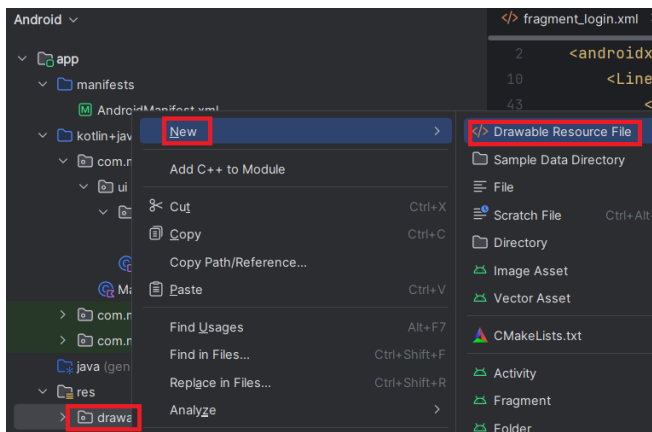


Figura 17

19. No campo File name, digite **bg_edit_text** e, no campo Root element, selecione **shape**, conforme ilustrado na Figura 18. Esse shape será utilizado como fundo para os campos de texto, exibindo uma borda arredondada.

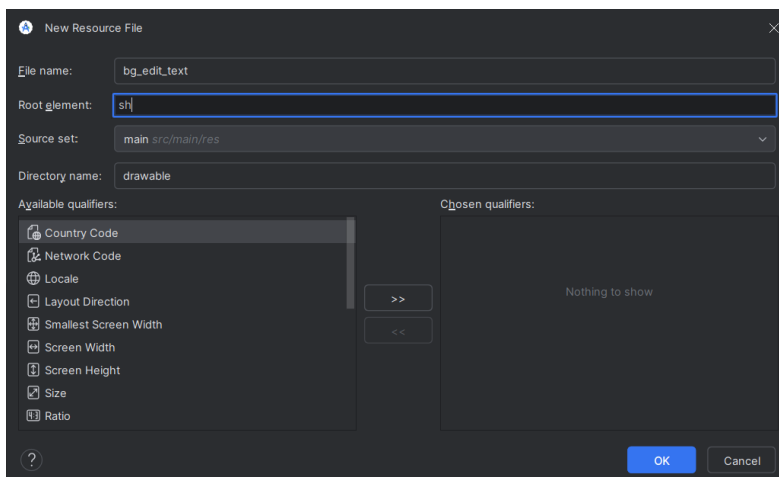


Figura 18

20. No arquivo shape **bg_edit_text.xml** inclua o seguinte XML conforme descrito na Figura 19.

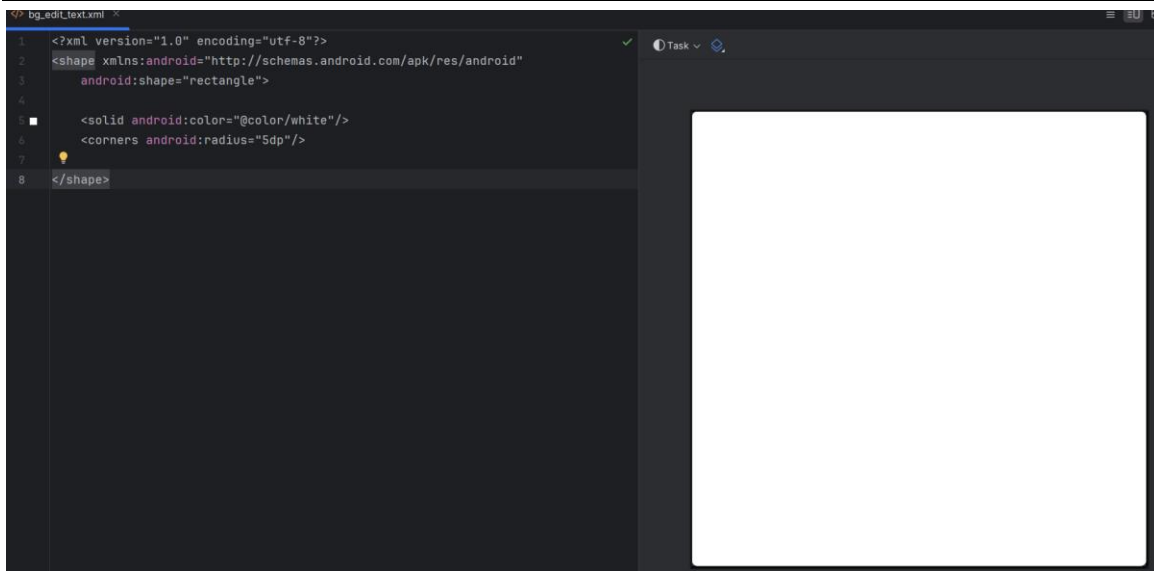


Figura 19

21. Conforme ilustrado na Figura 20, atribua ao **background** do EditText o shape criado anteriormente na tela fragment_login.xml.

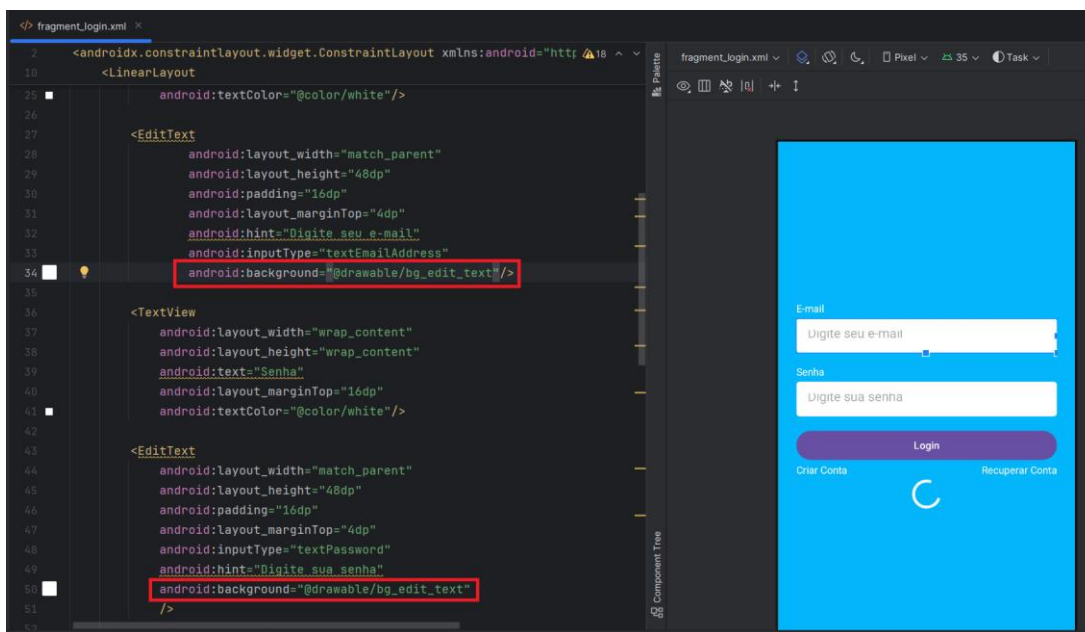


Figura 20

22. Clique com o botão direito na **pasta drawable**, e selecione a opção **New>>Drawable Resource File**. Crie um novo **shape** com o nome **bg_btn.xml** e root element com o valor **shape**, conforme mostrado na Figura 21. Esse shape será semelhante ao utilizado no EditText, porém será aplicado aos botões.

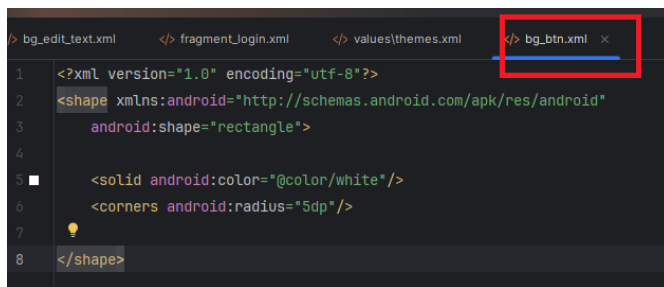


Figura 21

23. Inclua o atributo **android:background="@drawable/bg_btn"** e **app:backgroundTint="@null"** no botão, conforme Figura 22. O layout da tela de login está finalizada!

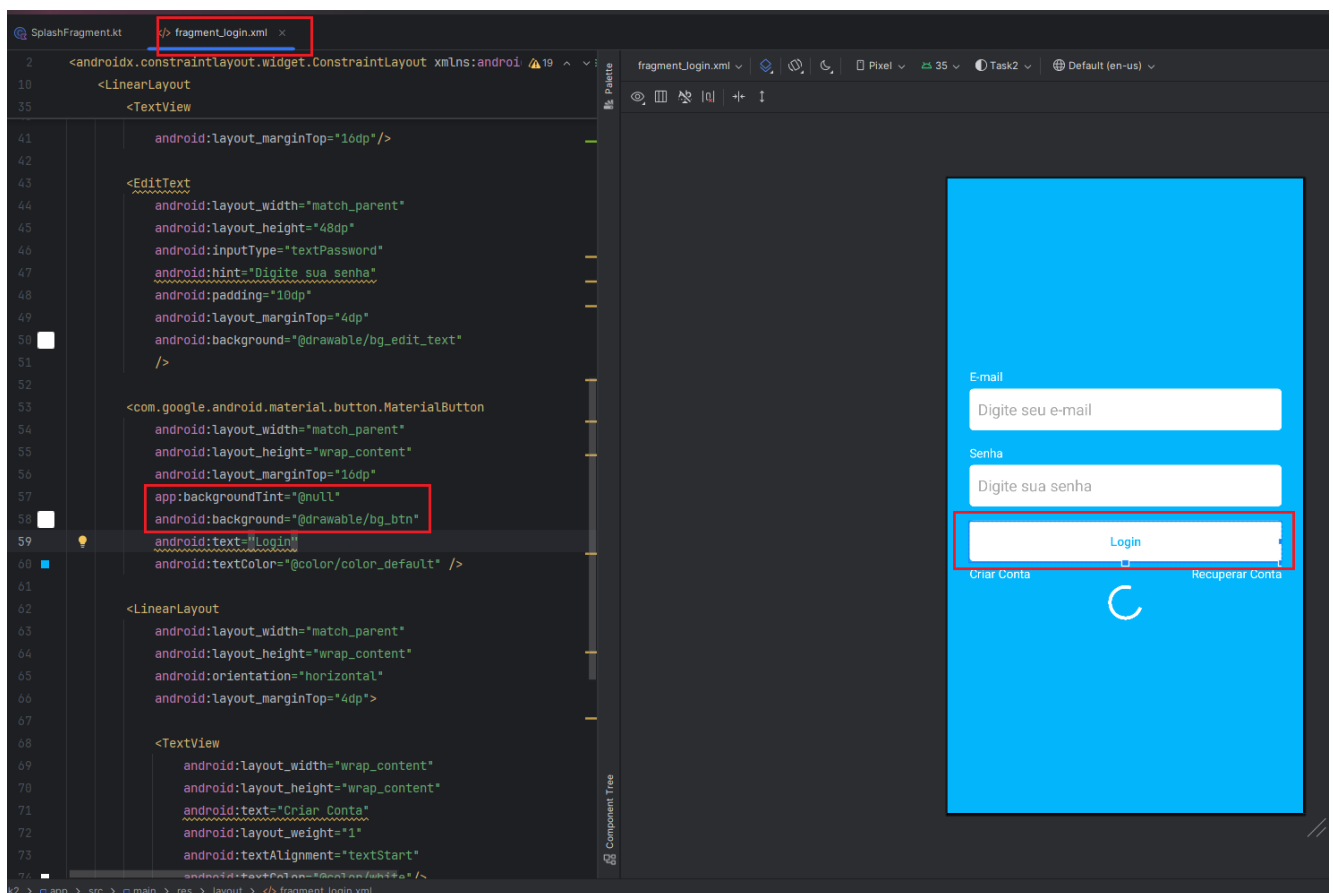


Figura 22

24. Agora vamos trabalhar na tela de cadastro. Dentro da pasta **auth**, crie um novo fragment selecionando a opção **New>>Fragment>>Fragment (Blank)**. Nomeie o fragment como **RegisterFragment**, conforme mostrado na Figura 23.

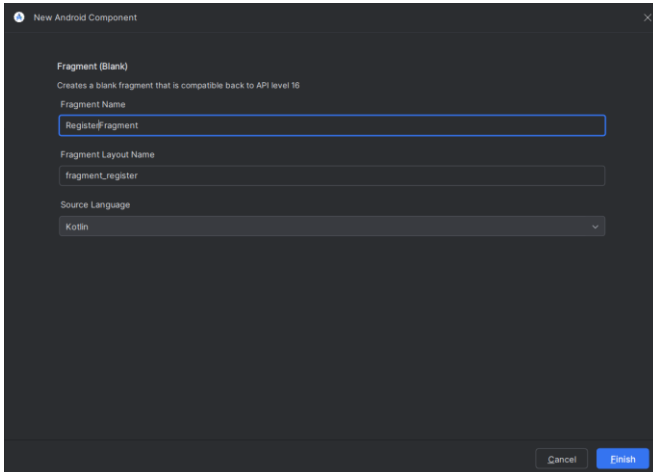


Figura 23

25. Remova o código Kotlin não utilizado do arquivo **RegisterFragment.kt**, conforme ilustrado na Figura 24.

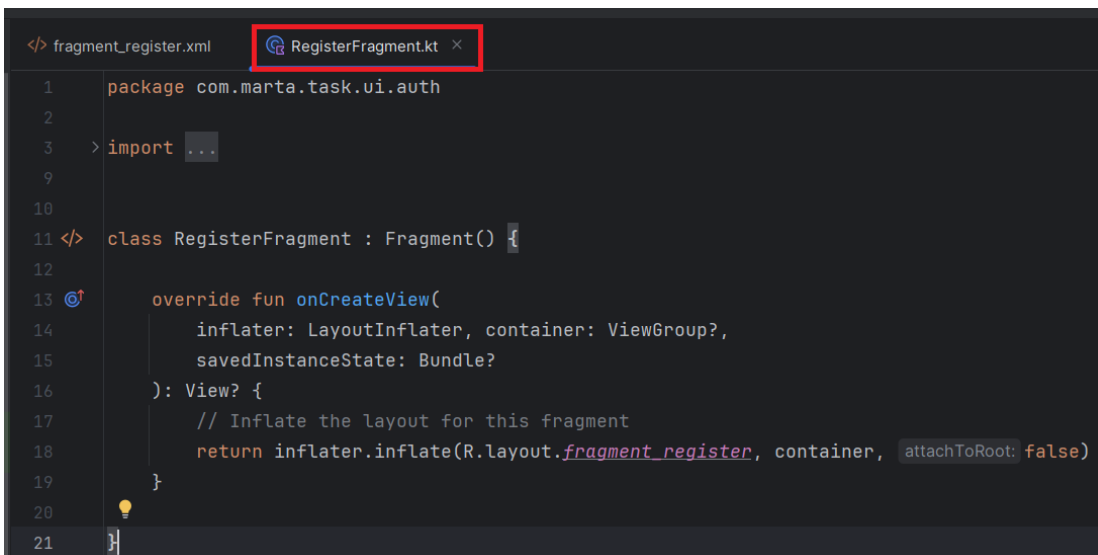


Figura 24

26. Copie todo o conteúdo XML da tela **fragment_login.xml** e cole-o no arquivo **fragment_register.xml**. Em seguida, atualize o atributo **tools:context** para referenciar a classe Kotlin correspondente à tela de cadastro, conforme ilustrado na Figura 25. **Lembre-se também de remover os TextViews "Criar Conta", "Recuperar Conta" e mudar o rótulo do botão para "Enviar".**

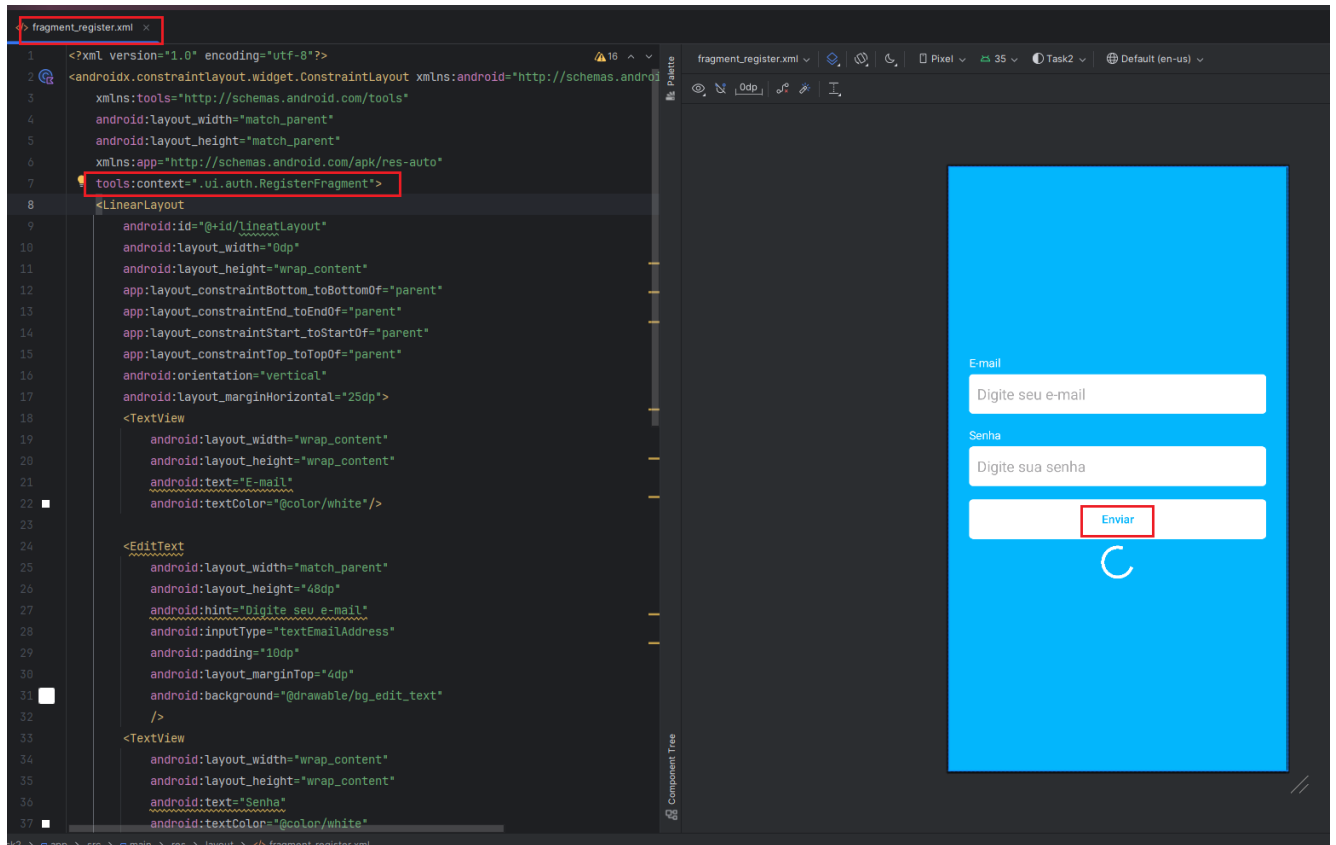


Figura 25

27. Agora vamos incluir um componente Toolbar, utilizado para navegação na interface. Antes disso, será necessário realizar algumas alterações no arquivo **fragment_register.xml**, conforme destacado em vermelho e verde na Figura 28. As modificações incluem: Substituição do ConstraintLayout por um ScrollView; Inclusão do atributo `fillViewport="true"` no ScrollView; Adição de um novo LinearLayout, identificado como `@+id/linearLayout2`, com `layout_width="match_parent"`; Inclusão de um TextView dentro do componente MaterialToolbar. Além disso, não se esqueça de adicionar um novo ícone ao projeto com o nome `ic_back`, para que a propriedade `navigationIcon` da Toolbar possa reconhecê-lo corretamente. Por fim, todos os demais componentes da tela deverão ficar dentro do LinearLayout2, garantindo a organização correta da hierarquia de layouts.

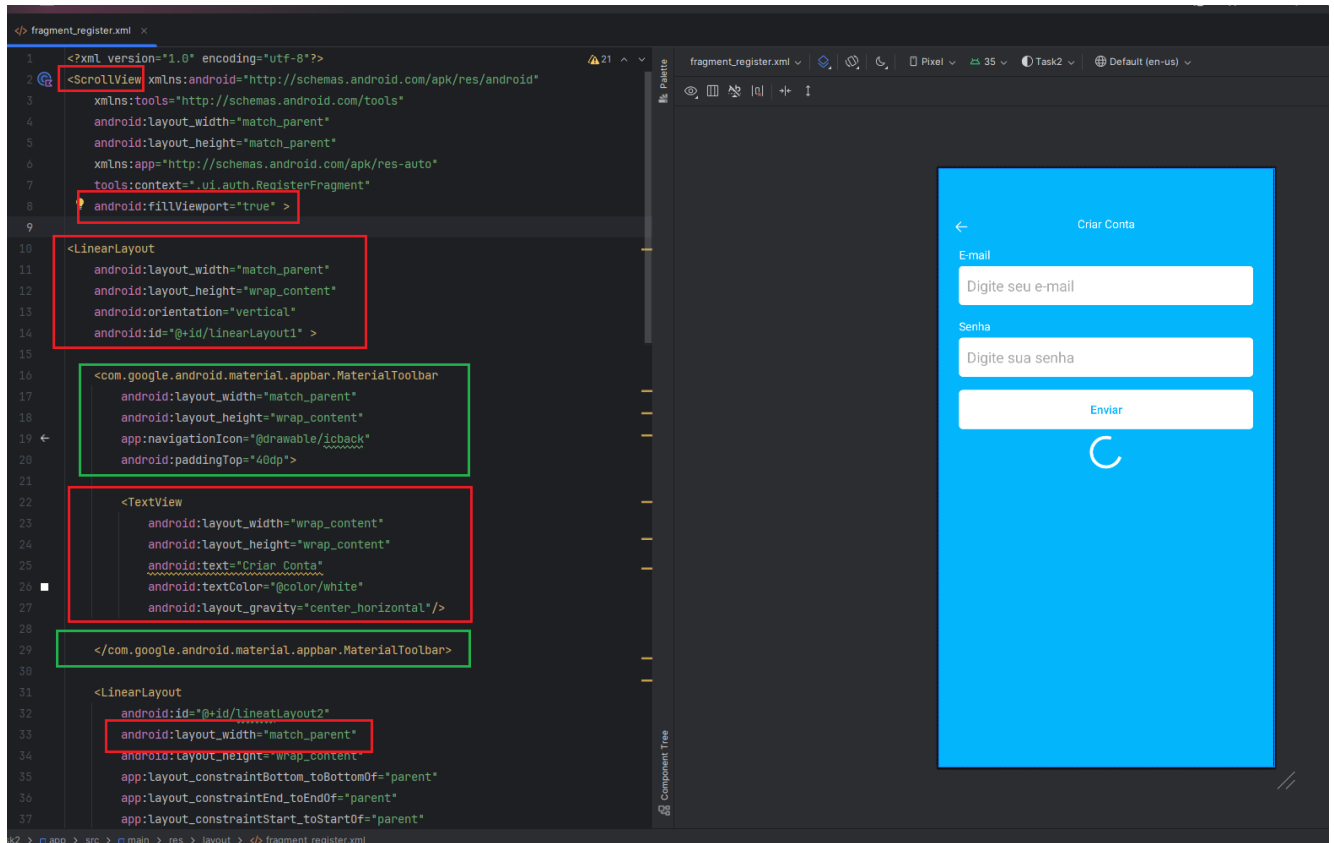


Figura 28

O objetivo de usar `ScrollView` é permitir a rolagem do conteúdo quando este ultrapassa o tamanho visível da tela. O atributo `fillViewport` controla como o conteúdo ocupa o espaço disponível. **Consulte o capítulo 6** do conteúdo para mais informações sobre o container `ScrollView`.

Não esqueça de remover os atributos de alinhamento do `LinearLayout2`:
`layout_constraintBottom_toBottomOf`, `layout_constraintEnd_toEndOf`,
`layout_constraintStart_toStartOf` e `layout_constraintTop_toTopOf`.

28. Ainda no arquivo `fragment_register.xml`, ajuste o atributo `marginTop` do `Progressbar` do, conforme Figura 30.

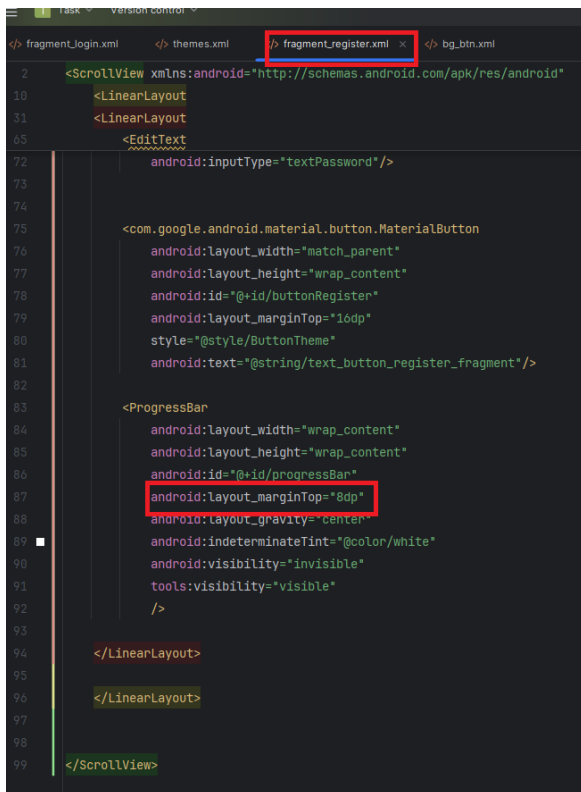


Figura 30

29. Configure o viewBinding para o código em kotlin para as telas **RegisterFragment.kt** e **LoginFragment.kt**, conforme Figuras 31 e 32. Não esqueça de remover o ponto de interrogação do View {}.

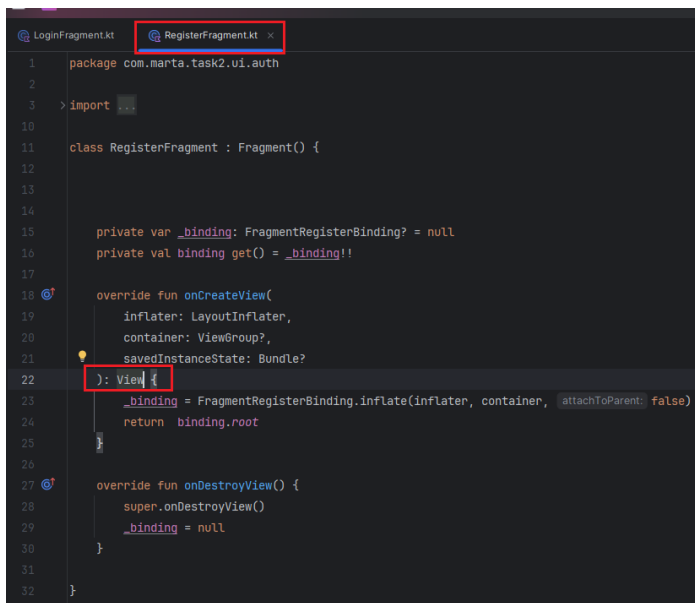
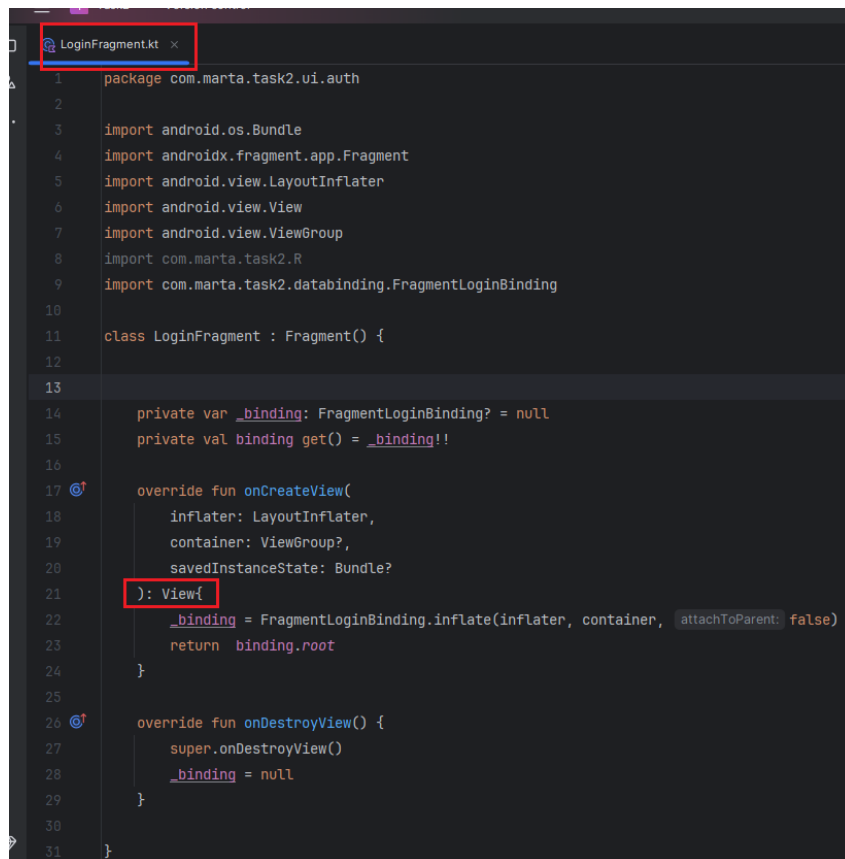


Figura 31



```
1 package com.marta.task2.ui.auth
2
3 import android.os.Bundle
4 import androidx.fragment.app.Fragment
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import com.marta.task2.R
9 import com.marta.task2.databinding.FragmentLoginBinding
10
11 class LoginFragment : Fragment() {
12
13
14     private var _binding: FragmentLoginBinding? = null
15     private val binding get() = _binding!!
16
17     override fun onCreateView(
18         inflater: LayoutInflater,
19         container: ViewGroup?,
20         savedInstanceState: Bundle?
21     ): View? {
22         _binding = FragmentLoginBinding.inflate(inflater, container, attachToParent: false)
23         return binding.root
24     }
25
26     override fun onDestroyView() {
27         super.onDestroyView()
28         _binding = null
29     }
30
31 }
```

Figura 32

30. Dentro da pasta **auth**, vamos criar a tela de Recuperação de conta. Crie um novo fragment selecionando a opção **New>>Fragment>>Fragment (Blank)**. Nomeie o fragment como **RecoverAccountFragment**, conforme mostrado na Figura 33.

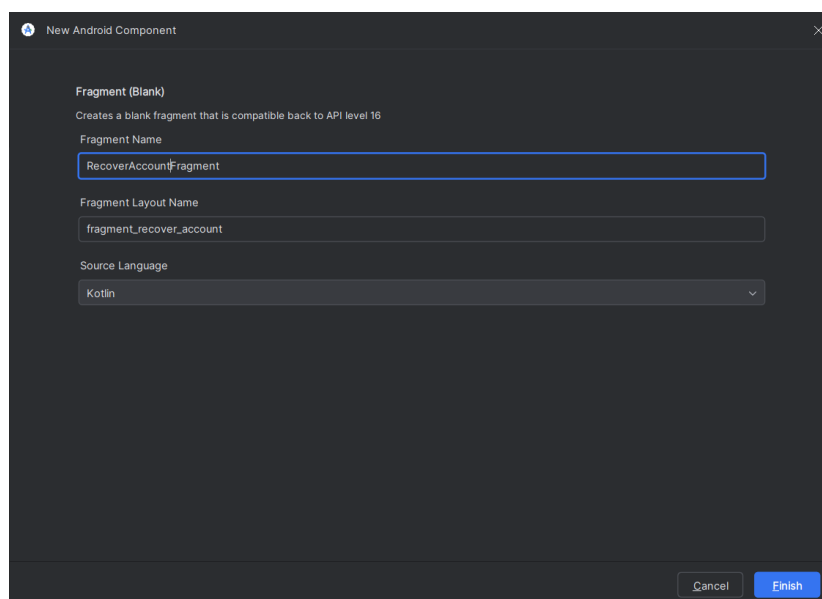
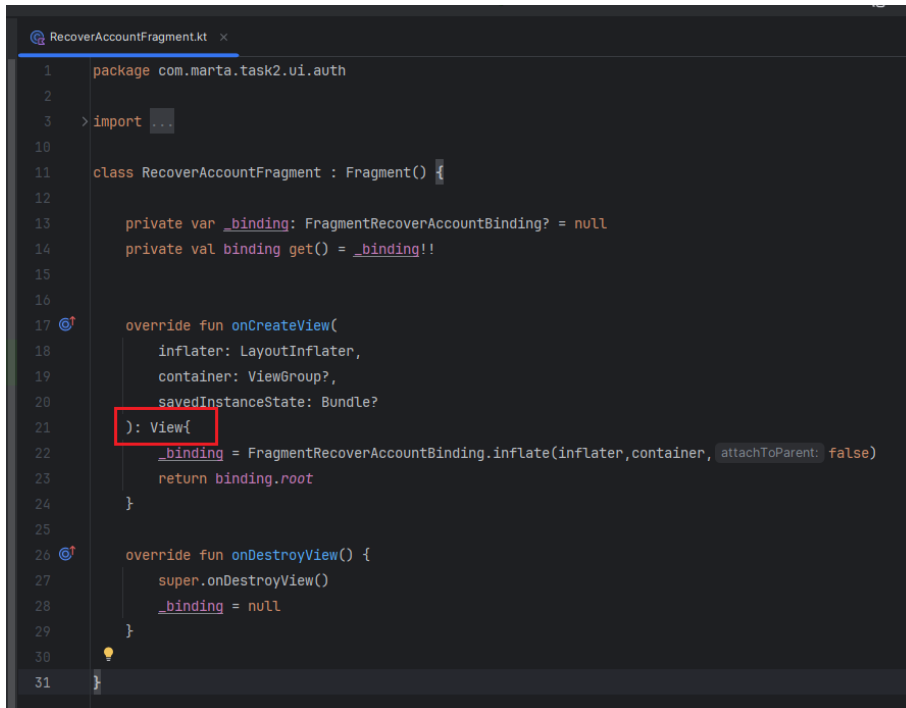


Figura 33

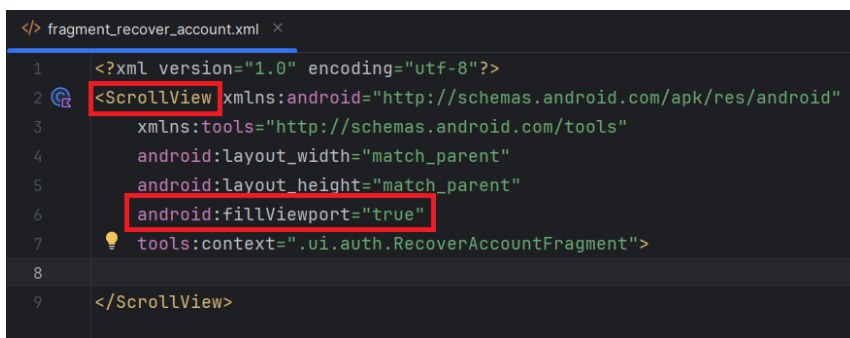
31. Remova o código Kotlin não utilizado do arquivo **RemoveAccount.kt** e inclua configuração do viewbinding, conforme ilustrado na Figura 34.



```
1 package com.marta.task2.ui.auth
2
3 > import ...
4
5
6
7
8
9
10
11 class RecoverAccountFragment : Fragment() {
12
13     private var _binding: FragmentRecoverAccountBinding? = null
14     private val binding get() = _binding!!
15
16
17     override fun onCreateView(
18         inflater: LayoutInflater,
19         container: ViewGroup?,
20         savedInstanceState: Bundle?
21     ): View {
22         _binding = FragmentRecoverAccountBinding.inflate(inflater, container, attachToParent: false)
23         return binding.root
24     }
25
26     override fun onDestroyView() {
27         super.onDestroyView()
28         _binding = null
29     }
30
31 }
```

Figura 34

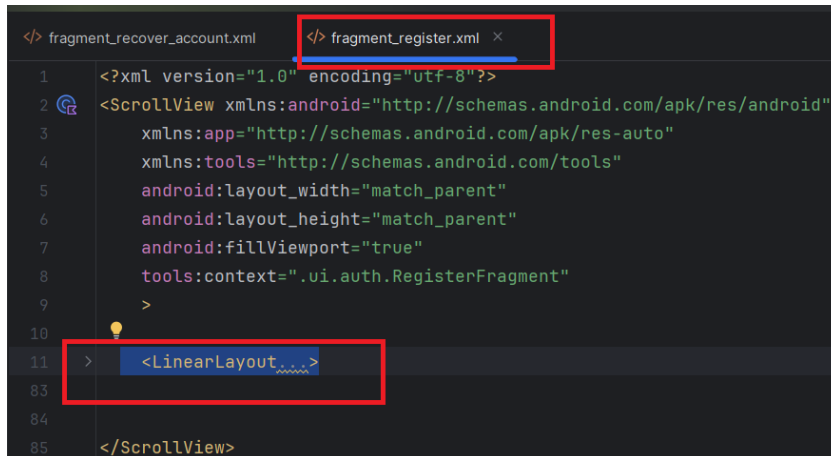
32. Abra o arquivo **fragment_recover_account.xml** e altere o código conforme Figura 35.



```
<?xml version="1.0" encoding="utf-8"?>
1 <ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
2     xmlns:tools="http://schemas.android.com/tools"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:fillViewport="true"
6     tools:context=".ui.auth.RecoverAccountFragment">
7
8
9 </ScrollView>
```

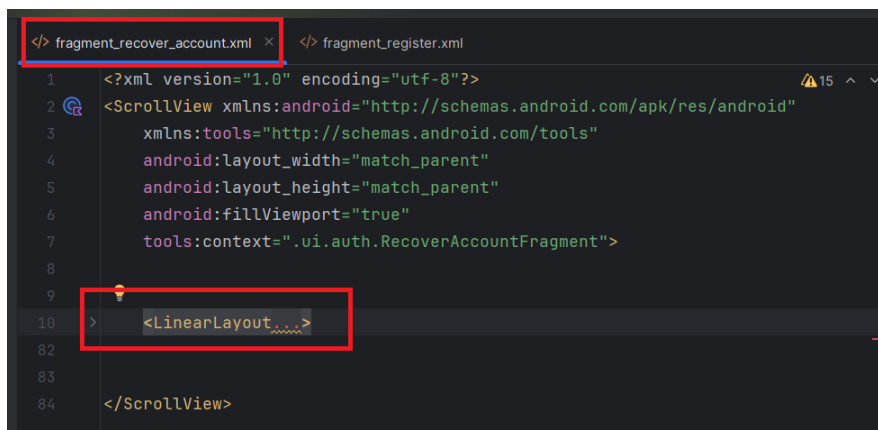
Figura 35

33. Abra o arquivo **fragment_register.xml**, copie todo o código do **<LinearLayout>** conforme Figura 36, e cole no arquivo **fragment_recover_account.xml**, conforme Figura 37.



```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    tools:context=".ui.auth.RegisterFragment"
    >
    >
</ScrollView>
```

Figura 36



```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    tools:context=".ui.auth.RecoverAccountFragment">
    >
</ScrollView>
```

Figura 37

Caso algum namespace não seja reconhecido, aperte as teclas **ALT + ENTER**.

34. No arquivo **fragment_recover_account.xml**, remova o **EditText** e o **TextView** relacionados ao campo Senha. Em seguida, atualize o valor do atributo **text** para "Recuperar Conta" no **TextView** que está dentro do componente **MaterialToolbar**, conforme ilustrado na Figura 38.

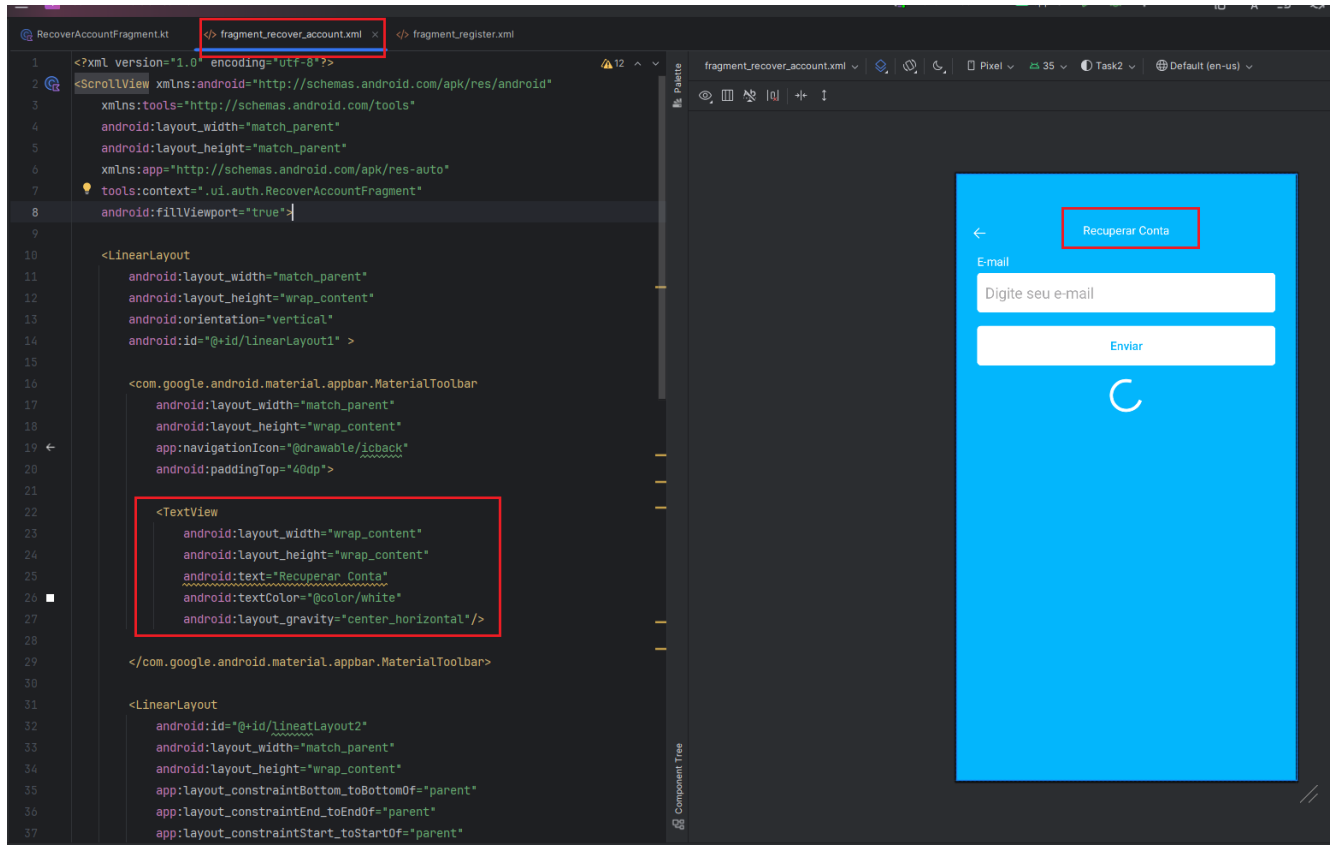


Figura 38

35. Caso o prefixo (namespace) **app** apareça em vermelho, posicione o mouse sobre ele e clique no link Create namespace declaration **ou** pressione as teclas ALT + ENTER para importar automaticamente o namespace, conforme ilustrado na Figura 39.

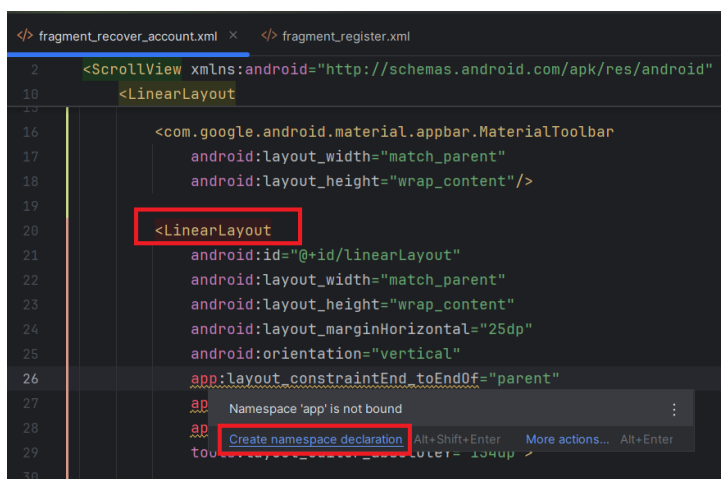


Figura 39

36. Realize as seguintes alterações no arquivo **fragment_recover_account.xml**, conforme destacado em vermelho na Figura 40:

- Adicione um TextView com o ID textViewMensagem; O valor do atributo text será android:text="Informe seu e-mail de cadastro para receber \n um link para alteração de senha."

- Inclua a propriedade `layout_marginTop` no `TextView` com o ID `textViewEmail`.

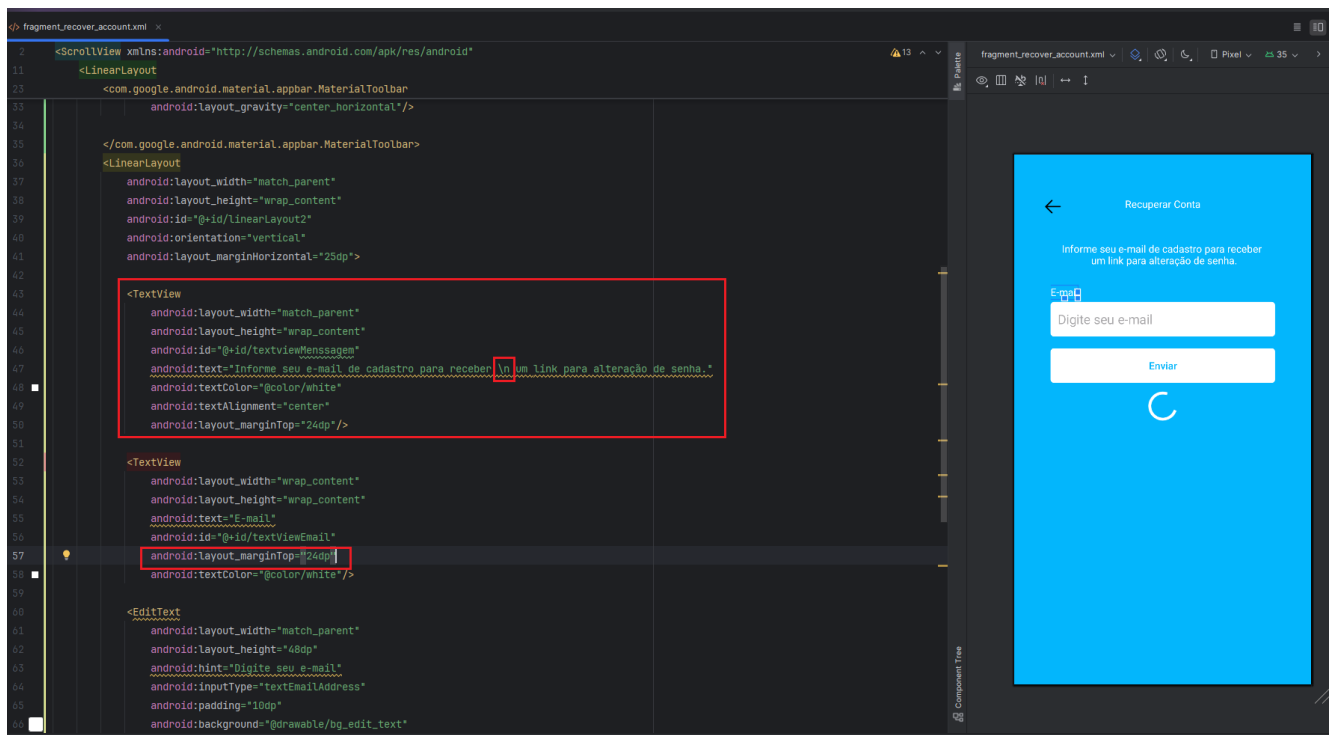


Figura 40

Observe o `\n` dentro do texto do `TextView`. O caracter `\n` renderiza uma quebra de linha no texto.

37. Configure o `viewBinding` do fragment **RecoverAccountFragment.kt** conforme Figura 41.

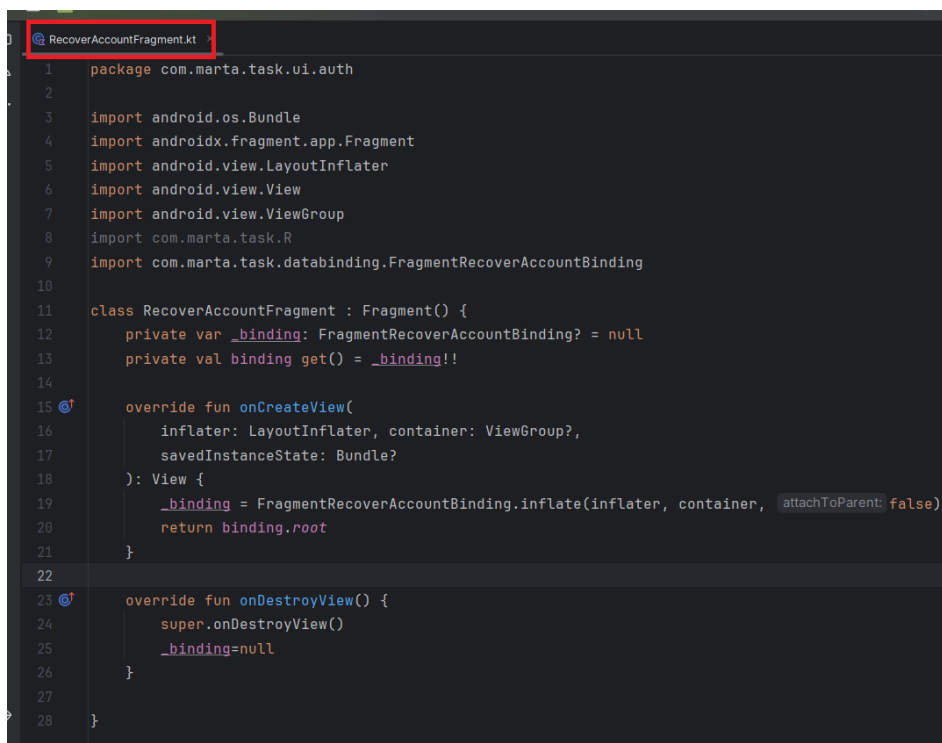
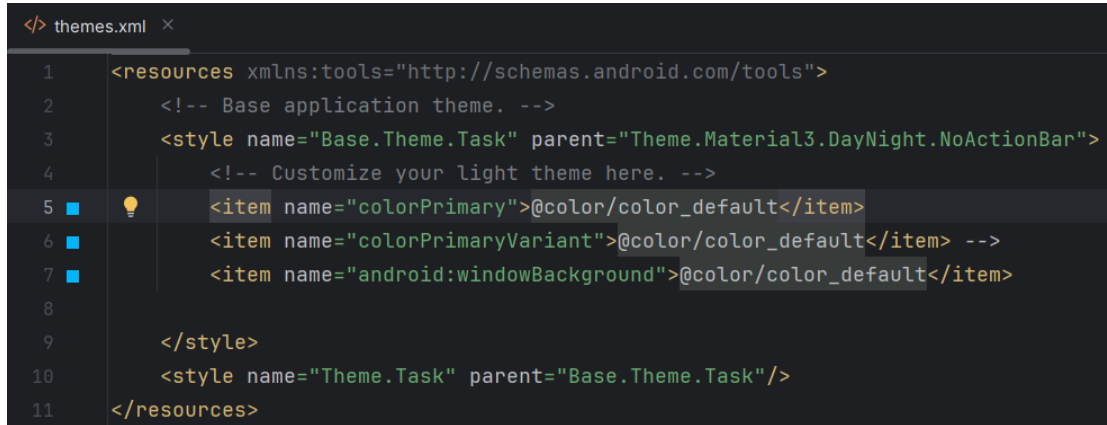


Figura 41

38. Inclua novas configurações de cores no arquivo **themes.xml**. Inclua os itens **ColorPrimaryVariant** e **ColorPrimary**, conforme Figura 42. Execute o emulador para observar as alterações.



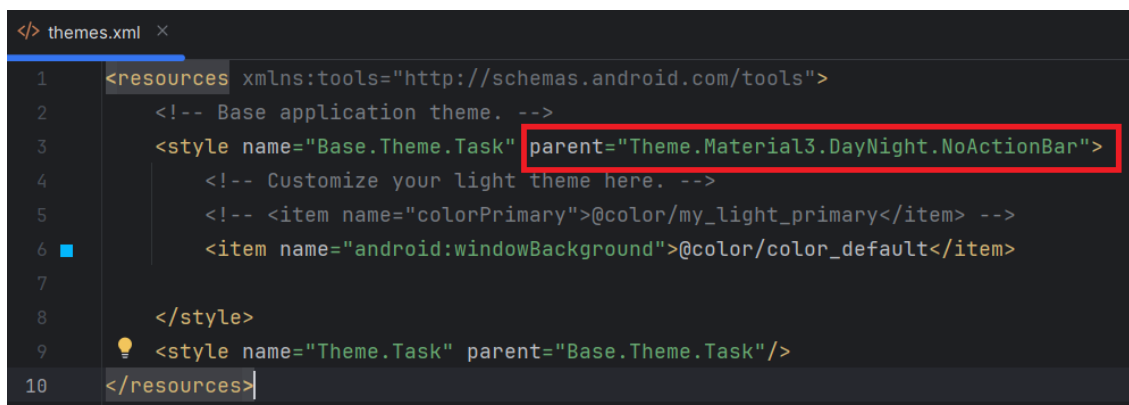
```
<?xml version="1.0" encoding="utf-8"?>
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Task" parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your light theme here. -->
        <item name="colorPrimary">@color/color_default</item>
        <item name="colorPrimaryVariant">@color/color_default</item> -->
        <item name="android:windowBackground">@color/color_default</item>
    </style>
    <style name="Theme.Task" parent="Base.Theme.Task"/>
</resources>
```

Figura 42

A **colorPrimary** define a cor principal do aplicativo, sendo utilizada nos elementos mais destacados da interface, como a AppBar (Toolbar), botões e outros componentes-chave. Já a **colorPrimaryVariant** representa uma variação mais clara ou mais escura da cor principal, geralmente aplicada para criar contraste, profundidade e reforçar a hierarquia visual. Um exemplo comum de uso da **colorPrimaryVariant** é na Status Bar, entre outros componentes da interface.

PARTE 2 – NAVEGABILIDADE ENTRE TELAS

39. Agora vamos fazer a navegabilidade entre as telas **Splash**, **Login** e **Cadastro**. Antes é importante remover a barra de navegação padrão do Android. Para isso, acesse o arquivo de **themes.xml** e coloque a opção **NoActionBar** conforme Figura 43.



```
<?xml version="1.0" encoding="utf-8"?>
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Task" parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
        <item name="android:windowBackground">@color/color_default</item>
    </style>
    <style name="Theme.Task" parent="Base.Theme.Task"/>
</resources>
```

Figura 43

O atributo parent é composto por diferentes partes, cada uma com um significado específico:

- Theme.Material3 → Indica que o tema base segue as diretrizes do Material Design 3, trazendo os novos padrões visuais do Android.
- DayNight → Garante suporte automático ao modo claro e escuro, permitindo que o sistema escolha a aparência com base nas configurações do dispositivo.
- NoActionBar → Define que o tema não incluirá a ActionBar por padrão, uma configuração comum quando se utiliza o Toolbar ou outras abordagens modernas de navegação na interface.

40. Agora vamos, de fato, iniciar a criação do gráfico de navegação do aplicativo. Esse gráfico definirá os caminhos possíveis que o usuário poderá seguir dentro do app. Para isso, clique com o botão direito na pasta **res** e selecione a opção **New>>Android Resource Directory**, conforme ilustrado na Figura 44.

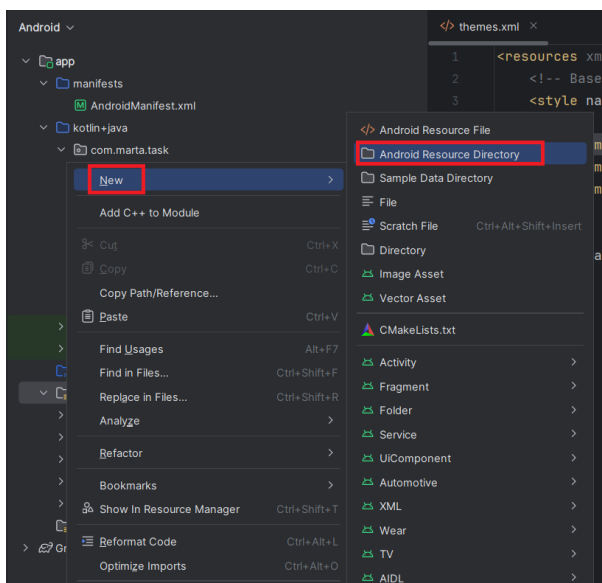


Figura 44

41. Uma nova janela será exibida. Nela, selecione a opção **navigation** no campo **Resource Type** e clique no botão **OK**, conforme mostrado na Figura 45. Após isso, será criada uma pasta chamada navigation.

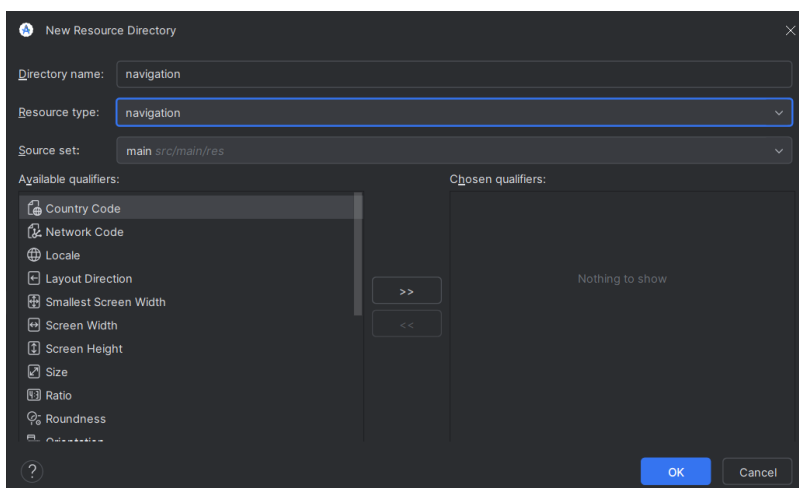


Figura 45

42. Dentro desta pasta **navigation**, criaremos nosso gráfico de navegação.

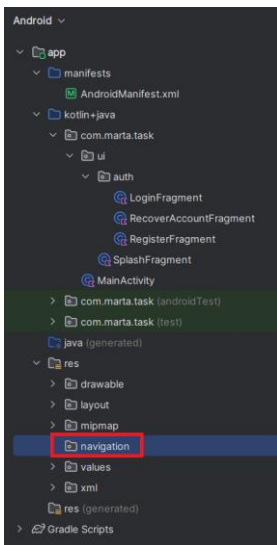


Figura 46

43. Sobre a pasta **Navigation** clique com o botão direito e selecione a opção **New>>Navigation Resource File** para criar o mapa de navegação conforme Figura 47.

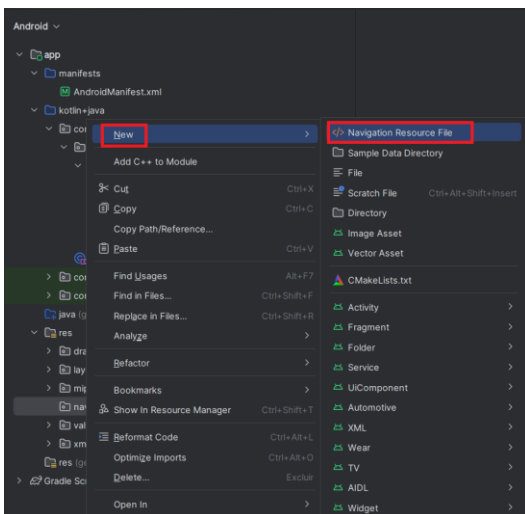


Figura 47

44. Uma tela será aberta conforme Figura 48. Informe o nome do gráfico de navegação **main_graph**.

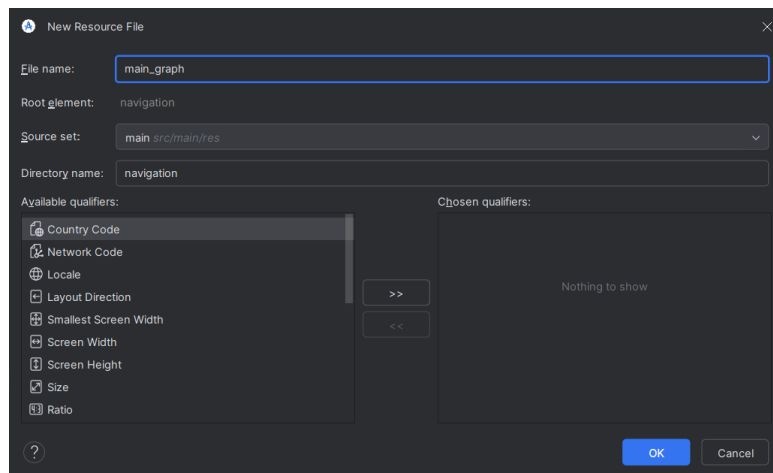


Figura 48

45. Uma janela pop-up será exibida, conforme mostrado na Figura 49. Essa mensagem informa sobre a adição de novas dependências ao projeto. Clique no botão **OK** para confirmá-las e adicioná-las automaticamente.

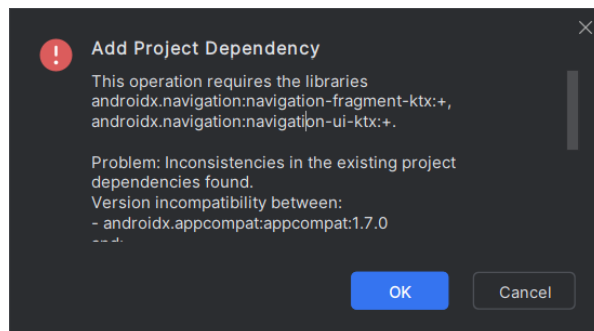


Figura 49

46. A primeira ação é adicionar a tela inicial de navegação. Para isso clique no ícone destacado na Figura 50, e selecione o **fragment_splash**.

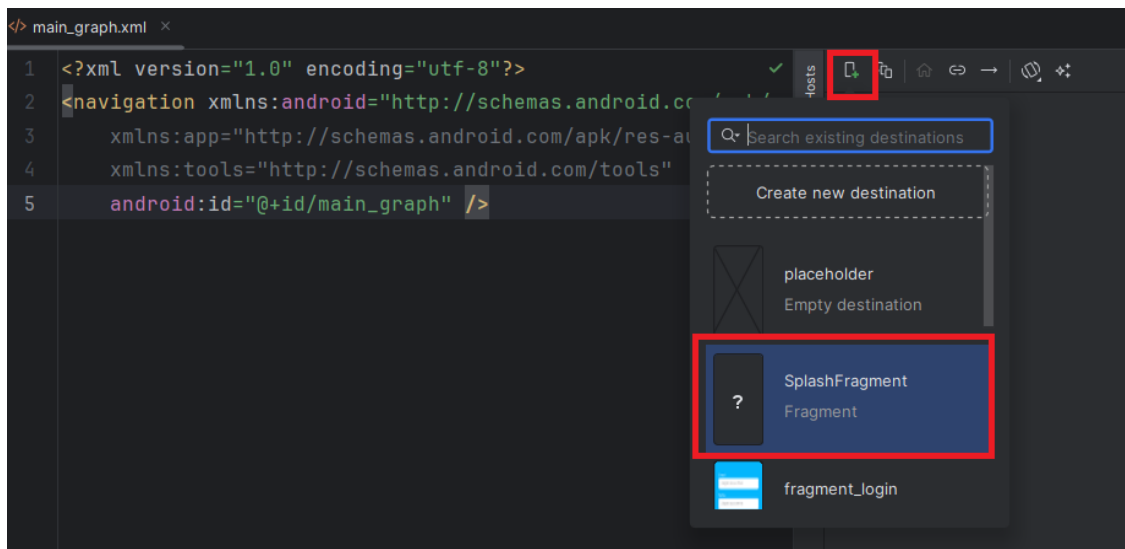


Figura 50

47. Observe que o fragment será adicionado ao gráfico e exibirá o ícone de uma casinha, indicando que se trata da tela inicial, conforme mostrado na Figura 51. A Activity que utilizar esse gráfico de navegação terá o **SplashFragment** definido como sua tela de entrada.

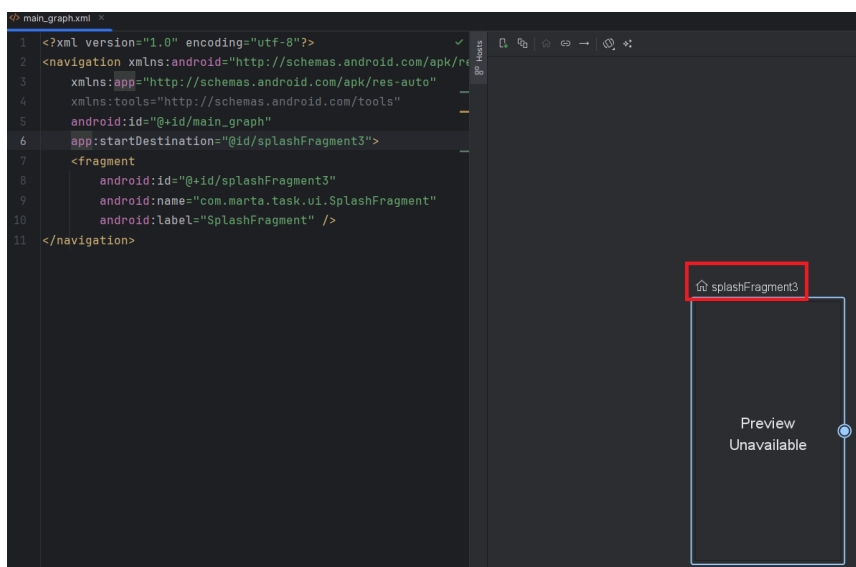


Figura 51

48. Feche o gráfico de navegação. Em seguida, **abra o arquivo activity_main.xml** para adicionar o `fragmentSplash` e vincular o gráfico de navegação. Insira o componente **FragmentContainerView** com os atributos mostrados nas Figuras 52 e 53.

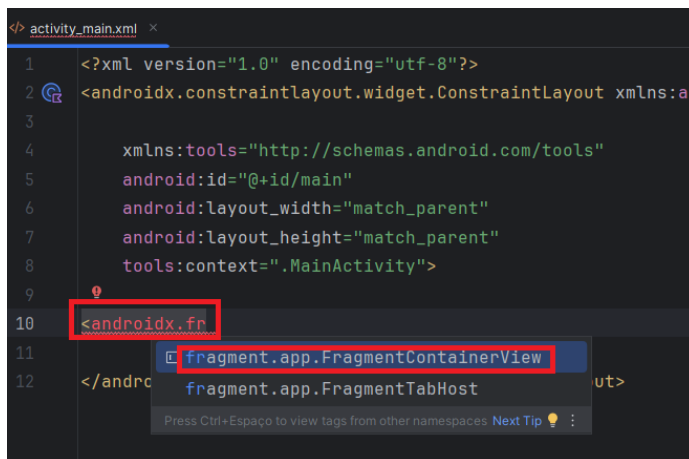


Figura 52

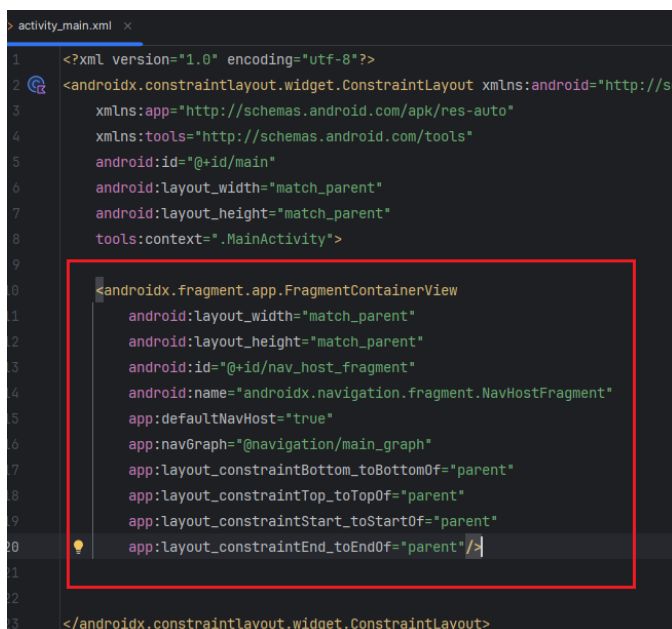


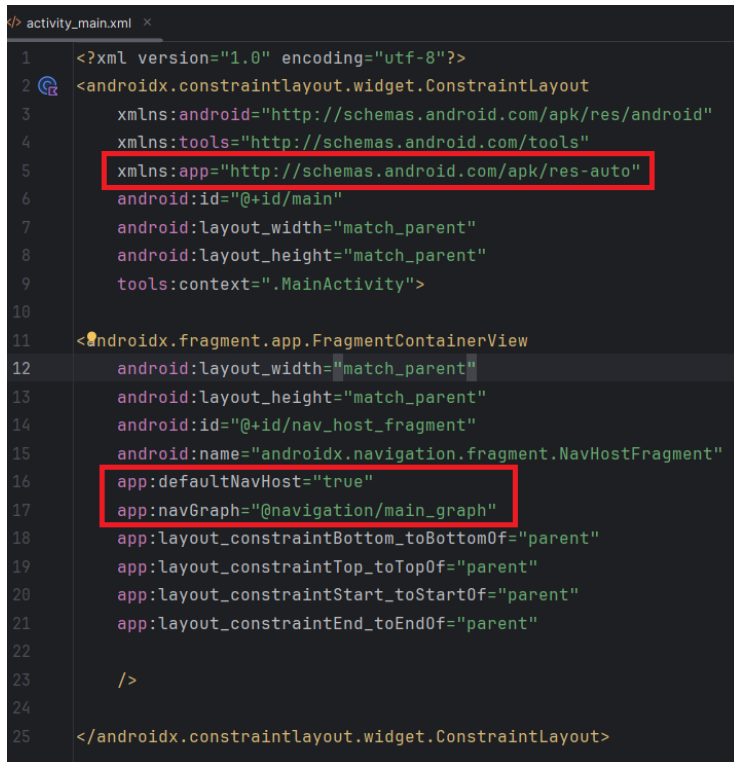
Figura 53

O atributo **android:name="androidx.navigation.fragment.NavHostFragment"** indica a biblioteca gerenciadora dos fragments de navegação.

O atributo **defaultNavHost="true"** garante que o botão Voltar do dispositivo (seja físico ou por navegação por gestos) funcione corretamente, retornando ao fragmento anterior dentro da pilha de navegação. Se esse atributo estiver definido como false, o comportamento muda: o botão Voltar não navega entre os fragments. Em vez disso, ele encerra a Activity, como se o usuário tivesse pressionado "Sair" do aplicativo.

O atributo **app:navGraph** define qual gráfico de navegação será carregado pelo componente de navegação. No caso, ele especifica o arquivo main_graph.xml como o gráfico a ser utilizado.

49. Clique nas teclas ALTER +ENTER para importar os namespaces. Dessa forma, os erros nos namespaces dos atributos defaultNavHost e navGraph serão corrigidos, conforme Figura 54.



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:tools="http://schemas.android.com/tools"
5     xmlns:app="http://schemas.android.com/apk/res-auto"
6     android:id="@+id/main"
7     android:layout_width="match_parent"
8     android:layout_height="match_parent"
9     tools:context=".MainActivity">
10
11     <androidx.fragment.app.FragmentContainerView
12         android:layout_width="match_parent"
13         android:layout_height="match_parent"
14         android:id="@+id/nav_host_fragment"
15         android:name="androidx.navigation.fragment.NavHostFragment"
16         app:defaultNavHost="true"
17         app:navGraph="@navigation/main_graph"
18         app:layout_constraintBottom_toBottomOf="parent"
19         app:layout_constraintTop_toTopOf="parent"
20         app:layout_constraintStart_toStartOf="parent"
21         app:layout_constraintEnd_toEndOf="parent"
22     />
23
24 </androidx.constraintlayout.widget.ConstraintLayout>
```

Figura 54

50. O próximo passo é fazer as restrições de alinhamento do `fragmentContainerView`, para isso alinhe todas as extremidades direita, esquerda, superior e inferior ao container pai, conforme Figura 55.

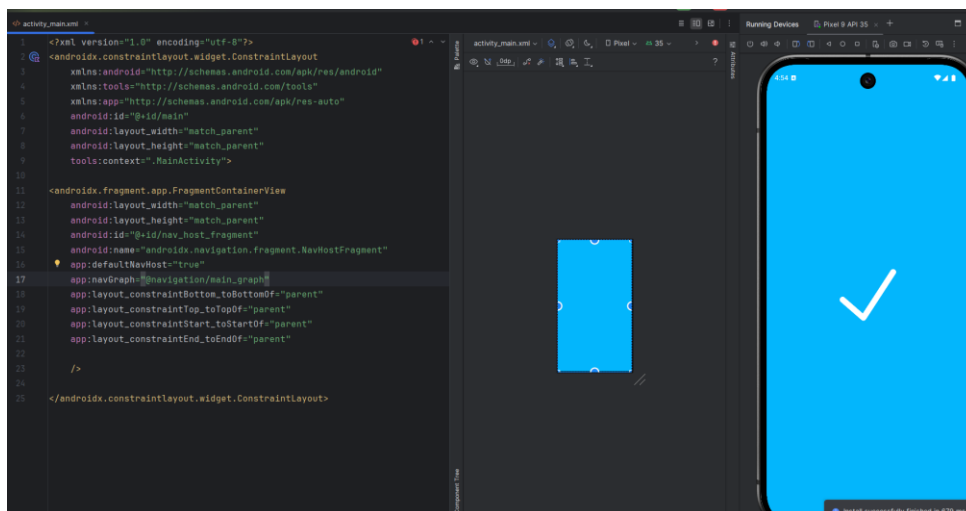


Figura 55

51. **Execute o aplicativo** e observe o carregamento do `SplashFragment` dentro da `MainActivity`. Conforme mostrado na Figura 56, a `MainActivity` é responsável por exibir o `SplashFragment` como tela inicial.

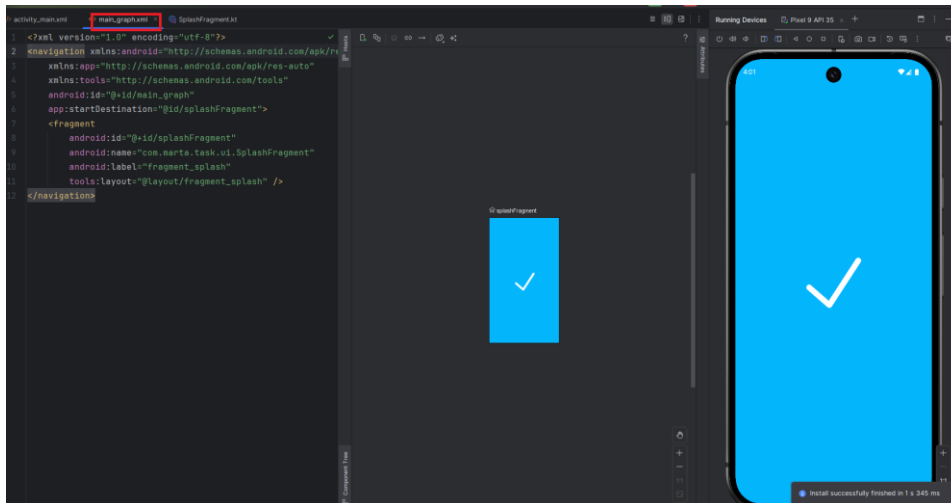


Figura 56

Qual é o objetivo do aplicativo? Após exibir o SplashFragment por 3 segundos, a aplicação deve redirecionar automaticamente para a tela de Login. Para isso, será necessário adicionar novos fragments de tela ao arquivo main_graph.xml.

52. Abra novamente o arquivo de navegação **main_graph.xml**, e inclua os demais fragments para realizar os links de navegabilidade, conforme Figura 57.

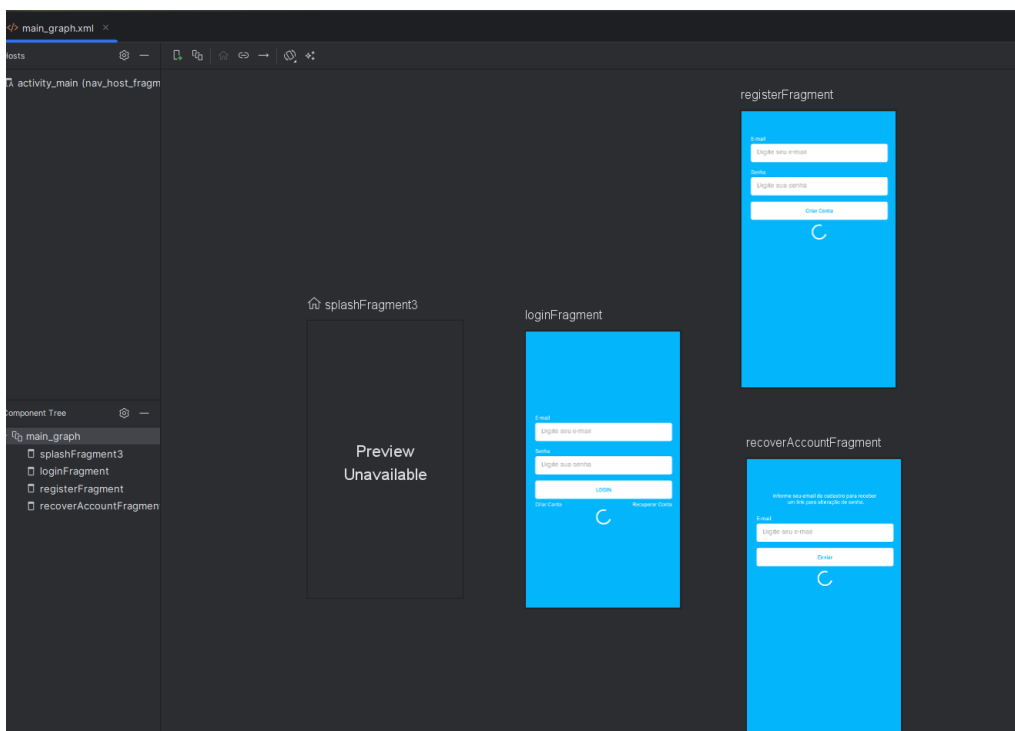


Figura 57

Observe que depois da tela de login o usuário tem as opções de acessar a tela de criar conta ou recuperar conta. Esses são os caminhos possíveis.

53. Selecione o fragment **loginFragment** e arraste uma seta de navegabilidade até o **registerFragment**, conforme Figura 58.

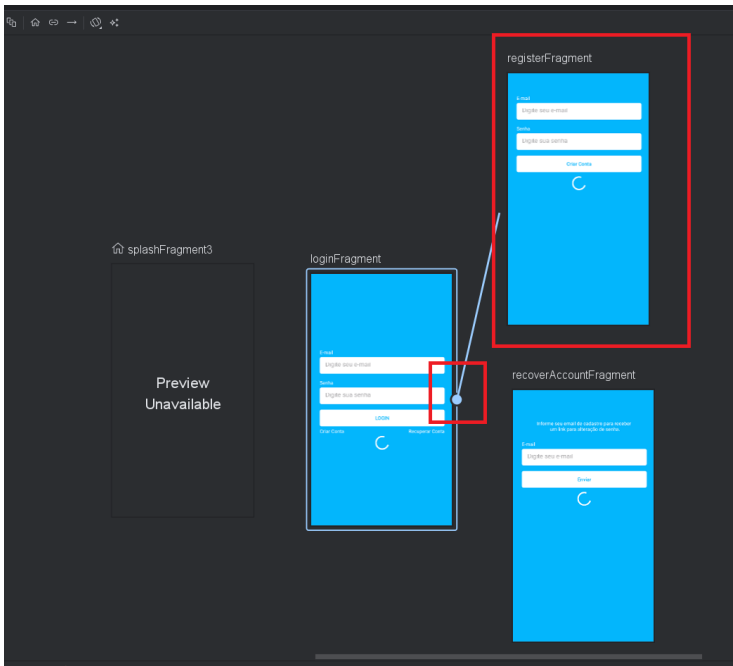


Figura 58

54. Selecione o fragmento **loginFragment** e arraste uma seta de navegação até o **RecoverAccountFragment**, conforme ilustrado na Figura 59. Com essa ação, dois caminhos de navegação foram criados.

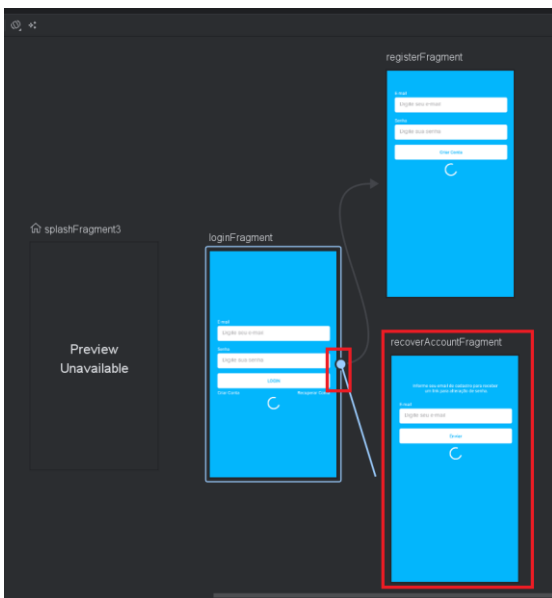


Figura 59

55. Pode ser útil que você precise reutilizar apenas o gráfico de navegação: loginFragment, registerFragment e recoverAccountFragment. Por exemplo: uma tela de e-commerce que ao final dos itens adicionados no carrinho você precise efetuar login e registrar uma conta. Para isso existe um recurso no Android que você pode criar subgráficos de navegação para reaproveitá-los em outras situação do seu App. Para isso, aperta a Tecla SHIFT e selecione os fragments: loginFragment, registerFragment e recoverAccountFragment. Depois clique com o botão direito, depois selecione a opção

Move to Nested Graph>>New Graph. Será criado um subgráfico de navegação conforme Figura 60.

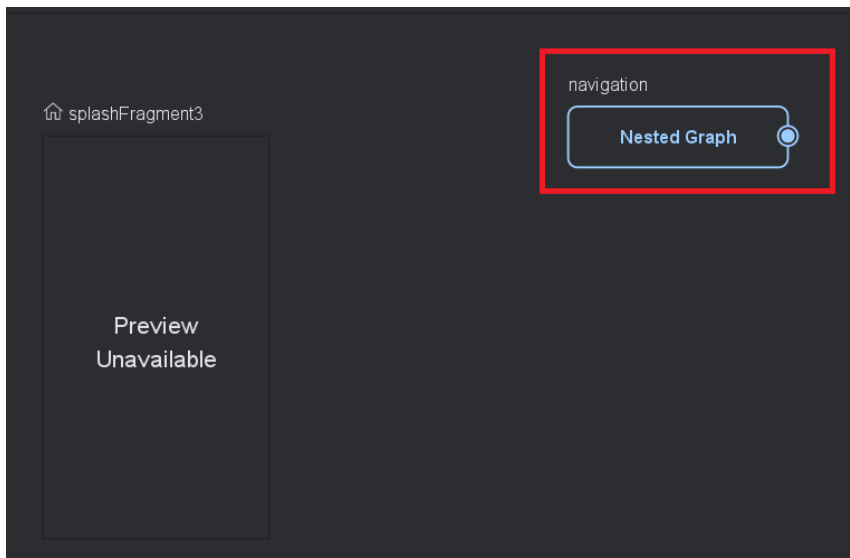


Figura 60

56. Altere o id do subgrafico de navegação para **authentication** conforme Figura 61.

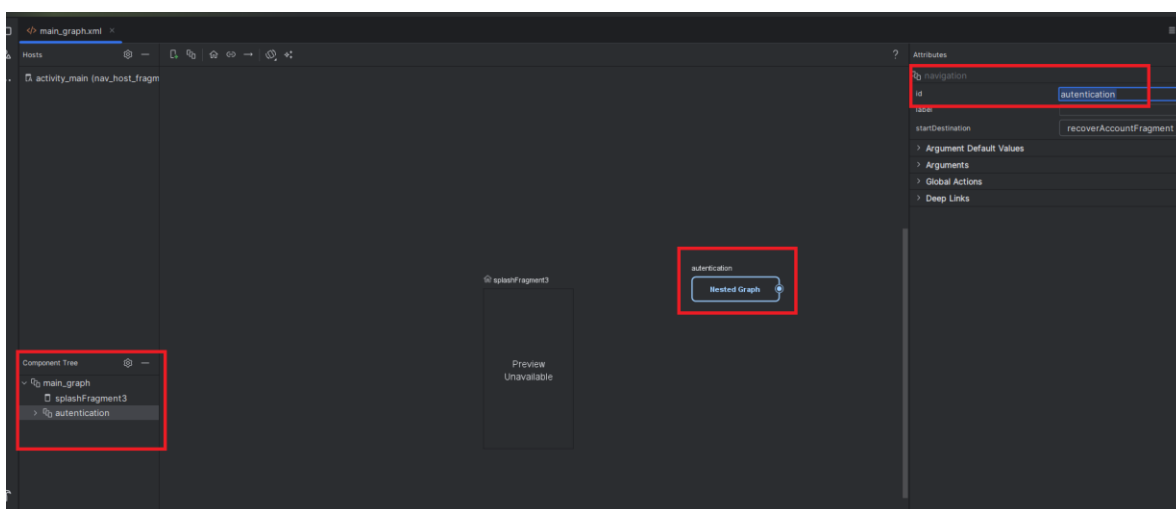


Figura 61

57. Dê um duplo clique no retângulo Nested Graph para acessar o subgráfico de navegação. Dentro do subgráfico, selecione o **loginFragment** e clique no ícone da casinha para defini-lo como a tela inicial. Para retornar ao gráfico principal, utilize a **Component Tree** e clique em **main_graph**, conforme ilustrado na Figura 62.

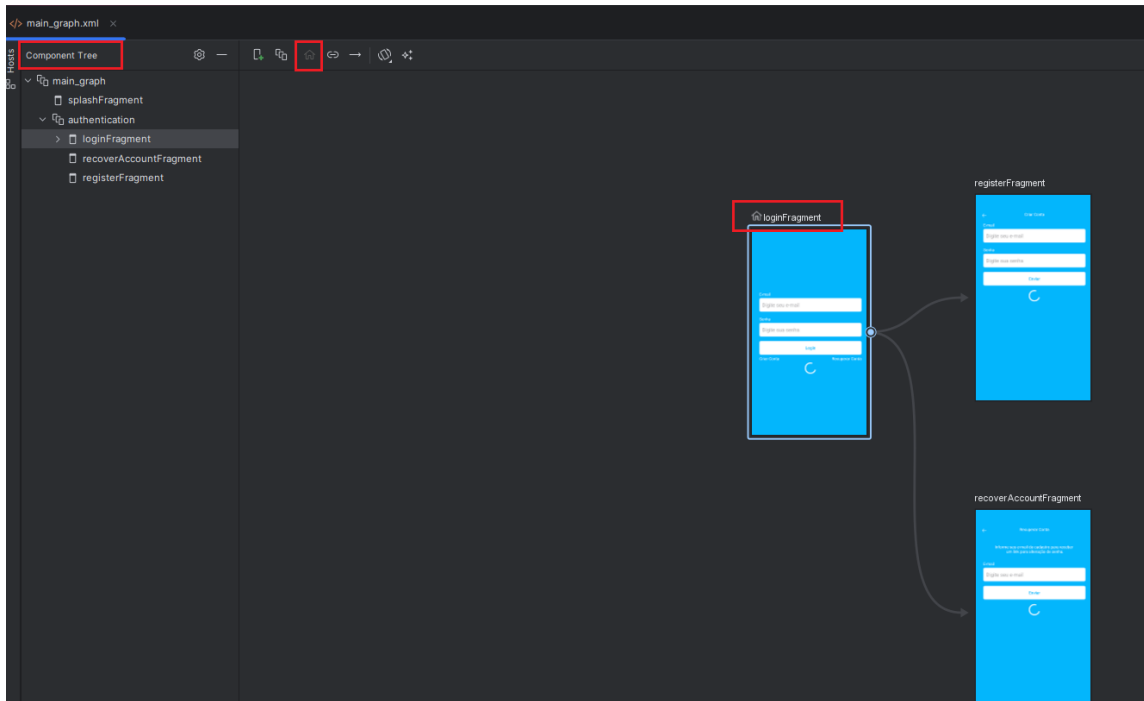


Figura 62

58. Crie uma seta de navegabilidade entre o **splashFragment** e o subgráfico **authentication**, conforme ilustrado na Figura 63. Vale ressaltar que a criação dessa seta de navegação, por si só, não faz com que a transição ocorra automaticamente ao executar o aplicativo no emulador. Para que a navegação aconteça, é necessário programar o seu início explicitamente.

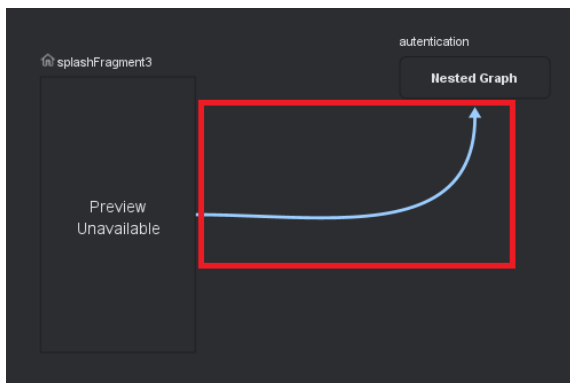
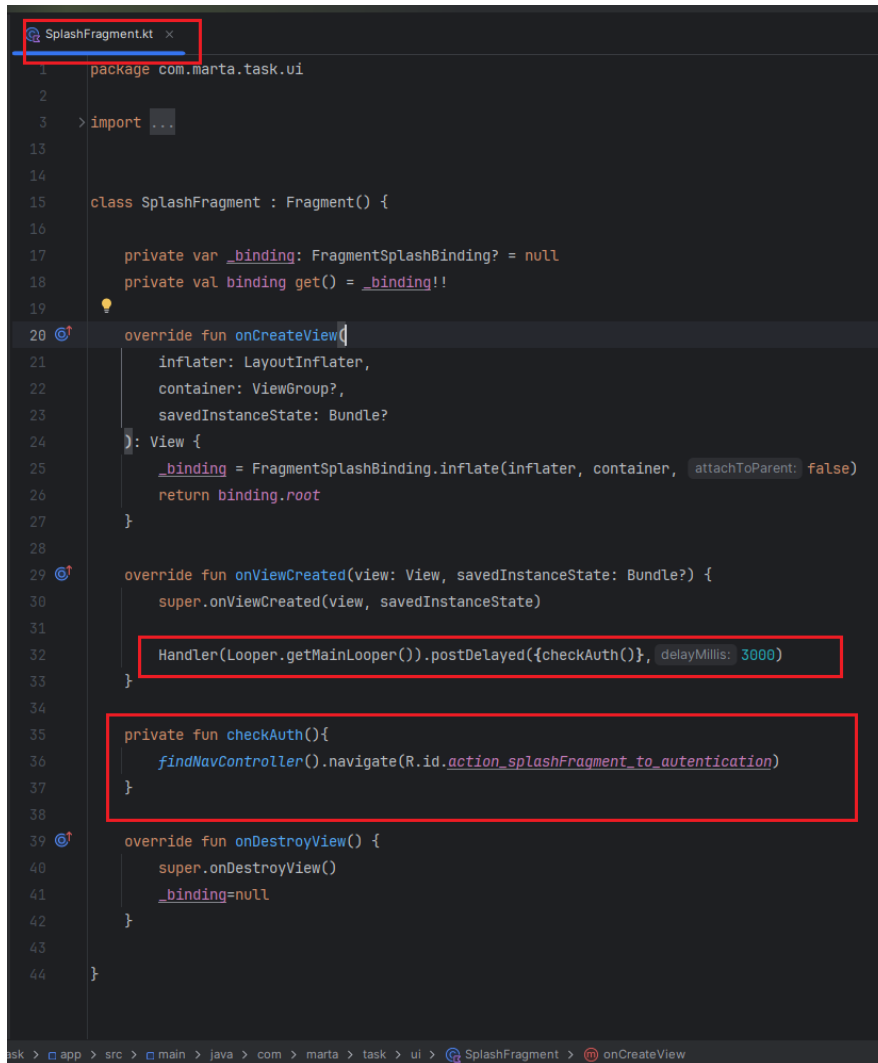


Figura 63

59. Abra o arquivo **SplashFragment.kt**, e inclua o código destacado na Figura 64.



```
1 package com.marta.task.ui
2
3 > import ...
4
5
6
7
8
9
10
11
12
13
14
15 class SplashFragment : Fragment() {
16
17     private var _binding: FragmentSplashBinding? = null
18     private val binding get() = _binding!!
19
20     override fun onCreateView(
21         inflater: LayoutInflater,
22         container: ViewGroup?,
23         savedInstanceState: Bundle?
24     ): View {
25         _binding = FragmentSplashBinding.inflate(inflater, container, attachToParent: false)
26         return binding.root
27     }
28
29     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
30         super.onViewCreated(view, savedInstanceState)
31
32         Handler(Looper.getMainLooper()).postDelayed({checkAuth()}, delayMillis: 3000)
33     }
34
35     private fun checkAuth(){
36         findNavController().navigate(R.id.action_splashFragment_to_authentication)
37     }
38
39     override fun onDestroyView() {
40         super.onDestroyView()
41         _binding=null
42     }
43
44 }
```

Figura 64

O método **Handler** agenda a execução da função `checkAuth()` após um atraso de 3 segundos (3000 milissegundos), na thread principal do Android.

O método **findNavController** é uma função do Navigation Component do Android que navega do `loginFragment` para o `homeFragment`, usando uma ação definida no gráfico de navegação. O `NavController` é um objeto do Navigation Component do Android que gerencia a navegação entre fragments da sua aplicação. Pense nele como um "controlador de rotas". Ele sabe qual tela (fragment) deve ser exibida em seguida, como voltar para a tela anterior, quais dados enviar entre telas, entre outras ações.

R.id.action_splashFragment3_to_authentication refere-se ao ID da ação de navegação, conforme ilustrado na Figura 65. O método futuramente verificará a autenticação do usuário, embora neste trecho ela apenas redirecione para outra tela.

O método **onCreateView** é responsável por inflar (ou seja, criar) o layout XML do fragmento. Já o método **onViewCreated** é chamado quando a interface do fragmento já está completamente criada e pronta para manipulação dos componentes visuais, como botões, textos e outras lógicas de interface. Por esse motivo, é dentro de `onViewCreated` que chamamos o `Handler`, garantindo que todos os elementos da tela já estejam disponíveis.

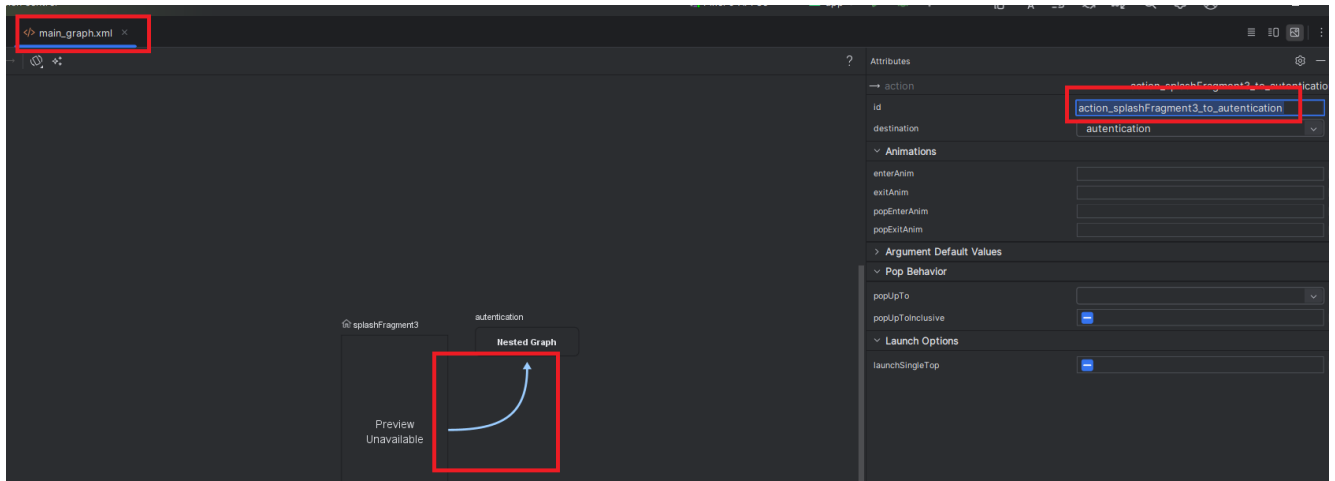


Figura 65

60. Abra o **fragment_login.xml** e mude o atributo visibility da progressBar conforme Figura 66.

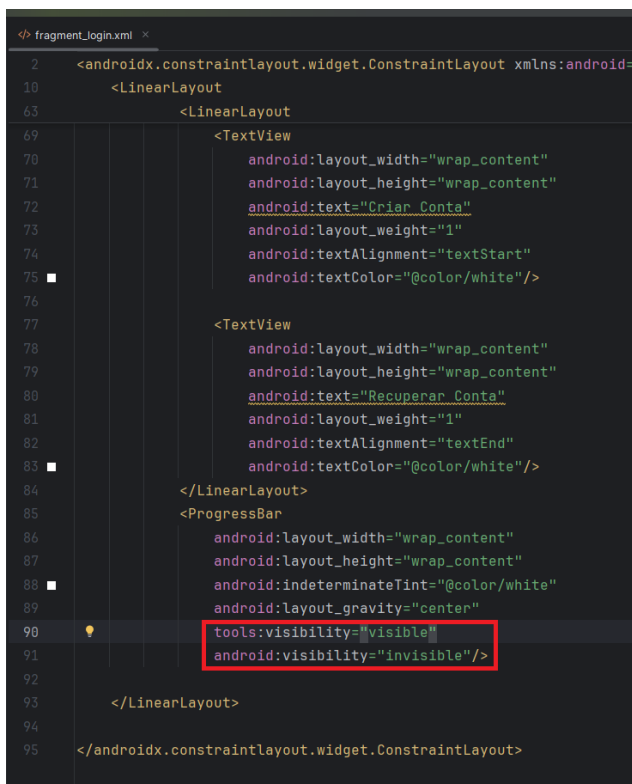


Figura 66

61. Abra o arquivo **fragment_login.xml** e coloque id nos TextViews Criar Conta e Recuperar Senha conforme Figura 67.

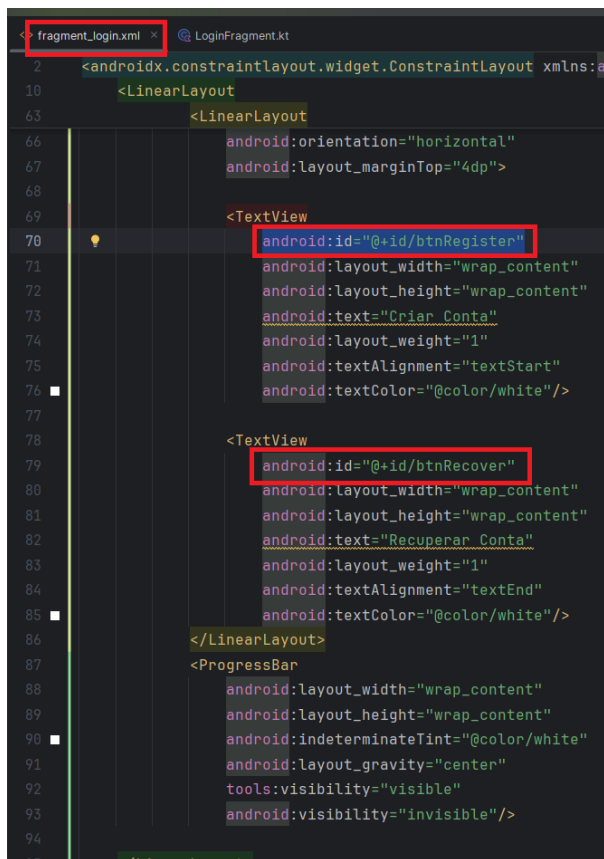


Figura 67

62. Vamos ativar a navegação na tela de login. Ao clicar no botão "Criar Conta", o usuário será direcionado para a tela de criação de conta. Da mesma forma, ao clicar no botão "Recuperar Conta", será redirecionado para a tela de recuperação de conta. Para isso digite o código da Figura 68.

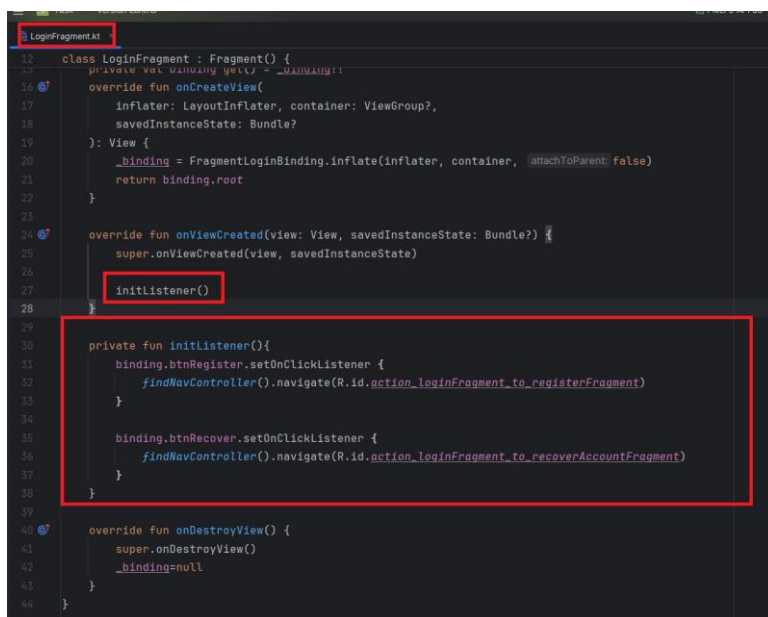


Figura 68

63. Abra o arquivo **fragment_register.xml** e inclua os atributos **visibility** conforme Figura 69. Execute o App no emulador para verificar o funcionamento.

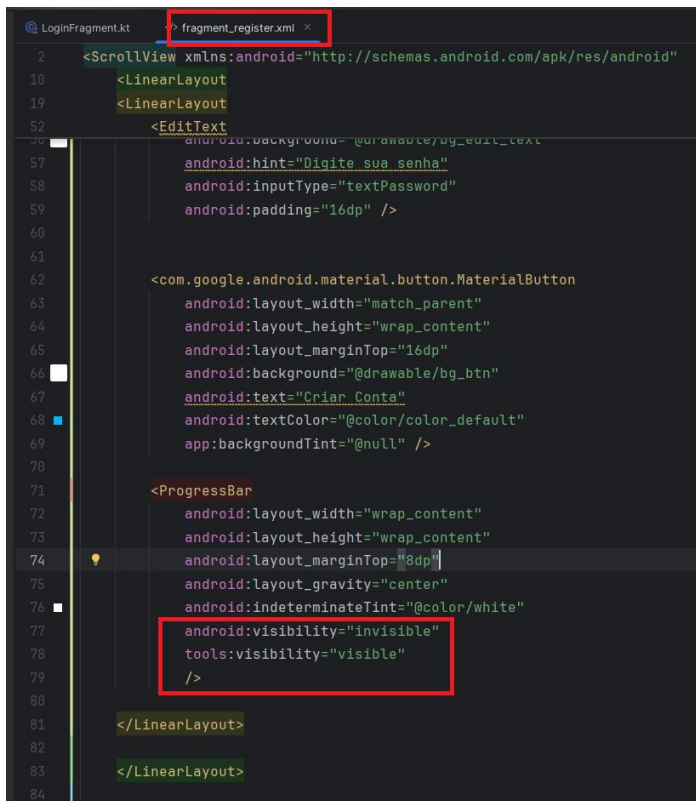


Figura 69

64. Agora vamos desabilitar o botão "voltar" físico do dispositivo enquanto o usuário estiver na tela de login. Isso é necessário porque, ao pressionar esse botão na tela de login, o usuário retornaria para a tela de splash, **o que não é adequado**. A tela de splash tem como única função exibir uma imagem, logotipo ou animação enquanto o aplicativo carrega seus recursos iniciais.

65. Para desabilitar o botão voltar da tela de login, abra o arquivo **main_graph.xml** e inclua os atributos **app:popUpTo="@id/splashFragment"** e **app:popUpToInclusive="true"** dentro da tag action do fragment splashFragment, conforme Figura 70.

```
18         tools:layout="@layout/fragment_register" />
19     <fragment
20         android:id="@+id/loginFragment"
21         android:name="com.marta.task.ui.auth.LoginFragment"
22         android:label="fragment_login"
23         tools:layout="@layout/fragment_login">
24         <action
25             android:id="@+id/action_loginFragment_to_registerFragment"
26             app:destination="@id/registerFragment" />
27         <action
28             android:id="@+id/action_loginFragment_to_recoverAccountFragment"
29             app:destination="@id/recoverAccountFragment" />
30     </fragment>
31 </navigation>
32 <fragment
33     android:id="@+id/splashFragment"
34     android:name="com.marta.task.ui.SplashFragment"
35     android:label="SplashFragment" >
36     <action
37         android:id="@+id/action_splashFragment_to_authentication"
38         app:destination="@id/authentication"
39         app:popUpTo="@id/splashFragment"
40         app:popUpToInclusive="true"
41     />
42 </fragment>
43 </navigation>
```

Figura 70

O atributo **app:popUpTo="@id/splashFragment"** indica que, ao executar a ação de navegação de voltar da tela de login para a de Splash, a pilha de telas (back stack) será esvaziada até o fragmento splashFragment, ou seja, todos os fragmentos que estão no topo da pilha até encontrar o splashFragment serão removidos.

O atributo **app:popUpToInclusive="true"** complementa o popUpTo. Quando ele está definido como true, além dos fragmentos acima do splashFragment, o próprio splashFragment também é removido da pilha. Significa: A tela de splash será completamente retirada da pilha de navegação, impedindo que o usuário volte para ela usando o botão "voltar" do dispositivo.

Isso faz com que, ao navegar, o aplicativo remova da pilha tanto a tela de splash quanto qualquer outra que estiver acima dela. Assim, se você está, por exemplo, indo para a tela de login ou para a tela principal, o botão "voltar" não levará mais o usuário de volta à splash screen.

66. Crie o **HomeFragment** dentro da pasta **ui**, conforme Figura 71.

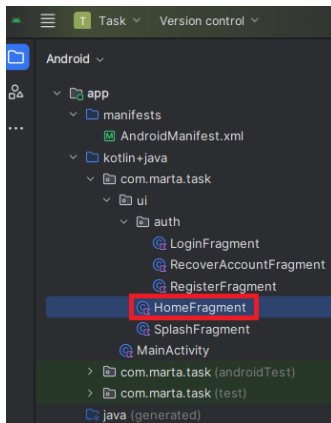


Figura 71

67. Limpe o código do **HomeFragment** para que fique apenas o evento **onCreateView**, conforme Figura 72.

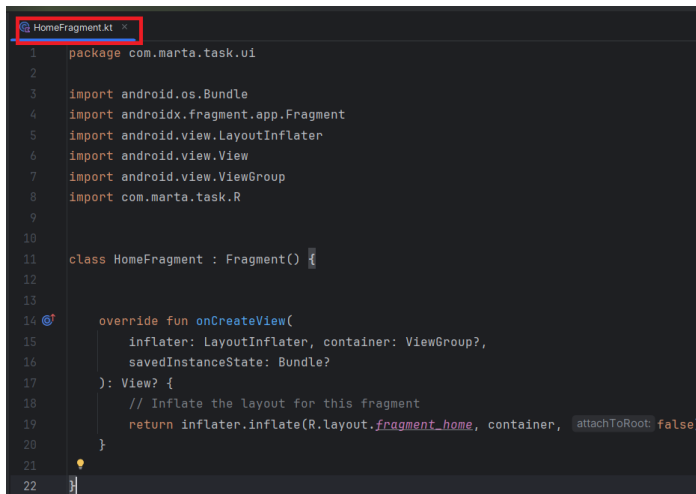


Figura 72

68. Crie o **TodoFragment** dentro da pasta **ui**, conforme ilustrado na Figura 73. Essa tela será responsável por exibir **tarefas pendentes** do Kanban. Limpe o código Kotlin do arquivo **TodoFragment.kt**, mantendo apenas o evento **onCreateView**.

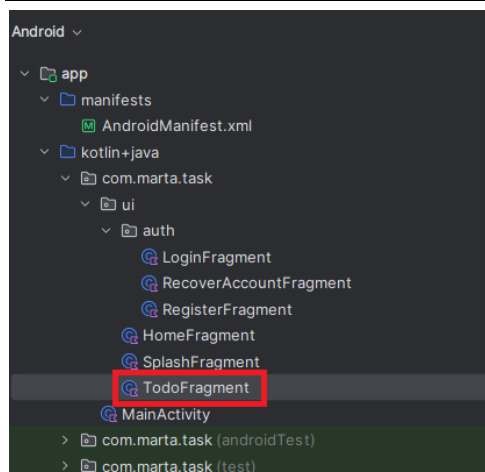


Figura 73

69. Crie o **DoingFragment** dentro da pasta **ui**, conforme ilustrado na Figura 74. Essa tela será responsável por exibir a listagem das **tarefas** que o usuário está **realizando no momento**. Limpe o código Kotlin do arquivo DoingFragment.kt, mantendo apenas o evento onCreateView.

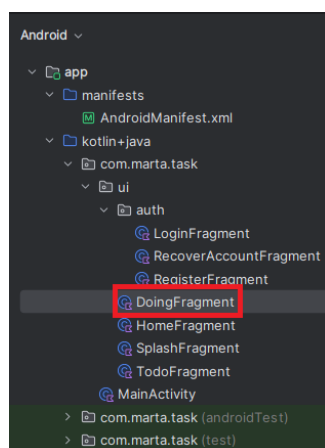


Figura 74

70. Crie o **DoneFragment** dentro da pasta **ui**, conforme ilustrado na Figura 75. Essa tela será responsável por exibir a listagem das **tarefas concluídas** pelo usuário. Limpe o código Kotlin do arquivo DoneFragment.kt, mantendo apenas o evento onCreateView.

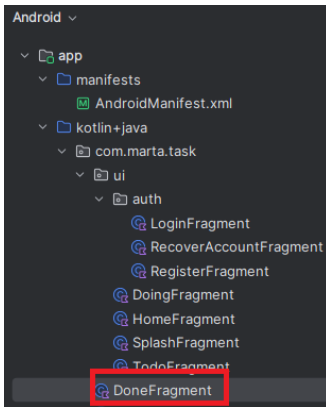


Figura 75

71. Vamos habilitar o viewBinding nos fragments: HomeFragment, TodoFragment, DoingFragment e DoneFragment conforme Figura 76.

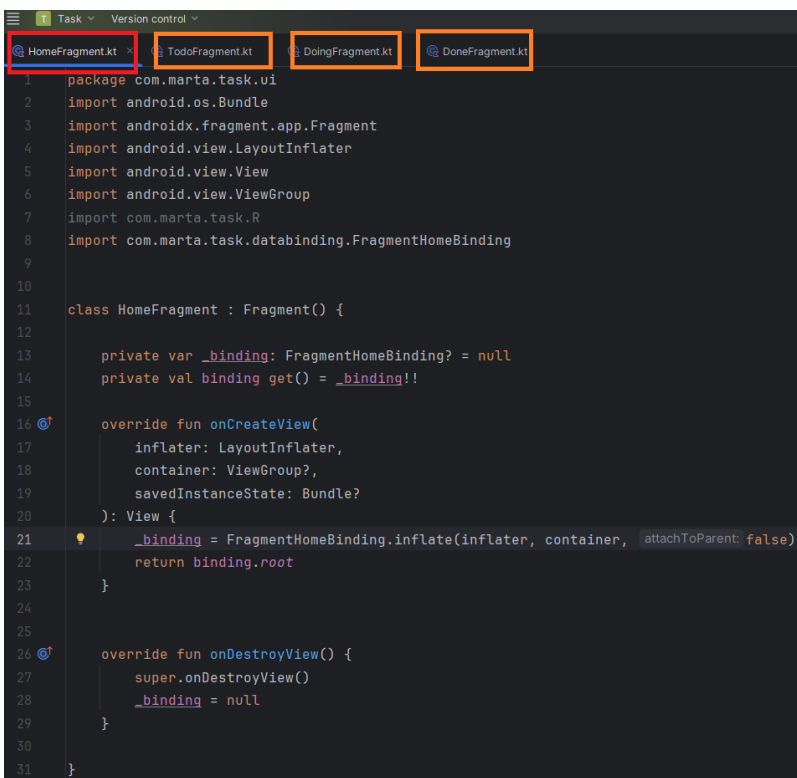


Figura 76

72. Precisamos fazer um ajuste de configuração na navegabilidade. Para isso, abra o arquivo **main_graph.xml**, adicione o **homeFragment** e conecte o **nested graph authentication** ao homeFragment, conforme ilustrado na Figura 77. A partir de qualquer tela que está dentro do nested graph poderá nevegar para a homeFragment.

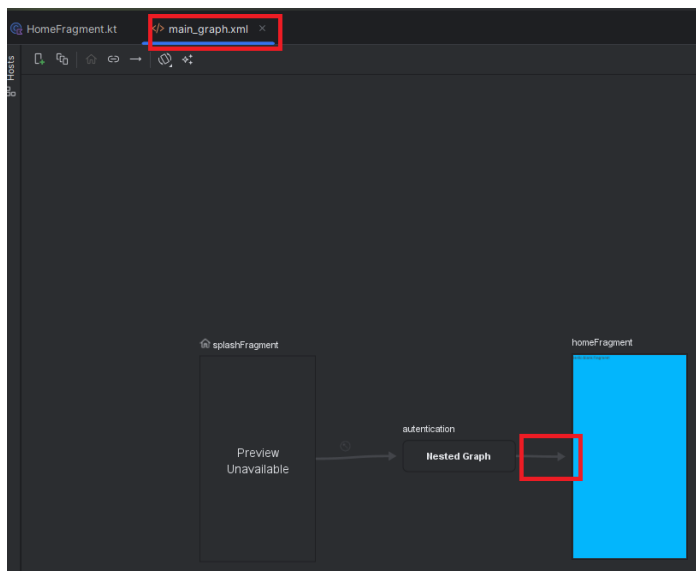


Figura 77

73. Abra o arquivo `fragment_login.xml` e atribua um ID `buttonLogin` ao botão de login. Em seguida, acesse o arquivo `LoginFragment.kt` e, dentro do método `initListener`, adicione o código destacado em vermelho na Figura 78. Esse código é responsável por iniciar a navegação da tela de login para a tela Home.

```
10 import com.marta.task.databinding.FragmentLoginBinding
11
12 class LoginFragment : Fragment() {
13
14     private var _binding: FragmentLoginBinding? = null
15     private val binding get() = _binding!!
16
17     override fun onCreateView(
18         inflater: LayoutInflater, container: ViewGroup?,
19         savedInstanceState: Bundle?
20     ): View {
21         _binding = FragmentLoginBinding.inflate(inflater, container, attachToParent = false)
22         return binding.root
23     }
24
25     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
26         super.onViewCreated(view, savedInstanceState)
27
28         initListener()
29     }
30
31     private fun initListener(){
32         binding.buttonLogin.setOnClickListener {
33             findNavController().navigate(R.id.action_global_homeFragment)
34         }
35
36         binding.btnRegister.setOnClickListener {
37             findNavController().navigate(R.id.action_loginFragment_to_registerFragment)
38         }
39
40         binding.btnRecover.setOnClickListener {
41             findNavController().navigate(R.id.action_loginFragment_to_recoverAccountFragment)
42         }
43     }
44 }
```

Figura 78

74. Abra o emulador e teste a navegação, clicando no botão de login para verificar se a transição para a tela Home está funcionando corretamente.

75. Vamos incluir um novo ícone no projeto. Para isso, clique com o botão direito na pasta **Drawable** e selecione **New >> Vector Asset**, conforme ilustrado na Figura 79. Em seguida, defina a cor do ícone como branco. Esse ícone será utilizado posteriormente em um `FloatingActionButton` dentro do `TabLayout`. Caso necessário, a cor também pode ser ajustada diretamente no XML, garantindo um contraste adequado com o fundo e uma melhor legibilidade do elemento.

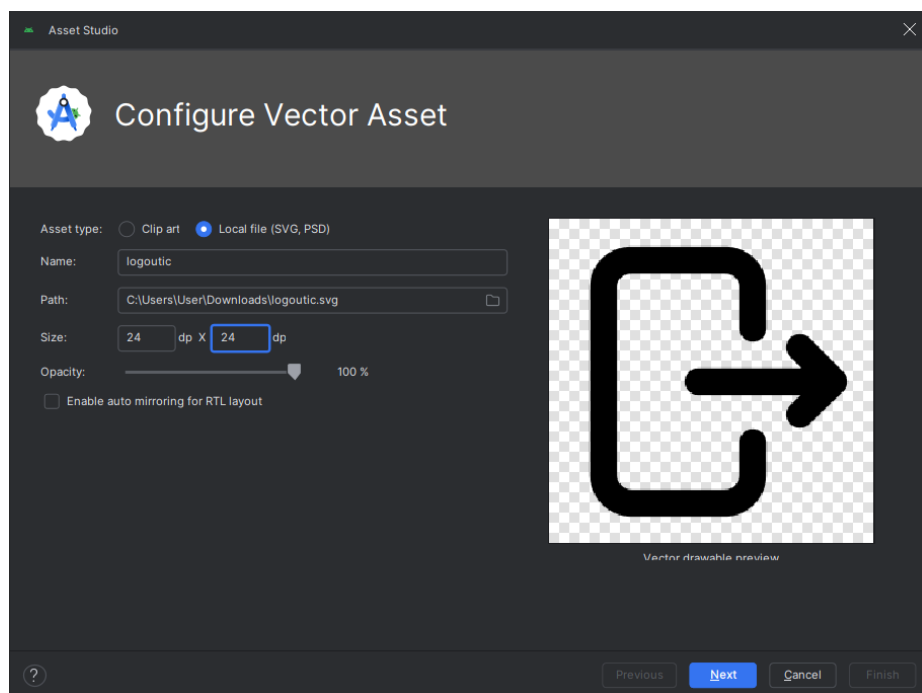


Figura 79

76. Agora vamos configurar o componente `TabLayout` e realizar algumas alterações no arquivo **homeFragment.xml**, conforme destacado em vermelho na Figura 80. As modificações incluem:

- Substituir o `FrameLayout` por um `CoordinatorLayout`;
- Adicionar o componente `AppBarLayout`;
- Incluir um `ConstraintLayout` com o ID `ConstraintLayout1`;
- Inserir o componente `ViewPager2`.

Essas mudanças são necessárias para estruturar a interface com abas (tabs) e permitir a navegação entre diferentes fragmentos dentro da tela principal.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout 1
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    tools:context=".ui.HomeFragment" >

    <!-- Topo com comportamento de scroll --> 2
    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@color/color_default"
        android:elevation="4dp">

        <androidx.constraintlayout.widget.ConstraintLayout 3
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:id="@+id/ConstraintLayout1">

            <!-- Tablayout ficará aqui -->

        </androidx.constraintlayout.widget.ConstraintLayout>
    </com.google.android.material.appbar.AppBarLayout>

    <!-- Conteúdo que responde ao scroll -->
    <androidx.viewpager2.widget.ViewPager2 4
        android:id="@+id/viewPager"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:layout_behavior="@string/appbar_scrolling_view_behavior"/>

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Figura 80

O AppBarLayout é um componente do Material Design, fornecido pela biblioteca `com.google.android.material.appbar`. Ele é um **container vertical que organiza elementos da parte superior da tela**, como: Toolbars (barras de ferramentas), TabLayout (navegação por abas) e CollapsingToolbarLayout (barra expansível e recolhível). É frequentemente utilizado para criar barras de navegação superiores

O ViewPager2 é um componente do Android que mostra diferentes fragmentos ou layouts em páginas separadas. Ele é a versão atualizada e recomendada do antigo ViewPager, com melhorias e mais flexibilidade.

Para que o ViewPager2 “converse” com o AppBarLayout — ou seja, para que o topo da tela reaja ao movimento de rolagem (scroll inteligente), podendo subir, desaparecer ao rolar ou permanecer fixo quando necessário — é essencial que o TabLayout esteja contido dentro do AppBarLayout.

Além disso, o uso do CoordinatorLayout é fundamental quando se deseja que os elementos da interface interajam entre si, especialmente em cenários que envolvem rolagem e animações, permitindo um comportamento mais dinâmico e alinhado ao padrão do Material Design.

O atributo `app:layout_behavior` é responsável por conectar o scroll do conteúdo — como o `ViewPager2` — ao `AppBarLayout`. Em outras palavras, ele indica ao sistema que essa view deve reagir ao comportamento do `AppBarLayout` durante a rolagem.

Sem esse atributo, o topo da interface, como as abas (`TabLayout`) ou a toolbar, permanece estático, sem qualquer interação com o scroll, o que impede a ocorrência do chamado “scroll inteligente”. Por outro lado, ao utilizá-lo, o `AppBarLayout` passa a subir e descer em sincronia com o movimento da tela, permitindo que as animações e transições funcionem de forma fluida e conforme o padrão do Material Design.

77. Ainda no arquivo **fragment_home.xml**, insira dentro do `ConstraintLayout` os seguintes componentes, conforme destacado em vermelho na Figura 81:

- Um `TabLayout`, que será utilizado para exibir as abas de navegação;
- Um `ImageButton`, que pode ser utilizado para ações adicionais, como configurações ou filtros.

Esses elementos contribuirão para uma interface mais interativa e funcional na tela principal do aplicativo.

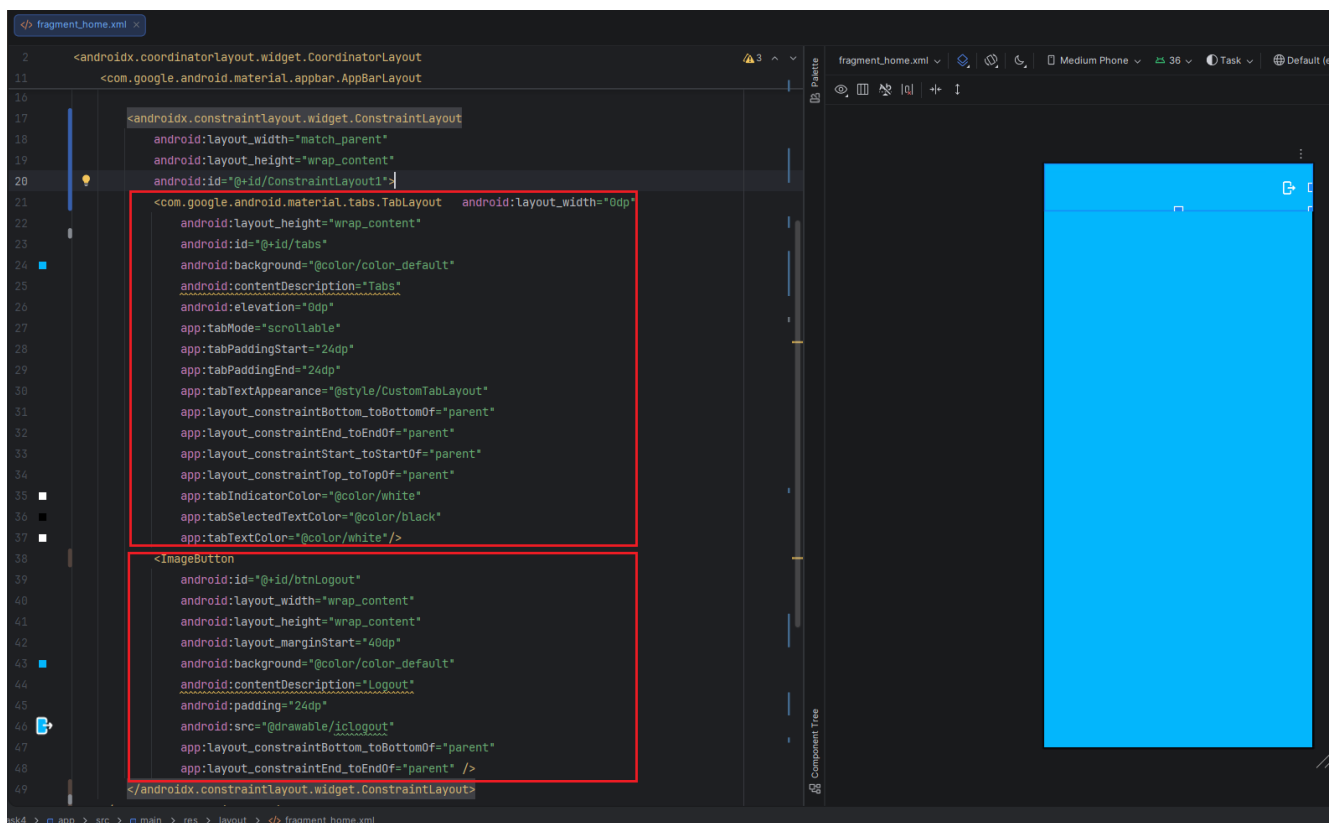


Figura 81

O atributo `android:elevation="0dp"` remove a sombra (elevação) do `TabLayout`, deixando-o no mesmo nível dos outros elementos.

O atributo `app:tabIndicatorColor="@color/white"` define a cor da aba ativa como branco.

PARTE 3 – UTILIZANDO O COMPONENTE TABLAYOUT

78. Agora, vamos configurar e criar o componente TabLayout, que permitirá a navegação por abas na interface. Abra o arquivo `fragment_home.xml`, insira dentro do `ConstraintLayout` os seguintes componentes, conforme destacado em vermelho nas Figuras 81 e 82:

- Um `TabLayout`, que será utilizado para exibir as abas de navegação;
- Um `ImageButton`, que pode ser utilizado para ações adicionais, como configurações ou filtros.

Esses elementos contribuirão para uma interface mais interativa e funcional na tela principal do aplicativo.

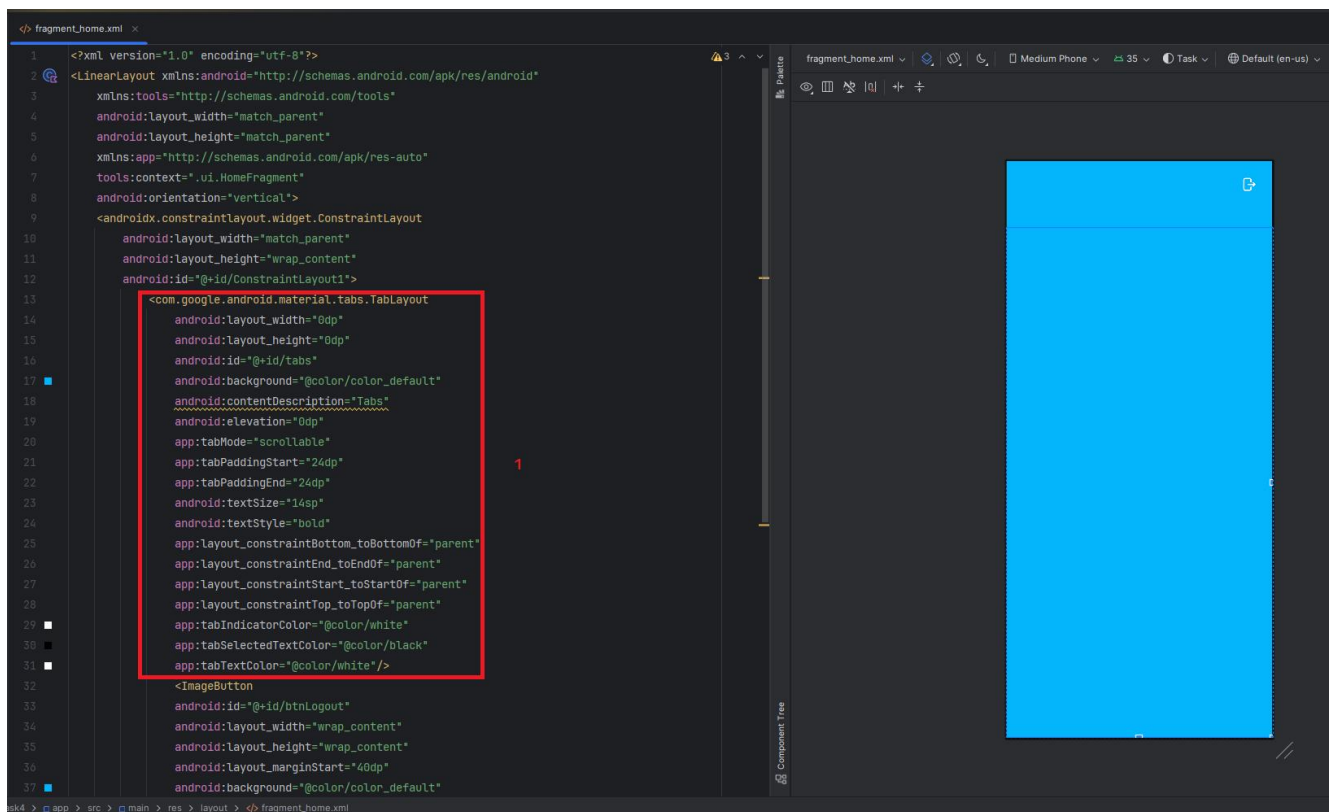


Figura 81

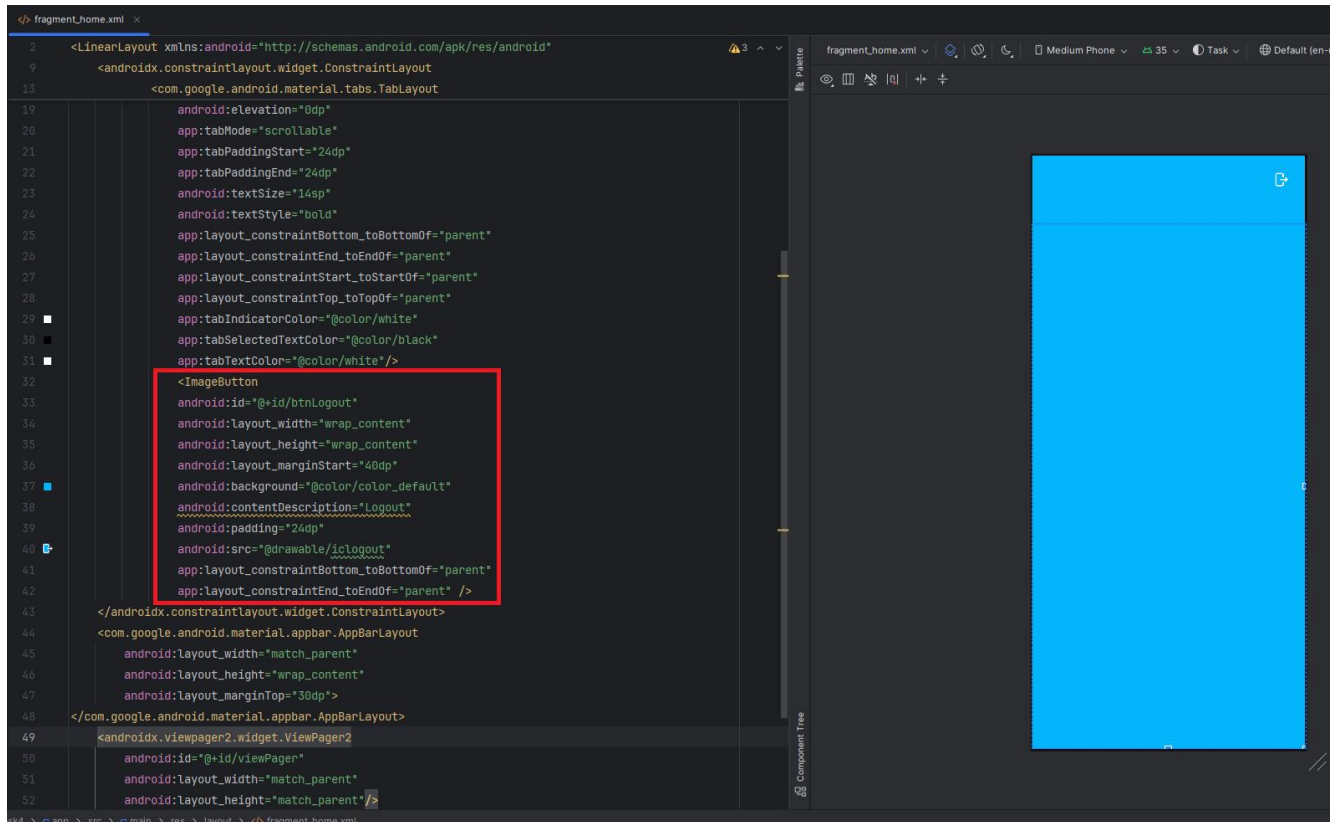


Figura 82

Vamos as explicações de cada atributo do TabLayout:

- O atributo `android:elevation="0dp"` remove a sombra (elevação) do TabLayout, deixando-o no mesmo nível dos outros elementos.
- O atributo `app:tabIndicatorColor="@color/white"` define a cor da aba ativa como branco.
- O atributo `android:contentDescription` é usado para fornecer uma descrição textual de um elemento da interface, principalmente para acessibilidade. Isso significa que, para leitores de tela (como o TalkBack no Android), o componente TabLayout será anunciado como "Tabs" para usuários com deficiência visual.
- O atributo `app:tabMode` controla como as abas são distribuídas horizontalmente, podendo assumir dois valores principais: `fixed` e `scrollable`. O valor `fixed` (valor padrão) todas as abas têm largura igual e são distribuídas para ocupar 100% da largura do TabLayout. Não permite rolagem horizontal. O valor `scrollable` cada aba tem largura mínima necessária para caber o conteúdo (texto/ícone). Se o conjunto de abas ultrapassar a largura da tela, o TabLayout se torna rolável horizontalmente.

79. Mude a cor do ícone de logout conforme Figura 83

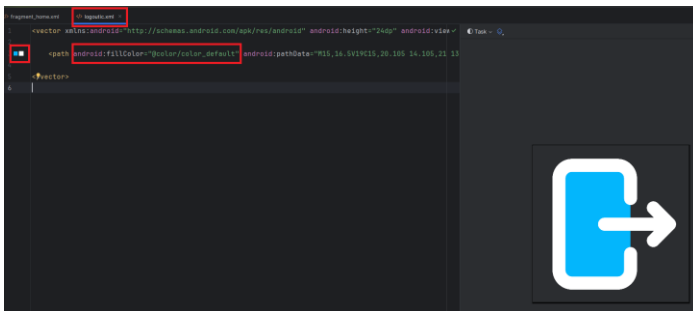


Figura 83

80. Clique com o botão direito na **pasta ui** e crie um novo **package** chamado **adapter**, conforme ilustrado na Figura 84. Dentro desse pacote, será criada uma classe responsável por gerenciar uma lista de dados. Essa lista armazenará as telas (Fragments) que representam as tarefas do Kanban e será utilizada para exibi-las no componente ViewPager2.

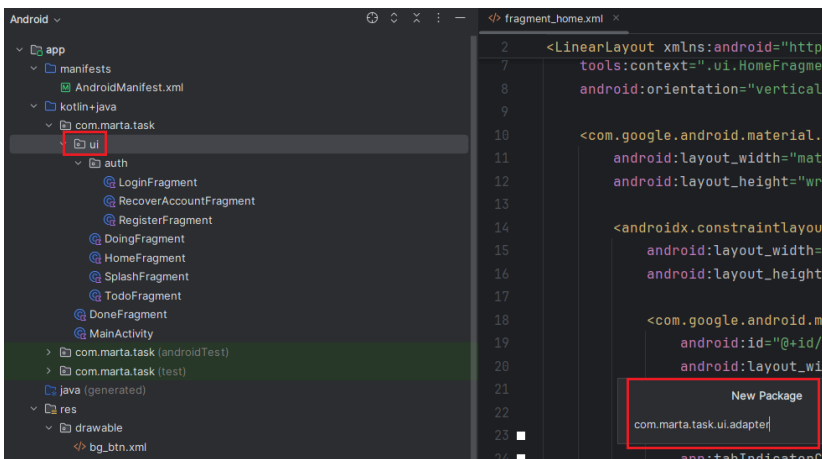


Figura 84

81. Clique com o botão direito no **package adapter**, e escolha a opção **New>> Kotlin/class File** conforme Figura 85.

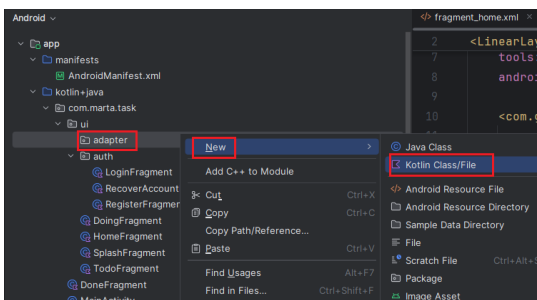


Figura 85

82. Crie a **classe** com o nome **ViewPagerAdapter** conforme Figura 86.

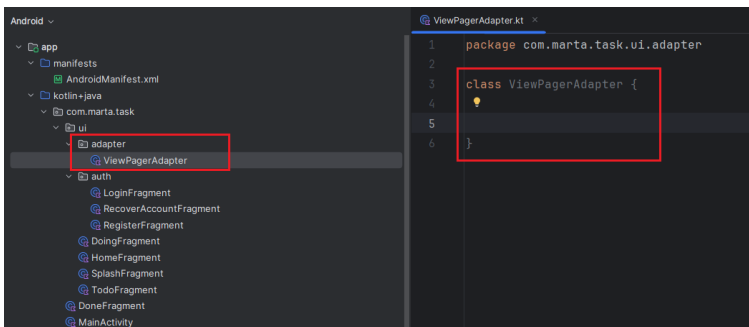


Figura 86

83. O código da classe ViewPagerAdapter está ilustrado na Figura 87.

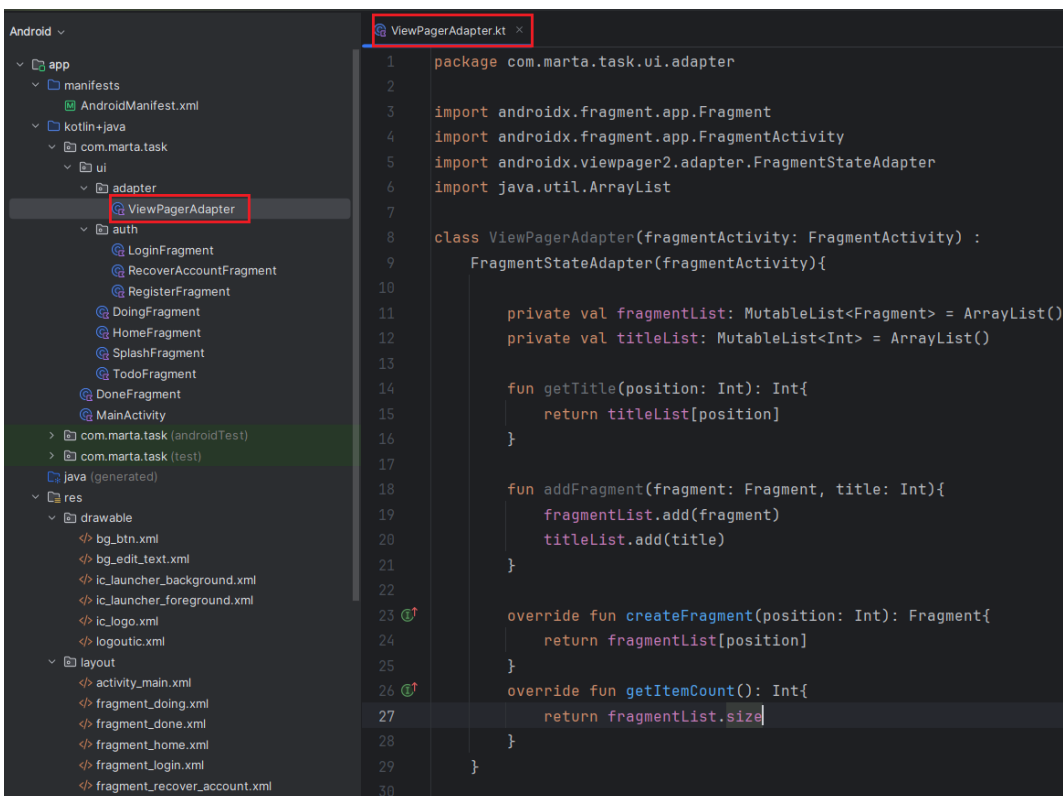


Figura 87

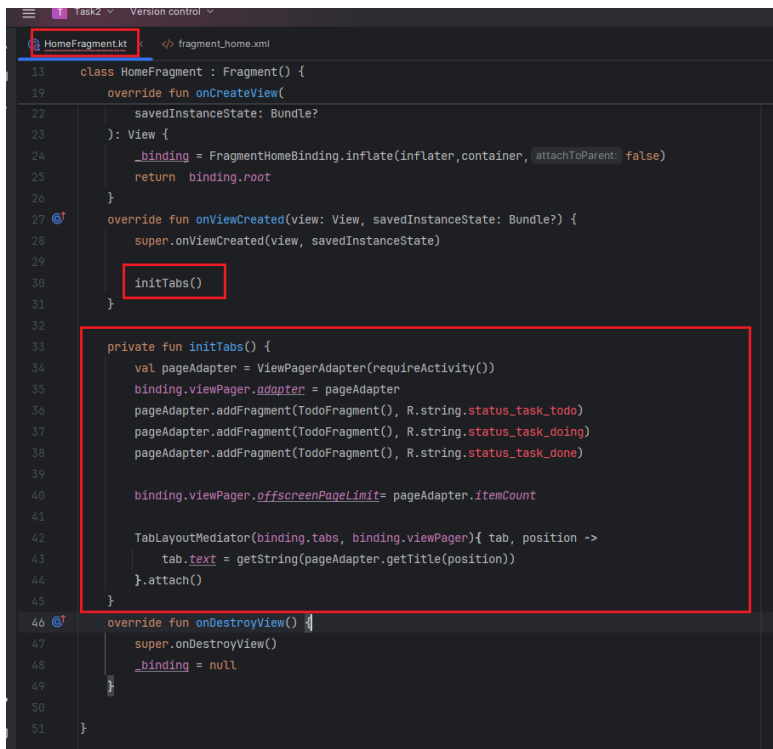
Esse código define uma classe chamada ViewPagerAdapter, que serve para gerenciar uma lista de fragments dentro do componente ViewPager2.

A classe ViewPagerAdapter herda de FragmentStateAdapter, que é a classe recomendada para trabalhar com ViewPager2. A classe recebe como parâmetro um FragmentActivity, que fornece o contexto necessário para criar os fragments. Resumo do funcionamento da classe:

- O método addFragment adiciona os fragments que deseja mostrar e seus títulos.
- O componente ViewPager2 chamará automaticamente o método createFragment conforme o usuário desliza entre as páginas, dessa forma os fragments são criados e exibidos nas abas.

- O título de cada fragment é retornado pelo método `getTitle(position)` — geralmente para exibir em abas (`TabLayout`).
- O atributo `fragmentList` é uma lista que armazena os fragments que serão exibidos no `ViewPager`.
- O atributo `titleList` armazena os títulos de cada fragment. Observe que seu tipo é `Int`, pois ele contém os IDs dos recursos de string (por exemplo, `R.string.alguma_coisa`).
- O método `getItemCount` retorna o número total de fragments adicionados. Isso define quantas páginas o `ViewPager` terá.

84. Abra o arquivo **HomeFragment.kt** para configurar a classe `ViewPagerAdapter`, responsável por armazenar e gerenciar a lista de fragments exibidos nas abas do aplicativo. O código necessário estará descrito na Figura 88. **Atenção:** para que o código funcione corretamente, o componente `ViewPager2` no arquivo `fragment_home.xml` deve estar com o ID definido como `viewPager`.



```
13 class HomeFragment : Fragment() {
14     override fun onCreateView(
15         savedInstanceState: Bundle?
16     ): View {
17         _binding = FragmentHomeBinding.inflate(inflater, container, attachToParent: false)
18         return binding.root
19     }
20     @Override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
21         super.onViewCreated(view, savedInstanceState)
22         initTabs()
23     }
24     private fun initTabs() {
25         val pageAdapter = ViewPagerAdapter(requireActivity())
26         binding.viewPager.adapter = pageAdapter
27         pageAdapter.addFragment(TodoFragment(), R.string.status_task_todo)
28         pageAdapter.addFragment(TodoFragment(), R.string.status_task_doing)
29         pageAdapter.addFragment(TodoFragment(), R.string.status_task_done)
30         binding.viewPager.offscreenPageLimit = pageAdapter.itemCount
31         TabLayoutMediator(binding.tabs, binding.viewPager) { tab, position ->
32             tab.text = getString(pageAdapter.getTitle(position))
33         }.attach()
34     }
35     @Override fun onDestroyView() {
36         super.onDestroyView()
37         _binding = null
38     }
39 }
```

Figura 88

A Figura 88 apresenta o código responsável por anexar os fragments ao `TabLayout`. A seguir, é feita a explicação do funcionamento.

- O evento `onViewCreated` é disparado quando o `FragmentHome` está completamente criado. Nesse momento, é chamado o método `initTabs()` para configurar o `ViewPager` e as abas (`TabLayout`).
- No método `initTabs`, a linha `val pageAdapter = ViewPagerAdapter(requireActivity())` cria um adaptador de páginas (`ViewPagerAdapter`), passando o `Fragment` como

contexto. Aqui, o contexto se refere à instância da tela que está sendo executada. Um adaptador é um componente intermediário que conecta os dados aos elementos visuais, como no nosso exemplo, que vincula uma lista de dados ao componente visual ViewPager. Em seguida, o método `addFragment` é chamado para adicionar três instâncias de fragments.

- A propriedade `binding.viewPager.offscreenPageLimit` define o número de páginas que o ViewPager deve manter pré-carregadas em memória. Essa configuração evita a recriação dos Fragments ao alternar entre abas, o que é vantajoso quando a quantidade de Fragments é pequena, reduzindo a sobrecarga de ciclo de vida.
- A classe `TabLayoutMediator(binding.tabs, binding.viewPager)` é responsável por conectar o TabLayout ao ViewPager2, permitindo que eles funcionem de forma sincronizada. Com essa ligação ao deslizar o ViewPager, a aba correspondente no TabLayout é atualizada automaticamente; Ao tocar em uma aba, o fragmento correto é exibido no ViewPager. Dentro da configuração do TabLayoutMediator, o atributo `tab.text` é utilizado para definir o texto exibido em cada aba, com base nos títulos associados a cada fragmento do ViewPager.
- O trecho `{ tab, position -> }` representa um bloco lambda (ou arrow function) e um callback (também conhecido como arrow function) utilizado na configuração de cada aba do TabLayout dentro do TabLayoutMediator. Um bloco lambda é uma forma curta de escrever uma função anônima (sem nome). Ou seja, lambda é apenas sintaxe para representar uma função. Um callback (pode ser lambda, função anônima ou nomeada) é uma função passada como parâmetro para outra função ou objeto, para que seja executada em um momento específico, geralmente durante um processo controlado. O bloco `{ tab, position -> ... }` é um lambda. Também é usado como callback, pois é passado para o TabLayoutMediator, que vai chamá-lo para cada aba. Com esse callback o componente TabLayout poderá observar e gerenciar eventos internos (como mudança de página ou seleção de aba) para manter tudo sincronizado.
- A propriedade `position` é um valor inteiro (Int) que indica a posição atual da aba e do fragmento associado. A contagem começa em zero. Exemplo: `position = 0` → Primeira aba (ex: "A Fazer"), `position = 1` → Segunda aba (ex: "Fazendo") e `position = 2` → Terceira aba (ex: "Concluído").
- A propriedade `tab` é uma instância da classe TabLayout, que representa a aba que está sendo configurada. Através dela, é possível personalizar diversas propriedades, como: `tab.text` → Define o texto exibido na aba; `tab.setIcon(R.drawable.seu_icone)` → Define um ícone, etc.
- O método `.attach()`: faz a conexão ser efetivada entre o TabLayout e o ViewPager.

85. Os erros das linhas `R.string.status_task_todo`, `R.string.status_task_doing` e `R.string.status_task_done` serão removidos. Para isso, abra o arquivo `string.xml` e adicione as linhas conforme Figura 89. Remova a linha destacada em azul.

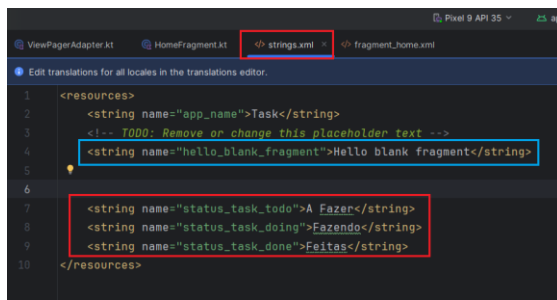


Figura 89

86. Para otimizar o gerenciamento de memória, feche todos os arquivos atualmente abertos no Android Studio. Em seguida, abra os arquivos fragment_doing.xml, fragment_done.xml e fragment_todo.xml e remova o Textview e o comentário TODO conforme Figura 90.

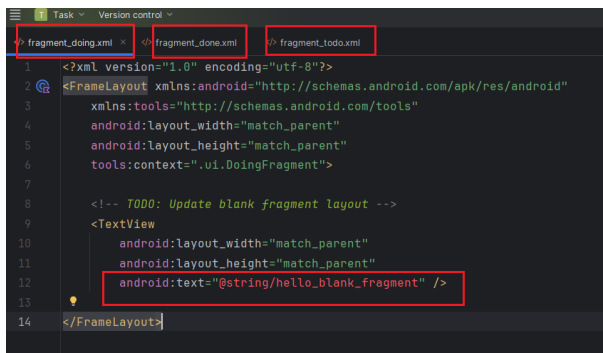


Figura 90

87. Execute o emulador para verificar o funcionamento do App conforme Figura 91.

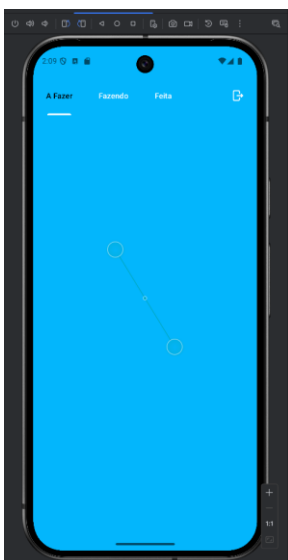


Figura 91

88. Vamos estilizar ainda mais o TabLayout. Abra o arquivo Themes.xml e informe o código de estilo para o TabLayout conforme Figura 92. A parte destacada em amarelo refere-se

ao estilo principal do App. A parte destacada em vermelho é o estilo aplicado ao TabLayout.

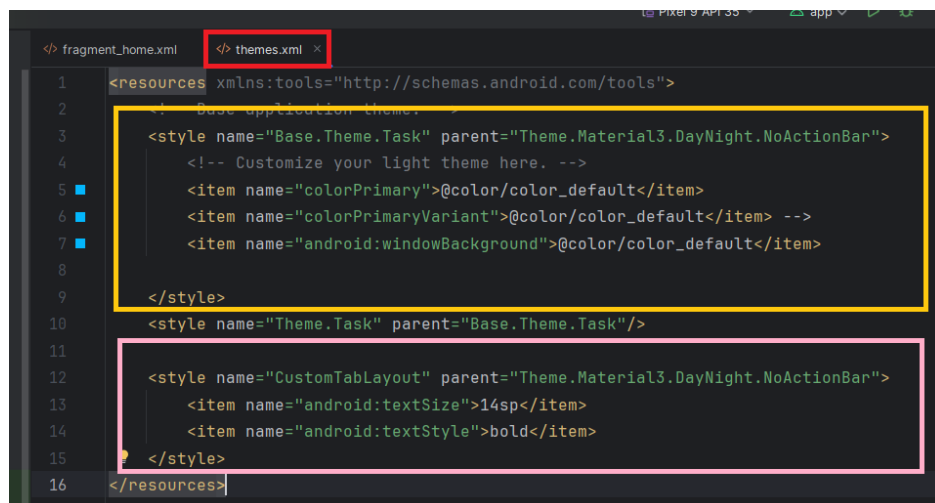


Figura 92

O código define um estilo personalizado chamado **CustomTabLayout**, que herda propriedades do tema **Theme.Material3.DayNight.NoActionBar**. A propriedade **parent** define o estilo base a partir do qual o **CustomTabLayout** será construído. Ele pertence à biblioteca Material 3 (Material You) e possui algumas características específicas:

- **Material3:** É a nova versão do Material Design, com componentes visuais atualizados, suporte a theming dinâmico e foco em acessibilidade.
- **DayNight:** Permite que o tema alterne automaticamente entre modo claro (day) e escuro (night), dependendo das configurações do sistema.
- **NoActionBar:** Indica que o tema não inclui uma ActionBar padrão, permitindo ao desenvolvedor usar alternativas como Toolbar, Jetpack Compose, BottomNavigation, etc. A ActionBar (ou barra de ações) é um componente da interface do usuário do Android que fica normalmente na parte superior da tela e oferece elementos comuns como: Título da tela, Botões de navegação (como "voltar"), Ícones de ação (ex: busca, salvar, configurações) e Menu de opções

89. Abra o arquivo **fragment_home.xml** e aplique o estilo ao atributo **tabTextAppearance** conforme Figura 93. Além disso, remova as propriedades **textSize** e **textStyle** do XML do TabLayout que está no **fragment_home.xml**. Não esqueça de remover os atributos **texSize** e **textStyle** do TabLayout.

```
<com.google.android.material.appbar.AppBarLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="30dp"
    >

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <com.google.android.material.tabs.TabLayout
            android:id="@+id/tabs"
            android:layout_width="match_parent"
            android:layout_height="70dp"
            android:background="@color/color_default"
            android:contentDescription="Tabs"
            android:elevation="0dp"
            android:paddingEnd="24dp"
            app:tabTextAppearance="@style/CustomTabLayout"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toStartOf="@+id/btnLogout"
            app:layout_constraintStart_toStartOf="parent"
            app:tabIndicatorColor="@color/white"
            app:tabTextColor="@color/white" />

        <androidx.viewpager2.widget.ViewPager2
```

Figura 93

PARTE 4 – UTILIZANDO EXTENSIONS

90. Vamos criar uma tela para criar uma nova tarefa. Clique com o botão direito na pasta **ui**, na opção **New>>Fragment>>Fragment (blank)** conforme Figura 94.

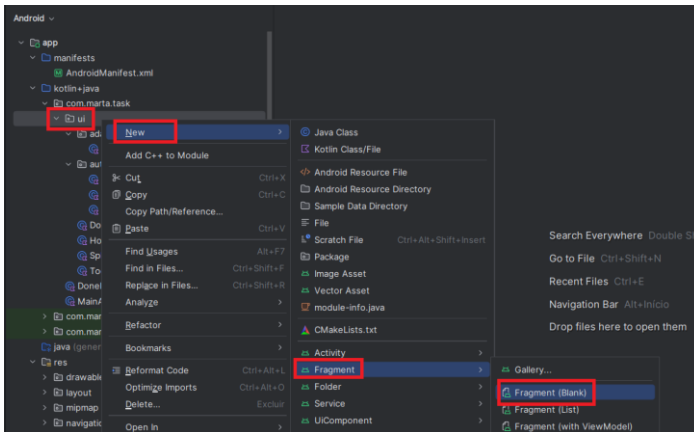


Figura 94

91. Informe o nome do Fragment como **FormTaskFragment** conforme Figura 95.

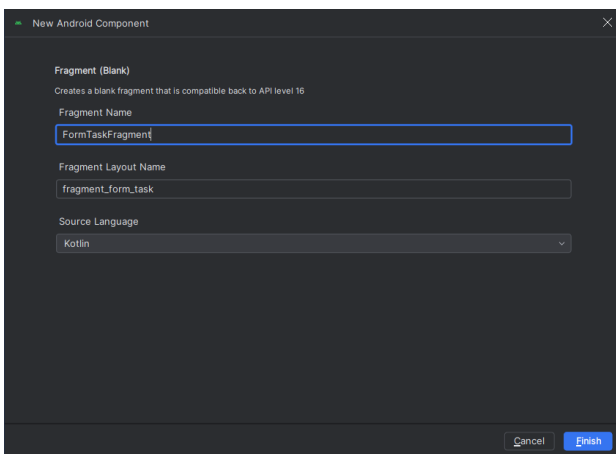


Figura 95

92. Limpe o código Kotlin do **FormTaskFragment** conforme Figura 96.

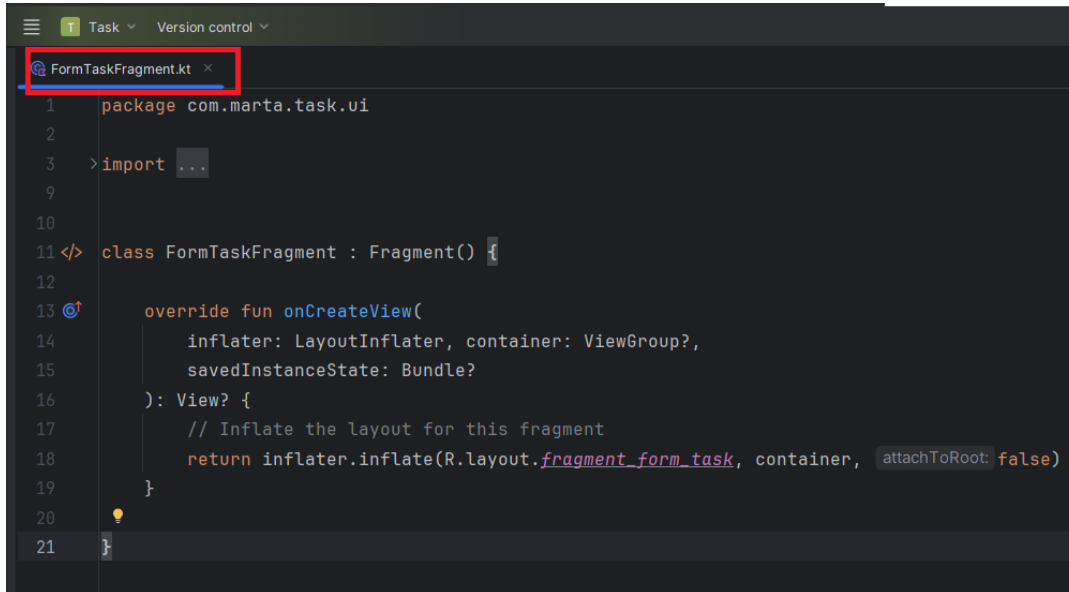


Figura 96

93. Inclua o código XML no arquivo fragment_form_task.xml conforme Figura 97.

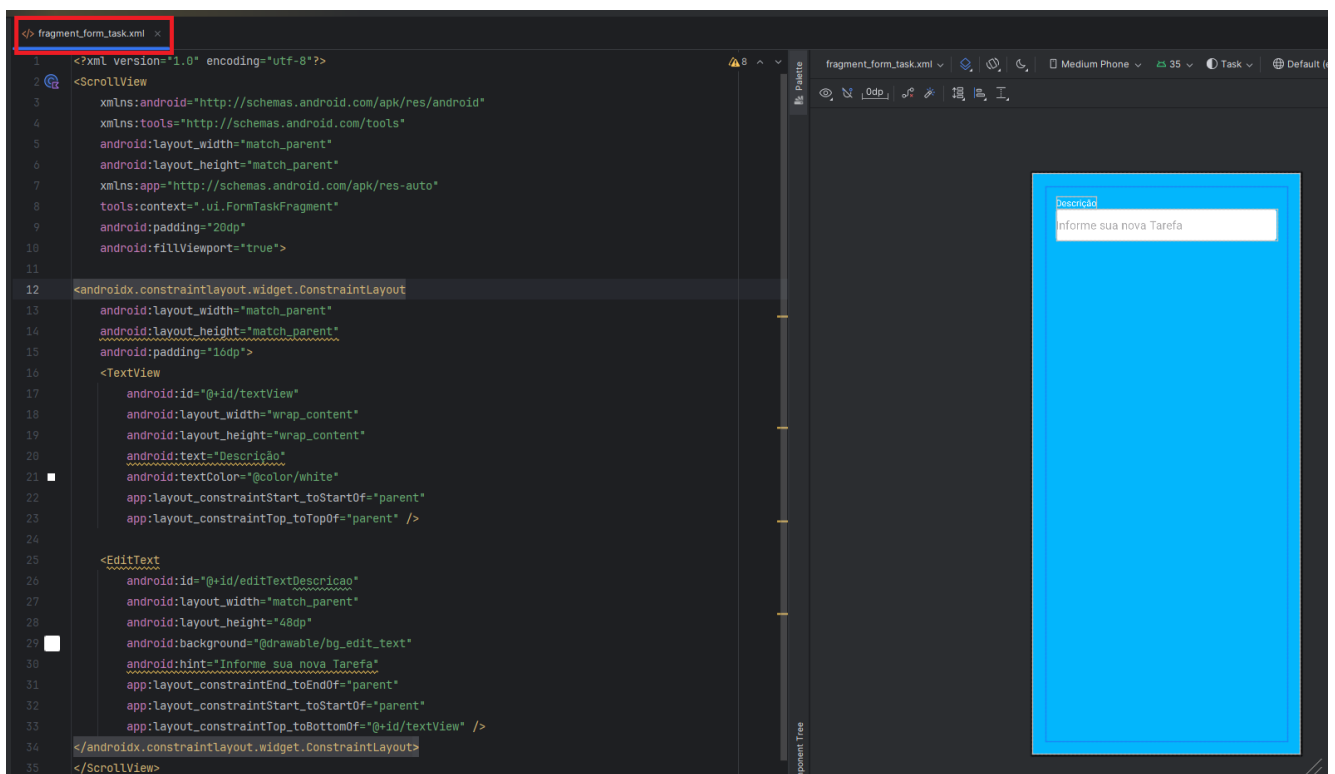


Figura 97

94. No fragment_form_task.xml, inclua mais um TextView e um Radio group conforme Figura 98.

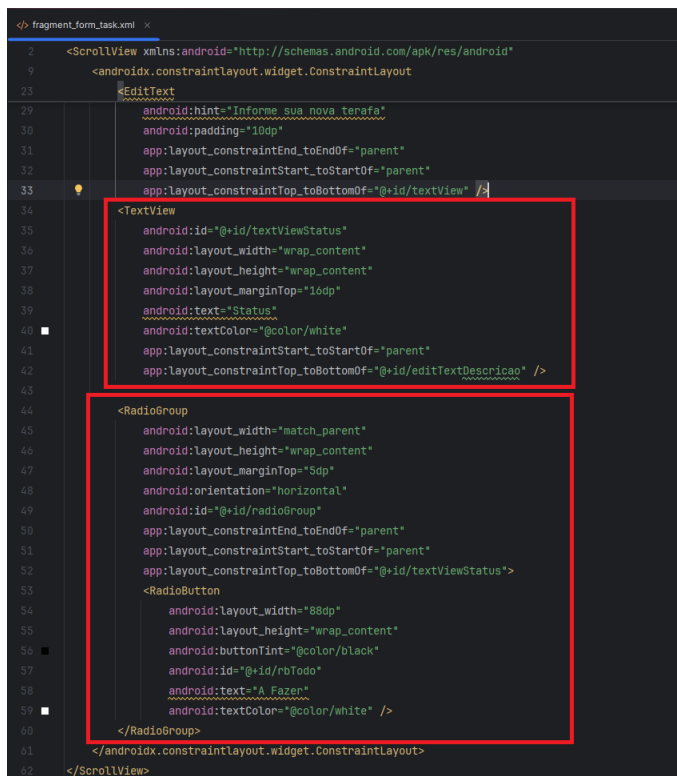


Figura 98

95. Adicione mais dois RadioButtons, conforme ilustrado na Figura 99. O atributo theme deve receber o valor MyRadioButton, previamente configurado no arquivo themes.xml, conforme o código apresentado na Tabela 2.

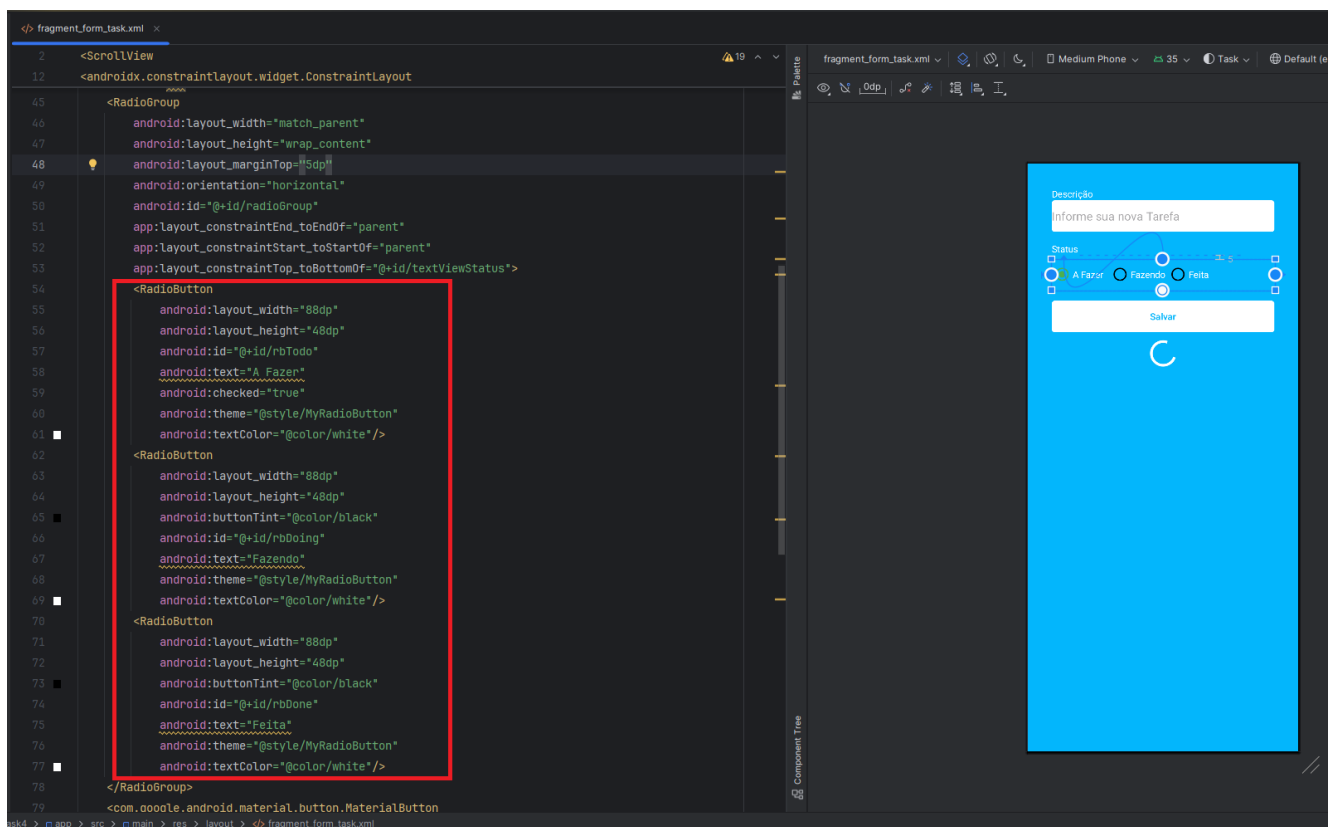


Figura 99


```
<style name="MyRadioButton" parent="Theme.AppCompat.Light">

    <item name="colorControlNormal">@color/black</item>

    <item name="colorControlActivated">@color/green</item>

</style>
```

Tabela 2

96. Alinhe o radiogroup conforme Figura 100.

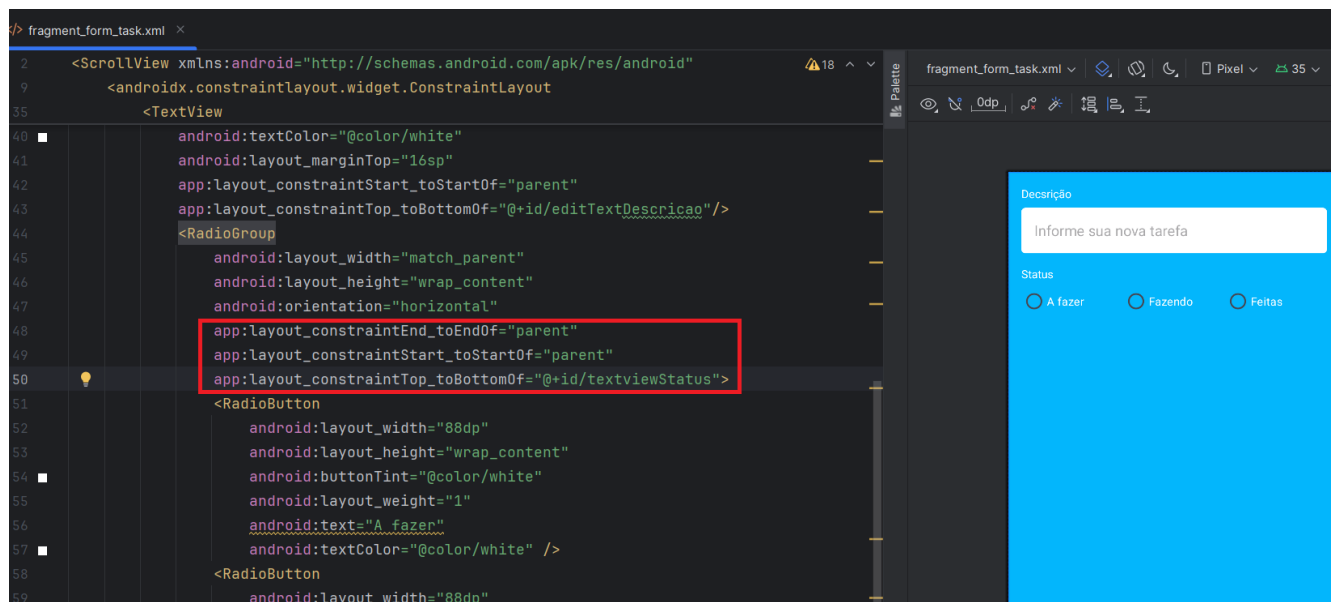


Figura 100

97. Na tela **fragment_form_task.xml**, adicione um novo botão logo abaixo do RadioGroup conforme Figura 101. Esse botão possui a propriedade `android:textAllCaps`, que controla se o texto exibido no componente será mostrado em letras maiúsculas, independentemente de como foi escrito no código ou no arquivo de strings. Se o valor da propriedade for `true`, o texto será exibido inteiramente em maiúsculas, mesmo que tenha sido definido em minúsculas. Se o valor for `false`, o texto será exibido exatamente como foi definido no XML, no código ou no arquivo de strings.

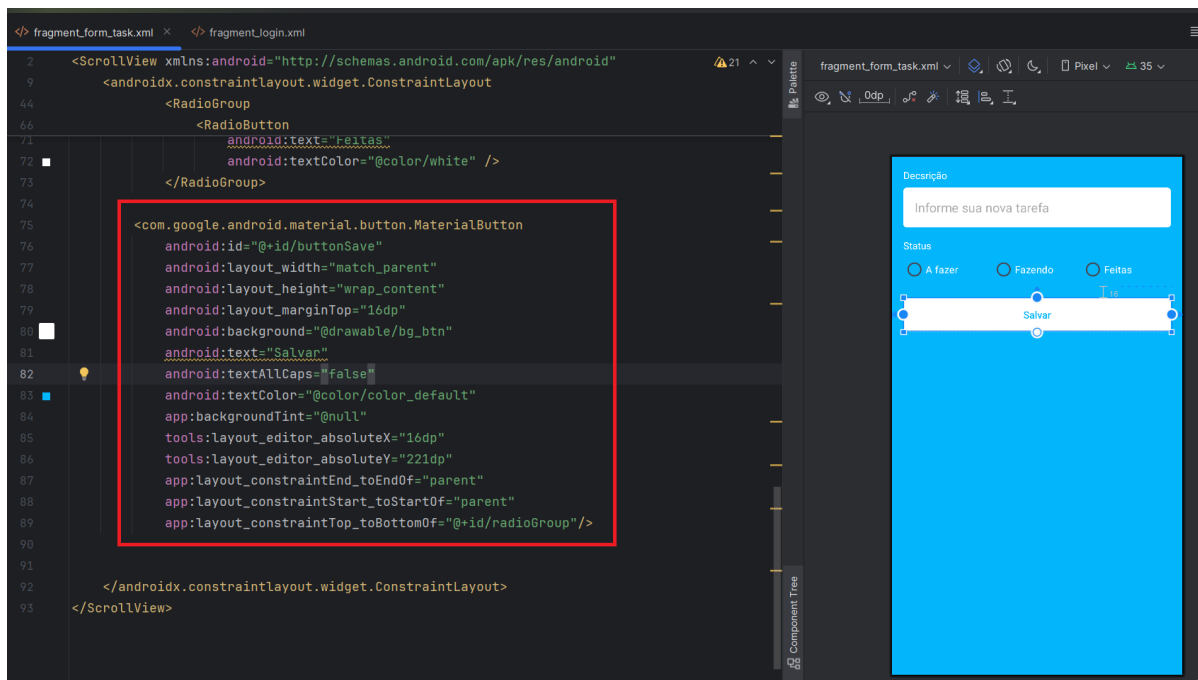


Figura 101

98. Ainda no arquivo **fragment_form_task.xml**, abaixo do botão Salvar, inclua o componente `progressBar` conforme Figura 102.

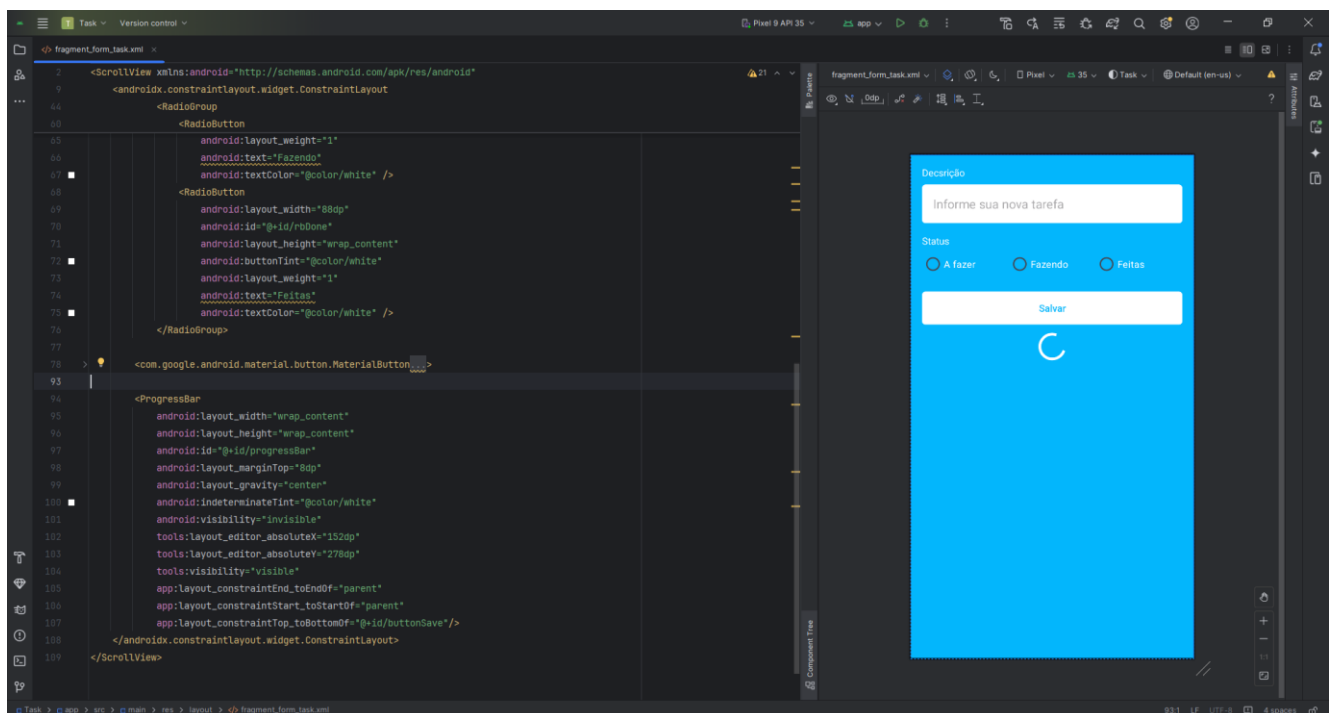


Figura 102

99. Abra os arquivos **fragment_todo.xml**, **fragment_done.xml** e **fragment_doing.xml** e altere o container pai para `ConstraintLayout` conforme na Figura 103.

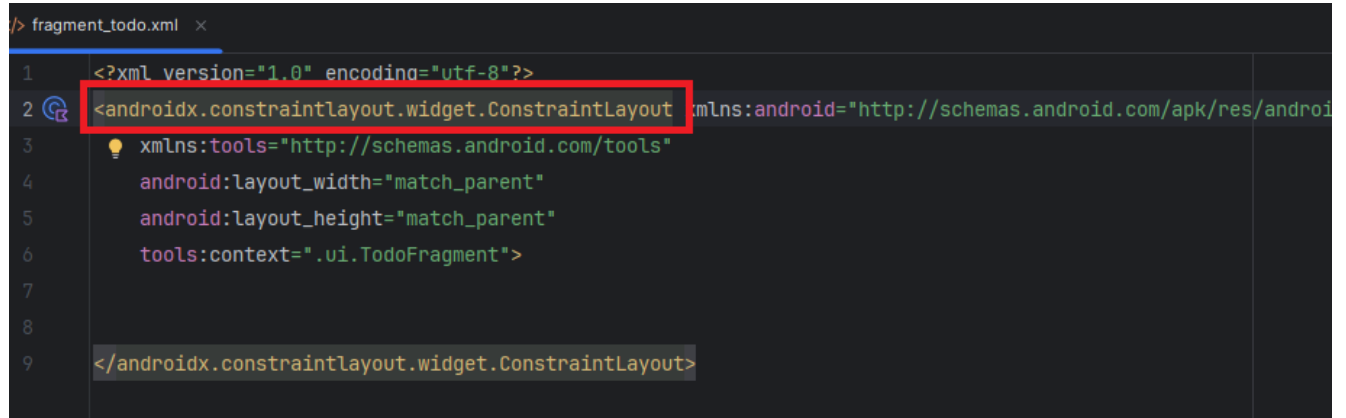


Figura 103

100. Abra o arquivo **fragment_todo.xml** e inclua um componente **FloatActionButton** no XML, conforme Figura 104. Clique e arraste o botão no canto inferior da tela conforme Figuras 104 e 105.

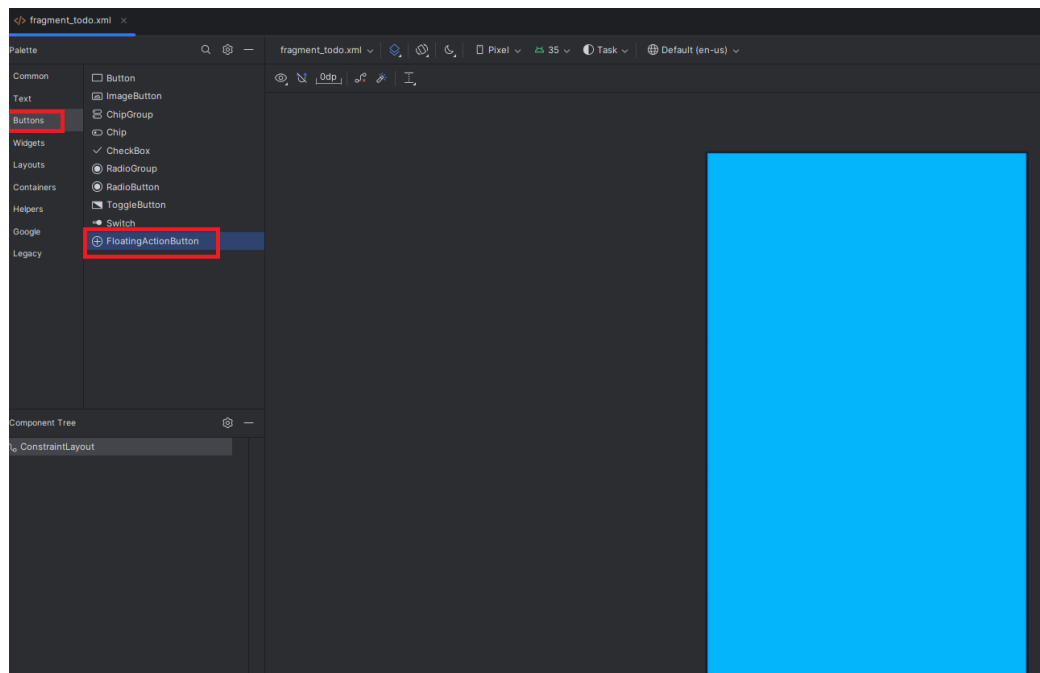


Figura 104

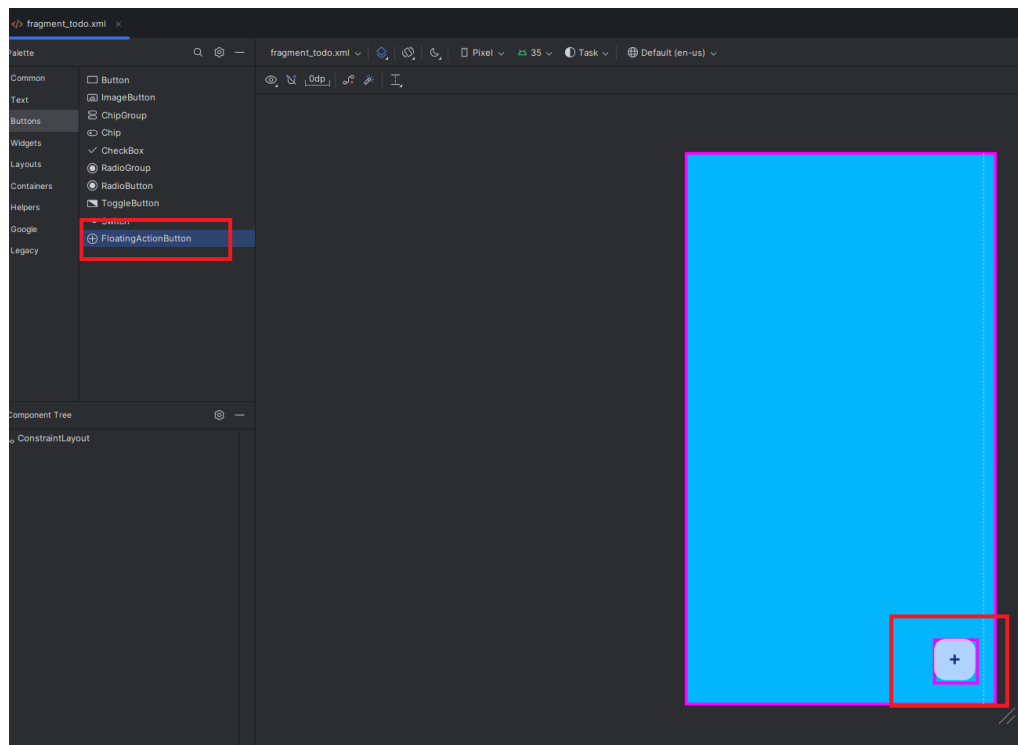


Figura 105

101. Uma janela será aberta para selecionar um ícone para o botão **FloatingActionButton**, conforme mostrado na Figura 106. Caso ainda não tenha um ícone disponível, clique com o botão direito na pasta drawable e selecione a opção **New >> Vector Asset**.

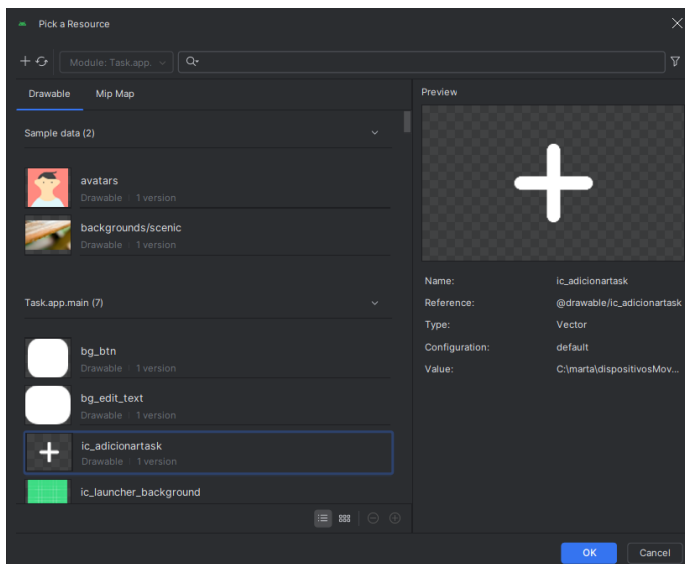
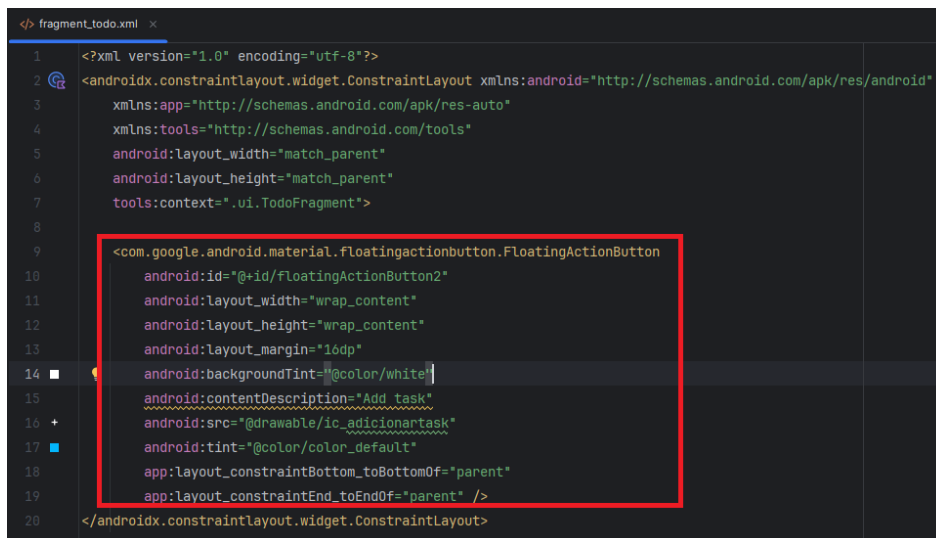


Figura 106

102. Permaneça no arquivo **fragment_todo.xml**, e ajuste algumas propriedades do FloatingActionButton, conforme Figura 107.



```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ui.TODOFragment">

    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/floatingActionButton2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_margin="16dp"
        android:backgroundTint="@color/white"
        android:contentDescription="Add task"
        android:src="@drawable/ic_adicionartask"
        android:tint="@color/color_default"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Figura 107

Algumas propriedades do FloatingActionButton são descritas a seguir:

- A propriedade **android:contentDescription** é uma descrição textual do propósito do componente, usada principalmente por tecnologias assistivas, como leitores de tela (ex: TalkBack, para pessoas com deficiência visual). Isso significa que quando um usuário com leitor de tela tocar nesse botão, ele ouvirá: “Add task” — ajudando-o a entender que esse botão serve para adicionar uma tarefa.
- A propriedade **android:tint** é semelhante à **backgroundTint**, aplica uma cor sobre o ícone ou imagem definida em **android:src**, funcionando como um filtro de cor sobre esse elemento visual.

103. Execute o aplicativo no emulador e verifique se o FloatingActionButton é exibido corretamente na tela, conforme mostrado na Figura 108.

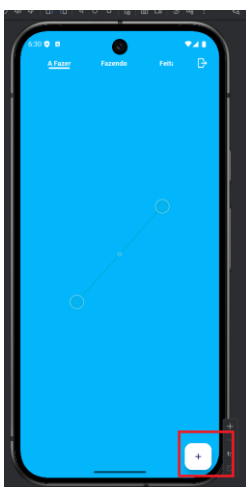


Figura 108

104. Agora vamos configurar a navegação entre as telas HomeFragment e FormTaskFragment. Quando o usuário clicar no botão FloatingActionButton presente no TodoFragment (que, na verdade, está contido dentro do HomeFragment), ele deverá ser direcionado para a tela FormTaskFragment. Para isso, abra o arquivo **main_graph.xml**, adicione o FormTaskFragment ao gráfico de navegação e crie a ligação entre as telas, conforme ilustrado na Figura 109.

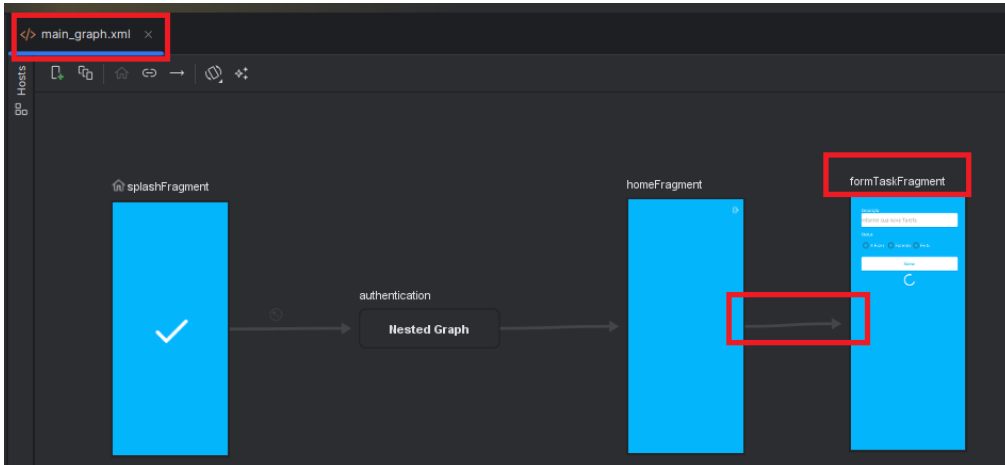


Figura 109

105. Configure o evento de clique do botão ActionFloat. Para isso, abra o código Kotlin **TodoFragment.kt** e inclua o código conforme Figura 110.

```
1  import androidx.navigation.fragment.findNavController
2
3  import com.marta.task.R
4  import com.marta.task.databinding.FragmentTodoBinding
5
6  class TodoFragment : Fragment() {
7      private var _binding: FragmentTodoBinding? = null
8      private val binding get() = _binding!!
9
10     override fun onCreateView(
11         inflater: LayoutInflater,
12         container: ViewGroup?,
13         savedInstanceState: Bundle?
14     ): View {
15         _binding = FragmentTodoBinding.inflate(inflater, container, attachToParent: false)
16         return binding.root
17     }
18
19     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
20         super.onViewCreated(view, savedInstanceState)
21         initListeners()
22     }
23
24     private fun initListeners() {
25         binding.floatingActionButton2.setOnClickListener {
26             findNavController().navigate(R.id.action_homeFragment_to_formTaskFragment)
27         }
28     }
29
30     override fun onDestroyView() {
31         super.onDestroyView()
32         _binding = null
33     }
34 }
```

Figura 110

O evento `onViewCreated` é chamado automaticamente logo após o Fragment ter sua interface criada por completo. Dentro do método `initListeners`, o `setOnClickListener` define o comportamento que deve ocorrer quando o usuário clica no botão `FloatingActionButton`. Nesse caso, ao clicar no botão, acontecem os seguintes passos: `findNavController()` obtém o `NavController` associado ao Fragment. Em seguida, `navigate(R.id.action_homeFragment_to_formTaskFragment)` executa a navegação para outro Fragment (nesse exemplo, o `formTaskFragment`). Por final, a ação `R.id.action_homeFragment_to_formTaskFragment` é previamente definida no Navigation Graph do Android, que gerencia as rotas de navegação entre os Fragments.

106. Execute o App e veja se tudo está funcionando, conforme Figura 111.

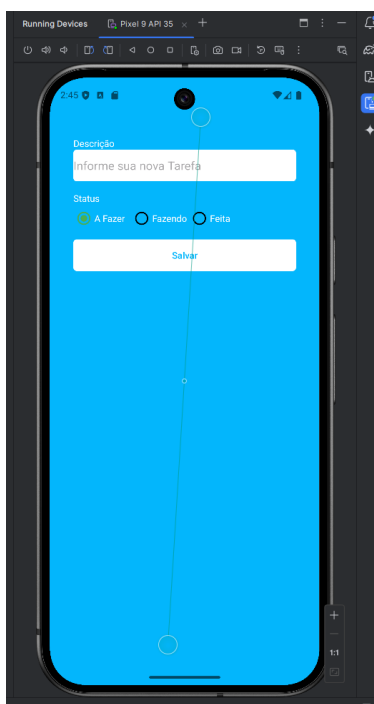


Figura 111

107. Agora, vamos configurar o componente `Toolbar`, que funciona como elemento de navegação da interface. Nele, será exibido o título da tela atual e um botão de retorno (voltar). Embora seja possível adicionar menus laterais à `Toolbar`, neste momento configuraremos apenas o título e o botão de navegação para retornar à tela anterior.

108. Abra a tela **fragment_register.xml**, e insira o componente `Toolbar` conforme ilustrado na Figura 112. Além disso, adicione a propriedade `android:paddingTop="20dp"` ao `TextView` com o texto "Criar conta".

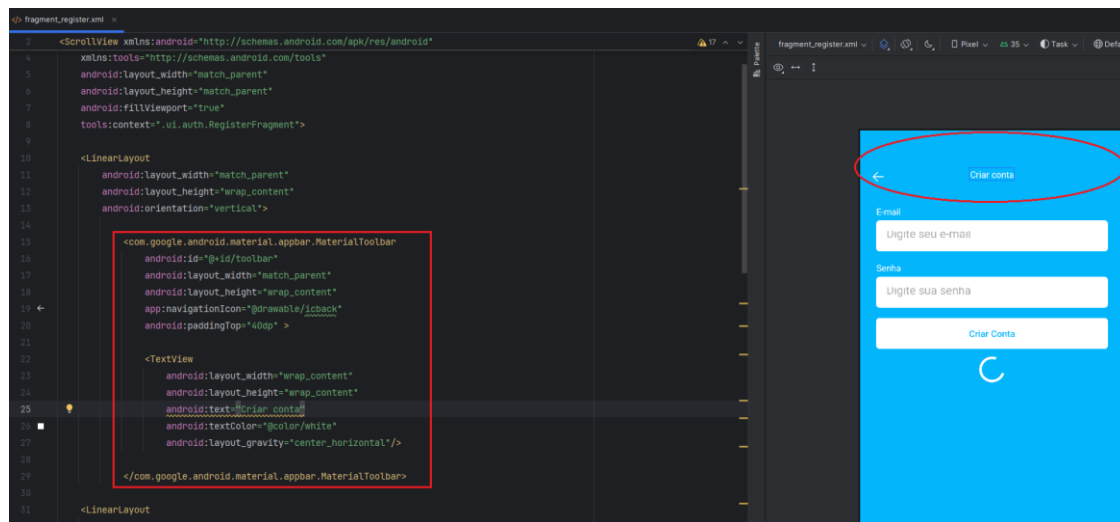


Figura 112

109. Replique a Toolbar na tela `fragment_recover_account.xml` conforme Figura 113.

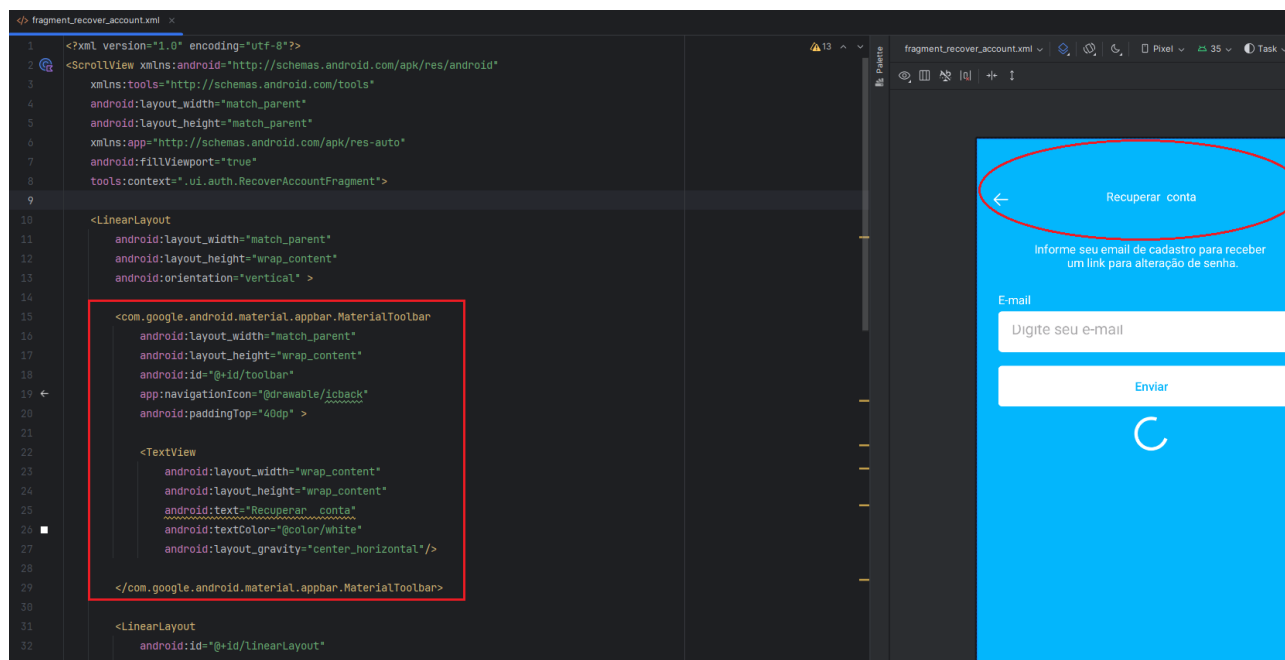


Figura 113

110. Replique a Toolbar na tela `fragment_form_task.xml` conforme Figura 114. Inclua um novo `LinearLayout` acima da Toolbar conforme Figura 114.

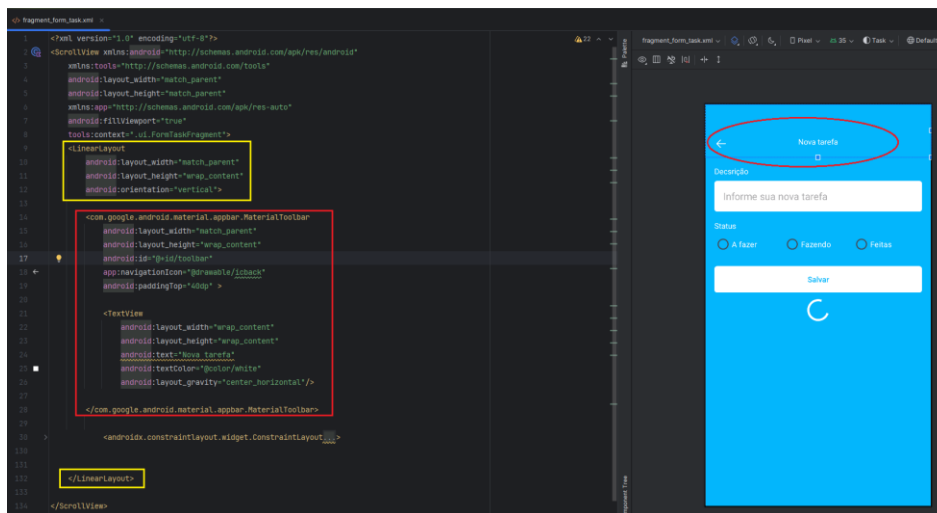


Figura 114

111. Execute no emulador seu App para verificar se sua Toolbar está funcionando.

112. Vamos utilizar um recurso chamado Extension. Para leitura complementar, acesse a documentação oficial do Kotlin: <https://kotlinlang.org/docs/extensions.html>. Além disso, o Capítulo 7, disponível no AVA, apresenta informações detalhadas sobre Extensions.

113. A proposta é criar uma **Extension Function** para configurar o comportamento do botão 'voltar' da Toolbar. Para isso, comece criando um **package** chamado **util** dentro do pacote principal do projeto. Em seguida, dentro do package util, clique com o botão direito e selecione **New >> Kotlin Class/File**. Crie um arquivo com o nome **Extensions.kt** e o tipo **File**, conforme ilustrado nas Figuras 115 e 116.

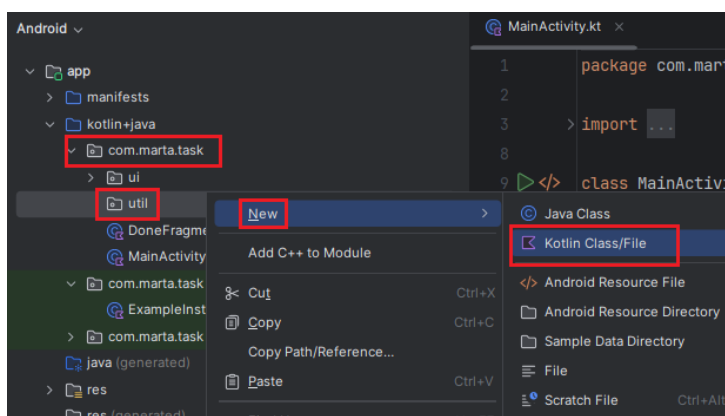


Figura 115

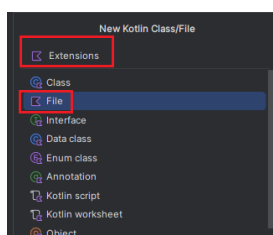


Figura 116

114. Crie uma função utilizando a palavra-chave `fun` e, em seguida, declare o tipo ao qual essa função estará associada. No nosso caso, a função `initToolbar` — responsável pela inicialização do componente `Toolbar` — será criada como uma `Extension` de um `Fragment`, ou seja, será chamada diretamente a partir de um `Fragment`. O parâmetro de entrada da função será um objeto do tipo `Toolbar`, conforme ilustrado nas Figuras 117 e 118.

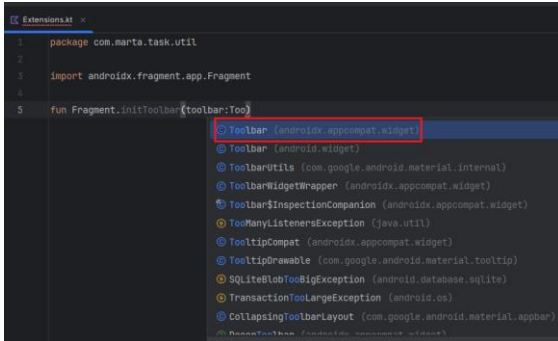


Figura 117

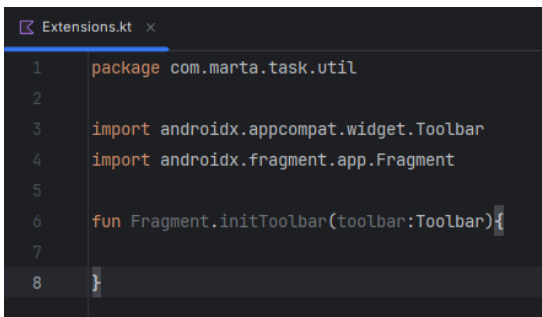


Figura 118

115. Abra o arquivo **RegisterFragment.kt** e digite “init”, você verá o método `initToolbar`, conforme Figura 119. Esse método aparecerá para qualquer classe do tipo `Fragment`. O mesmo não ocorre para a classe `MainActivity.kt`, pois a classe não é um `Fragment`.

```

1 package com.marta.task.ui.auth
2
3 import android.os.Bundle
4 import androidx.fragment.app.Fragment
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import com.marta.task.R
9 import com.marta.task.databinding.FragmentRegisterBinding
10
11
12 class RegisterFragment : Fragment() {
13
14     private var _binding: FragmentRegisterBinding? = null
15     private val binding get() = _binding!!
16
17     override fun onCreateView(
18         inflater: LayoutInflater, container: ViewGroup?,
19         savedInstanceState: Bundle?
20     ): View {
21         // Inflate the layout for this fragment
22         initToolbar(toolbar: Toolbar) for Fragment in com.marta.task.util Unit ent: false)
23         setInitialSavedState(state: Fragment.SavedState?) Unit
24         Press Enter to insert, Ctrl to replace Next Tip
25         init
26     }
27
28     override fun onDestroyView() {
29         super.onDestroyView()
30         _binding = null
31     }
32 }

```

Figura 119

116. Abra novamente o arquivo **Extensions.kt** e adicione o código conforme Figura 120.

```

1 package com.marta.task.util
2
3 import androidx.appcompat.app.AppCompatActivity
4 import androidx.appcompat.widget.Toolbar
5 import androidx.fragment.app.Fragment
6
7 fun Fragment.initToolbar(toolbar: Toolbar){
8     (activity as AppCompatActivity).setSupportActionBar(toolbar)
9     (activity as AppCompatActivity).title=""
10    (activity as AppCompatActivity).supportActionBar?.setDisplayHomeAsUpEnabled(true)
11    toolbar.setNavigationOnClickListener {
12        activity?.onBackPressedDispatcher?.onBackPressed()
13    }
14
15 }

```

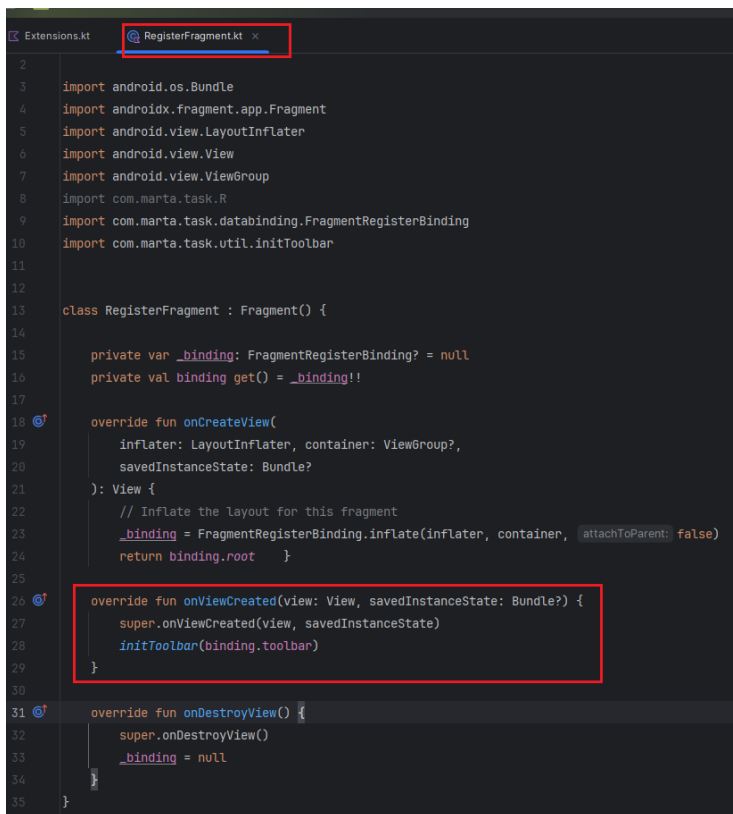
Figura 120

Vamos a explicação do código do arquivo `Extension.kt`:

- arquivo `Extensions.kt` define uma `Extension Function` para a classe `Fragment` chamada `initToolbar`, que configura a `Toolbar` dentro de um fragmento.
- O comando `activity as AppCompatActivity` realiza um `cast` (conversão) da `Activity` que hospeda o fragmento para `AppCompatActivity`. Mas por que é necessário esse `cast`? A classe `Fragment` possui acesso à propriedade `activity`, porém ela é do tipo genérico `Activity`. A classe `Activity`, por sua vez, não oferece suporte nativo para usar a `Toolbar`. Já o método `setSupportActionBar(toolbar)` e outros relacionados à `SupportActionBar` pertencem à classe `AppCompatActivity`, que é uma subclasse de `Fragmente`, consequentemente, de `Activity`. Portanto, para acessar métodos como `setSupportActionBar(toolbar)`, `supportActionBar` e `title` (quando utilizados no contexto de `AppCompatActivity`), é obrigatório fazer o `cast` de `activity` para `AppCompatActivity`. O método `.setSupportActionBar(toolbar)` define a `Toolbar` como a `ActionBar` da `Activity`, permitindo usar recursos como título, botões de navegação e menus.

- método `.title = ""` define o título da Toolbar como vazio. Isso limpa qualquer título padrão que poderia ser exibido. Pode ser personalizado, se desejar.
- O método `.supportActionBar` é uma propriedade que dá acesso à ActionBar (barra superior da tela) quando você está utilizando a Toolbar como ActionBar. O método `?.setDisplayHomeAsUpEnabled(true)` Ativa o botão de voltar (ícone de seta para trás) na Toolbar. Isso não define o comportamento do botão, apenas exibe o ícone.
- O operador `?.` no Kotlin é chamado de safe call (chamada segura). Ele é utilizado para evitar `NullPointerException` caso a propriedade seja nula. Sem o uso do `?`, o compilador não permite o acesso direto, pois não há garantia de que o valor não seja nulo.
- O evento `.setNavigationOnClickListener` define o comportamento do botão de navegação (seta voltar). Quando o usuário clica, ele dispara o comando: `onBackPressedDispatcher.onBackPressed()`, que simula o pressionamento do botão físico de voltar do celular. Isso faz com que o fragmento atual seja removido da pilha e a navegação retorne para a tela anterior.

117. Abra o arquivo **RegisterFragment.kt** e coloque o código no `onViewCreated` conforme Figura 121.



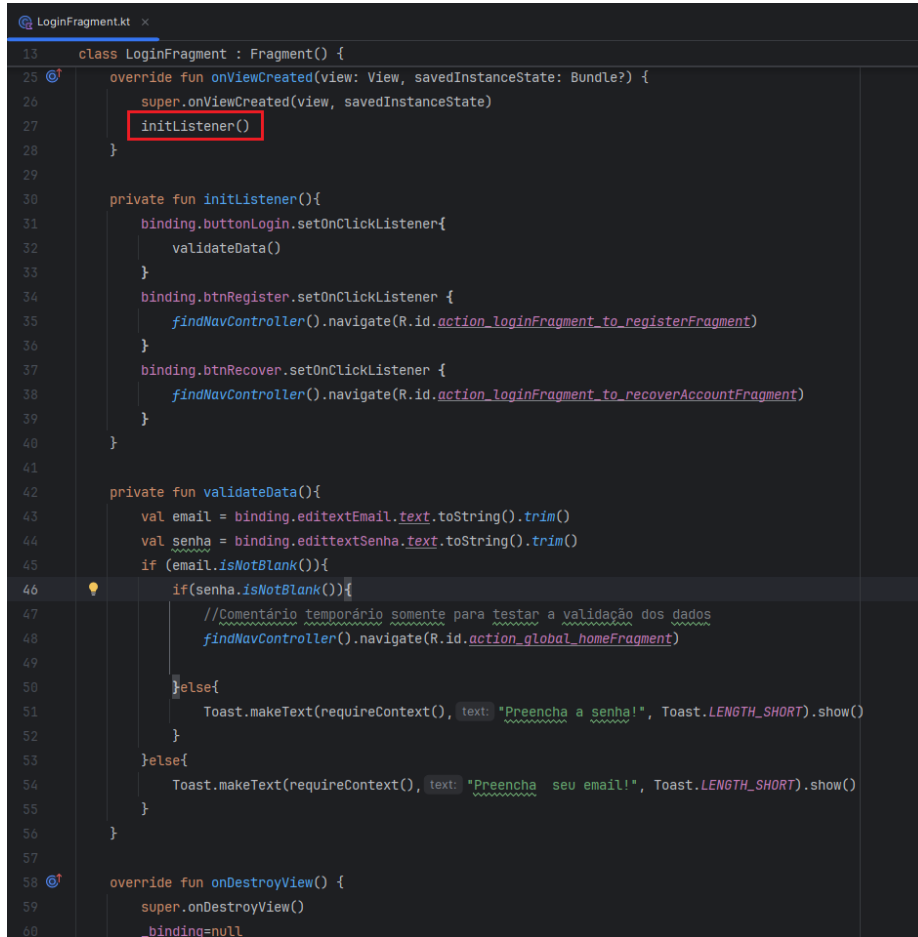
```
2
3 import android.os.Bundle
4 import androidx.fragment.app.Fragment
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import com.marta.task.R
9 import com.marta.task.databinding.FragmentRegisterBinding
10 import com.marta.task.util.initToolbar
11
12
13 class RegisterFragment : Fragment() {
14
15     private var _binding: FragmentRegisterBinding? = null
16     private val binding get() = _binding!!
17
18     override fun onCreateView(
19         inflater: LayoutInflater, container: ViewGroup?,
20         savedInstanceState: Bundle?
21     ): View {
22         // Inflate the layout for this fragment
23         _binding = FragmentRegisterBinding.inflate(inflater, container, attachToParent: false)
24         return binding.root
25     }
26
27     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
28         super.onViewCreated(view, savedInstanceState)
29         initToolbar(binding.toolbar)
30     }
31
32     override fun onDestroyView() {
33         super.onDestroyView()
34         _binding = null
35     }
36 }
```

Figura 121

Faça o mesmo código da Figura 121, nos arquivos **RecoverAccountFragment.kt** e no **FormTaskFragment.kt**.

118. Inicie o emulador e o App e verifique se o botão voltar da Toolbar funciona!

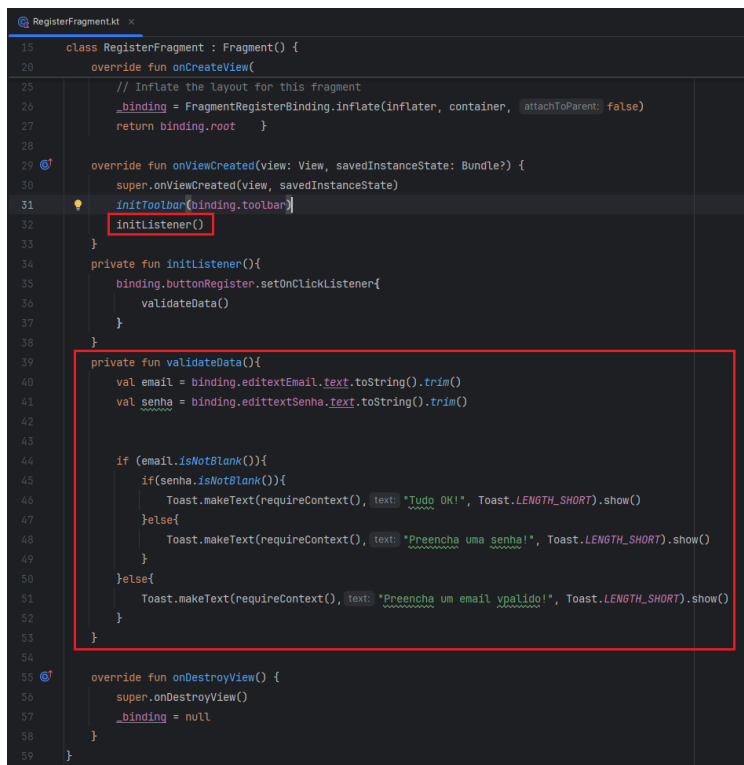
119. Agora vamos validar as informações inseridas na tela de login. Portanto abra a tela **fragment_login.xml**. Inclua um id em todos os campos EditText e no botão.
120. Abra o arquivo **LoginFragment.kt** e inclua o código do método `validateData`, conforme Figura 122.



```
13 class LoginFragment : Fragment() {
25     override fun onCreateView(view: View, savedInstanceState: Bundle?) {
26         super.onCreateView(view, savedInstanceState)
27         initListener()
28     }
29
30     private fun initListener(){
31         binding.buttonLogin.setOnClickListener{
32             validateData()
33         }
34         binding.btnRegister.setOnClickListener {
35             findNavController().navigate(R.id.action_loginFragment_to_registerFragment)
36         }
37         binding.btnRecover.setOnClickListener {
38             findNavController().navigate(R.id.action_loginFragment_to_recoverAccountFragment)
39         }
40     }
41
42     private fun validateData(){
43         val email = binding.edittextEmail.text.toString().trim()
44         val senha = binding.edittextSenha.text.toString().trim()
45         if (email.isNotBlank()){
46             if(senha.isNotBlank()){
47                 //Comentário temporário somente para testar a validação dos dados
48                 findNavController().navigate(R.id.action_global_homeFragment)
49             }else{
50                 Toast.makeText(requireContext(), text: "Preencha a senha!", Toast.LENGTH_SHORT).show()
51             }
52         }else{
53             Toast.makeText(requireContext(), text: "Preencha seu email!", Toast.LENGTH_SHORT).show()
54         }
55     }
56 }
57
58     override fun onDestroyView() {
59         super.onDestroyView()
60         _binding=null
61     }
```

Figura 122

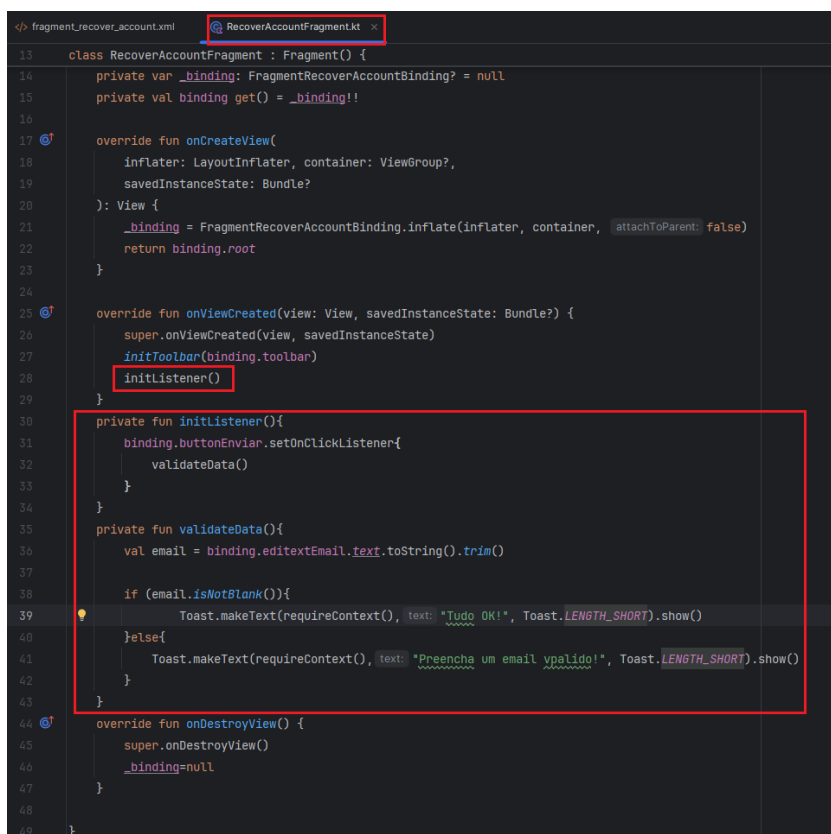
121. Abra o emulador e teste o App para verificar se a validação está funcionando.
122. Abra o arquivo **RegisterFragment.kt** e faça a mesma validação conforme Figura 123.



```
15 class RegisterFragment : Fragment() {
16     override fun onCreateView(
17         inflater: LayoutInflater, container: ViewGroup?,
18         savedInstanceState: Bundle?
19     ): View? {
20         // Inflate the layout for this fragment
21         _binding = FragmentRegisterBinding.inflate(inflater, container, attachToParent: false)
22         return binding.root
23     }
24
25     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
26         super.onViewCreated(view, savedInstanceState)
27         initToolBar(binding.toolbar)
28         initListener()
29     }
30
31     private fun initListener() {
32         binding.buttonRegister.setOnClickListener {
33             validateData()
34         }
35     }
36
37     private fun validateData() {
38         val email = binding.editTextEmail.text.toString().trim()
39         val senha = binding.editTextSenha.text.toString().trim()
40
41         if (email.isNotBlank()) {
42             if (senha.isNotBlank()) {
43                 Toast.makeText(requireContext(), text: "Tudo OK!", Toast.LENGTH_SHORT).show()
44             } else {
45                 Toast.makeText(requireContext(), text: "Preencha uma senha!", Toast.LENGTH_SHORT).show()
46             }
47         } else {
48             Toast.makeText(requireContext(), text: "Preencha um email vpalido!", Toast.LENGTH_SHORT).show()
49         }
50     }
51
52     override fun onDestroyView() {
53         super.onDestroyView()
54         _binding = null
55     }
56 }
```

Figura 123

123. Faça a mesma validação dos campos da tela **RecoverAccountFragment.kt** conforme Figura 124.



```
13 class RecoverAccountFragment : Fragment() {
14     private var _binding: FragmentRecoverAccountBinding? = null
15     private val binding get() = _binding!!
16
17     override fun onCreateView(
18         inflater: LayoutInflater, container: ViewGroup?,
19         savedInstanceState: Bundle?
20     ): View? {
21         _binding = FragmentRecoverAccountBinding.inflate(inflater, container, attachToParent: false)
22         return binding.root
23     }
24
25     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
26         super.onViewCreated(view, savedInstanceState)
27         initToolBar(binding.toolbar)
28         initListener()
29     }
30
31     private fun initListener() {
32         binding.buttonEnviar.setOnClickListener {
33             validateData()
34         }
35     }
36
37     private fun validateData() {
38         val email = binding.editTextEmail.text.toString().trim()
39
40         if (email.isNotBlank()) {
41             Toast.makeText(requireContext(), text: "Tudo OK!", Toast.LENGTH_SHORT).show()
42         } else {
43             Toast.makeText(requireContext(), text: "Preencha um email vpalido!", Toast.LENGTH_SHORT).show()
44         }
45     }
46
47     override fun onDestroyView() {
48         super.onDestroyView()
49         _binding = null
50     }
51 }
```

Figura 124

124. Abra o arquivo **fragment_form_task.xml** e coloque o atributo `checked=true` no radiobutton “A fazer”, conforme Figura 125.

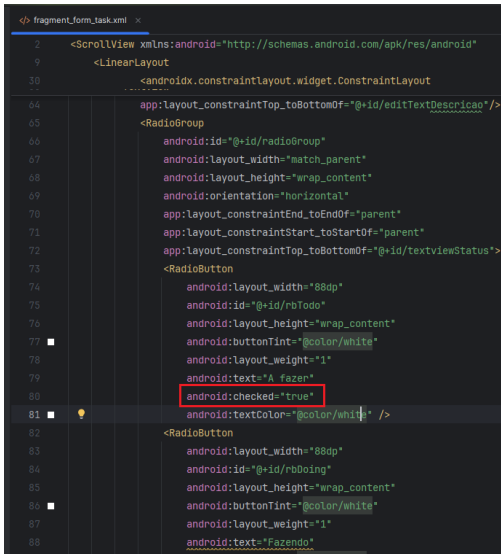


Figura 125

125. Abra o arquivo **FormTaskFragment.kt** e inclua o seguinte código de validação dos dados, conforme Figura 126.

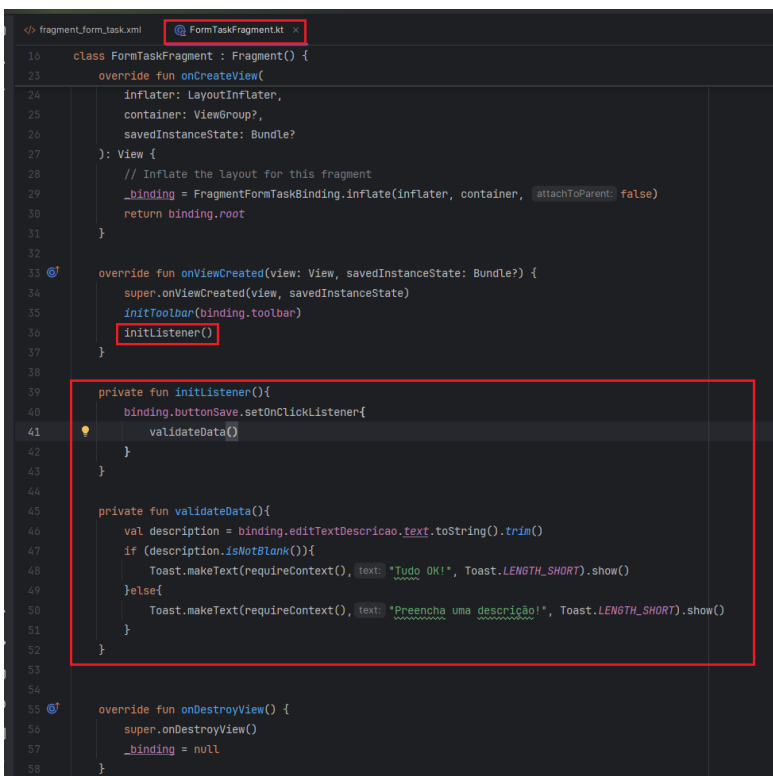


Figura 126

126. Execute o emulador e teste o botão voltar de todas as telas.

PARTE 5 – UTILIZANDO LISTAS VISUAIS

127. Vamos aprender um novo componente chamado BottomSheetDialog que vai substituir o componente Toast. Clique com o botão direito do mouse na pasta layout, depois selecione a opção **New>>Layout Resource File**, conforme Figura 127.

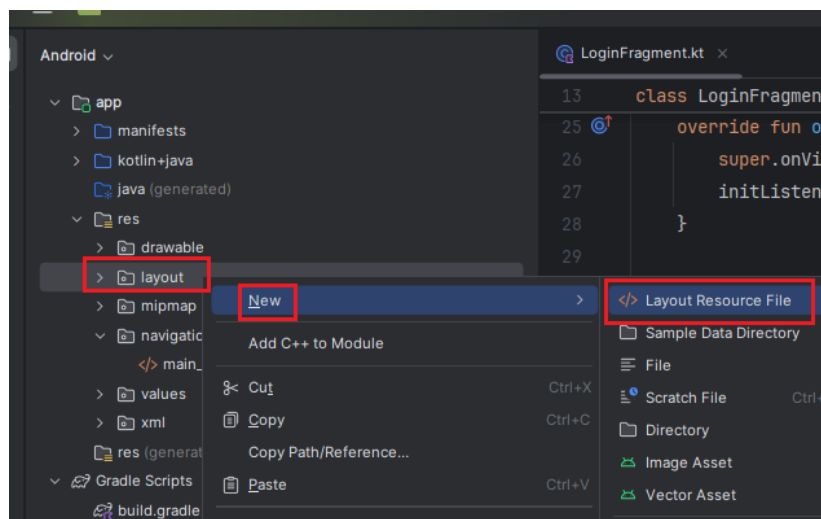


Figura 127

Assim como o Fragment é utilizado para o reaproveitamento de código da interface visual, o Layout Resource File também tem esse propósito.

A principal diferença entre eles está na composição: o Layout Resource File consiste apenas em um arquivo XML que define a estrutura visual da interface. Já o Fragment é composto por dois elementos — um arquivo XML responsável pelo layout visual e uma classe em Kotlin ou Java, que implementa o comportamento, a lógica e o ciclo de vida desse componente dentro da aplicação. Usaremos o Layout Resource File para criar o componente BottomSheetDialog. Esse componente simulará um popup com mensagens informativas sobre o sistema.

128. Na tela “New Resource File”, informe o file name **bottom_sheet**. Informe em root element **LinearLayout**. Por último clique no botão Ok, conforme Figura 128.

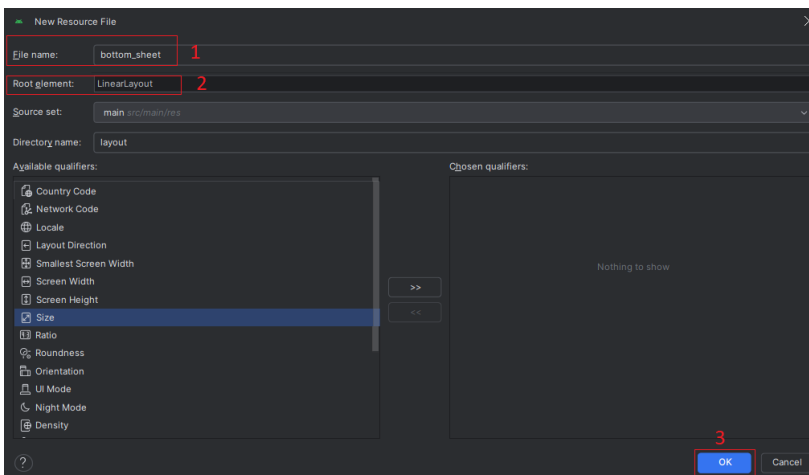


Figura 128

129. Abra o arquivo **bottom_sheet.xml** e digite o código da Figura 129. Observe na Figura 129 que os componentes estão numerados de 1 a 4.

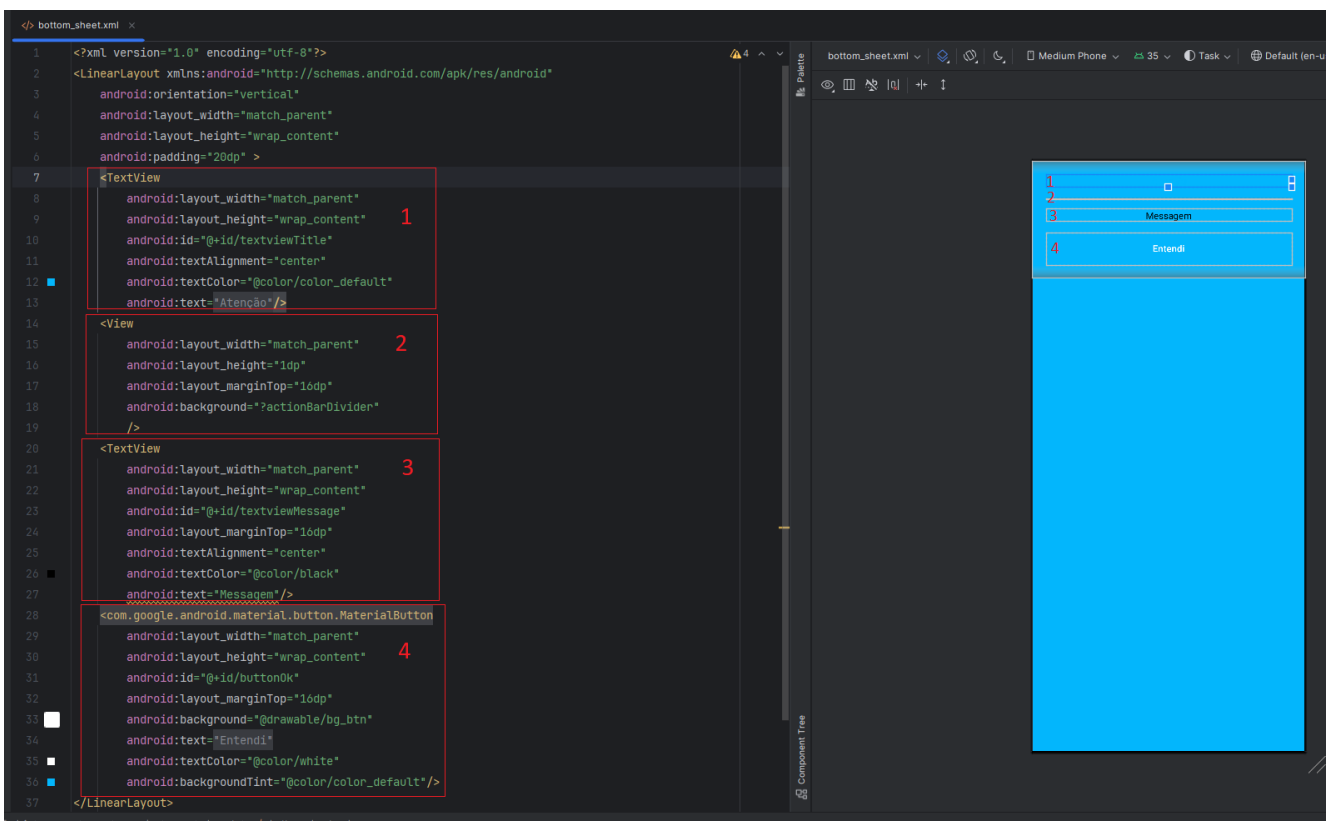


Figura 129

130. Vamos criar um arquivo para produzir arredondamento das bordas do arquivo **bottom_sheet.xml**. Clique com o botão direito na pasta **drawable**, depois selecione a opção **New>>Drawable Resource File**. Informe no campo **File Name** o valor **bg_bottom_sheet** e no campo **Root element** o valor **shape**, conforme Figura 130 e depois clique no botão Ok.

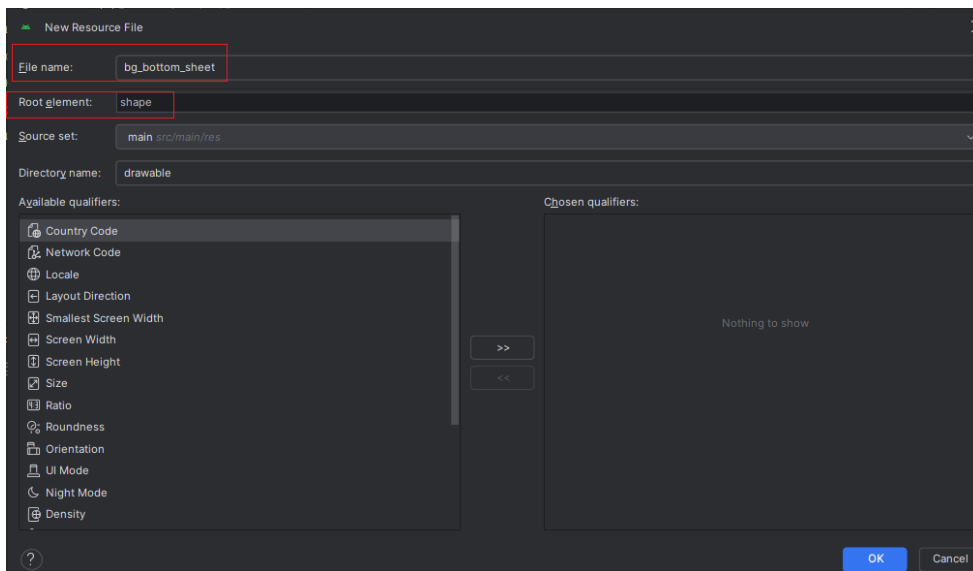


Figura 130

131. O arquivo **bg_bottom_sheet** será codificado conforme Figura 131.

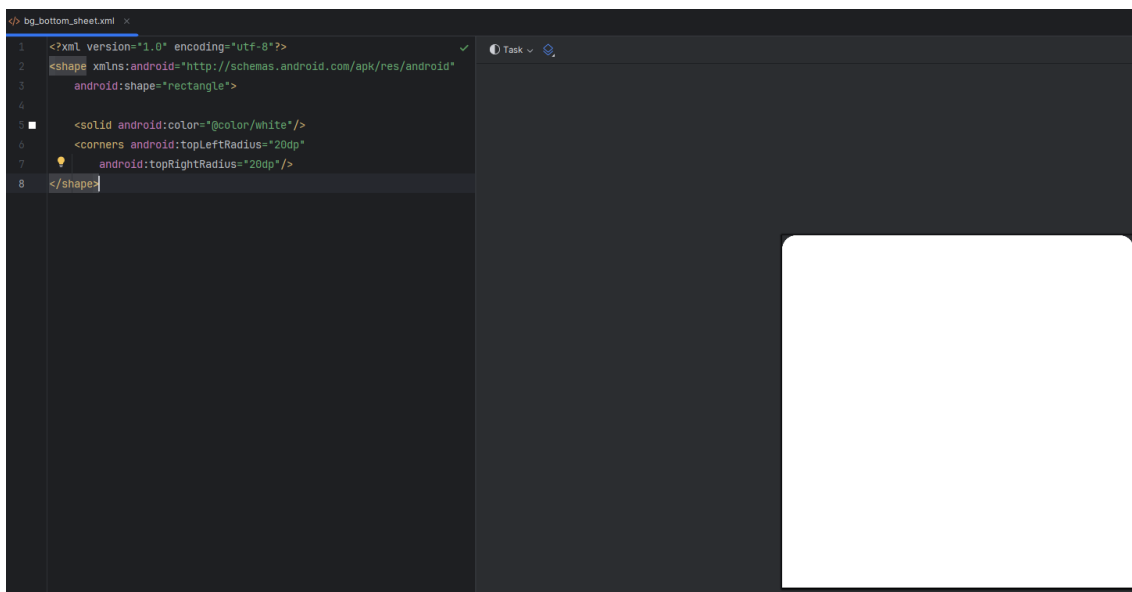


Figura 131

Os atributos `topLeftRadius` e `topRightRadius` arredondará apenas a parte superior do retângulo.

132. Volte ao arquivo **bottom_sheet.xml** para aplicarmos arredondamento das bordas conforme Figura 132. O atributo `background` será preenchido com o valor `@drawable/bg_bottom_sheet`.

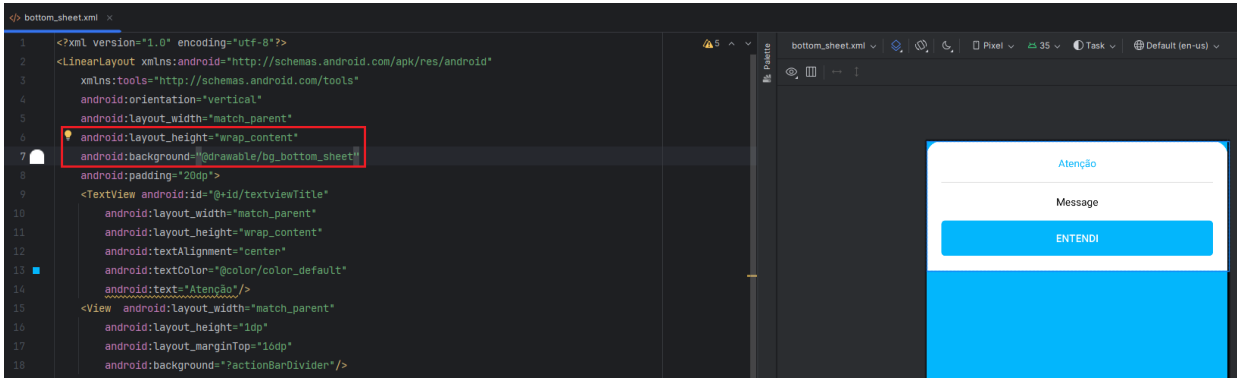


Figura 132

133. Vamos implementar o código responsável por exibir o componente **bottom_sheet**. Para isso, abra o arquivo **Extensions.kt**, localizado no pacote **util**, e implemente o método **showBottomSheet** conforme Figura 133.

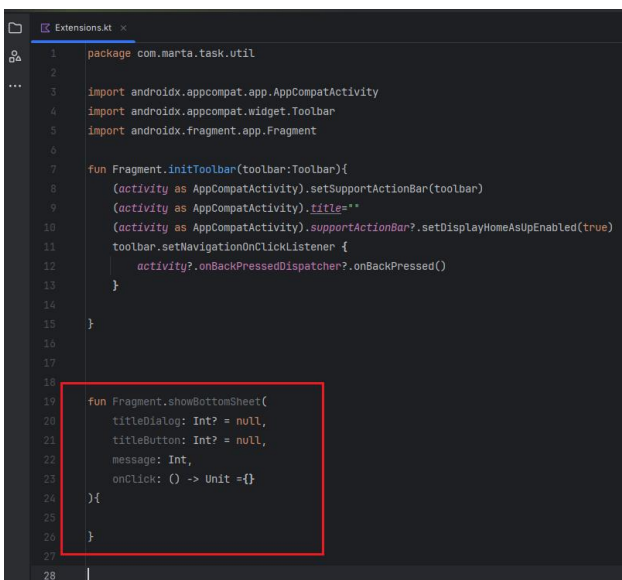


Figura 133

O método **showBottomSheet** recebe quatro parâmetros de entrada:

- O **titleDialog** define o título da tela;
- O **titleButton** corresponde ao rótulo do botão;
- O **message** exibe a mensagem informativa para o usuário;
- O **onClick** representa o evento disparado no clique do botão.

Os parâmetros **titleDialog** e **titleButton** são opcionais, portanto é utilizado a interrogação (?) após o tipo, indicando que eles podem receber valores nulos ou serem omitidos.

134. No arquivo **Extension.kt**, continue a codificação do método **showBottomSheet** conforme Figura 134.

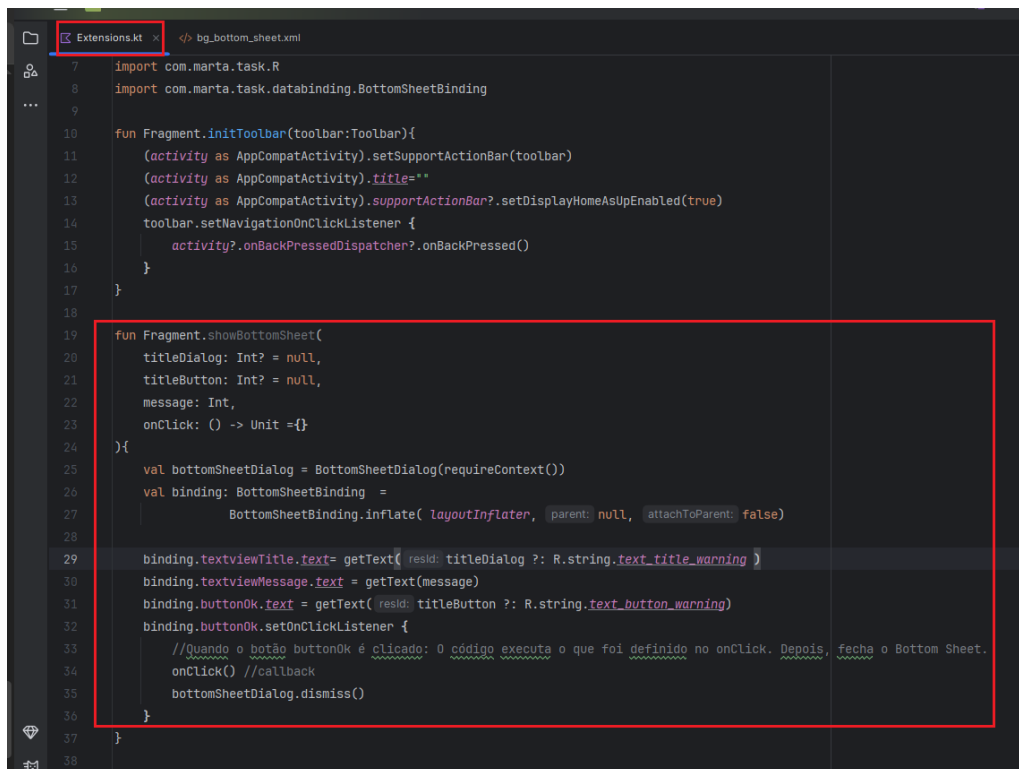
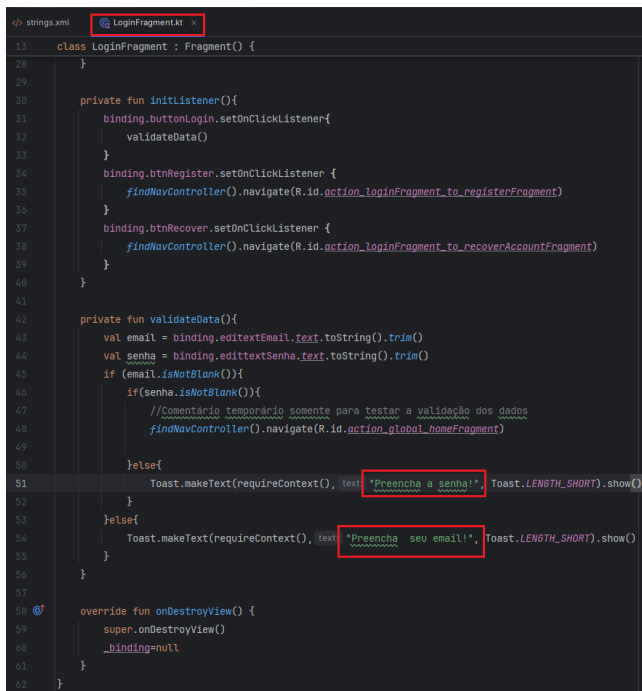


Figura 134

O método `showBottomSheet` é uma função de extensão da classe `Fragment`. Ele exibe o layout definido em `bottom_sheet.xml`, que representa um Bottom Sheet Dialog – um painel que desliza a partir da parte inferior da tela no Android. Esse painel contém um título, uma mensagem, um botão e um comportamento personalizado associado ao clique nesse botão. Os parâmetros da função `showBottomSheet` são descritos a seguir:

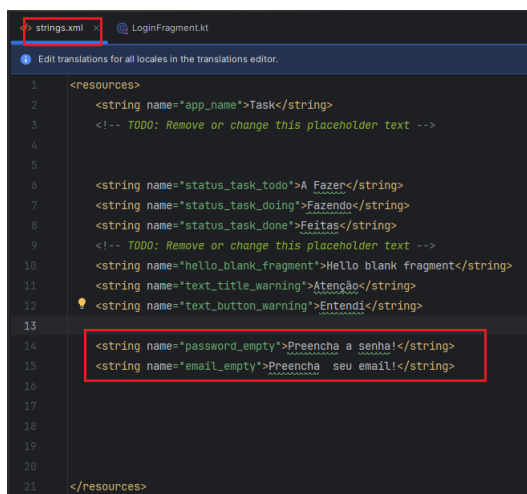
- O `titleDialog` define o título do `bottom_sheet.xml`. Esse parâmetro é opcional e, caso não seja informado, será utilizado um valor padrão. Se `titleDialog` for informado, usa ele. Caso contrário, usa o texto padrão `R.string.text_title_warning`.
- O `titleButton` define o rótulo do botão. Se `titleButton` for informado, usa ele. Caso contrário, usa o texto padrão `R.string.titleButton`.
- A `message` define uma mensagem informativa
- O evento `onClick` é uma função de callback executada quando o botão é clicado. Por padrão, faz nada se não for passado. Uma função de callback é uma função que você passa como parâmetro para outra função, com o objetivo de ser executada em algum momento específico, geralmente em resposta a um evento, como o clique de um botão. Quando o botão `buttonOk` é clicado: O código executa o que foi definido no `onClick`. Depois, fecha o Bottom Sheet.
- A variável `BottomSheetDialog` é iniciada com a instância do `BottomSheetDialog`, associado ao contexto atual da tela (Activity/Fragment) em que está sendo exibido.
- O método `bottomSheetDialog.dismiss` fecha o botão.

135. Antes criar as chamadas ao `bottom_sheet`, vamos fazer alguns ajustes das strings no código **LoginFragment.kt**, copie as strings “Preencha sua senha” e “Preencha seu e-mail” e cole no arquivo de **strings.xml** conforme Figuras 135 e Figura 136.



```
13 class LoginFragment : Fragment() {
28
29 }
30
31 private fun initListener(){
32     binding.buttonLogin.setOnClickListener{
33         validateData()
34     }
35     binding.btnRegister.setOnClickListener {
36         findNavController().navigate(R.id.action_loginFragment_to_registerFragment)
37     }
38     binding.btnRecover.setOnClickListener {
39         findNavController().navigate(R.id.action_loginFragment_to_recoverAccountFragment)
40     }
41 }
42
43 private fun validateData(){
44     val email = binding.editTextEmail.text.toString().trim()
45     val senha = binding.editTextSenha.text.toString().trim()
46     if (email.isNotBlank()){
47         if(senha.isNotBlank()){
48             //Comentário temporário somente para testar a validação dos dados
49             findNavController().navigate(R.id.action_global_homeFragment)
50         }else{
51             Toast.makeText(requireContext(), text "Preencha a senha!", Toast.LENGTH_SHORT).show()
52         }
53     }else{
54         Toast.makeText(requireContext(), text "Preencha seu email!", Toast.LENGTH_SHORT).show()
55     }
56 }
57
58 override fun onDestroyView() {
59     super.onDestroyView()
60     _binding=null
61 }
62 }
```

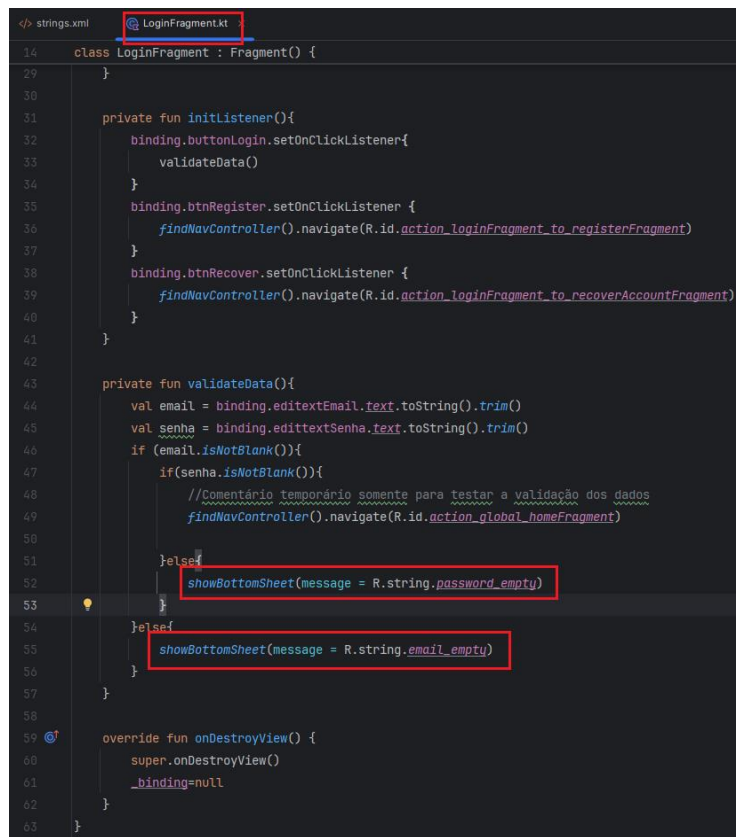
Figura 135



```
1 <resources>
2 <string name="app_name">Task</string>
3 <!-- TODO: Remove or change this placeholder text -->
4
5
6 <string name="status_task_todo">A Fazer</string>
7 <string name="status_task_doing">Fazendo</string>
8 <string name="status_task_done">Feitas</string>
9 <!-- TODO: Remove or change this placeholder text -->
10 <string name="hello_blank_fragment">Hello blank fragment</string>
11 <string name="text_title_warning">Atenção</string>
12 <string name="text_button_warning">Entendi</string>
13
14 <string name="password_empty">Preencha a senha!</string>
15 <string name="email_empty">Preencha seu email!</string>
16
17
18
19
20
21 </resources>
```

Figura 136

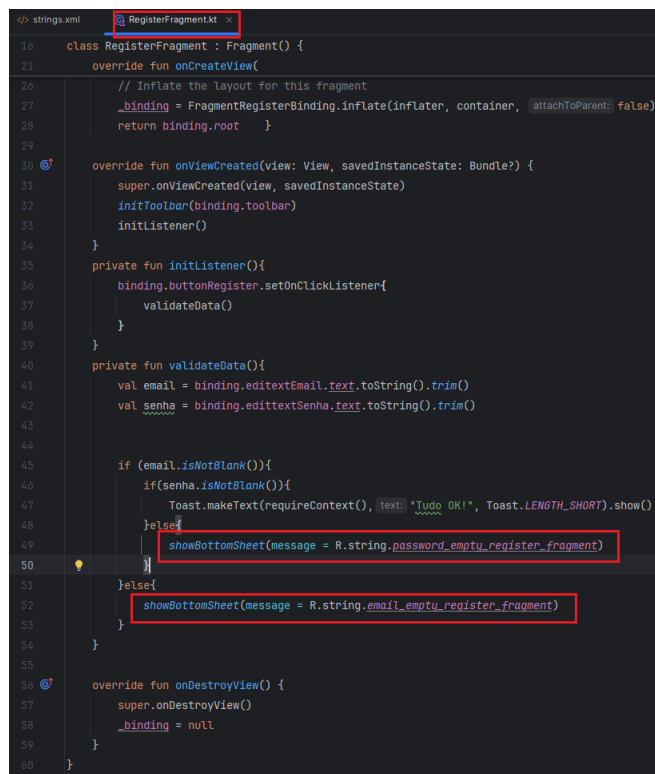
136. No código **LoginFragment.kt** substitua os Toasts pelo `bottom_sheet` conforme Figura 137.



```
14 class LoginFragment : Fragment() {
29 }
30
31 private fun initListener(){
32     binding.buttonLogin.setOnClickListener{
33         validateData()
34     }
35     binding.btnRegister.setOnClickListener {
36         findNavController().navigate(R.id.action_loginFragment_to_registerFragment)
37     }
38     binding.btnRecover.setOnClickListener {
39         findNavController().navigate(R.id.action_loginFragment_to_recoverAccountFragment)
40     }
41 }
42
43 private fun validateData(){
44     val email = binding.edittextEmail.text.toString().trim()
45     val senha = binding.edittextSenha.text.toString().trim()
46     if (email.isNotBlank()){
47         if(senha.isNotBlank()){
48             //Comentário temporário somente para testar a validação dos dados
49             findNavController().navigate(R.id.action_global_homeFragment)
50         }else{
51             showBottomSheet(message = R.string.password_empty)
52         }
53     }else{
54         showBottomSheet(message = R.string.email_empty)
55     }
56 }
57
58
59 override fun onDestoryView() {
60     super.onDestoryView()
61     _binding=null
62 }
63 }
```

Figura 137

137. No código **RegisterFragment.kt** substitua os Toasts pelo bottom_sheet conforme Figura 138. Crie no arquivo **strings.xml** novas entradas com os nomes **password_empty_register_fragment** e **email_empty_register_fragment**.



```
16 class RegisterFragment : Fragment() {
21     override fun onCreateView(
22         // Inflate the layout for this fragment
23         inflater: LayoutInflater, container: ViewGroup?,
24         savedInstanceState: Bundle?
25     ): View? {
26         _binding = FragmentRegisterBinding.inflate(inflater, container, false)
27         return binding.root
28     }
29
30     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
31         super.onViewCreated(view, savedInstanceState)
32         initToolBar(binding.toolbar)
33         initListener()
34     }
35     private fun initListener(){
36         binding.buttonRegister.setOnClickListener{
37             validateData()
38         }
39     }
40     private fun validateData(){
41         val email = binding.edittextEmail.text.toString().trim()
42         val senha = binding.edittextSenha.text.toString().trim()
43
44         if (email.isNotBlank()){
45             if(senha.isNotBlank()){
46                 Toast.makeText(requireContext(), "Tudo OK!", Toast.LENGTH_SHORT).show()
47             }else{
48                 showBottomSheet(message = R.string.password_empty_register_fragment)
49             }
50         }else{
51             showBottomSheet(message = R.string.email_empty_register_fragment)
52         }
53     }
54
55
56 override fun onDestoryView() {
57     super.onDestoryView()
58     _binding = null
59 }
60 }
```

Figura 138

138. Por enquanto o arquivo de **string.xml** está ficando como da Figura 139.

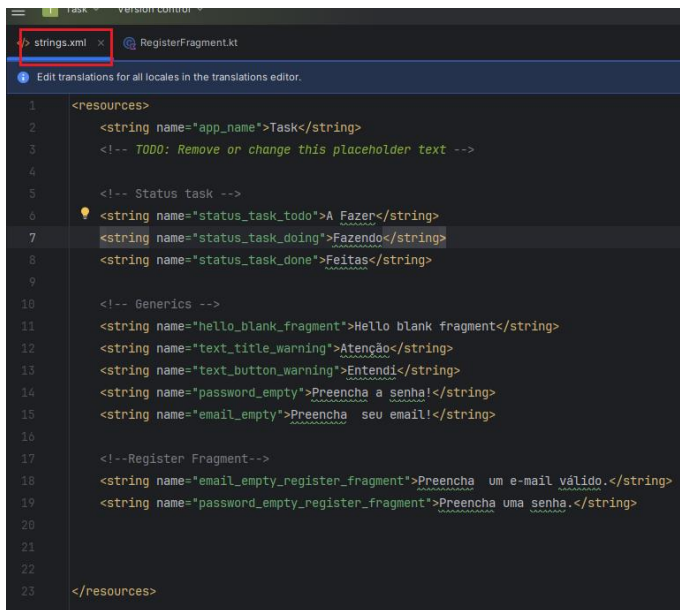


Figura 139

139. No código **RecoverAccountFragment.kt** substitua os Toasts pelo bottom_sheet conforme Figura 140.

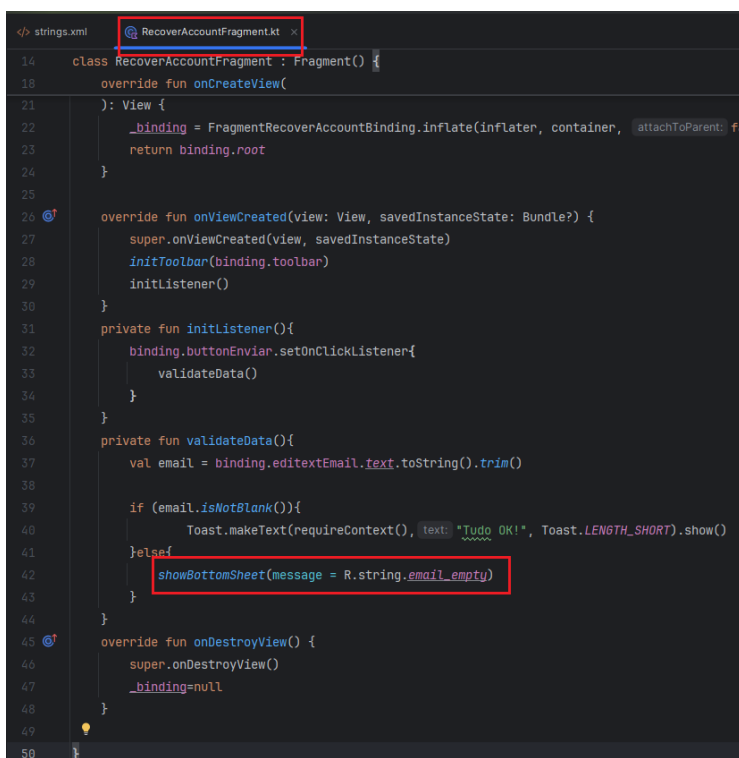
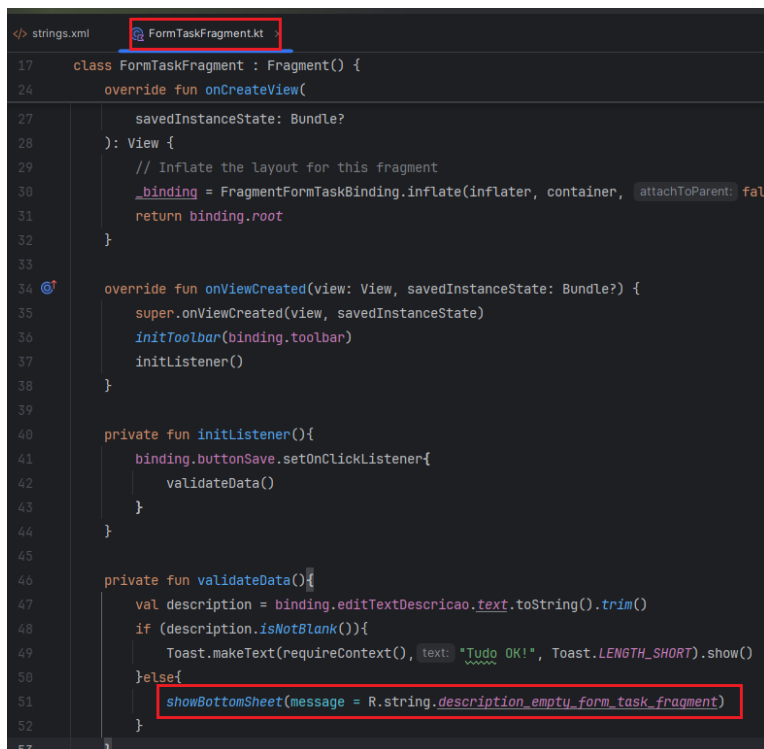


Figura 140

140. No código **FormTaskFragment.kt** substitua os Toasts pelo bottom_sheet conforme Figura 141.



```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="title_task_fragment" type="string">Tarefa</string>
</resources>

class FormTaskFragment : Fragment() {
    override fun onCreateView(
        savedInstanceState: Bundle?
    ): View {
        // Inflate the layout for this fragment
        binding = FragmentFormTaskBinding.inflate(inflater, container, attachToParent: false)
        return binding.root
    }

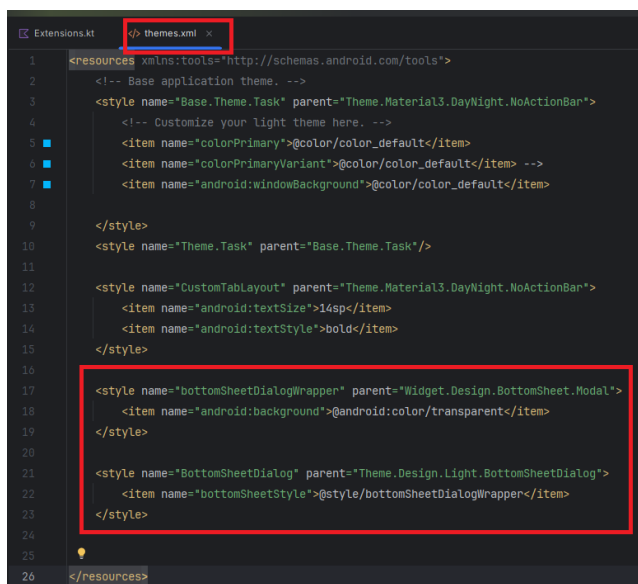
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        initToolbar(binding.toolbar)
        initListener()
    }

    private fun initListener(){
        binding.buttonSave.setOnClickListener{
            validateData()
        }
    }

    private fun validateData(){
        val description = binding.editTextDescricao.text.toString().trim()
        if (description.isNotBlank()){
            Toast.makeText(requireContext(), text: "Tudo OK!", Toast.LENGTH_SHORT).show()
        }else{
            showBottomSheet(message = R.string.description_empty_form_task_fragment)
        }
    }
}
```

Figura 141

141. Vamos configurar o BottomSheet para utilizar os estilos definidos no arquivo themes.xml. Para isso, abra o arquivo **themes.xml** e adicione o código mostrado na Figura 142.



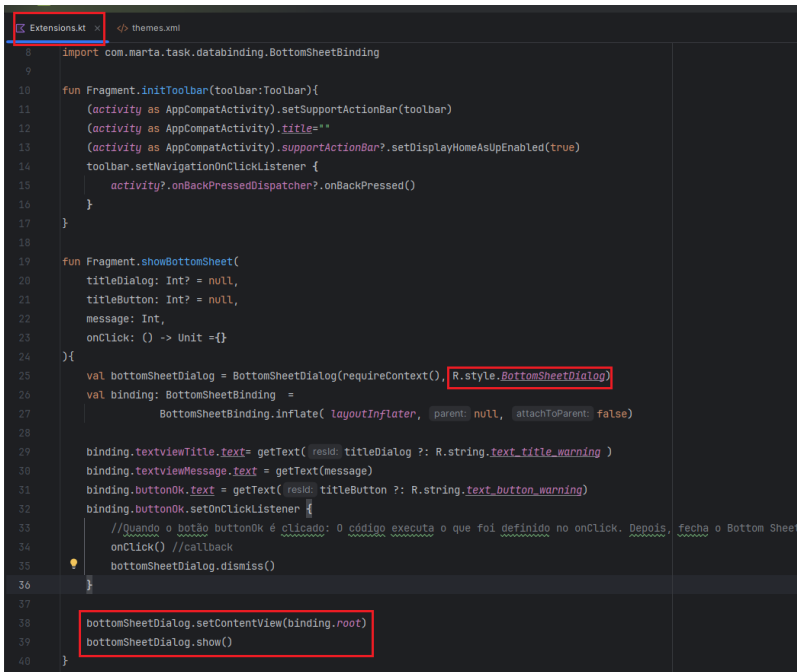
```
<?xml version="1.0" encoding="utf-8"?>
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme, -->
    <style name="Base.Theme.Task" parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your light theme here. -->
        <item name="colorPrimary">@color/color_default</item>
        <item name="colorPrimaryVariant">@color/color_default</item> -->
        <item name="android:windowBackground">@color/color_default</item>
    </style>
    <style name="Theme.Task" parent="Base.Theme.Task"/>
    <style name="CustomTabLayout" parent="Theme.Material3.DayNight.NoActionBar">
        <item name="android:textSize">14sp</item>
        <item name="android:textStyle">bold</item>
    </style>
    <style name="bottomSheetDialogWrapper" parent="Widget.Design.BottomSheet.Modal">
        <item name="android:background">@android:color/transparent</item>
    </style>
    <style name="BottomSheetDialog" parent="Theme.Design.Light.BottomSheetDialog">
        <item name="bottomSheetStyle">@style/bottomSheetDialogWrapper</item>
    </style>
</resources>
```

Figura 142

Esse trecho de código da Figura 142 define estilos personalizados para um Bottom Sheet Dialog no Android, controlando principalmente sua aparência — no caso, deixando o fundo transparente.

O estilo `bottomSheetDialogWrapper` herda do estilo padrão do componente modal Bottom Sheet: `Widget.Design.BottomSheet.Modal`. Além disso, o estilo `bottomSheetDialogWrapper` modifica a propriedade para a cor do fundo transparente. O estilo `BottomSheetDialog` herda de: `Theme.Design.Light.BottomSheetDialog` (tema claro para Bottom Sheets).

142. Abra o arquivo **Extensions.kt** e inclua o estilo ao componente BottomSheet, conforme arquivo 143.



```
1 import com.marta.task.databinding.BottomSheetBinding
2
3
4
5
6
7
8
9
10 fun Fragment.initToolBar(toolbar: Toolbar){
11     (activity as AppCompatActivity).setSupportActionBar(toolbar)
12     (activity as AppCompatActivity).title=""
13     (activity as AppCompatActivity).supportActionBar?.setDisplayHomeAsUpEnabled(true)
14     toolbar.setNavigationOnClickListener {
15         activity?.onBackPressedDispatcher?.onBackPressed()
16     }
17 }
18
19
20 fun Fragment.showBottomSheet(
21     titleDialog: Int? = null,
22     titleButton: Int? = null,
23     message: Int,
24     onClick: () -> Unit ={}
25 ){
26     val bottomSheetDialog = BottomSheetDialog(requireContext(), R.style.BottomSheetDialog)
27     val binding: BottomSheetBinding =
28         BottomSheetBinding.inflate( layoutInflater, parent: null, attachToParent: false)
29
30     binding.textViewTitle.text= getText( resId: titleDialog ? R.string.text_title_warning )
31     binding.textViewMessage.text = getText(message)
32     binding.buttonOk.text = getText( resId: titleButton ? R.string.text_button_warning)
33     binding.buttonOk.setOnClickListener {
34         //Quando o botão buttonOk é clicado: o código executa o que foi definido no onClick. Depois, fecha o Bottom Sheet
35         onClick() //callback
36         bottomSheetDialog.dismiss()
37     }
38
39     bottomSheetDialog.setContentView(binding.root)
40     bottomSheetDialog.show()
41 }
```

Figura 143

O método `setContentView(binding.root)` define qual será o conteúdo visual (layout) que o `BottomSheetDialog` vai exibir. Nesse código está sendo passando a view raiz do seu layout inflado pelo View Binding (`binding.root`). Em outras palavras, a linha de código `bottomSheetDialog.setContentView(binding.root)` está dizendo ao Bottom Sheet Dialog: "Use esse layout aqui como conteúdo da janela".

O trecho de código `bottomSheetDialog.show()` exibe o `BottomSheetDialog` na tela.

143. Execute o App para testar conforme Figura 144.

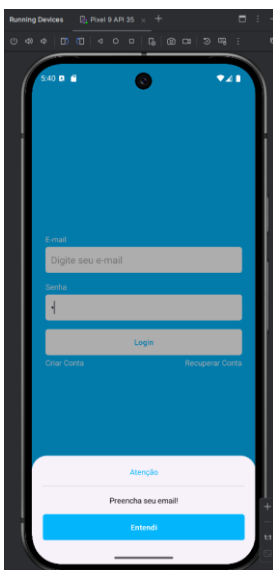


Figura 144

144. Vamos continuar configurando mais alguns estilos. Dessa vez para os componentes EditText. Para isso abra o arquivo **Themes.xml** e crie o estilo conforme Figura 145.

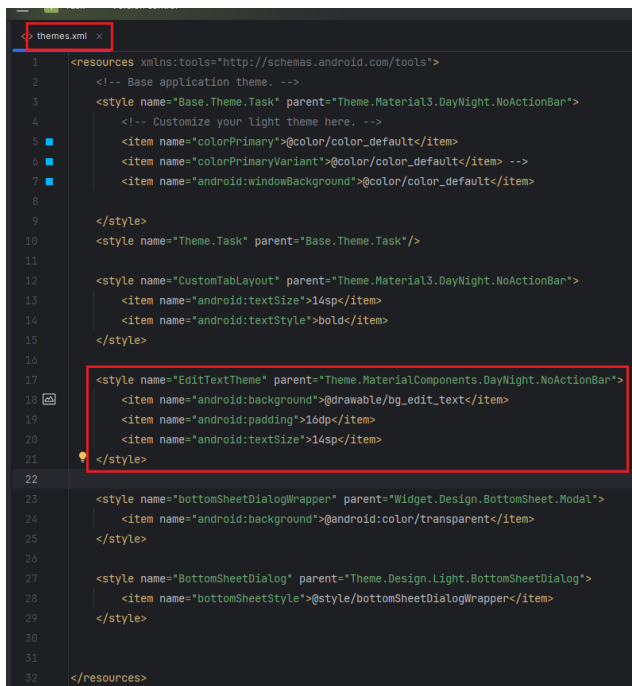


Figura 145

145. Abra o arquivo **fragment_login.xml**. Remova as propriedades background e padding que estão se repetindo nos componentes editTexts. Substitua pela propriedade style conforme Figura 146.

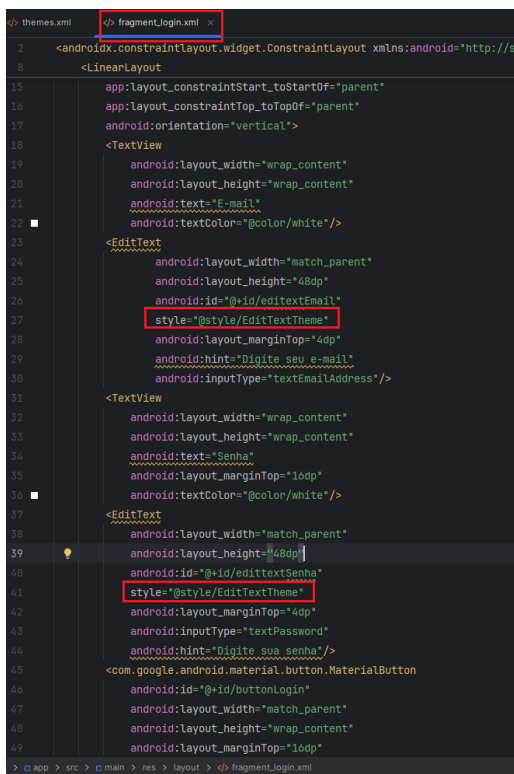


Figura 146

146. Vamos fazer o mesmo procedimento feito na tela `fragment_login.xml`, e replicar nas telas `fragment_form_task.xml`, `fragment_register.xml` e `fragment_recover_account.xml`.

147. Vamos continuar configurando mais alguns estilos. Dessa vez para o componente Button. Para isso abra o arquivo **Themes.xml** e crie o estilo conforme Figura 147.

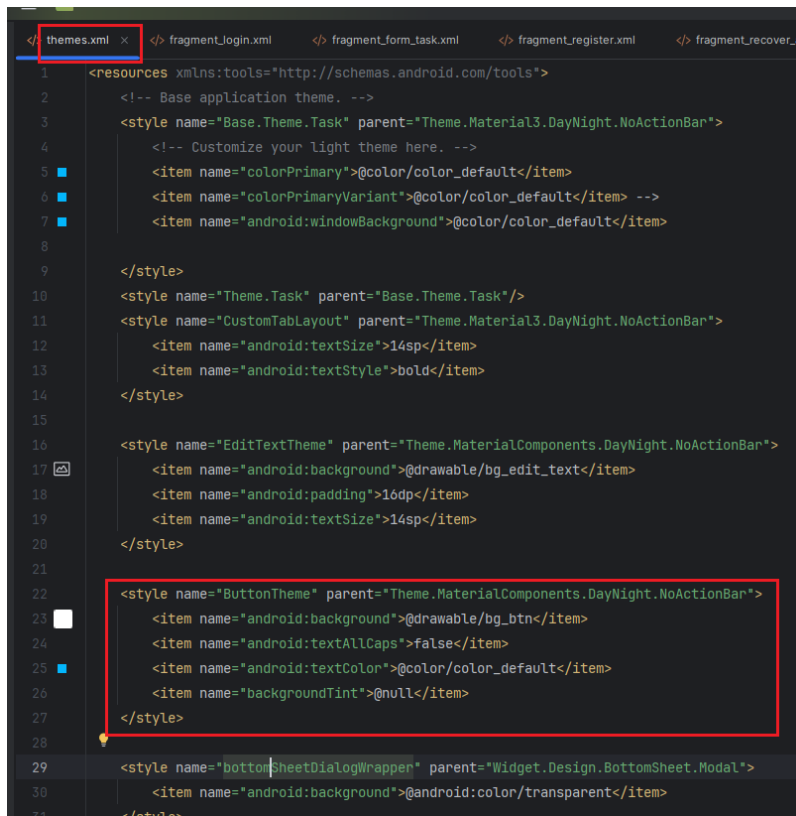
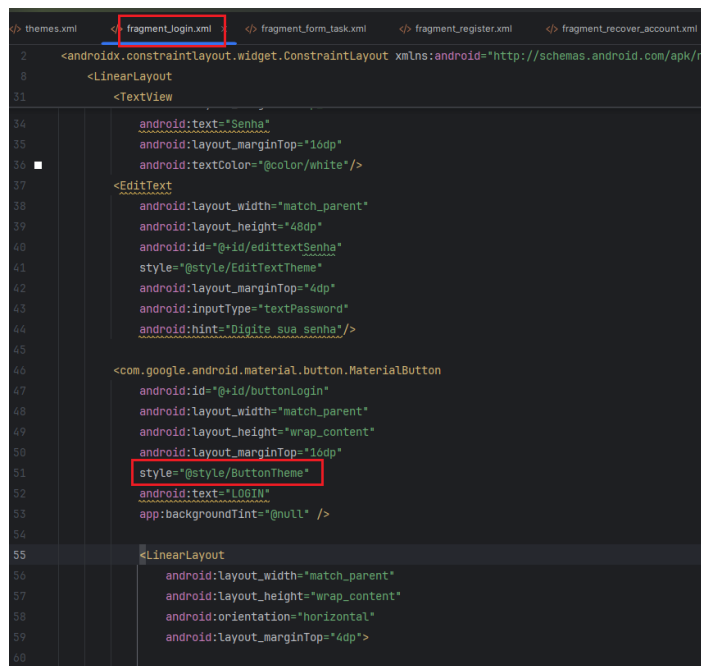


Figura 147

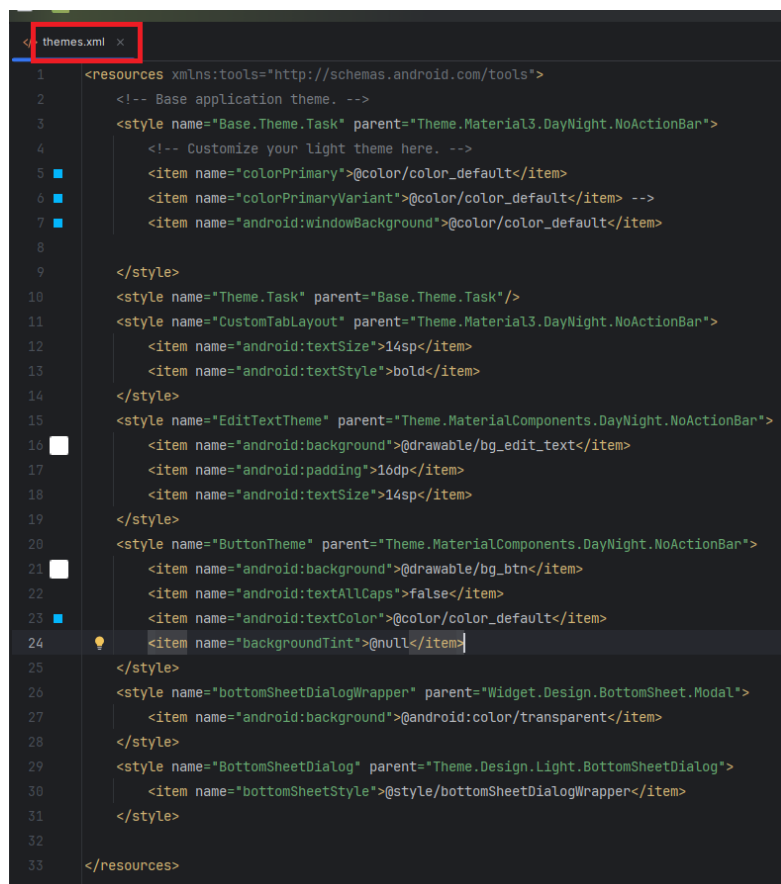
148. Abra o arquivo `fragment_login.xml`. Remova as propriedades `background`, `textColor` e `textAllCaps` que estão se repetindo nos componentes Button. Substitua pela propriedade `style` conforme Figura 148. Depois, faça o mesmo procedimento feito na tela `fragment_login.xml`, e replique nas telas `fragment_form_task.xml`, `fragment_register.xml` e `fragment_recover_account.xml`.



```
<?xml version="1.0" encoding="utf-8" android:windowBackground="@color/white"
2   <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
3       android:background="@color/white"
4       android:padding="16dp"
5       android:theme="@style/Base.Theme.Task"
6       >
7       <TextView
8           android:id="@+id/textView"
9           android:layout_width="match_parent"
10          android:layout_height="48dp"
11          android:text="Senha"
12          android:layout_marginTop="16dp"
13          android:textColor="@color/white"
14          >
15          <EditText
16              android:id="@+id/editTextSenha"
17              android:layout_width="match_parent"
18              android:layout_height="48dp"
19              android:inputType="textPassword"
20              android:hint="Digite sua senha"
21              >
22              <com.google.android.material.button.MaterialButton
23                  android:id="@+id/buttonLogin"
24                  android:layout_width="match_parent"
25                  android:layout_height="wrap_content"
26                  android:layout_marginTop="16dp"
27                  style="@style/ButtonTheme"
28                  android:text="LOGIN"
29                  app:backgroundTint="@null"
30                  >
31          </LinearLayout>
32      </ConstraintLayout>
33  </?xml>
```

Figura 148

149. O arquivo themes.xml está descrito de forma completa na Figura 149.



```
<?xml version="1.0" encoding="utf-8"
1   <resources xmlns:tools="http://schemas.android.com/tools">
2       <!-- Base application theme. -->
3       <style name="Base.Theme.Task" parent="Theme.Material3.DayNight.NoActionBar">
4           <!-- Customize your light theme here. -->
5           <item name="colorPrimary">@color/color_default</item>
6           <item name="colorPrimaryVariant">@color/color_default</item> -->
7           <item name="android:windowBackground">@color/color_default</item>
8       </style>
9       <style name="Theme.Task" parent="Base.Theme.Task"/>
10      <style name="CustomTabLayout" parent="Theme.Material3.DayNight.NoActionBar">
11          <item name="android:textSize">14sp</item>
12          <item name="android:textStyle">bold</item>
13      </style>
14      <style name="EditTextTheme" parent="Theme.MaterialComponents.DayNight.NoActionBar">
15          <item name="android:background">@drawable/bg_edit_text</item>
16          <item name="android:padding">16dp</item>
17          <item name="android:textSize">14sp</item>
18      </style>
19      <style name="ButtonTheme" parent="Theme.MaterialComponents.DayNight.NoActionBar">
20          <item name="android:background">@drawable/bg_btn</item>
21          <item name="android:textAllCaps">false</item>
22          <item name="android:textColor">@color/color_default</item>
23          <item name="backgroundTint">@null</item>
24      </style>
25      <style name="bottomSheetDialogWrapper" parent="Widget.Design.BottomSheet.Modal">
26          <item name="android:background">@android:color/transparent</item>
27      </style>
28      <style name="BottomSheetDialog" parent="Theme.Design.Light.BottomSheetDialog">
29          <item name="bottomSheetStyle">@style/bottomSheetDialogWrapper</item>
30      </style>
31  </resources>
```

Figura 149

Ao criar um único estilo, qualquer alteração feita nele atualiza automaticamente todos os botões, pois eles seguem um padrão unificado.

150. Primeiro feche todas as telas. Em seguida será feito uma pequena adaptação na tela **Extension.kt**. Essas modificações serão necessárias quando integrarmos essa tela ao banco de dados. As mensagens de erro passarão a ser do tipo String, e não mais do tipo inteiro. Para isso, abra o arquivo **Extension.kt** e realize as seguintes alterações conforme Figura 150.

```
fun Fragment.showBottomSheet(
    titleDialog: Int? = null,
    titleButton: Int? = null,
    message: String,
    onClick: () -> Unit = {}
){
    val bottomSheetDialog = BottomSheetDialog(requireContext(), R.style.BottomSheetDialog)
    val binding: BottomSheetBinding =
        BottomSheetBinding.inflate(layoutInflater, parent: null, attachToParent: false)

    binding.textViewTitle.text = getText(resId: titleDialog ?: R.string.text_title_warning)

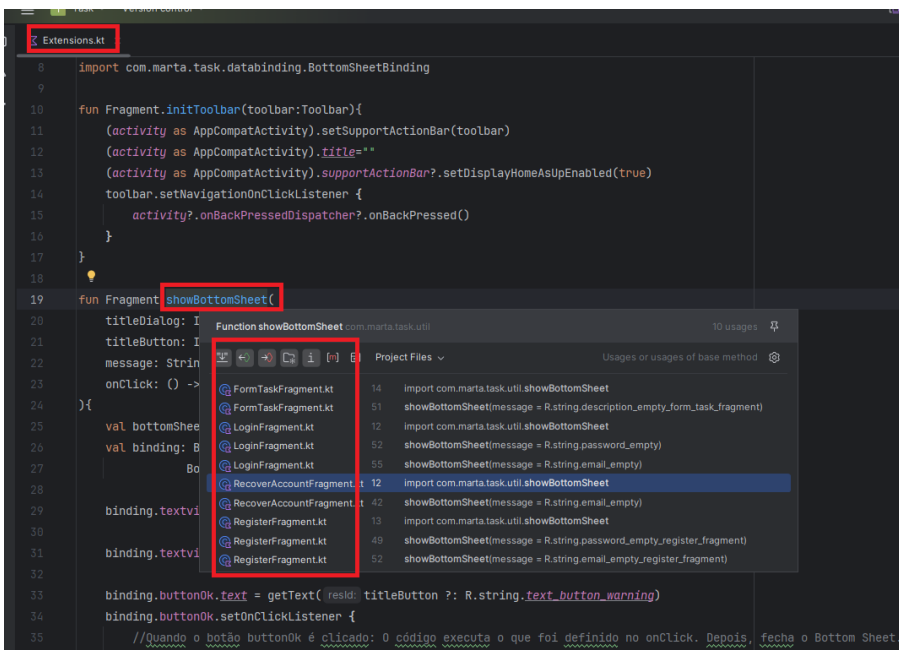
    binding.textViewMessage.text = message

    binding.buttonOk.text = getText(resId: titleButton ?: R.string.text_button_warning)
    binding.buttonOk.setOnClickListener {
        //Quando o botão buttonOk é clicado: O código executa o que foi definido no onClick. Depois, fecha o Bottom Sheet.
        onClick() //callback
        bottomSheetDialog.dismiss()
    }

    bottomSheetDialog.setContentView(binding.root)
    bottomSheetDialog.show()
}
```

Figura 150

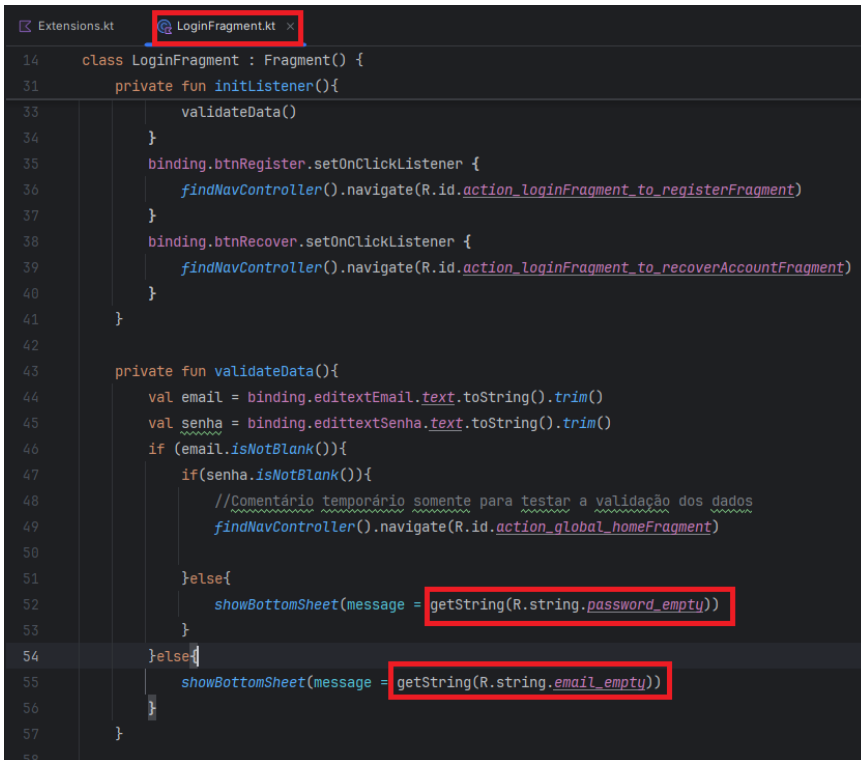
151. Com a alteração do tipo da variável **message**, será necessário ajustar todas as telas que utilizam o método **showBottomSheet**. Para localizar onde ele é usado, clique com o botão direito sobre o método **showBottomSheet** e selecione a opção **Go To > Declaration or Usages**. Como alternativa, você pode simplesmente pressionar o atalho **Ctrl + B**. Isso exibirá uma lista com todas as Activities e Fragments que utilizam essa extensão, conforme mostrado na Figura 151. Em seguida, localize o **LoginFragment** e clique para abri-lo.



The screenshot shows the **Extensions.kt** file in an IDE. The **showBottomSheet** method is defined, and a right-click context menu is open over it. The menu includes options like 'Go to Declaration', 'Go to Usages', and 'Project Files'. The 'Project Files' tab is selected, showing a list of files that use the **showBottomSheet** method, including **FormTaskFragment.kt**, **LoginFragment.kt**, **RecoverAccountFragment.kt**, and **RegisterFragment.kt**. The **LoginFragment.kt** file is highlighted in the list.

Figura 151

152. Na tela **LoginFragment.kt** faça as alterações conforme Figura 152. Faça essas mesmas alterações nos arquivos RegisterFragment.kt, RecoverAccount.kt e FormTaskFragment.kt.



```
14 class LoginFragment : Fragment() {
31     private fun initListener(){
33         validateData()
34     }
35     binding.btnRegister.setOnClickListener {
36         findNavController().navigate(R.id.action_loginFragment_to_registerFragment)
37     }
38     binding.btnRecover.setOnClickListener {
39         findNavController().navigate(R.id.action_loginFragment_to_recoverAccountFragment)
40     }
41 }
42
43 private fun validateData(){
44     val email = binding.edittextEmail.text.toString().trim()
45     val senha = binding.edittextSenha.text.toString().trim()
46     if (email.isNotBlank()){
47         if(senha.isNotBlank()){
48             //Comentário temporário somente para testar a validação dos dados
49             findNavController().navigate(R.id.action_global_homeFragment)
50         }else{
51             showBottomSheet(message = getString(R.string.password_empty))
52         }
53     }else{
54         showBottomSheet(message = getString(R.string.email_empty))
55     }
56 }
57
58 }
```

Figura 152

153. Agora vamos colocar todos os textos que estão hardcode no XML e transportá-los para o arquivo **strings.xml**. Basicamente os atributos text e hint serão alterados. Abra o arquivo strings.xml e inclua novas strings conforme destacado na Figura 153.

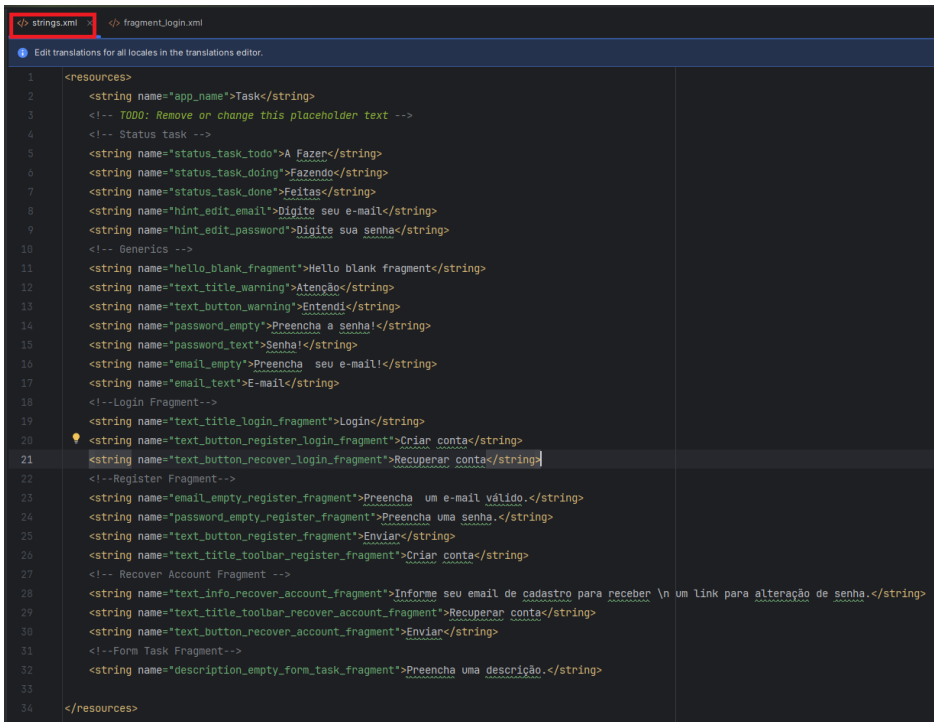


Figura 153

154. O arquivo **fragment_login.xml** está ilustrado na Figura 154, mostrando a substituição das strings. Isso deve ser feito em todas as telas.

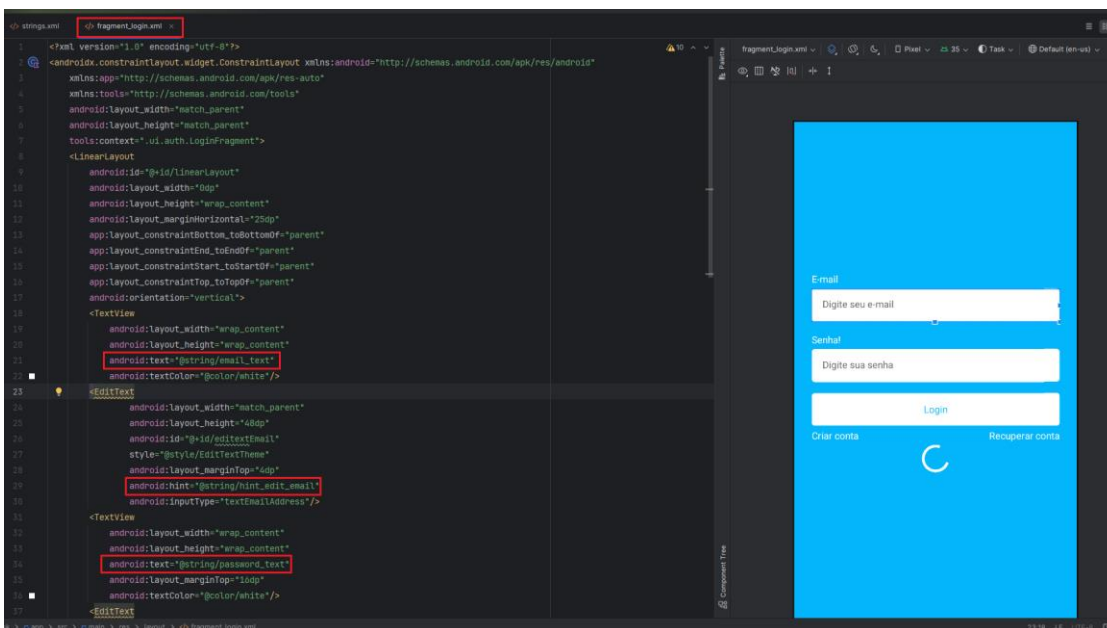


Figura 154

155. Execute seu APP para verificar se está tudo funcionando.

156. Agora vamos começar a trabalhar com listagem de dados. A listagem é usada para organizar e exibir conjuntos de dados. Podemos encontrá-la, por exemplo, no aplicativo YouTube, em que é apresentada uma lista de vídeos, ou no Instagram, que exibe uma lista de fotos, entre outros aplicativos conforme Figura 155.

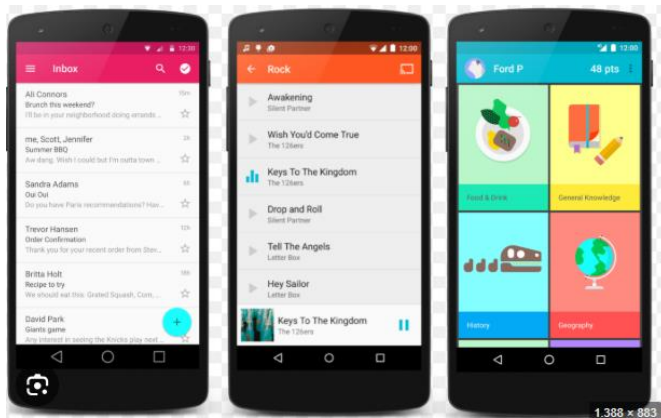


Figura 155

157. Em nosso App será uma listagem de tarefas, conforme ilustrado na Figura 156.

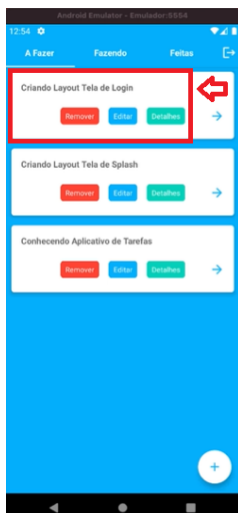


Figura 156

158. Clique com o botão direito do mouse sobre a pasta layout e selecione a opção **New > Layout Resource File**. Na janela que será exibida, informe item_task no campo File Name, conforme ilustrado na Figura 157. Depois clique no botão Ok.

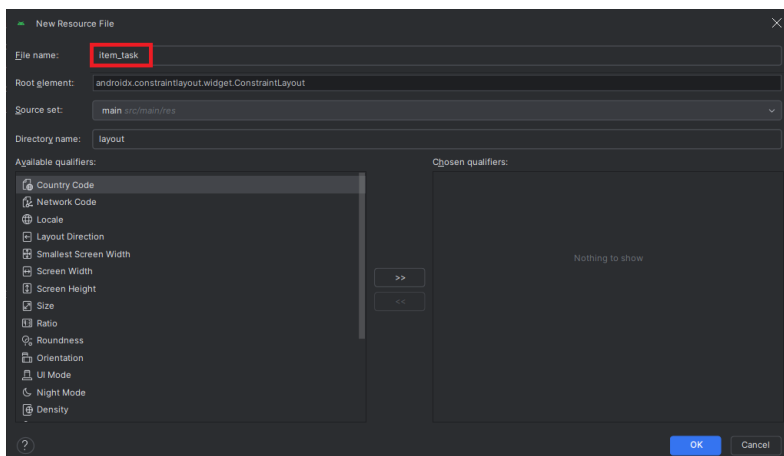


Figura 157

159. Abra o arquivo **item_task.xml**. Faça as seguintes alterações no arquivo: Altere o atributo `layout_height` para o valor `wrap_content`, inclua um componente `MaterialCardView` e inclua outro container `ConstraintLayout` conforme Figura 158.

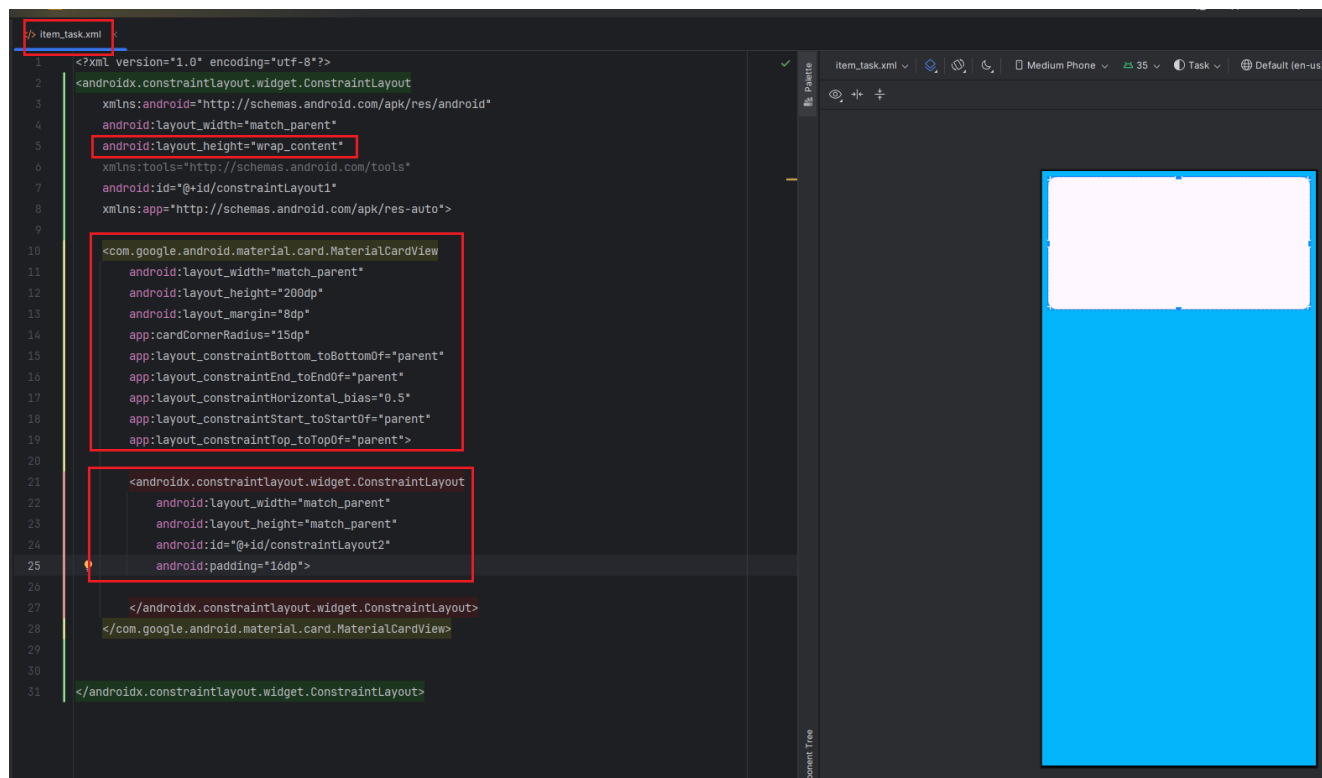


Figura 158

O `MaterialCardView` serve para exibir conteúdos em formato de “cartão” ou agrupar informações, seguindo as diretrizes do Material Design. Ele é ideal para destacar informações em blocos visuais organizados, com elevação, cantos arredondados, bordas e efeitos visuais.

160. Ainda no arquivo **item_task.xml**, adicione quatro componentes dentro do `ConstraintLayout` com o ID `constraintLayout2`, conforme especificado na Tabela 3.

```

<TextView

    android:layout_width="match_parent"

    android:layout_height="wrap_content"

    android:id="@+id/textDescription"

    tools:text="Criar layout da tela de login do app"

    android:textSize="16sp"

    android:textStyle="bold"

    app:layout_constraintEnd_toEndOf="parent"

    app:layout_constraintStart_toStartOf="parent"

    app:layout_constraintTop_toTopOf="parent"/>

```

```
<com.google.android.material.button.MaterialButton
```

```
    android:id="@+id/buttonDelete"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="48dp"
```

```
    android:text="Remover"
```

```
    android:background="@drawable/bg_btn"
```

```
    app:backgroundTint="@color/color_delete"
```

```
    android:textAllCaps="false"
```

```
    android:textSize="12sp"
```

```
    tools:layout_editor_absoluteX="16dp"
```

```
    tools:layout_editor_absoluteY="76dp" />
```

```
<com.google.android.material.button.MaterialButton
```

```
    android:id="@+id/buttonEditar"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="48dp"
```

```
    android:text="Editar"
```

```
    tools:layout_editor_absoluteX="153dp"
```

```
    android:background="@drawable/bg_btn"
```

```
    app:backgroundTint="@color/color_default"
```

```
    android:textAllCaps="false"
```

```
    android:textSize="12sp"
```

```
    tools:layout_editor_absoluteY="76dp" />
```

```
<com.google.android.material.button.MaterialButton
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="48dp"
```

```
    android:text="Detalhes"
```

```
    android:id="@+id/buttonDetails"
```

```
    android:background="@drawable/bg_btn"
```

```
    app:backgroundTint="@color/color_details"
```

```
    android:textAllCaps="false"
```

```
    android:textSize="12sp"
```

```
    tools:layout_editor_absoluteX="258dp"
```

```
tools:layout_editor_absoluteY="76dp" />
```

Tabela 3

O container pai está com o atributo `android:layout_height="wrap_content"` pois ao incorporar o componente na tela, queremos que seja mostrado apenas 1 item por vez.

161. Abra o arquivo **colors.xml**. Inclua as cores `color_delete` e `color_details` conforme Figura 159.

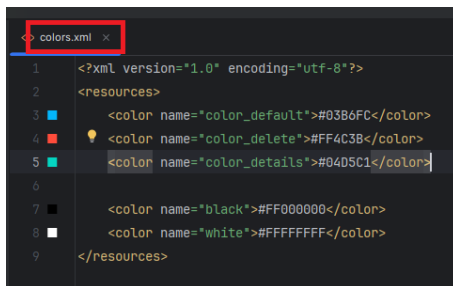


Figura 159

162. Abra o arquivo **item_task.xml**. Vamos alinhar os botões conforme mostrado na Figura 159. Primeiramente, foi realizado o alinhamento dos botões e, em seguida, criada uma chain horizontal entre eles. O código referente a esse alinhamento está apresentado na Tabela 4.

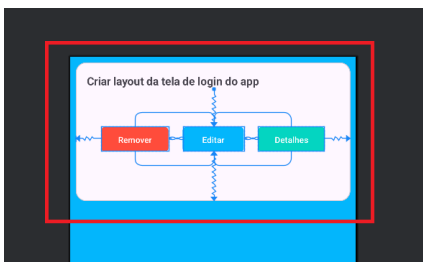


Figura 159

163. Depois selecione os botões e coloque um estilo `packed` nos botões conforme Figura 160.

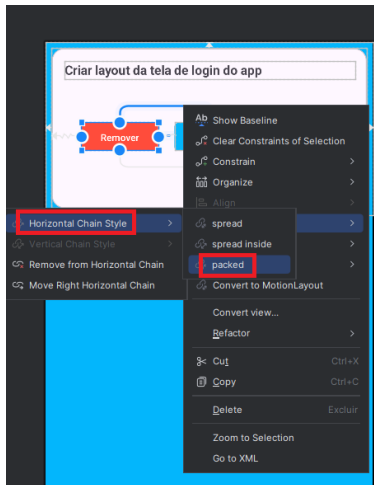


Figura 160

164. Selecione o botão editar e coloque uma `marginHorizontal` de 8dp conforme Figura 161.

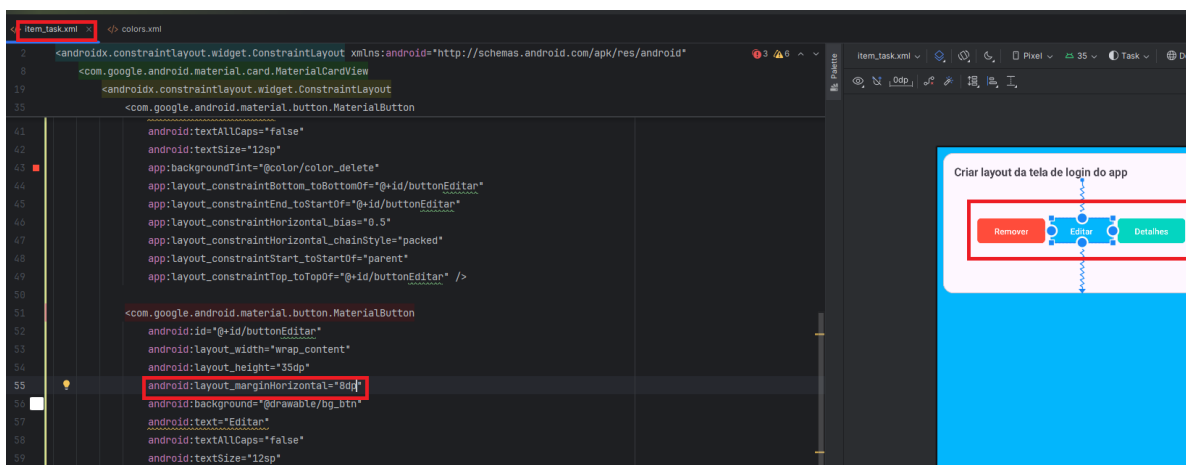
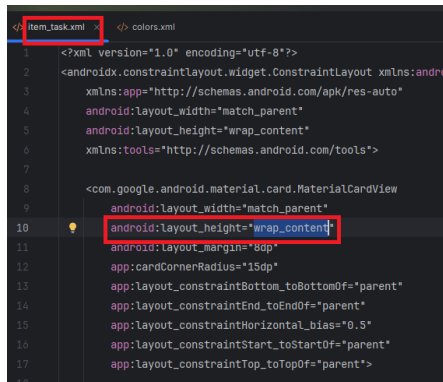


Figura 161

165. No componente `MaterialCardView` coloque o atributo `android:layout_height="wrap_content"` conforme Figura 162.

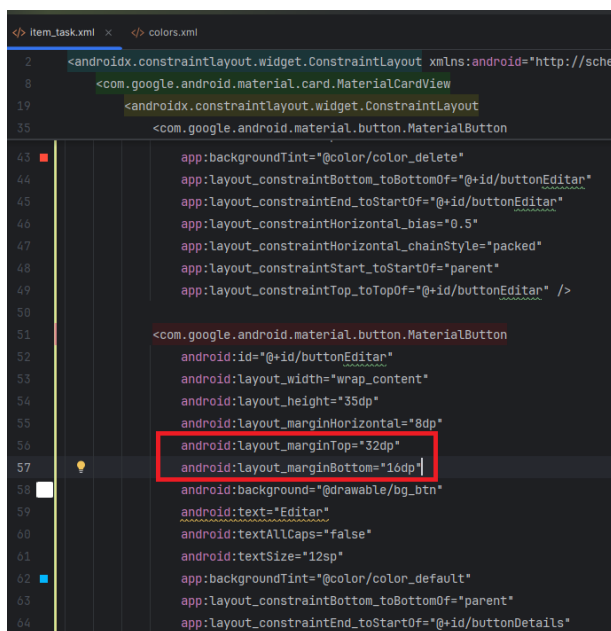


```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    xmlns:tools="http://schemas.android.com/tools">

    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="8dp"
        app:cardCornerRadius="15dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.5"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent">
```

Figura 162

166. No botão editar inclua as propriedades `android:layout_marginTop="32dp"` e `android:layout_marginBottom="16dp"` conforme Figura 163.



```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    xmlns:tools="http://schemas.android.com/tools">

    <com.google.android.material.button.MaterialButton
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="8dp"
        app:backgroundTint="@color/color_delete"
        app:layout_constraintBottom_toBottomOf="@+id/buttonEditar"
        app:layout_constraintEnd_toStartOf="@+id/buttonEditar"
        app:layout_constraintHorizontal_bias="0.5"
        app:layout_constraintHorizontal_chainStyle="packed"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="@+id/buttonEditar" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonEditar"
        android:layout_width="wrap_content"
        android:layout_height="35dp"
        android:layout_marginHorizontal="8dp"
        android:layout_marginTop="32dp"
        android:layout_marginBottom="16dp"
        android:background="@drawable/bg_btn"
        android:text="Editar"
        android:textAllCaps="false"
        android:textSize="12sp"
        app:backgroundTint="@color/color_default"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toStartOf="@+id/buttonDetails">
```

Figura 163

167. Inclua um novo ícone no projeto com o nome `icBack`. Esse ícone será uma seta que ficará ao lado do botão remover.

168. Inclua o componente `ImageButton` conforme Figura 164 e faça o alinhamento.

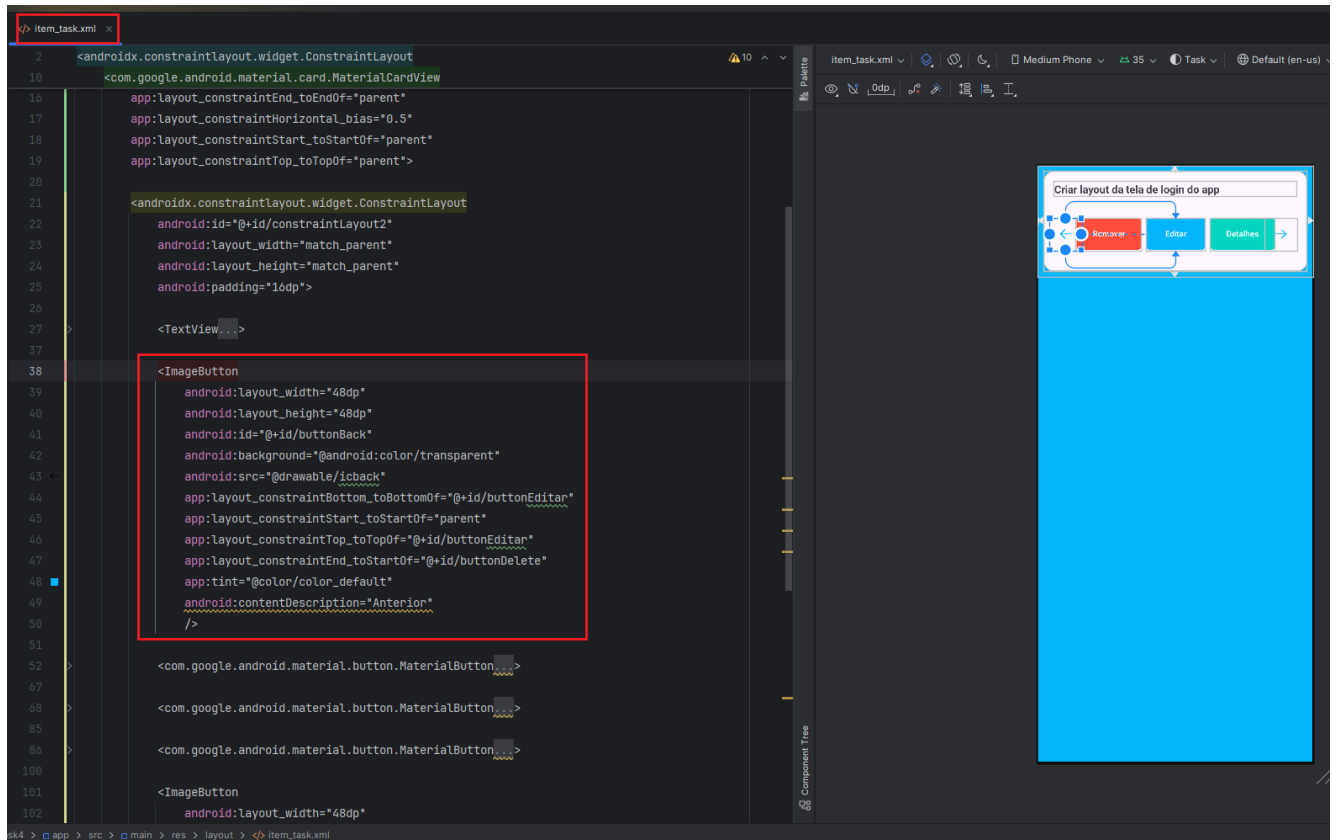


Figura 164

169. Inclua um novo ícone no projeto com o nome icForward. Esse ícone será uma seta que ficará ao lado do botão detalhes conforme Figura 165.

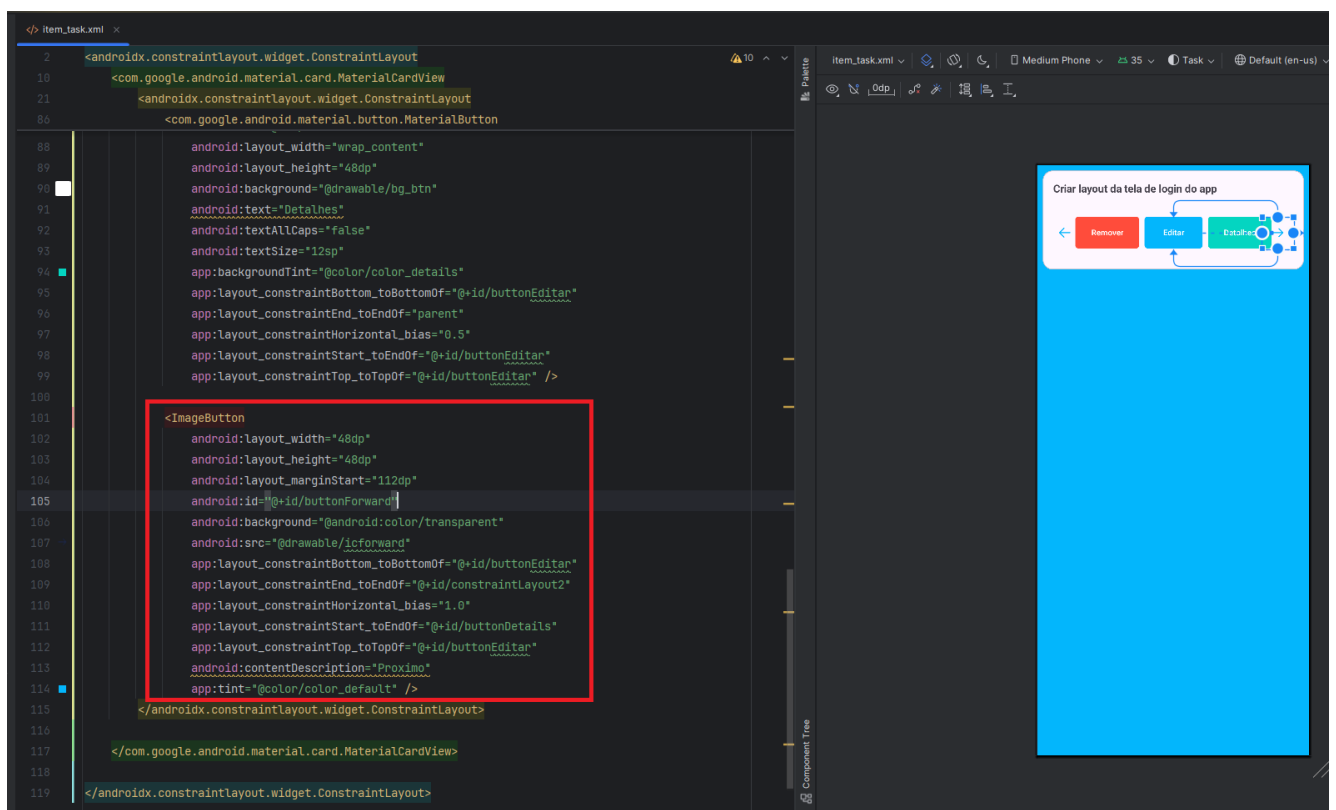


Figura 165

170. Adicione aos ImageButton das setas direita e esquerda o atributo id, utilizando respectivamente os nomes buttonForward e buttonBack

171. Finalizamos a criação do layout da lista de tarefas. Antes de incluir esse componente na tela, precisamos criar a classe que será responsável por manipular os dados da lista. Sempre que você for exibir uma listagem no aplicativo, é necessário implementar uma classe Adapter, que terá a função de gerenciar e apresentar os itens na interface. Para a lista de tarefas, crie uma classe chamada **TaskAdapter**, que ficará responsável por controlar e exibir os itens na tela. Para isso, clique com o botão direito na pasta adapter, selecione New > Kotlin Class/File e, na janela que será exibida, informe o nome TaskAdapter, conforme ilustrado na Figura 167. Em seguida, pressione Enter para criar a classe.

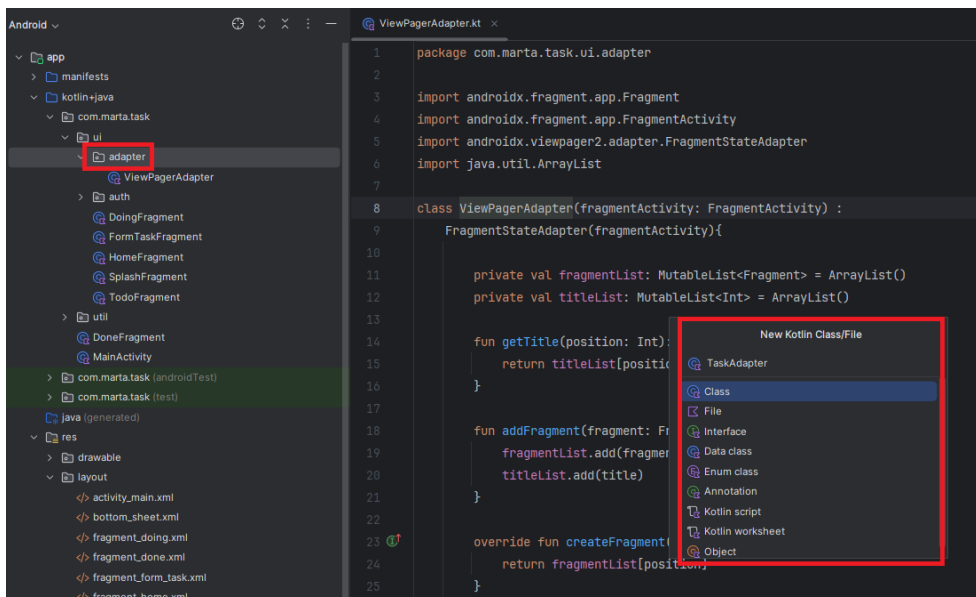


Figura 167

172. O código a ser implementado na TaskAdapter.kt está descrito na Figura 168.

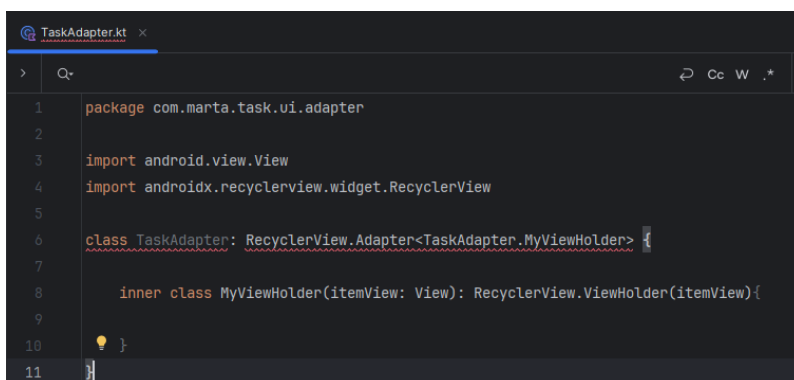


Figura 168

A classe TaskAdapter irá controlar como os dados da lista são exibidos na interface, criando e vinculando cada item da lista. A classe TaskAdapter herda da classe RecyclerView.Adapter. A classe RecyclerView.Adapter é responsável por fazer a ponte entre os dados (listas, objetos, textos, imagens) e a interface gráfica no componente RecyclerView, que exibe esses dados na tela em formato de lista, grade ou outro formato rolável. Ela controla como os dados são exibidos item por item na lista.

173. Clique sobre a classe **TaskAdapter**. Uma lâmpada vermelha deverá aparecer, conforme mostrado na Figura 169. Caso a lâmpada não seja exibida, posicione o cursor sobre o nome da classe TaskAdapter e pressione as teclas Alt + Enter no teclado. Será exibido um pop-up. No menu apresentado, selecione a opção “Implement Members”.

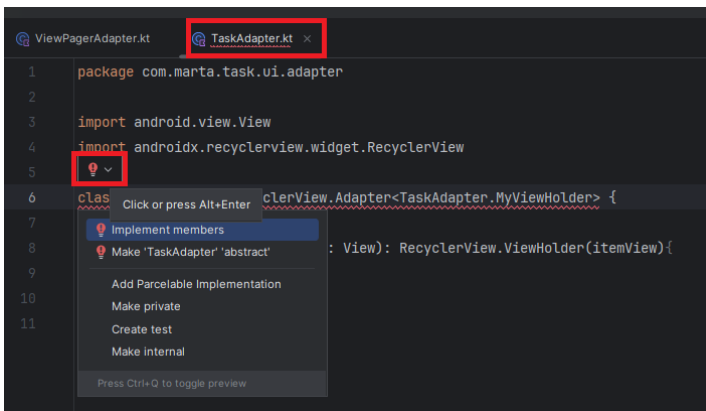


Figura 169

174. Selecione todos os métodos conforme Figura 170 e clique no botão Ok.

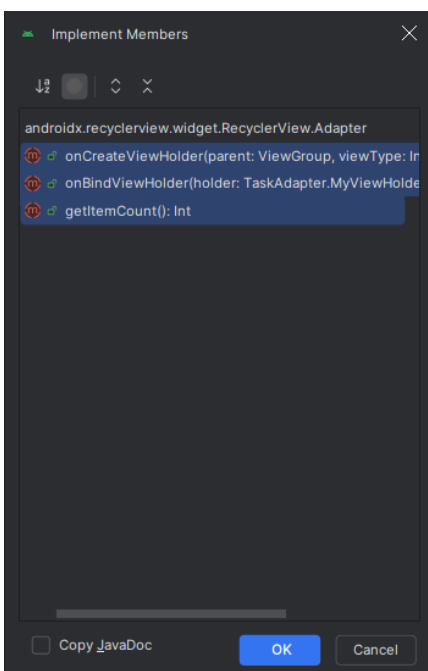
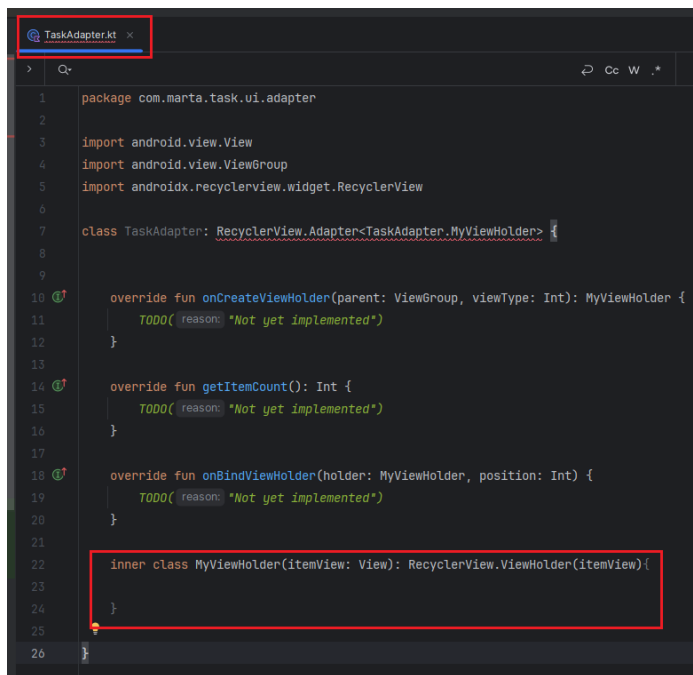


Figura 170

175. Coloque a classe MyViewHolder abaixo dos demais métodos conforme Figura 171.

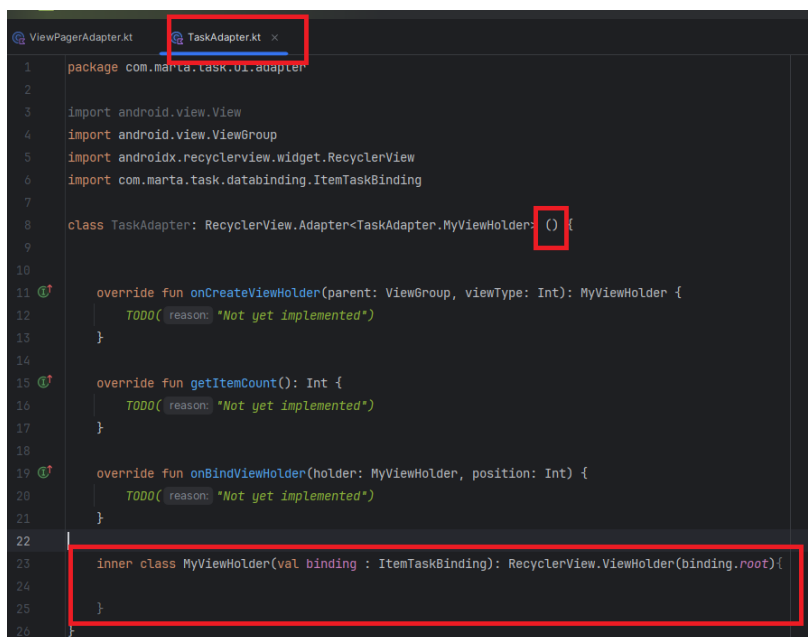


```
1 package com.marta.task.ui.adapter
2
3 import android.view.View
4 import android.view.ViewGroup
5 import androidx.recyclerview.widget.RecyclerView
6
7 class TaskAdapter: RecyclerView.Adapter<TaskAdapter.MyViewHolder> {
8
9
10     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {
11         TODO(reason: "Not yet implemented")
12     }
13
14     override fun getItemCount(): Int {
15         TODO(reason: "Not yet implemented")
16     }
17
18     override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
19         TODO(reason: "Not yet implemented")
20     }
21
22     inner class MyViewHolder(itemView: View): RecyclerView.ViewHolder(itemView){
23
24     }
25
26 }
```

Figura 171

A classe Interna MyViewHolder representa cada item individual da lista. Um itemView é a view que representa cada componente do layout (linha, card, etc.).

176. Vamos fazer algumas modificações na classe **TaskAdapter** conforme Figura 172.



```
1 package com.marta.task.ui.adapter
2
3 import android.view.View
4 import android.view.ViewGroup
5 import androidx.recyclerview.widget.RecyclerView
6 import com.marta.task.databinding.ItemTaskBinding
7
8 class TaskAdapter: RecyclerView.Adapter<TaskAdapter.MyViewHolder>() {
9
10
11     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {
12         TODO(reason: "Not yet implemented")
13     }
14
15     override fun getItemCount(): Int {
16         TODO(reason: "Not yet implemented")
17     }
18
19     override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
20         TODO(reason: "Not yet implemented")
21     }
22
23     inner class MyViewHolder(val binding : ItemTaskBinding): RecyclerView.ViewHolder(binding.root){
24
25     }
26 }
```

Figura 172

Os **parênteses ()** referem-se ao construtor da classe.

O uso do **inner**: Quando você cria uma classe dentro de outra no Kotlin, ela é, por padrão, uma classe aninhada (nested), que não tem acesso direto aos membros da classe externa. Se você quiser que essa classe interna tenha acesso aos membros da classe externa, você usa a palavra-chave inner. Em um Adapter, às vezes o ViewHolder precisa acessar informações que estão na classe Adapter, como: Lista de dados (val taskList = listOf<Task>()), Métodos auxiliares e Variáveis de contexto

A classe **ItemTaskBinding** representa o layout XML que está em `res/layout/item_task.xml`

177. Crie um pacote com o nome **data** conforme ilustrado na Figura 173.

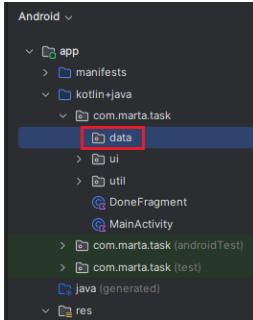


Figura 173

178. Crie um pacote com o nome **model** conforme Figura 174.

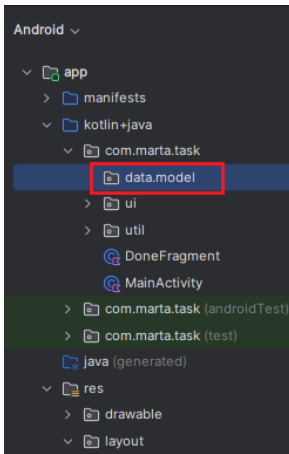


Figura 174

179. Crie a classe **Task** conforme Figura 175.

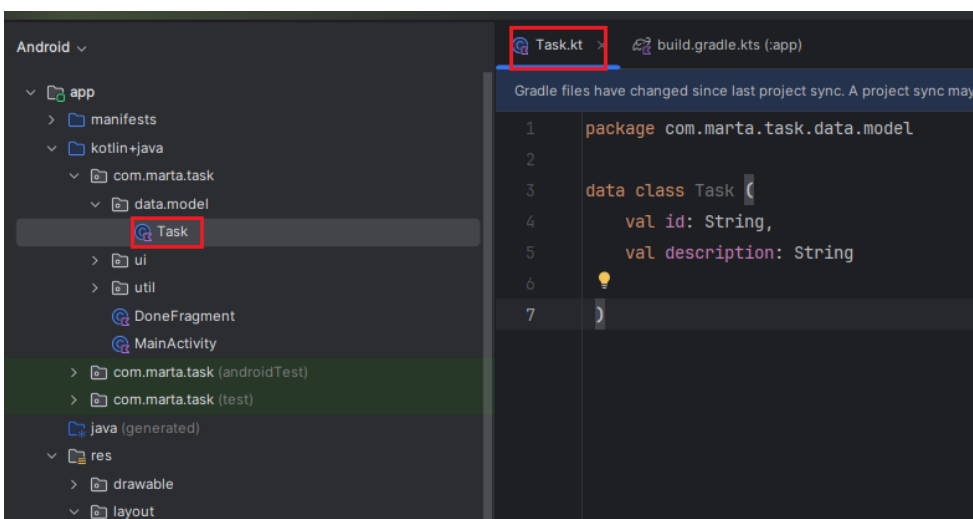


Figura 175

180. Para permitir o envio dos dados da classe **Task** entre telas e fragments, é necessário torná-la serializável. Vamos habilitar a biblioteca `kotlin-parcelize` no arquivo `build.gradle.kts` do módulo. Assim, poderemos utilizar a anotação `@Parcelize` para simplificar esse processo.

181. Abra o arquivo **build.gradle.kts** do módulo e adicione o código conforme mostra a Figura 176. Em seguida, clique no link **Sync Now** para sincronizar as dependências.

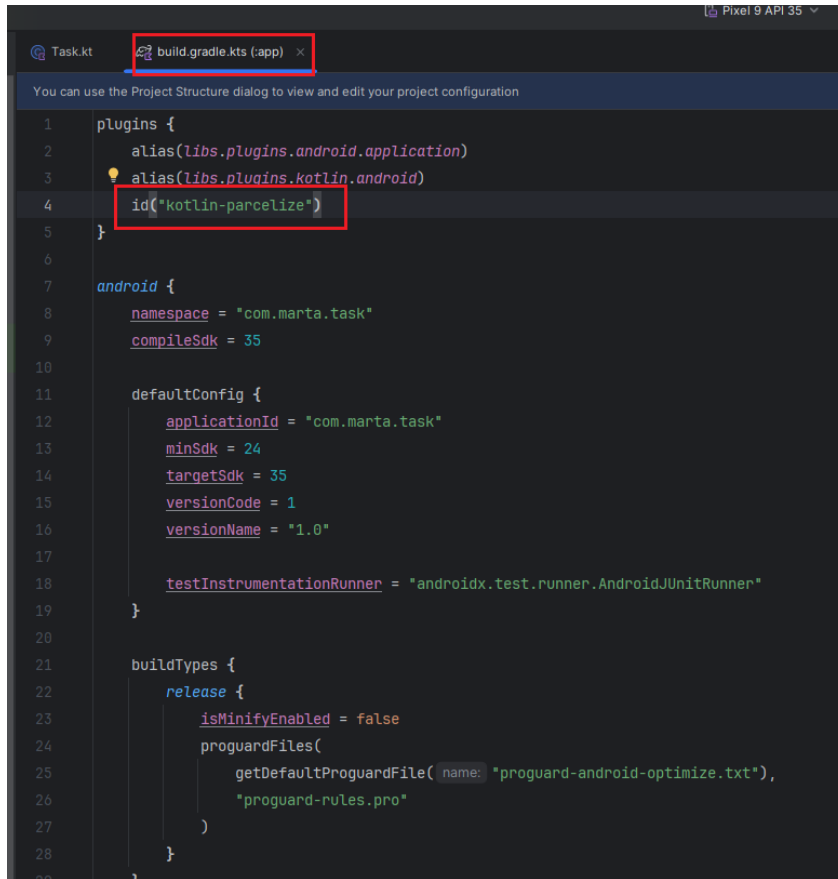


Figura 176

182. Abra o arquivo Task.kt e insira a extensão a classe Parcelable e o decorator (anotação) @parcelize conforme Figura 177.

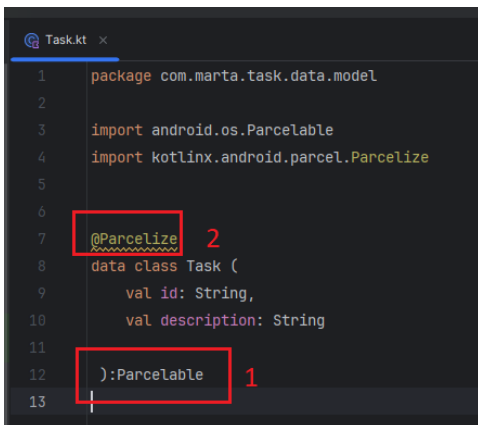


Figura 177

183. Abra a classe **TaskAdapter.kt** e implemente o método **onCreateViewHolder** e o **onBindViewHolder** conforme Figura 178.

```
TaskAdapter.kt
1 package com.marta.task.ui.adapter
2
3 import android.view.LayoutInflater
4 import android.view.ViewGroup
5 import androidx.recyclerview.widget.RecyclerView
6 import com.marta.task.data.model.Task
7 import com.marta.task.databinding.ItemTaskBinding
8
9 class TaskAdapter(
10     private val taskList: List<Task>
11 ): RecyclerView.Adapter<TaskAdapter.MyViewHolder> () {
12
13
14     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {
15         val view = ItemTaskBinding.inflate(LayoutInflater.from(parent.context), parent, attachToParent: false)
16         return MyViewHolder(view)
17     }
18
19     override fun getItemCount() = taskList.size
20
21     override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
22         val task = taskList[position]
23         holder.binding.textDescription.text = task.description
24     }
25
26     inner class MyViewHolder(val binding: ItemTaskBinding): RecyclerView.ViewHolder(binding.root) {
27     }
28 }
29
30 }
```

Figura 178

O método `onCreateViewHolder` cria a visualização (View) de cada item da lista quando ela precisa ser exibida na tela. Ele não preenche os dados no item (isso é função do `onBindViewHolder`). Ele apenas infla o layout XML que representa cada item e cria uma instância do `ViewHolder`, que guarda as referências às views desse item.

184. Depois da configuração da classe `TaskAdapter.kt`, que é classe que armazena a listagem de dados. Vamos configurar o componente `RecyclerView`. Ele é componente da tela que mostra a listagem de dados. Para isso Abra o XML `fragment_todo.xml` e inclua o código da Figura 179.

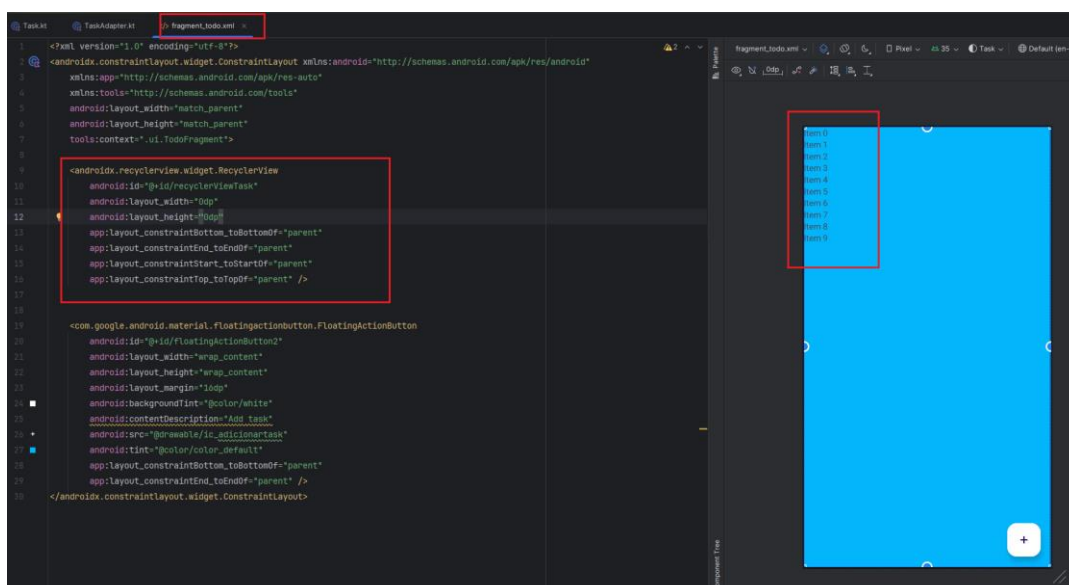


Figura 179

185. Ainda no arquivo `fragment_todo.xml`, adicione o atributo `tools:listitem` para visualizar, em tempo de desenvolvimento, como o `RecyclerView` será exibido. O código apresentado na Figura 180 ilustra esse atributo aplicado, permitindo ter uma prévia de como a interface ficará durante a execução do aplicativo.

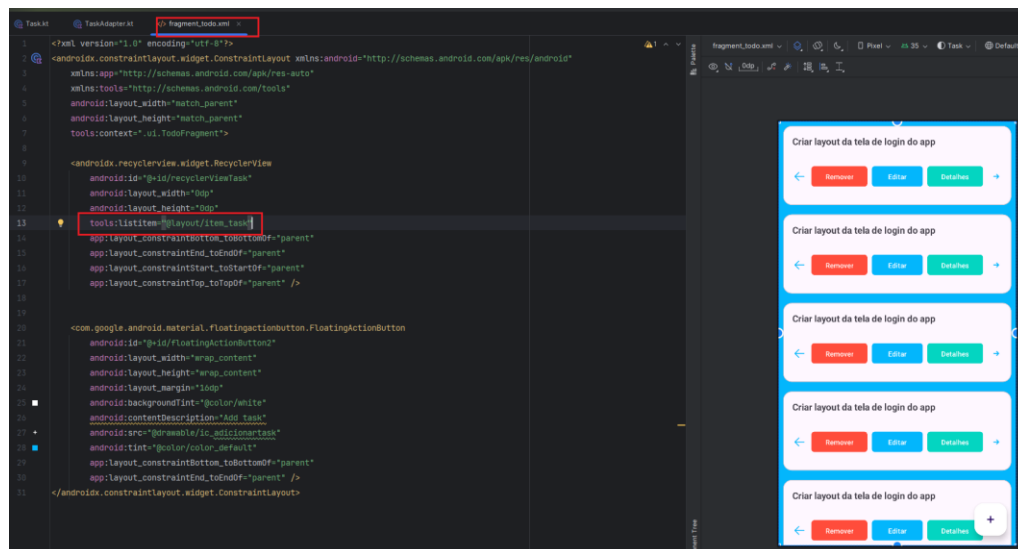
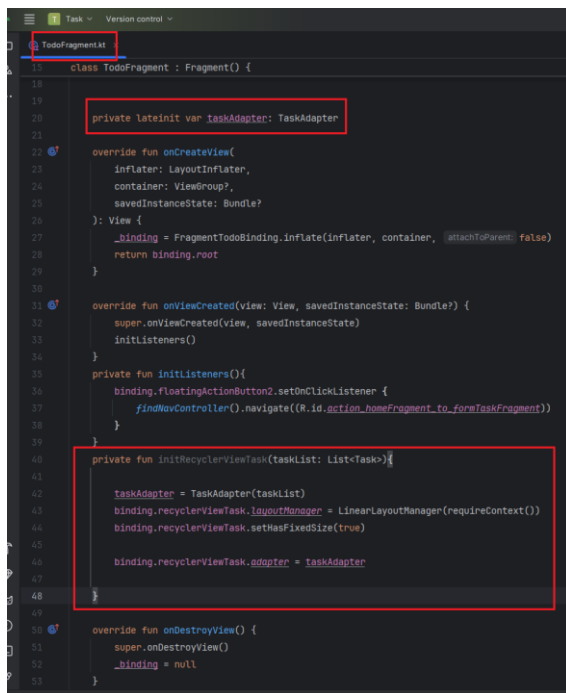


Figura 180

O atributo `listItem` possui o valor da tela `item_task.xml`.

186. Faça o mesmo procedimento das Figuras 179 e 180 nos arquivos `fragments_doing.xml` e no `fragment_done.xml`. Não esquecendo que o container desses fragments sempre será o `ConstraintLayout`.

187. Vamos trabalhar no código que irá carregar o componente `RecyclerView`. Abra o código `TodoFragment.kt` conforme Figura 181.



189. Faça o mesmo procedimento da Figura 182 nos arquivos **DoneFragment.kt** e no **DoingFragment.kt**.

190. Agora execute o App conforme Figura 183.



Figura 183

191. Dentro do pacote **data.model**. Crie a classe **Status**, conforme Figura 184.

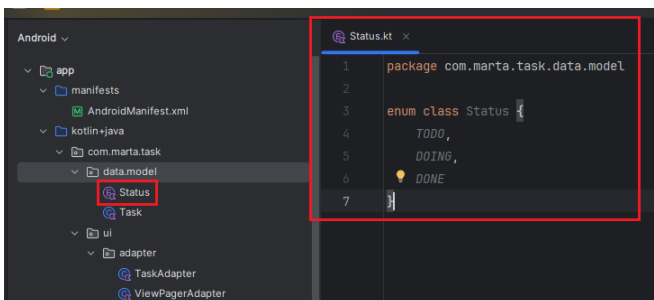


Figura 184

Uma enum class (ou simplesmente enumeração) é um tipo especial de classe que representa um conjunto fixo e limitado de valores constantes. Ela é usada quando você quer expressar opções pré-definidas, evitando o uso de números aleatórios ou strings soltas no código.

192. Abra a classe **Task.kt** e crie o atributo status conforme ilustrado na Figura 185.

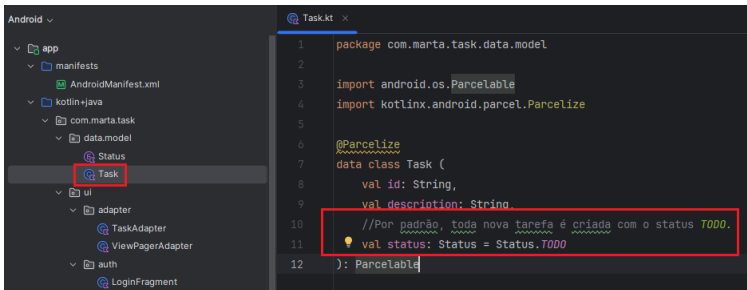


Figura 185

193. Abra o arquivo **TodoFragment.kt** e inclua o status na entrada do construtor das classes Task conforme Figura 186.

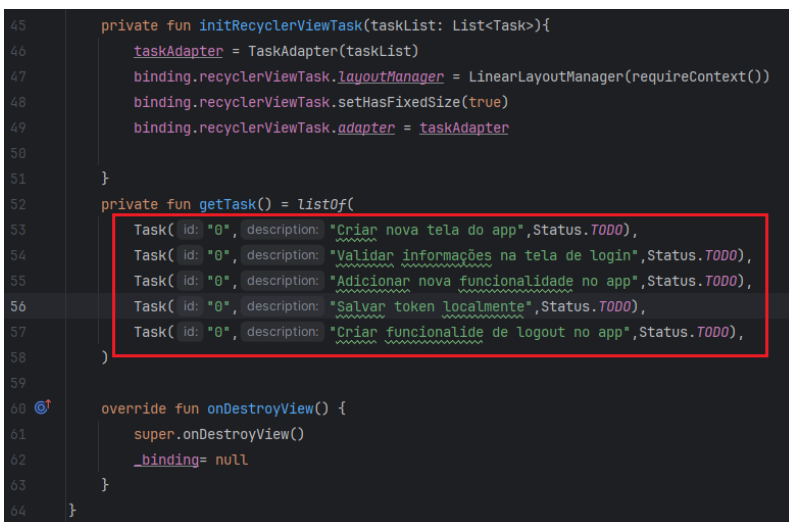


Figura 186

194. Faça o mesmo procedimento da Figura 186 nos arquivos **DoneFragment.kt** e no **DoingFragment.kt**. Altere o método getTask nos arquivos DoneFragment.kt e DoingFragment.kt: adicionando novas descrições para as tarefas, ids diferentes e alterando o status.

195. Execute o aplicativo e verifique se todas as abas estão funcionando conforme ilustrado nas Figuras 187, 188 e 189. Na Figura 187, a seta localizada à esquerda está destacada em vermelho. Essa seta deve ser removida, pois não há necessidade de navegação para o lado esquerdo. Na Figura 188, as setas à esquerda e à direita estão destacadas em vermelho. A seta da esquerda deve ter sua cor alterada para laranja, e a seta da direita para verde. Na Figura 189, a seta localizada à direita está destacada em vermelho. Essa seta também deve ser removida, pois não há necessidade de navegação para o lado direito.

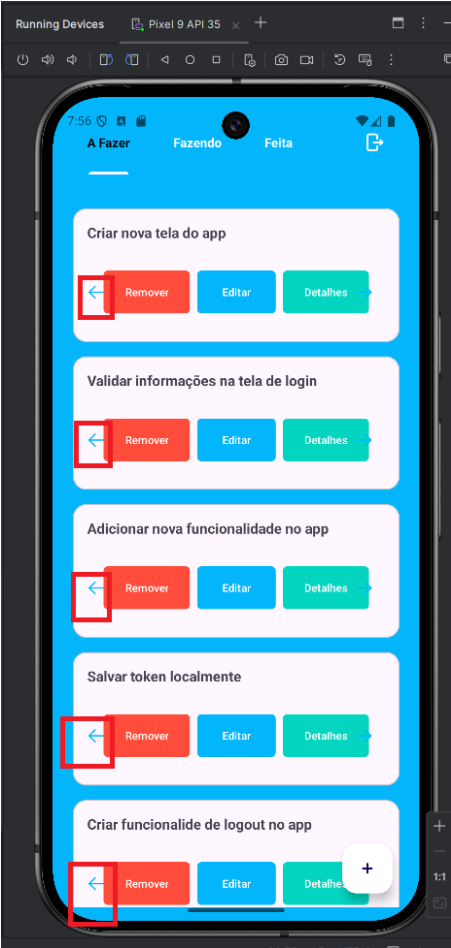


Figura 187

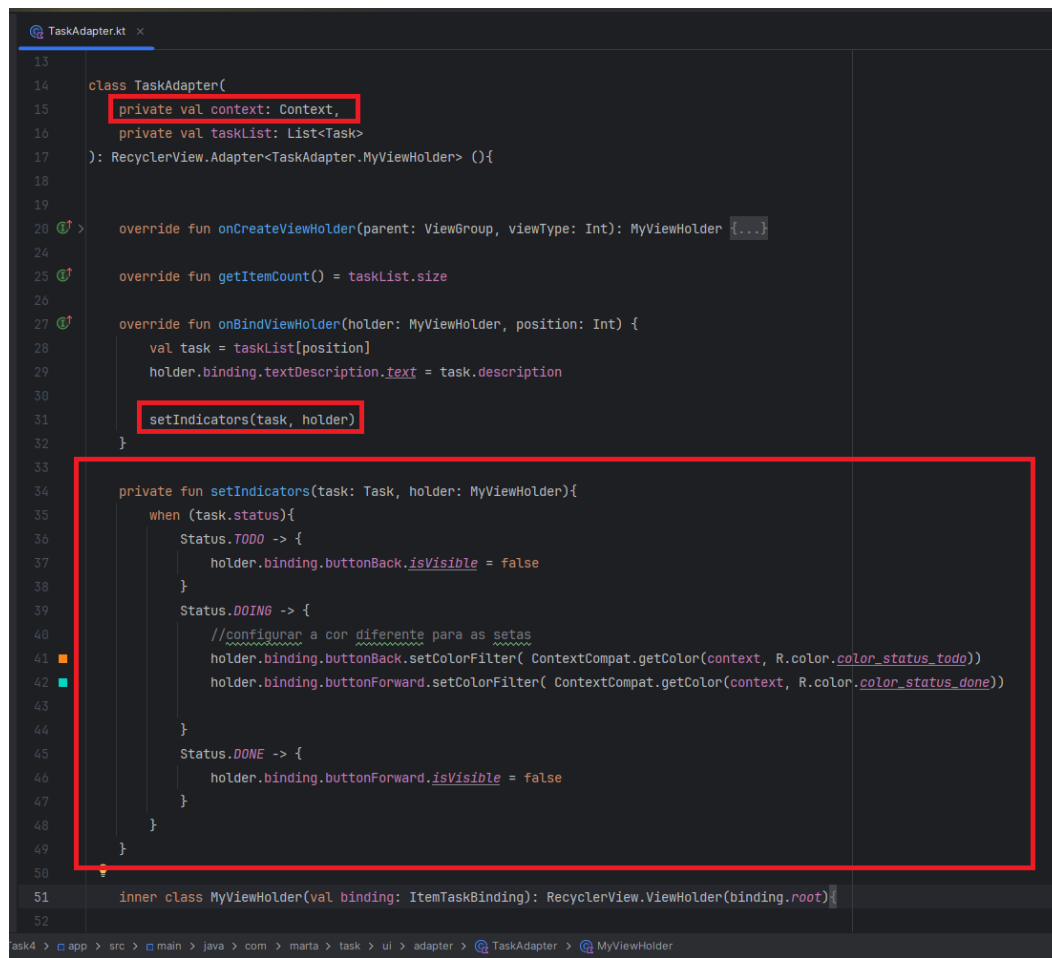


Figura 188



Figura 189

196. Para remover as setas nas respectivas telas, será necessário realizar alterações no arquivo `TaskAdapter.kt`. Abra o arquivo **`TaskAdapter.kt`** e crie o método `setIndicator`, conforme ilustrado na Figura 190. O método `setIndicators` é chamado dentro do evento `onBindViewHolder` porque é nesse ponto que cada item da lista (`RecyclerView`) está sendo ligado aos seus dados específicos.



```
13
14 class TaskAdapter(
15     private val context: Context,
16     private val taskList: List<Task>
17 ): RecyclerView.Adapter<TaskAdapter.MyViewHolder> () {
18
19
20 > override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {...}
24
25 > override fun getItemCount() = taskList.size
26
27 > override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
28     val task = taskList[position]
29     holder.binding.textDescription.text = task.description
30
31     setIndicators(task, holder)
32 }
33
34 private fun setIndicators(task: Task, holder: MyViewHolder){
35     when (task.status){
36         Status.TODO -> {
37             holder.binding.buttonBack.isVisible = false
38         }
39         Status.DOING -> {
40             //configurar a cor diferente para as setas
41             holder.binding.buttonBack.setColorFilter( ContextCompat.getColor(context, R.color.color_status_todo))
42             holder.binding.buttonForward.setColorFilter( ContextCompat.getColor(context, R.color.color_status_done))
43         }
44         Status.DONE -> {
45             holder.binding.buttonForward.isVisible = false
46         }
47     }
48 }
49
50
51 inner class MyViewHolder(val binding: ItemTaskBinding): RecyclerView.ViewHolder(binding.root){
52 }
```

Figura 190

197. Como incluímos o parâmetro context no construtor da classe TaskAdapter, será necessário ajustar todas as chamadas a essa classe. Para isso, abra os arquivos **DoingFragment.kt**, **DoneFragment.kt** e **TodoFragment.kt** e realize as alterações na instanciação da classe Taskadapter, conforme ilustrado na Figura 191.

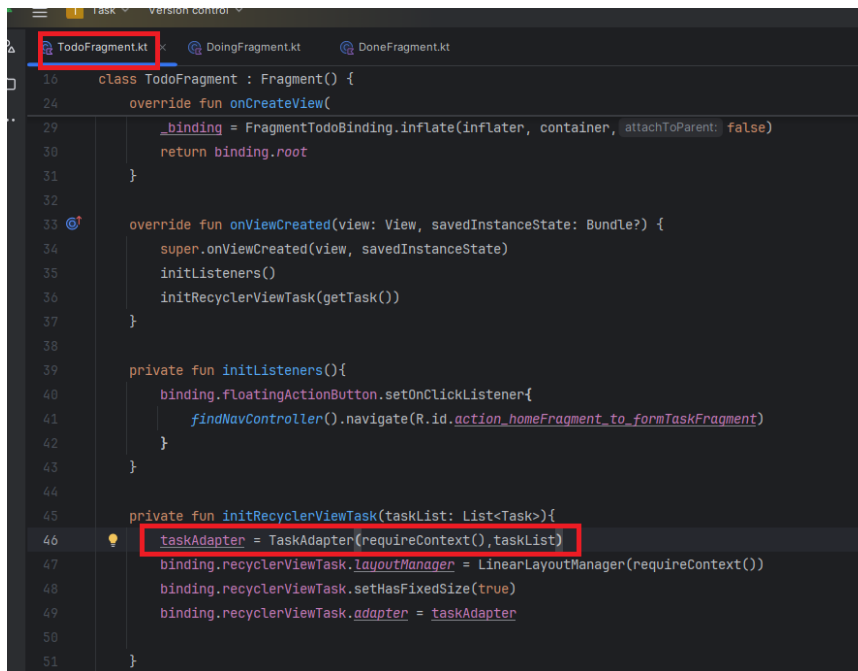


Figura 191

198. Para alterar a cor das setas da aba fazendo, será necessário incluir novas cores no arquivo **colors.xml**. Abra o arquivo **colors.xml** e inclua as cores conforme Figura 192.

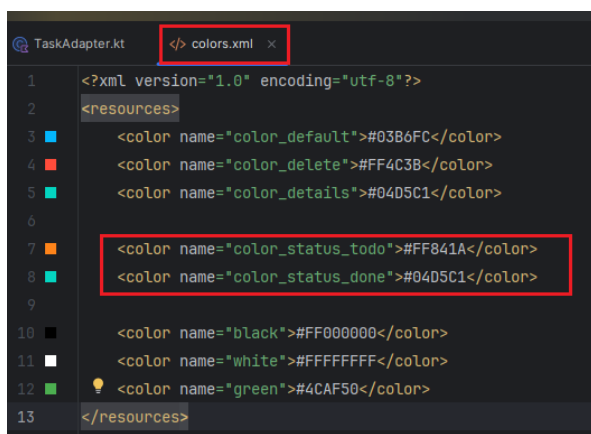


Figura 192

199. Abra novamente o arquivo **TaskAdapter.kt** e complemente o método **setIndicator**, adicionando a funcionalidade que permite alterar as cores dos botões programaticamente, conforme ilustrado na Figura 193. Observe que a alteração das cores será aplicada apenas à aba cujas tarefas estiverem com o status Fazendo.

```
private fun setIndicators(task: Task, holder: MyViewHolder){
    when (task.status){
        Status.TODO -> {
            holder.binding.buttonBack.isVisible = false
        }
        Status.DOING -> {
            //configurar a cor diferente para as setas
            holder.binding.buttonBack.setColorFilter( ContextCompat.getColor(context, R.color.color_status_todo))
            holder.binding.buttonForward.setColorFilter( ContextCompat.getColor(context, R.color.color_status_done))
        }
        Status.DONE -> {
            holder.binding.buttonForward.isVisible = false
        }
    }
}
```

Figura 193

200. Execute o App e verifique se as setas serão apresentadas conforme Figura 194.

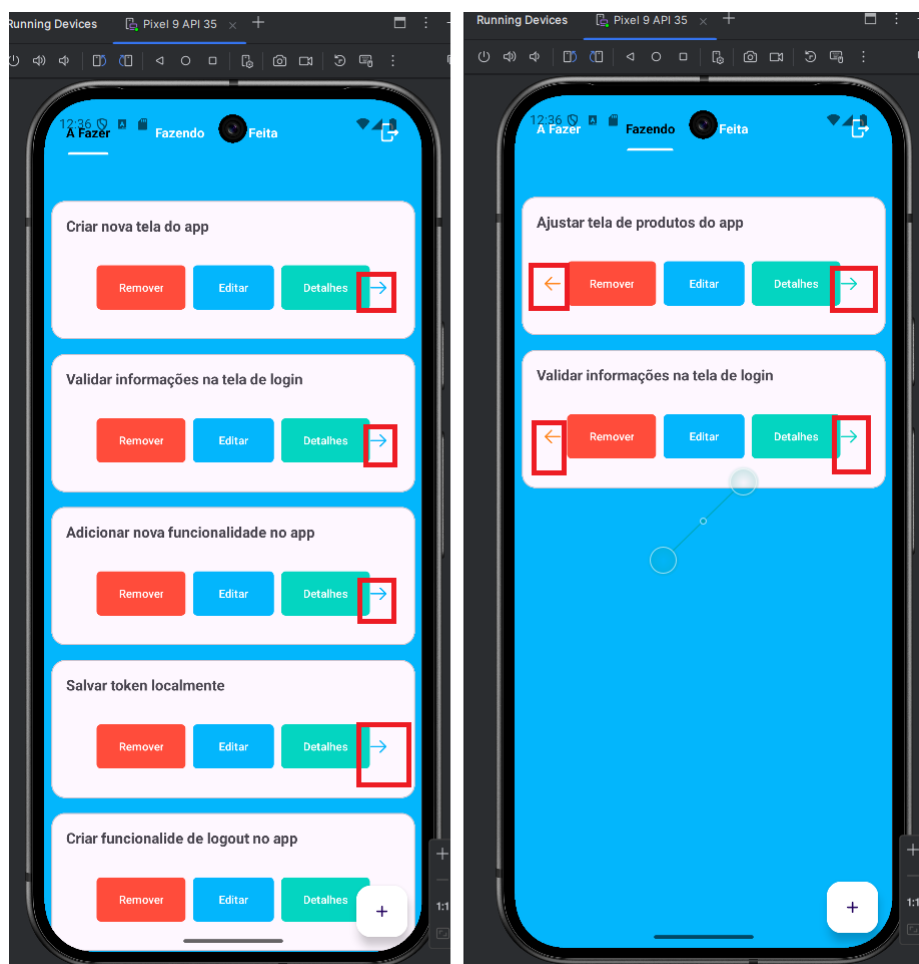
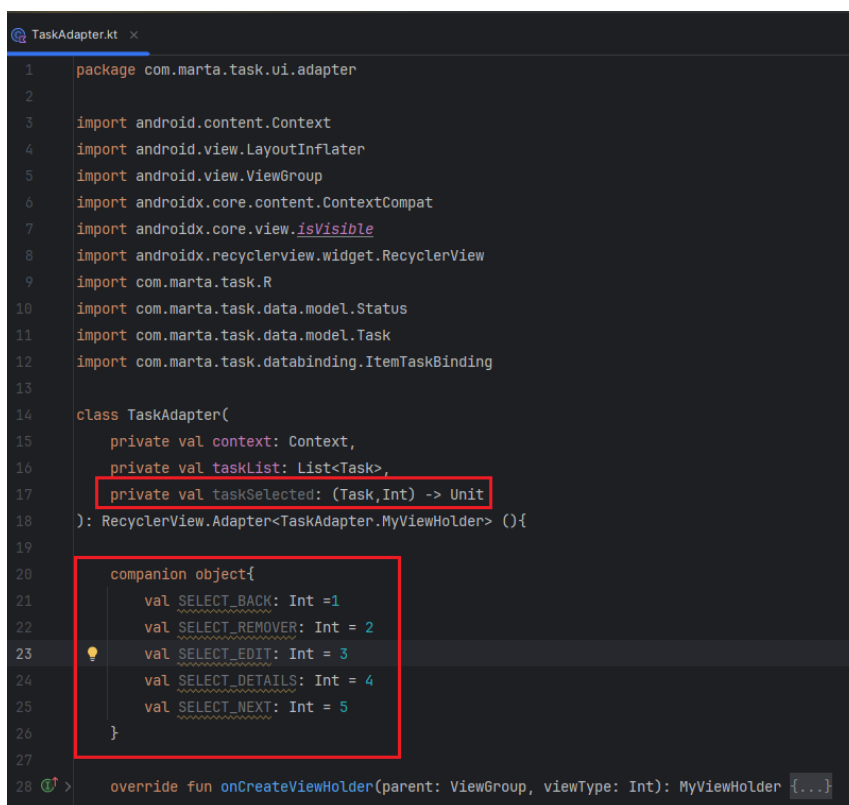


Figura 194

201. O próximo passo é capturar os eventos de clique nos componentes na tela de tarefas, que incluem: a seta à direita, a seta à esquerda, o botão de remover, o botão de editar e o botão para visualizar os detalhes da tarefa. Para isso, abra o arquivo **TaskAdapter.kt** e adicione a variável **taskSelector** no construtor da classe. Em seguida, crie um companion object, conforme ilustrado na Figura 195.



```
1 package com.marta.task.ui.adapter
2
3 import android.content.Context
4 import android.view.LayoutInflater
5 import android.view.ViewGroup
6 import androidx.core.content.ContextCompat
7 import androidx.core.view.isVisible
8 import androidx.recyclerview.widget.RecyclerView
9 import com.marta.task.R
10 import com.marta.task.data.model.Status
11 import com.marta.task.data.model.Task
12 import com.marta.task.databinding.ItemTaskBinding
13
14 class TaskAdapter(
15     private val context: Context,
16     private val taskList: List<Task>,
17     private val taskSelected: (Task, Int) -> Unit
18 ): RecyclerView.Adapter<TaskAdapter.MyViewHolder> () {
19
20     companion object {
21         val SELECT_BACK: Int = 1
22         val SELECT_REMOVER: Int = 2
23         val SELECT_EDIT: Int = 3
24         val SELECT_DETAILS: Int = 4
25         val SELECT_NEXT: Int = 5
26     }
27
28     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder { ... }
```

Figura 195

Vamos a explicações desses trechos de código. A variável `taskSelected: (Task, Int) -> Unit` → é uma função lambda que será chamada quando o usuário interagir com um item (ex.: clicar em um botão). Lembrando que uma função lambda é uma forma curta e direta de declarar uma função sem precisar dar um nome formal para ela. Em Kotlin (e em várias linguagens modernas), lambda é basicamente um bloco de código que pode ser passado como argumento para outra função.

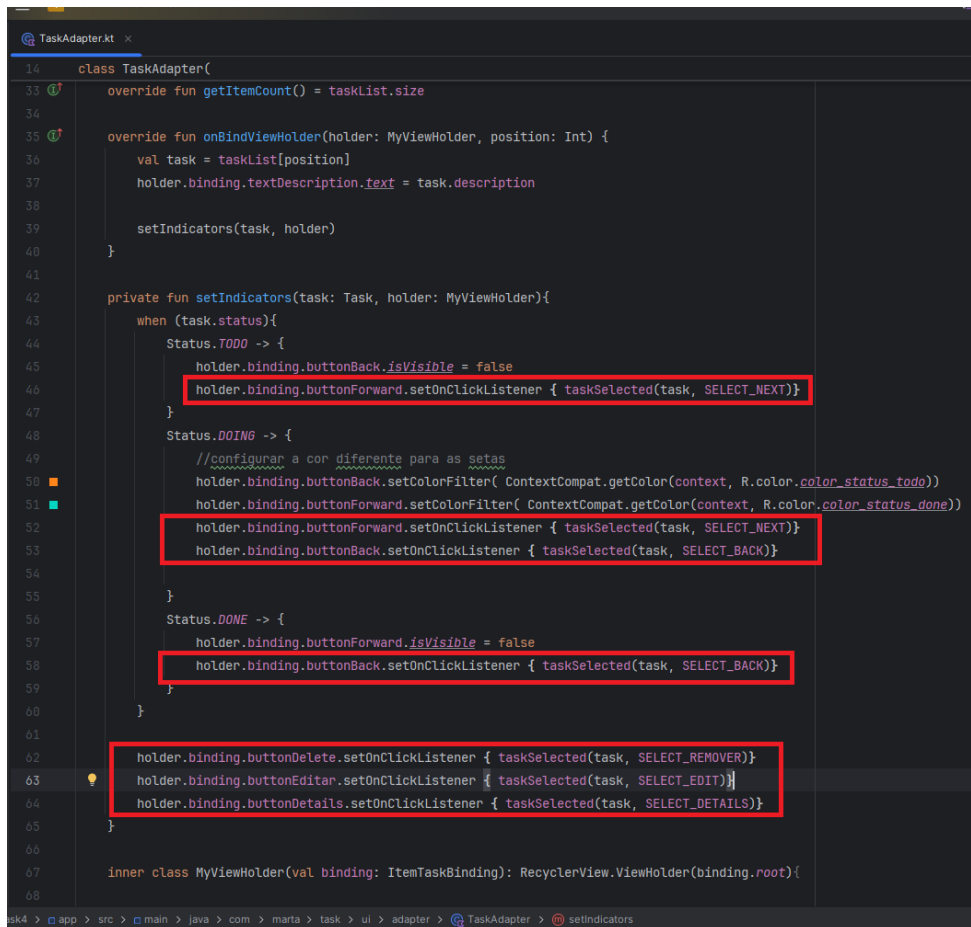
A função lambda executa um valor chamado `Unit`. Em Kotlin, `Unit` é o tipo de retorno que indica que a função não devolve nenhum valor útil – é equivalente ao `void` do Java. Em Python, usa-se simplesmente `return` (o retorno implícito é `None`).

A função Lambda recebe como parâmetros: `Task` → a tarefa selecionada e `Int` → um código que indica qual ação foi escolhida (editar, remover, etc.).

O `companion object` é como um OBJETO estático em Java. O `companion object` em Kotlin não precisa de instância para ser acessado. Ele é associado à classe, assim como os membros `static` em Java. Aqui, ele define constantes que representam as ações possíveis no item: `SELECT_BACK` → mover a tarefa para uma aba anterior ; `SELECT_REMOVER` → remover a tarefa; `SELECT_EDIT` → editar a tarefa; `SELECT_DETAILS` → abrir os detalhes da tarefa; `SELECT_NEXT` → mover para a próxima aba. Esses valores são usados como códigos de ação ao chamar o `taskSelected`.

Resumindo o funcionamento: A tela lista de tarefas. Quando o usuário interage com um item (ex.: clica em “Editar”), o adapter chama a função `taskSelected` com a tarefa e o código da ação. Quem criou o Adapter (Activity ou Fragment) é quem decide o que fazer com essa ação – essa lógica será implementada nos próximos passos.

202. O código do evento click de cada botão está codificado na Figura 196. Observe que o clique das setas são particulares a status de cada tarefas. Já o botões remover, editar e detalhes pertencem a qualquer status de tarefa.



```
14 class TaskAdapter(  
33 override fun getItemCount() = taskList.size  
34  
35 override fun onBindViewHolder(holder: MyViewHolder, position: Int) {  
36     val task = taskList[position]  
37     holder.binding.textDescription.text = task.description  
38  
39     setIndicators(task, holder)  
40 }  
41  
42 private fun setIndicators(task: Task, holder: MyViewHolder){  
43     when (task.status){  
44         Status.TODO -> {  
45             holder.binding.buttonBack.isVisible = false  
46             holder.binding.buttonForward.setOnClickListener { taskSelected(task, SELECT_NEXT)}  
47         }  
48         Status.DOING -> {  
49             //configurar a cor diferente para as setas  
50             holder.binding.buttonBack.setColorFilter( ContextCompat.getColor(context, R.color.color_status_todo))  
51             holder.binding.buttonForward.setColorFilter( ContextCompat.getColor(context, R.color.color_status_done))  
52             holder.binding.buttonForward.setOnClickListener { taskSelected(task, SELECT_NEXT)}  
53             holder.binding.buttonBack.setOnClickListener { taskSelected(task, SELECT_BACK)}  
54         }  
55         Status.DONE -> {  
56             holder.binding.buttonForward.isVisible = false  
57             holder.binding.buttonBack.setOnClickListener { taskSelected(task, SELECT_BACK)}  
58         }  
59     }  
60 }  
61  
62 holder.binding.buttonDelete.setOnClickListener { taskSelected(task, SELECT_REMOVER)}  
63 holder.binding.buttonEditar.setOnClickListener { taskSelected(task, SELECT_EDIT)}  
64 holder.binding.buttonDetails.setOnClickListener { taskSelected(task, SELECT_DETAILS)}  
65 }  
66  
67 inner class MyViewHolder(val binding: ItemTaskBinding): RecyclerView.ViewHolder(binding.root){  
68
```

Figura 196

Dentro do `setOnClickListener`, você está definindo qual ação deve ser executada quando o botão for clicado. Nesse caso, a ação é chamar a lambda `taskSelected`, passando como parâmetros a tarefa e o código da ação. Resumindo o fluxo: O `setOnClickListener` detecta o clique no botão. Ele executa a lambda `{ taskSelected(task, SELECT_NEXT) }`. Essa lambda chama a função `taskSelected`, que foi recebida no construtor do Adapter. Quem criou o Adapter (Fragment ou Activity) recebe a tarefa e o código da ação e decide o que fazer a seguir. Em outras palavras, `taskSelected` é uma função de callback (uma lambda) passada pelo Fragment ou Activity ao criar o Adapter.

Portanto, o Adapter apenas sinaliza o evento, enquanto o Fragment ou Activity é quem define a lógica do que acontecerá depois.

203. Como alteramos o construtor da classe TaskAdpter.kt , será necessário ajustar todas as outros arquivos que instanciam essa classe. Abra os arquivos **DoingFrament.kt**, **DoneFragment.kt** e **TodoFragment.kt**. Vamos começar os ajustes na tela DoingFragment.kt, e depois replicar para os outros arquivos.

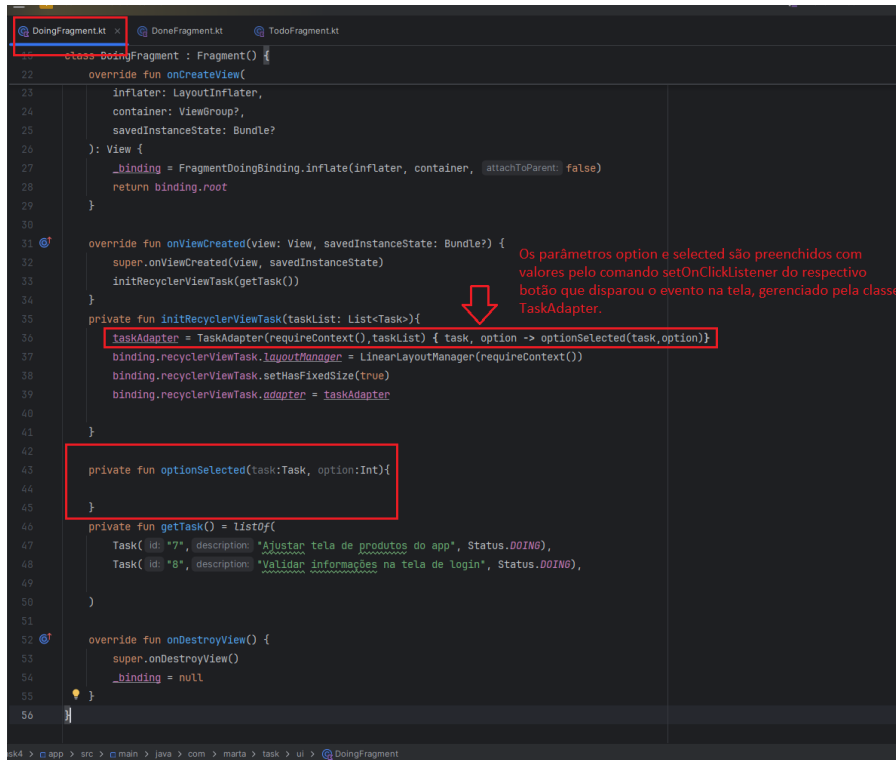


Figura 197

204. Vamos implementar o método optionSelect dos arquivos **DoingFrament.kt**, **DoneFragment.kt** e **TodoFragment.kt**, conforme Figura 197. Abra cada arquivo e coloque o código ajustando de acordo com os eventos dos botões disponíveis.


```
class TodoFragment : Fragment() {
    private fun initListeners(){
    }

    private fun initRecyclerViewTask(taskList: List<Task>){
        taskAdapter = TaskAdapter(requireContext(), taskList) { task, option -> optionSelected(task, option) }
        binding.recyclerViewTask.layoutManager = LinearLayoutManager(requireContext())
        binding.recyclerViewTask.setHasFixedSize(true)
        binding.recyclerViewTask.adapter = taskAdapter
    }

    private fun optionSelected(task: Task, option: Int){
        when (option){
            TaskAdapter.SELECT_REMOVER -> {
                Toast.makeText(requireContext(), "Removendo ${task.description}", Toast.LENGTH_SHORT).show()
            }
            TaskAdapter.SELECT_EDIT -> {
                Toast.makeText(requireContext(), "Editando ${task.description}", Toast.LENGTH_SHORT).show()
            }
            TaskAdapter.SELECT_DETAILS -> {
                Toast.makeText(requireContext(), "Detalhes ${task.description}", Toast.LENGTH_SHORT).show()
            }
            TaskAdapter.SELECT_NEXT -> {
                Toast.makeText(requireContext(), "Próximo", Toast.LENGTH_SHORT).show()
            }
        }
    }

    private fun getTask() = listOf(
        Task(id = "0", description = "Criar nova tela do app", Status.TODO),
        Task(id = "1", description = "Validar informações na tela de login", Status.TODO),
        Task(id = "2", description = "Adicionar nova funcionalidade no app", Status.TODO),
        Task(id = "3", description = "Salvar token localmente", Status.TODO),
        Task(id = "4", description = "Criar funcionalidade de logout no app", Status.TODO),
    )
}
```

Figura 197

205. Teste o aplicativo clicando nos botões das tarefas, conforme ilustrado na Figura 198. Em exercícios posteriores, esses botões serão integrados a ações que se conectarão ao banco de dados.

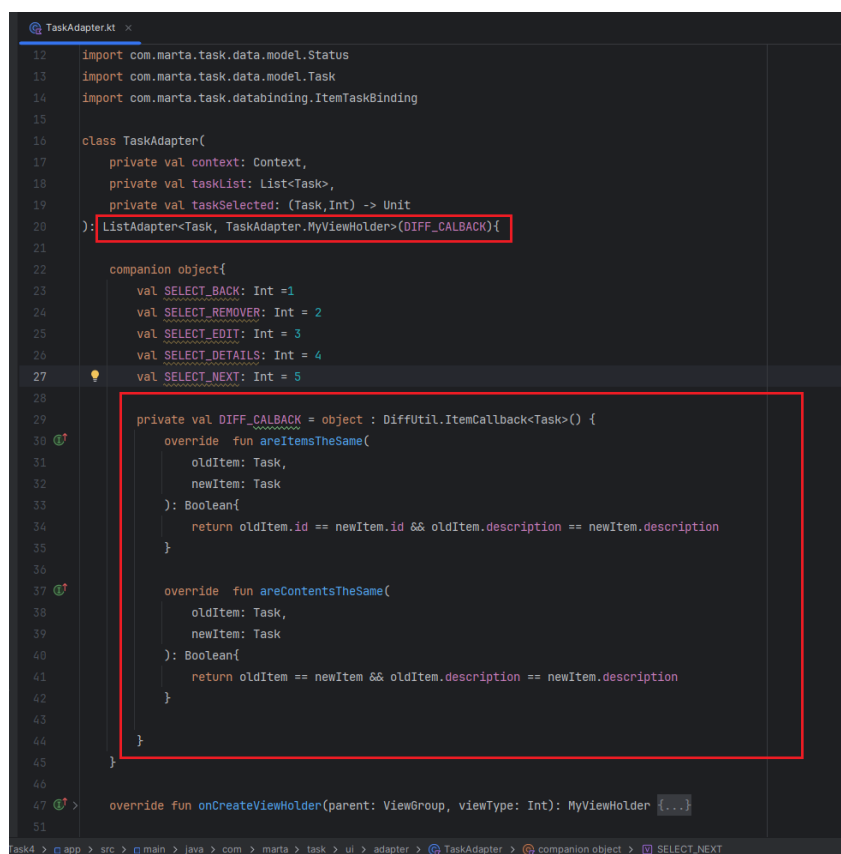


Figura 198

PARTE 6 – DiffUtil – Melhore o desempenho do RecyclerView

O ListAdapter e o DiffUtil trabalham juntos para melhorar o uso do RecyclerView. O ListAdapter é uma versão mais avançada do RecyclerView.Adapter, criado para facilitar a atualização eficiente da lista. O DiffUtil é uma biblioteca usada para comparar duas listas (antiga e nova) e descobrir: Quais itens foram adicionados, Quais itens foram removidos e Quais itens mudaram de posição ou conteúdo. Ele evita atualizar a lista inteira, o que melhora performance e permite **animações** automáticas.

206. Para implementar o DiffUtil, abra o arquivo **TaskAdapter.kt** e escreva o código conforme Figura 199.



```
12 import com.marta.task.data.model.Status
13 import com.marta.task.data.model.Task
14 import com.marta.task.databinding.ItemTaskBinding
15
16 class TaskAdapter(
17     private val context: Context,
18     private val taskList: List<Task>,
19     private val taskSelected: (Task, Int) -> Unit
20 ): ListAdapter<Task, TaskAdapter.MyViewHolder>(DIFF_CALLBACK){
21
22     companion object{
23         val SELECT_BACK: Int = 1
24         val SELECT_REMOVER: Int = 2
25         val SELECT_EDIT: Int = 3
26         val SELECT_DETAILS: Int = 4
27         val SELECT_NEXT: Int = 5
28
29         private val DIFF_CALLBACK = object : DiffUtil.ItemCallback<Task>() {
30             override fun areItemsTheSame(
31                 oldItem: Task,
32                 newItem: Task
33             ): Boolean{
34                 return oldItem.id == newItem.id && oldItem.description == newItem.description
35             }
36
37             override fun areContentsTheSame(
38                 oldItem: Task,
39                 newItem: Task
40             ): Boolean{
41                 return oldItem == newItem && oldItem.description == newItem.description
42             }
43         }
44     }
45
46     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {...
```

Figura 199

Vamos ao entendimento do código da Figura 199.

A classe TaskAdapter herda da classe ListAdapter, que é uma versão mais moderna do RecyclerView.Adapter e suporta atualização otimizada da lista usando DiffUtil. A ListAdapter já tem suporte nativo para calcular diferenças entre listas e atualizar apenas os itens que mudaram (mais eficiente).

A classe ListAdapter recebe por meio do construtor: context: para acessar recursos (cores, strings etc.); taskList: lista de tarefas a ser exibida e a taskSelected: uma função lambda de

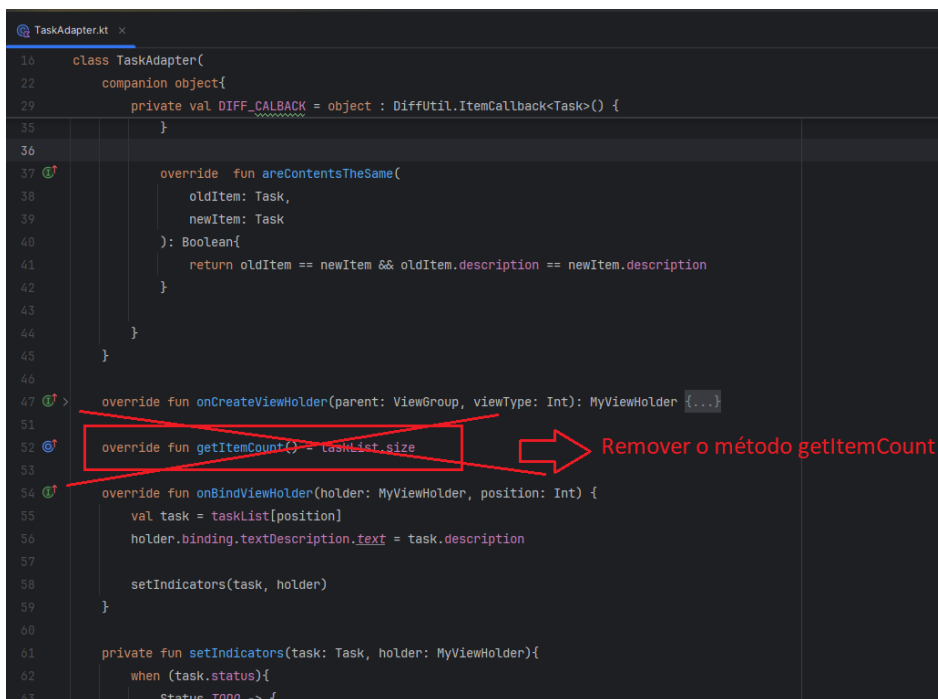
callback que é chamada quando o usuário interage com uma tarefa (ex.: clicar em editar ou remover).

Neste trecho de código `<Task, TaskAdapter.MyViewHolder>` são parâmetros genéricos usados pelo `ListAdapter`. Os parâmetros genéricos são os tipos genéricos esperados como entrada para o `ListAdapter`. Os dois tipos genéricos são: `T` → O tipo do dado que será exibido na lista e o `VH` → O tipo do `ViewHolder` que vai renderizar esses dados.

Quando você atualiza uma lista no `ListAdapter`, ele não recarrega tudo. Ele usa o `DiffUtil` para comparar o item antigo com o novo e atualizar só o que mudou → mais rápido e eficiente. Por isso os métodos `areItemsTheSame` e `areContentsTheSame` são implementados. O método `areItemsTheSame` verifica se é o mesmo item na lista. Normalmente compara apenas o id. Neste exemplo compara id e a description. O método `areContentsTheSame` compara todo o objeto (`oldItem == newItem`) e também a descrição.

Os métodos `areItemsTheSame` e `areContentsTheSame` estão dentro do companion object porque o `ListAdapter` precisa receber um objeto estático de comparação (`DiffUtil.ItemCallback`) quando é criado.

207. O próximo passo é remover o método `getItemCount` da classe `TaskAdapter.kt`, conforme ilustrado na Figura 200. No `RecyclerView.Adapter`, era obrigatório implementar o método `getItemCount` para informar o tamanho da lista. Já o `ListAdapter` gerencia automaticamente o tamanho da lista com base no `submitList`, portanto não é mais necessário implementar `getItemCount` manualmente.



```
10 class TaskAdapter(  
22     companion object {  
29         private val DIFF_CALLBACK = object : DiffUtil.ItemCallback<Task>() {  
35             }  
36  
37         override fun areContentsTheSame(  
38             oldItem: Task,  
39             newItem: Task  
40         ): Boolean {  
41             return oldItem == newItem && oldItem.description == newItem.description  
42         }  
43     }  
44 }  
45  
46  
47 > override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder { ... }  
51  
52 override fun getItemCount() = taskList.size  
53  
54 > override fun onBindViewHolder(holder: MyViewHolder, position: Int) {  
55     val task = taskList[position]  
56     holder.binding.textDescription.text = task.description  
57  
58     setIndicators(task, holder)  
59 }  
60  
61 private fun setIndicators(task: Task, holder: MyViewHolder) {  
62     when (task.status) {  
63         Status.TODO -> {
```

Figura 200

208. Ainda no arquivo **TaskAdapter.kt**, altere a linha destacada no método `onBindViewHolder` conforme Figura 201.

```
override fun onBindViewHolder(holder: MyViewHolder, position: Int) {  
    val task = getItem(position)  
    holder.binding.textDescription.text = task.description  
  
    setIndicators(task, holder)  
}  
  
> private fun setIndicators(task: Task, holder: MyViewHolder){...}
```

Figura 201

209. Os métodos `onBindViewHolder` e `getItemCount` não usam mais a variável lista de tarefas, portanto **remova** o parâmetro de entrada **taskList** do construtor da classe `TaskAdapter` conforme Figura 202.

```
16 class TaskAdapter(  
17     private val context: Context,  
18     private val taskList: List<Task>,  
19     private val taskSelected: (Task, Int) -> Unit  
20 ): ListAdapter<Task, TaskAdapter.MyViewHolder>(DIFF_CALLBACK){  
21  
22     companion object{  
23         val SELECT_BACK: Int = 1  
24         val SELECT_REMOVER: Int = 2  
25         val SELECT_EDIT: Int = 3  
26         val SELECT_DETAILS: Int = 4  
27         val SELECT_NEXT: Int = 5  
28  
29     private val DIFF_CALLBACK = object : DiffUtil.ItemCallback<Task>() {  
30         override fun areItemsTheSame(  
31             oldItem: Task,  
32             newItem: Task  
33         ): Boolean{  
34             return oldItem.id == newItem.id && oldItem.description == newItem.description  
35         }  
36  
37         override fun areContentsTheSame(  
38             oldItem: Task,  
39             newItem: Task  
40         ): Boolean{  
41             return oldItem.id == newItem.id && oldItem.description == newItem.description  
42         }  
43     }  
44 }
```

Remove variável taskList

Figura 202

210. Como removemos o parâmetro `taskList` do construtor da classe `TaskAdapter`, será necessário ajustar a criação da instância do adapter em alguns arquivos. Vamos começar pelo **TodoFragment.kt**. Remova os trechos de código destacados em vermelho, conforme ilustrado na Figura 203.

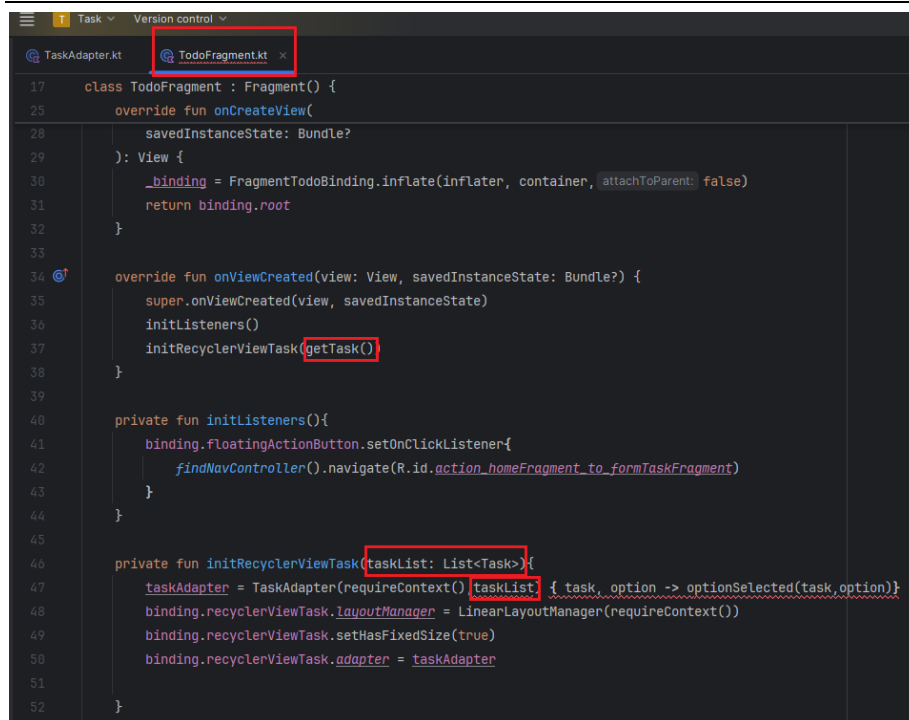


Figura 203

211. Outra alteração no arquivo TodoFragment.kt é a utilização do comando with para enrugamento de código. Faça conforme destacado na Figura 204.

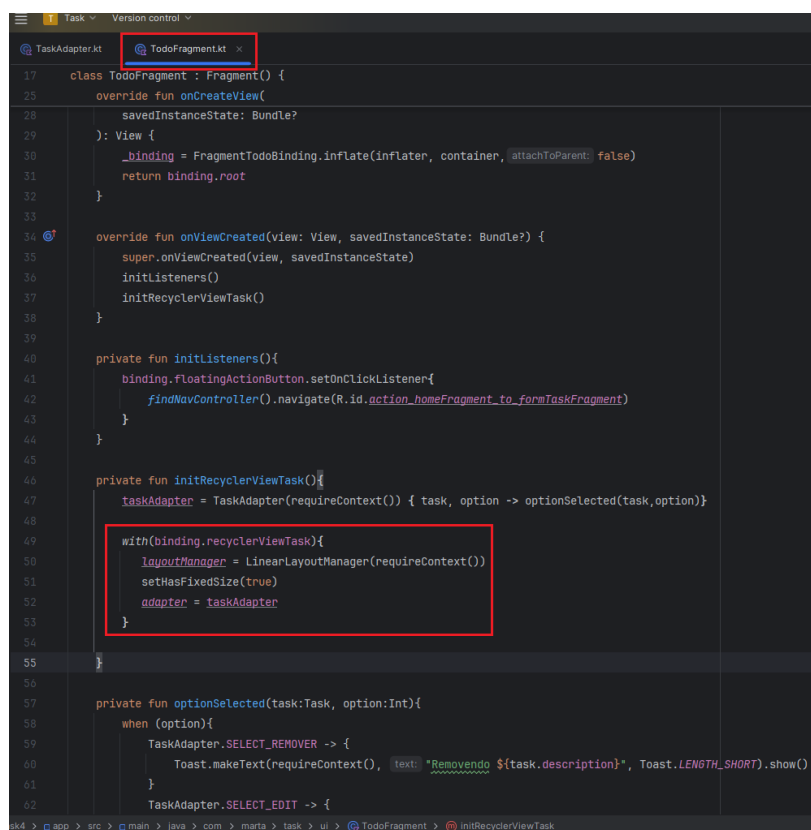


Figura 204

O `with` é uma função Kotlin que evita repetir o mesmo objeto várias vezes. Tudo que está dentro do bloco `{}` se refere ao contexto do `RecyclerViewTask`, que é o `RecyclerView` definido no layout.

212. No arquivo `TodoFragment.kt` altere o método `getTask` e inclua sua chamada no método `onViewCreated` conforme os trechos de códigos destacados em vermelho na Figura 205.

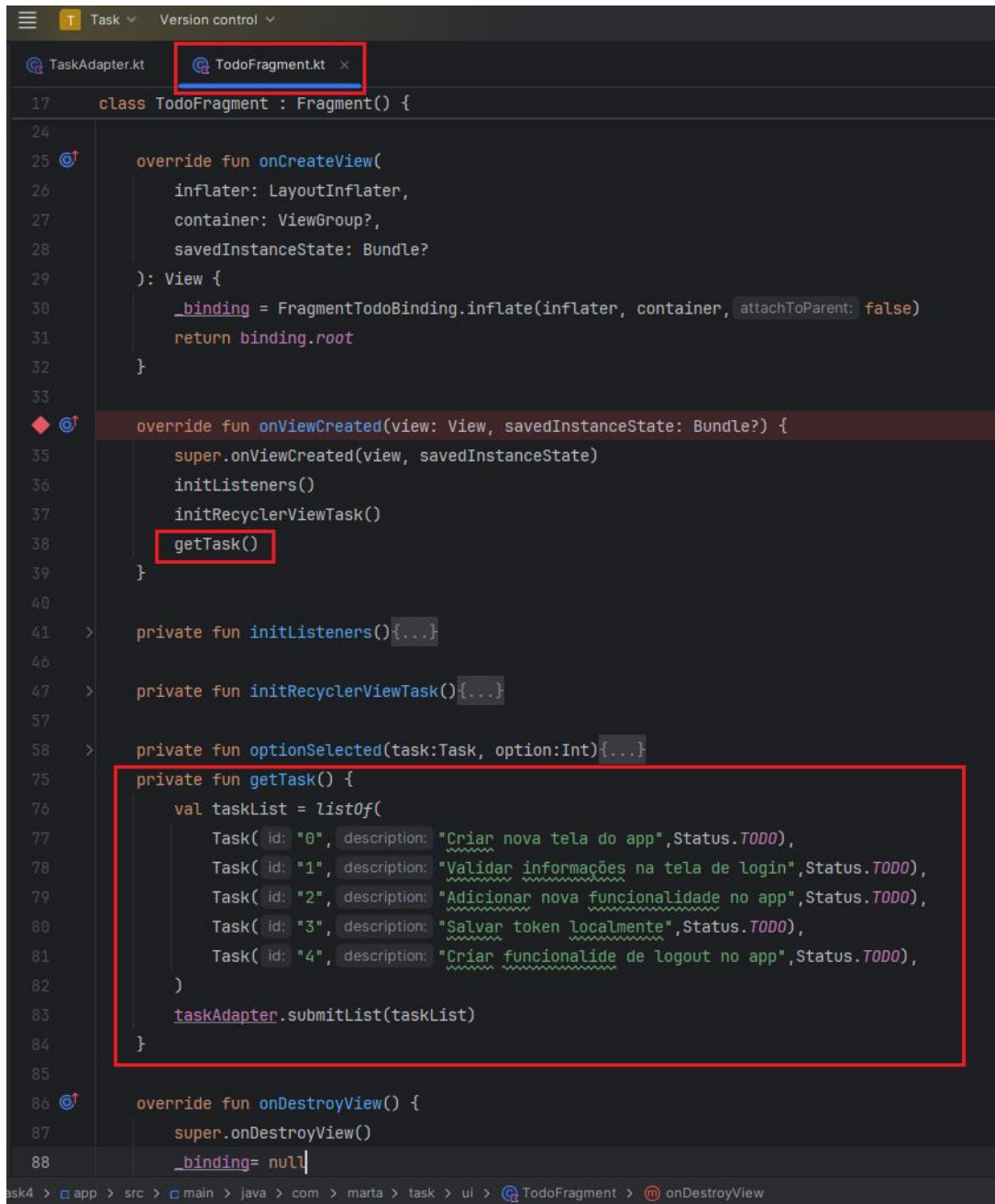


Figura 205

Observe a inclusão do método `submitList`. Esse método pertence a classe `ListAdapter`. O seu objetivo é substituir a lista atual por uma nova lista (`taskList`).

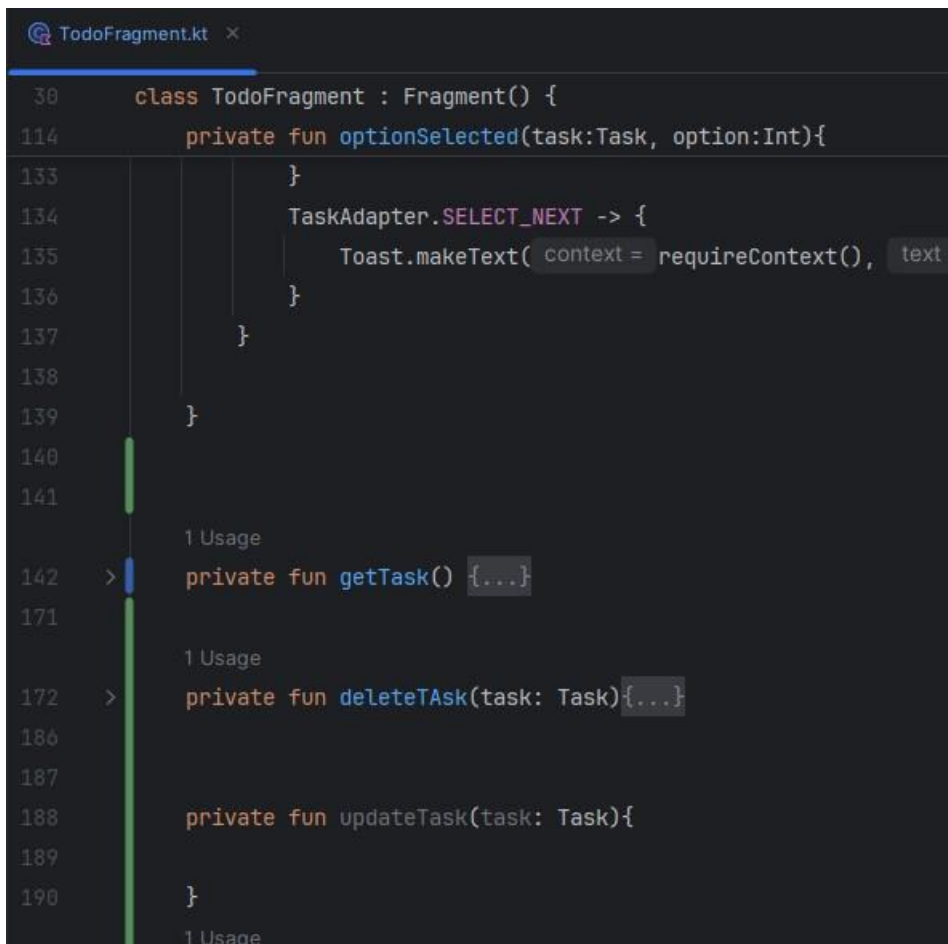
213. Repita os passos 209 até 211 nos arquivos **DoingFragment.kt** e **DoneFragment.kt**.

214. Execute o App para verificar se tudo está funcionando corretamente. Neste momento, as animações de atualização da lista ainda não serão visíveis, pois estamos trabalhando apenas com dados estáticos. Quando integrarmos o aplicativo ao banco de dados e houver alterações reais na lista (adição, remoção ou edição de tarefas), o ListAdapter exibirá automaticamente as animações de transição.

PARTE 7 – Finalizando Listagem das Tarefas

Para desenvolver esta seção, é necessário implementar previamente a classe `FirebaseHelper`, conforme apresentado nos slides da aula.

215. Abra o arquivo `TodoFragment.kt` e crie o esboço inicial da função `updateTask` conforme Figura 206.



```
30 class TodoFragment : Fragment() {
114     private fun optionSelected(task: Task, option: Int) {
133         }
134         TaskAdapter.SELECT_NEXT -> {
135             Toast.makeText(context = requireContext(), text =
136         }
137     }
138 }
139 }
140
141
142 > private fun getTask() {...}
171
172 > private fun deleteTask(task: Task) {...}
186
187
188     private fun updateTask(task: Task) {
189     }
190 }
1 Usage
```

Figura 206

216. Em seguida, realize a chamada da função `updateTask` dentro da função `optionSelected` e altere o status da tarefa para `Status.Doing`, conforme ilustrado na Figura

207 destacado em vermelho. Depois remova a linha de código `Toast.makeText(requireContext(), "Próximo", Toast.LENGTH_SHORT).show()`.

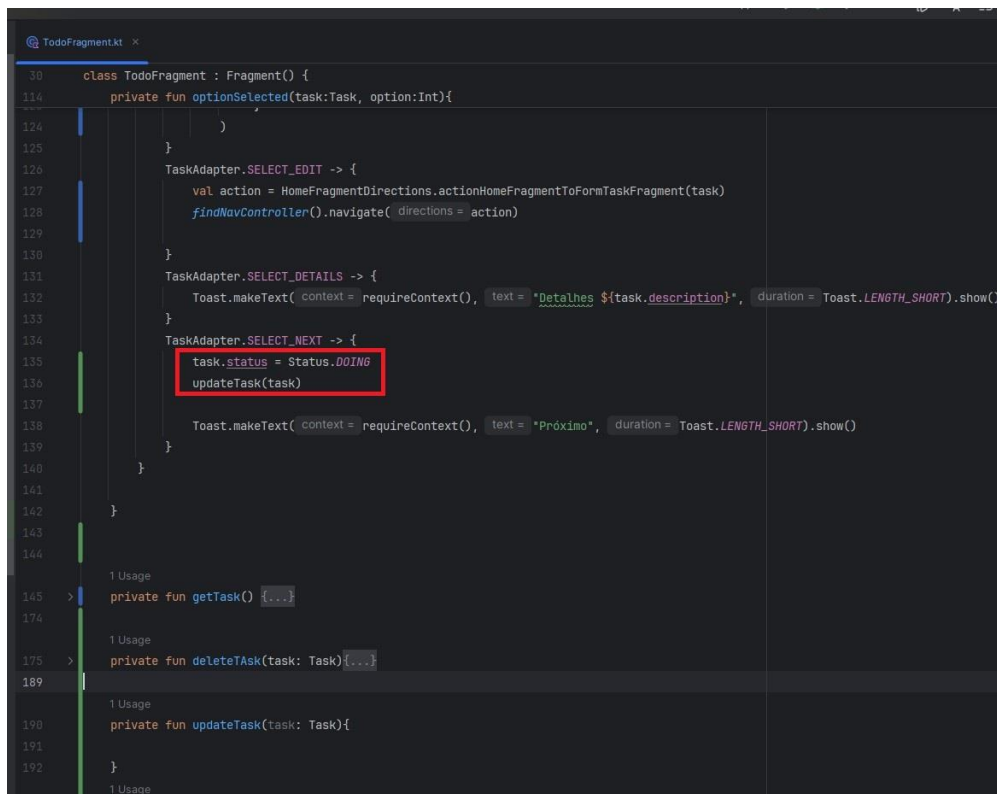


Figura 207

217. Ajuste o método deleteTask do TodoFragment.kt conforme Figura 208.

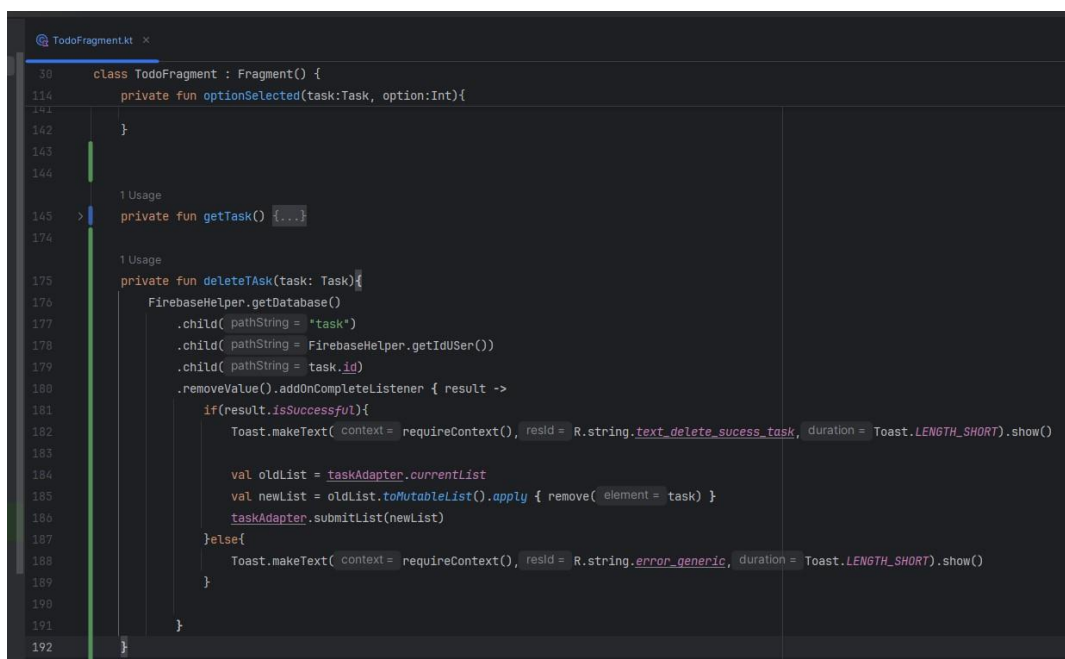
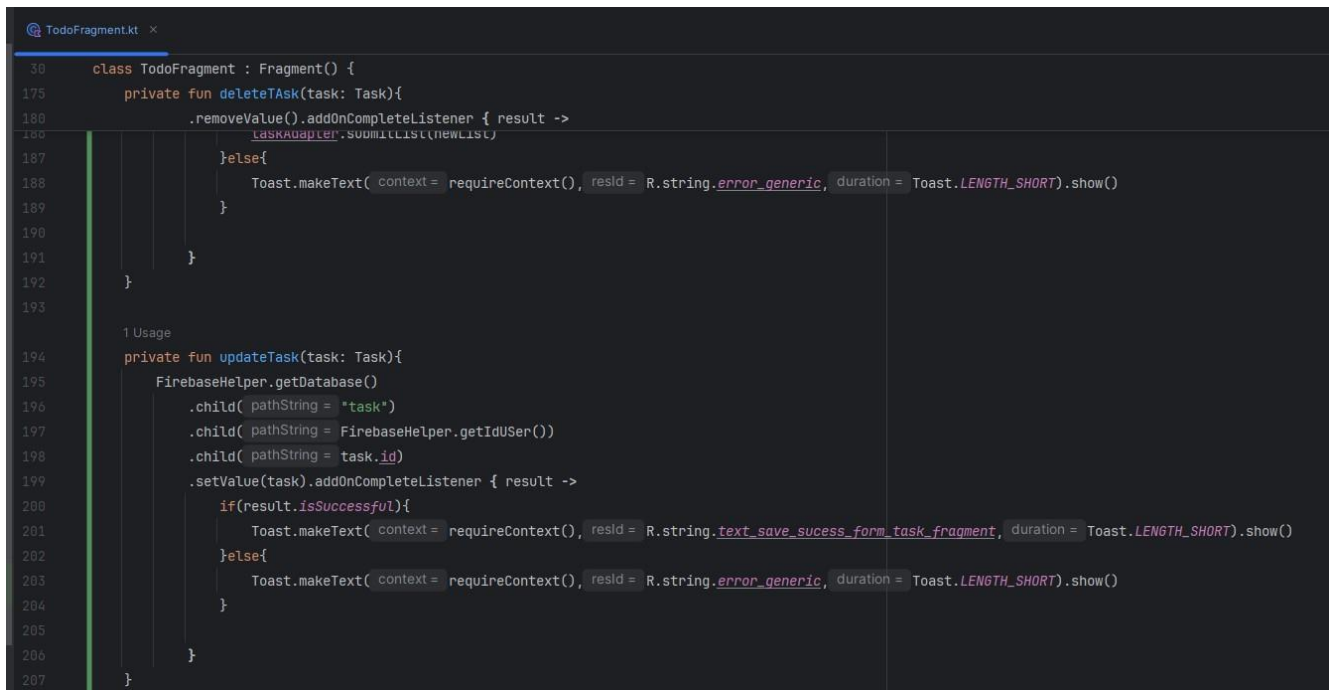


Figura 208

A função `deleteTask` é definida como privada e recebe como parâmetro um objeto do tipo `Task`, sendo responsável por realizar a exclusão de uma tarefa. Inicialmente, é obtida a instância do `Firebase Realtime Database` por meio da classe utilitária `FirebaseHelper`. Em seguida, o código navega pela estrutura do banco de dados utilizando os métodos `child("task")`, `child(FirebaseHelper.getIdUser())` e `child(task.id)`, onde `"task"` representa o nó principal que armazena as tarefas, `FirebaseHelper.getIdUser()` corresponde ao identificador do usuário logado e `task.id` identifica especificamente a tarefa que será removida. O método `removeValue()` é então utilizado para excluir o nó correspondente à tarefa no banco de dados. Para acompanhar o resultado dessa operação, é adicionado um listener com `addOnCompleteListener`, que permite verificar se a exclusão foi realizada com sucesso ou se ocorreu algum erro. Após a remoção no `Firebase`, a lista atualmente exibida no `RecyclerView` é obtida por meio de `taskAdapter.currentList`, sendo criada uma nova lista mutável a partir dela, na qual a tarefa excluída é removida. Por fim, o método `submitList` do `ListAdapter` é chamado para atualizar a interface, garantindo que a exclusão da tarefa seja refletida corretamente na tela.

218. Implemente o método `updateTask` do `TodoFragment.kt` conforme Figura 209.



```
30 class TodoFragment : Fragment() {
175     private fun deleteTask(task: Task){
180         .removeValue().addOnCompleteListener { result ->
181             taskAdapter.submitList(newList)
182         }else{
183             Toast.makeText( context = requireContext(), resId = R.string.error_generic, duration = Toast.LENGTH_SHORT).show()
184         }
185     }
186 }
187
188
189
190
191
192
193
194
195 1 Usage
196 private fun updateTask(task: Task){
197     FirebaseHelper.getDatabase()
198     .child( pathString = "task")
199     .child( pathString = FirebaseHelper.getIdUser())
200     .child( pathString = task.id)
201     .setValue(task).addOnCompleteListener { result ->
202         if(result.isSuccessful){
203             Toast.makeText( context = requireContext(), resId = R.string.text_save_sucess_form_task_fragment, duration = Toast.LENGTH_SHORT).show()
204         }else{
205             Toast.makeText( context = requireContext(), resId = R.string.error_generic, duration = Toast.LENGTH_SHORT).show()
206         }
207     }
208 }
```

Figura 209

O código define a função `updateTask`, que é privada e recebe como parâmetro um objeto do tipo `Task`. Essa função tem como objetivo atualizar os dados de uma tarefa no `Firebase Realtime Database`. Inicialmente, o método `FirebaseHelper.getDatabase()` é utilizado para obter a instância do banco de dados. Em seguida, o código percorre a estrutura do `Firebase` por meio das chamadas `child("task")`, `child(FirebaseHelper.getIdUser())` e `child(task.id)`. Nesse caminho, o nó `"task"` representa o agrupamento principal das tarefas,

FirestoreHelper.getIdUser() corresponde ao identificador do usuário atualmente logado e task.id identifica de forma única a tarefa que será atualizada.

Após alcançar o nó correto, o método setValue(task) é chamado para sobrescrever os dados existentes da tarefa com as informações contidas no objeto task. Para acompanhar o resultado dessa operação, é adicionado um listener com addOnCompleteListener, que verifica se a atualização foi concluída com sucesso. Caso a operação seja bem-sucedida, é exibida ao usuário uma mensagem de confirmação por meio de um Toast, indicando que os dados foram salvos corretamente. Caso contrário, uma mensagem genérica de erro é apresentada, informando que houve uma falha no processo de atualização.

219. Agora, vamos replicar todas as ações da tela fragment_todo.xml para as telas fragment_done.xml e fragment_doing.xml. Para isso, copie o trecho de código XML do arquivo fragment_todo.xml destacado na Figura 210 e, em seguida, cole-o nos arquivos fragment_done.xml e fragment_doing.xml, conforme ilustrado na Figura 211.

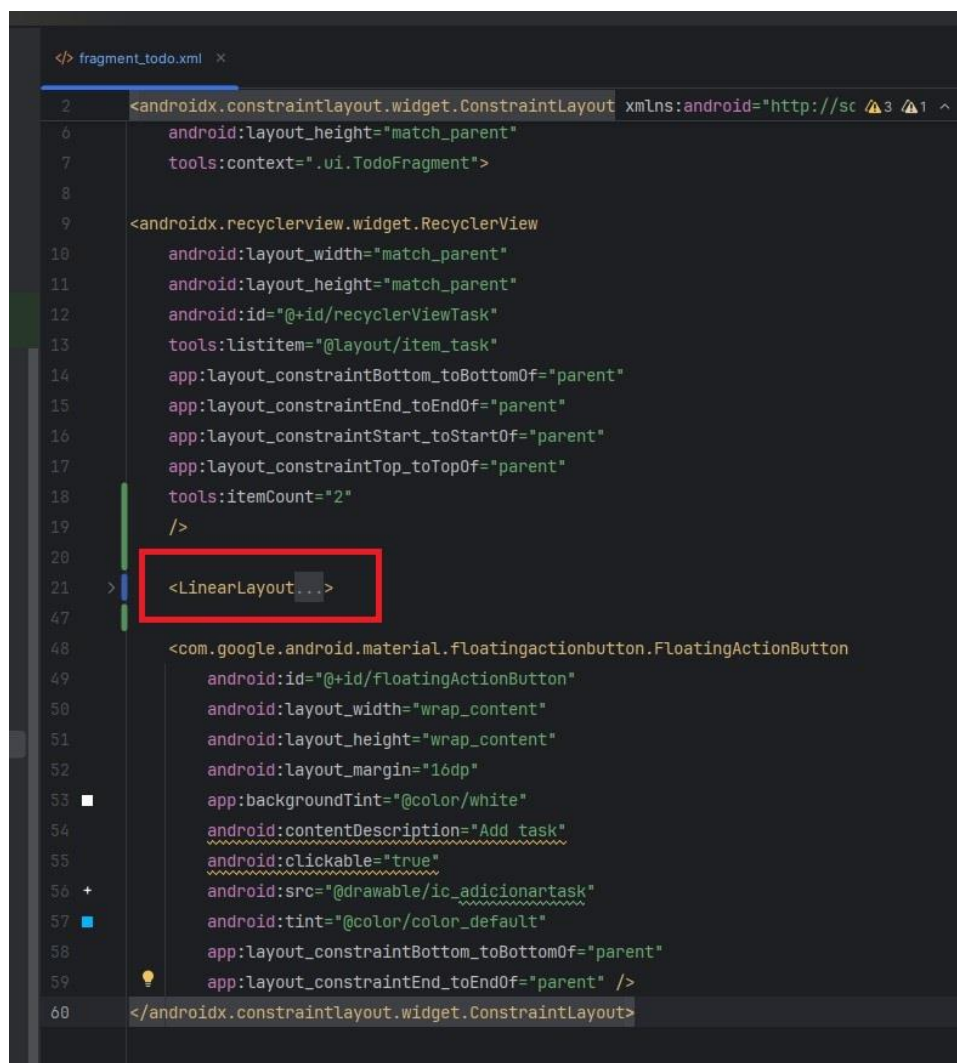


Figura 210

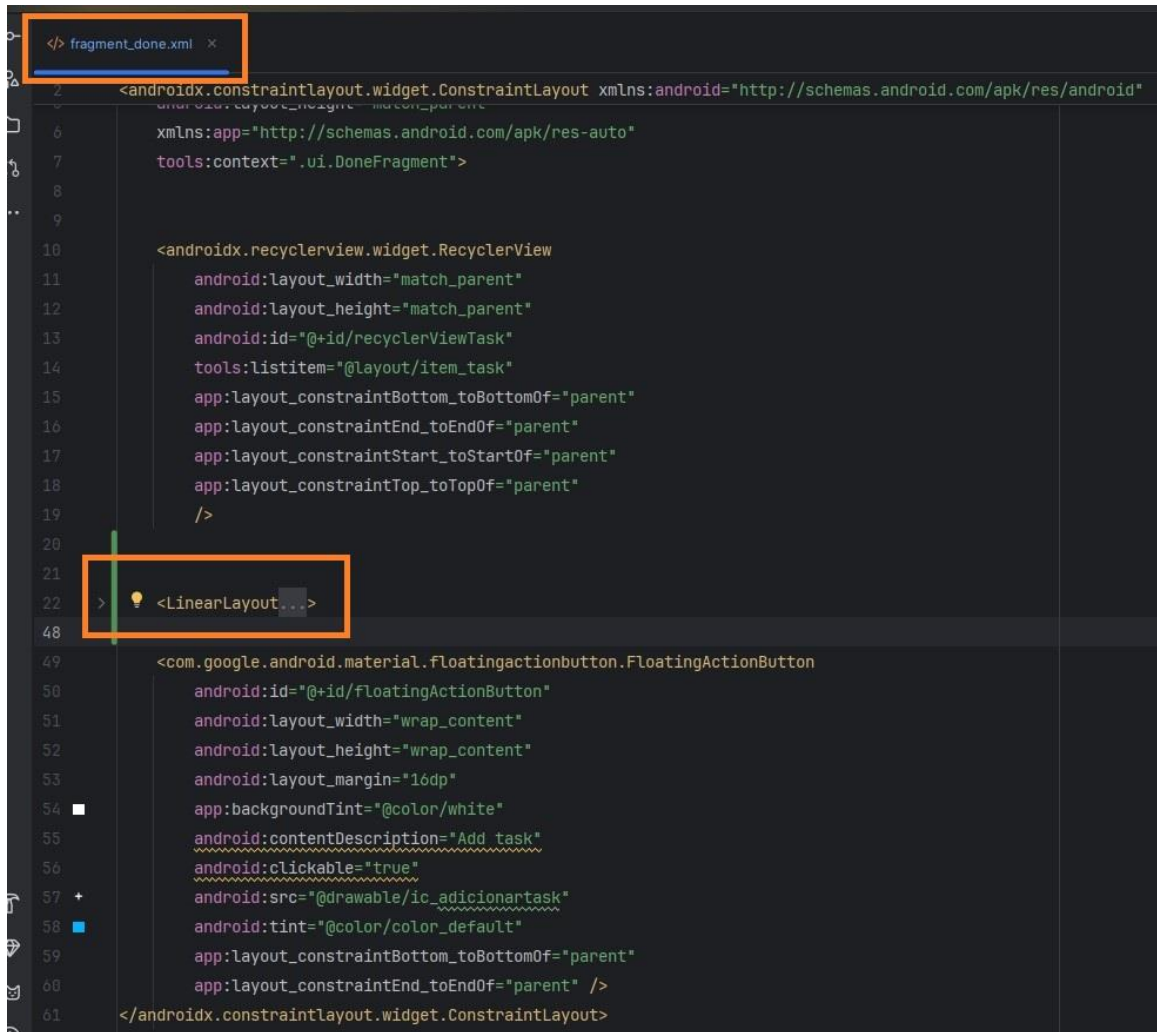


Figura 211

220. No componente XML do RecyclerView inclua o atributo itemCount igual a 2, conforme Figura 212. Isso facilitará a visualização das tarefas na tela. Inclua esse atributo no xml do fragment_doing.xml.

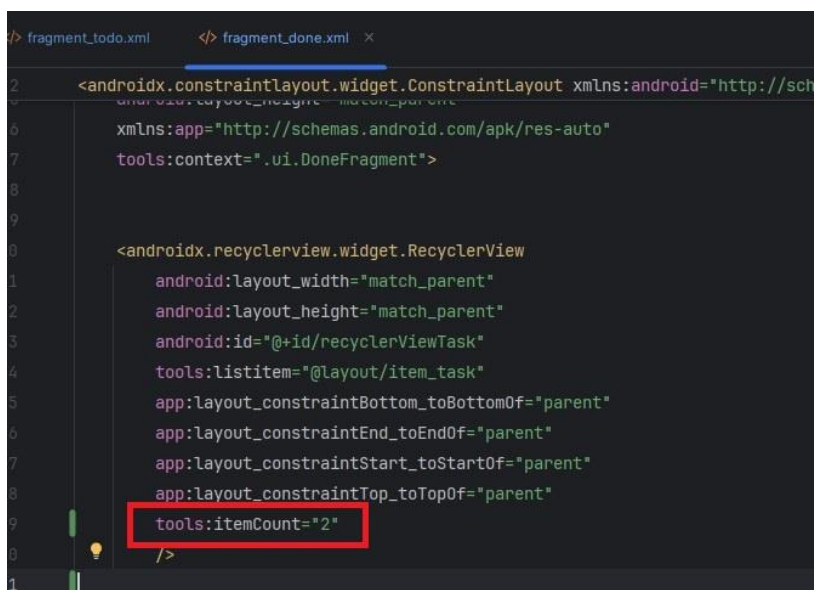


Figura 212

221. Copie os métodos `getTask` e `listEmpty` do `ToDoFragment.kt` e cole no `DoingFragment.kt` e no `DoneFragment` e faça ajuste conforme destacado na Figura 213.

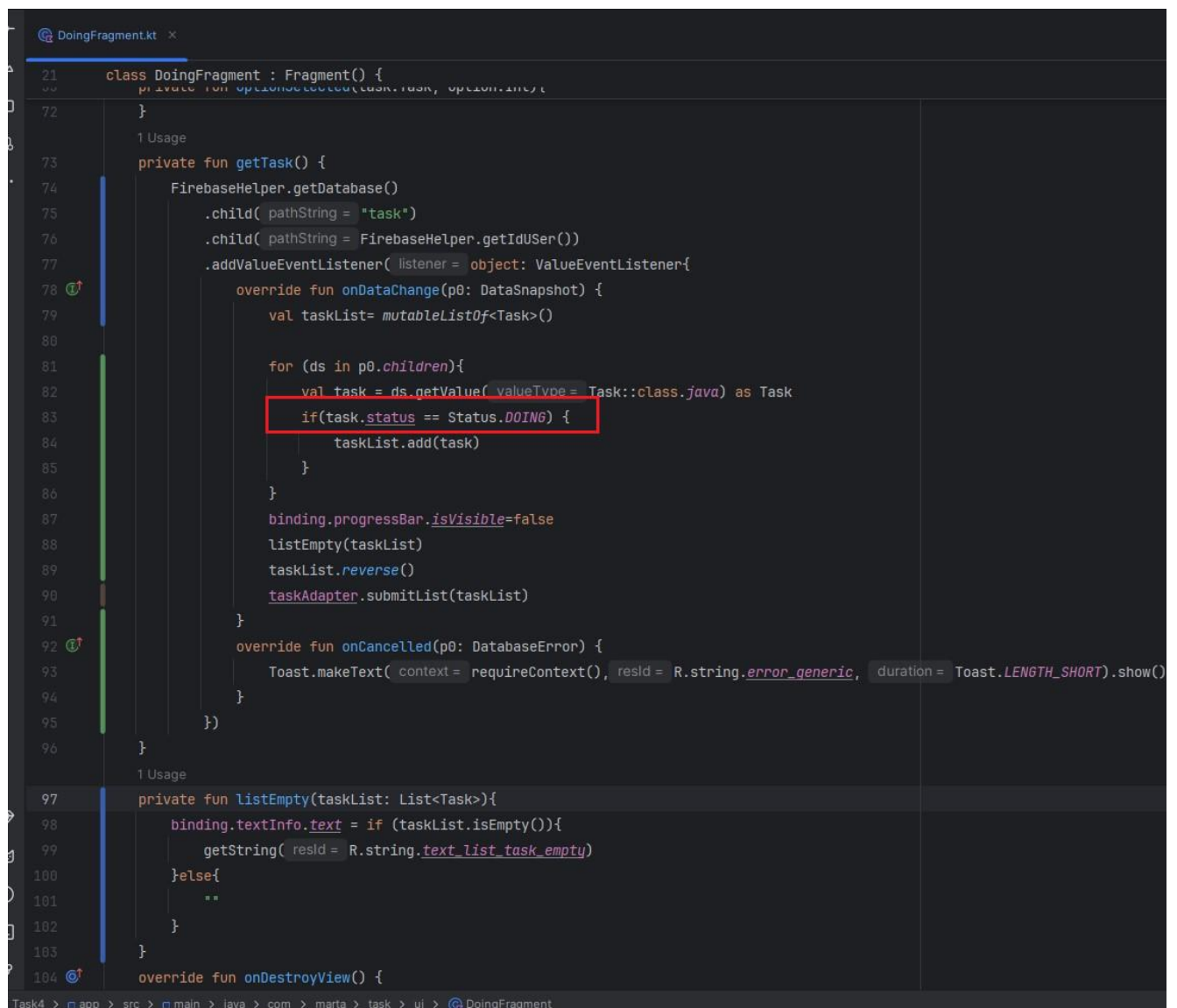


Figura 213

222. Copie o método `updateTask` da classe `ToDoFragment.kt` para a classe `DoingFragment.kt`. Altere o método `optionSelected` do `DoingFragment` conforme Figura 214..

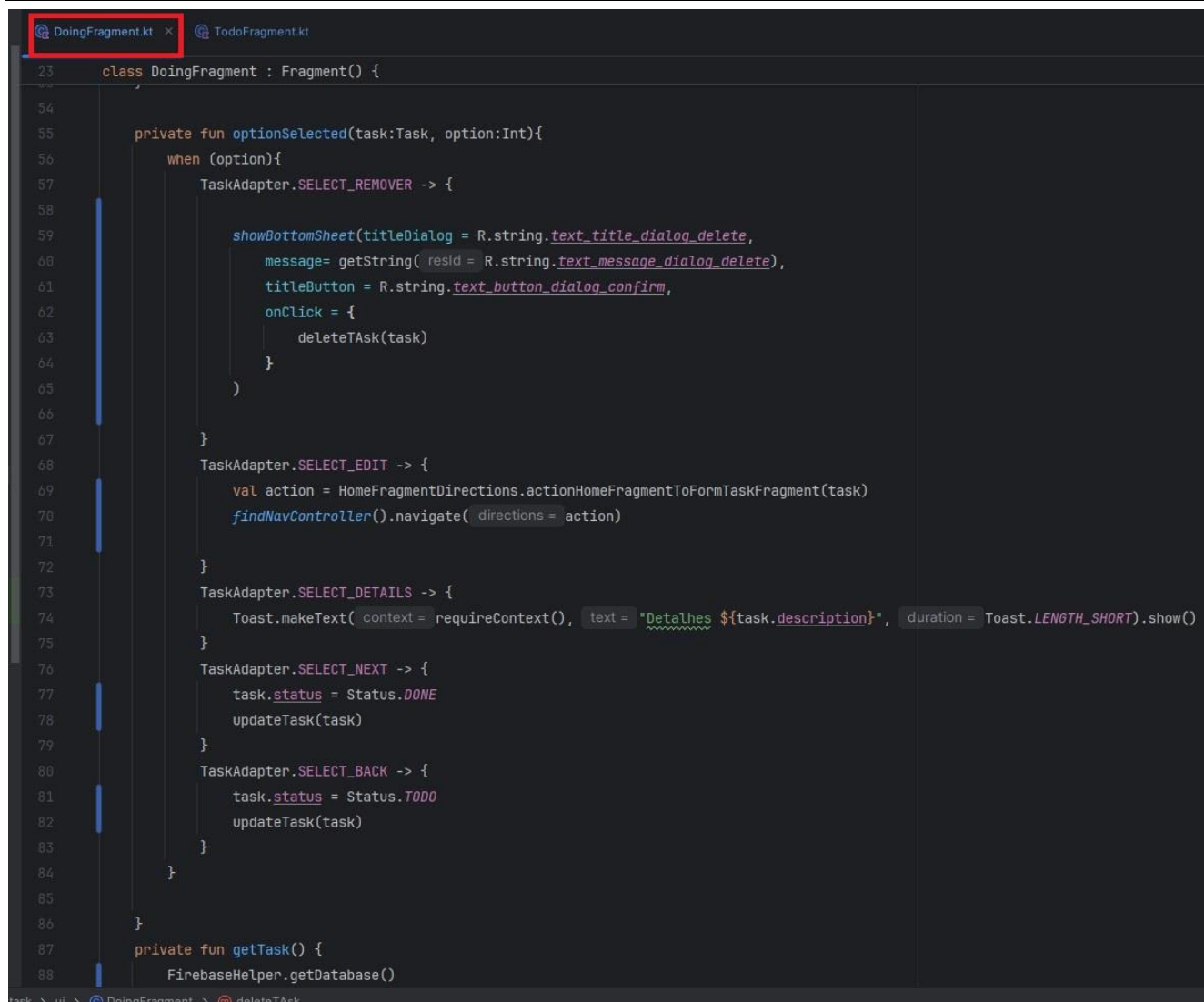
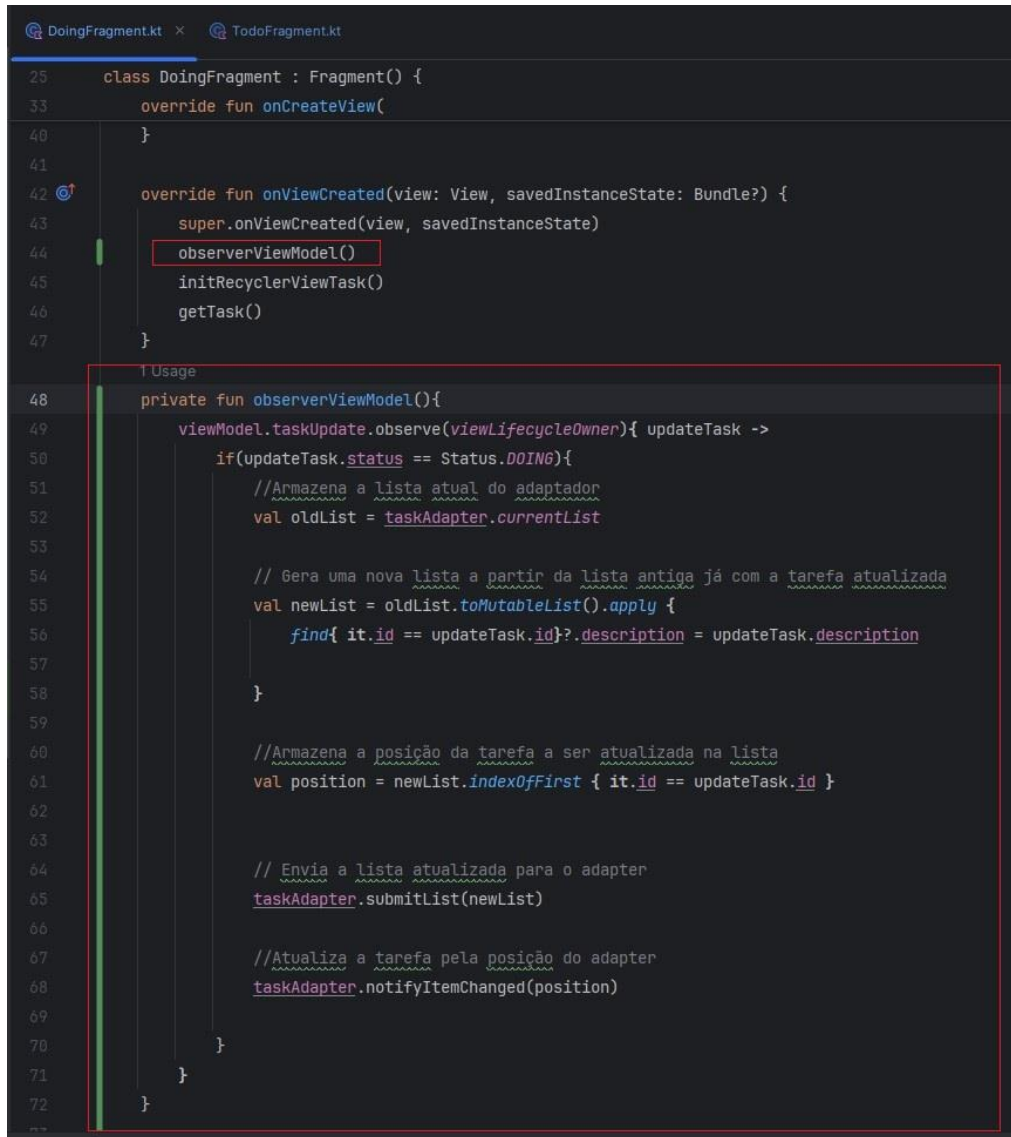


Figura 214

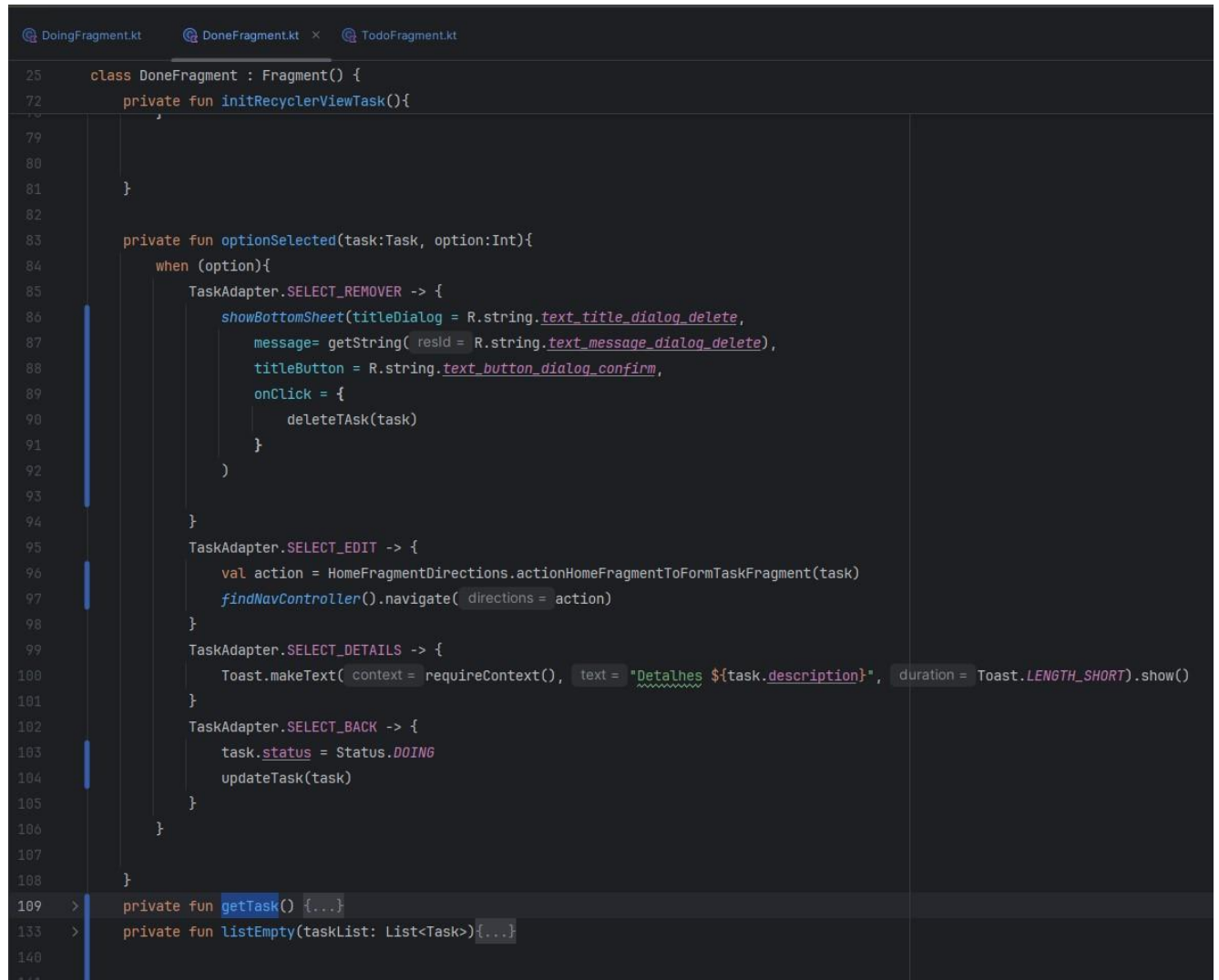
223. Copie o método `observerViewModel` do `TodoFragment.kt` para o `DoingFragment.kt`. Faça a chamada do método `observerViewModel` dentro do evento `onViewCreated` conforme Figura 215. Lembre-se que esse método é crucial na atualização dos dados da tela.



```
25 class DoingFragment : Fragment() {
33     override fun onCreateView(
40     ) {
41
42     @
43     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
44         super.onViewCreated(view, savedInstanceState)
45         observerViewModel()
46         initRecyclerViewTask()
47         getTask()
48     }
49
50     private fun observerViewModel(){
51         viewModel.taskUpdate.observe(viewLifecycleOwner){ updateTask ->
52             if(updateTask.status == Status.DOING){
53                 //Armazena a lista atual do adaptador
54                 val oldList = taskAdapter.currentList
55
56                 // Gera uma nova lista a partir da lista antiga já com a tarefa atualizada
57                 val newList = oldList.toMutableList().apply {
58                     find{ it.id == updateTask.id)?.description = updateTask.description
59                 }
60
61                 //Armazena a posição da tarefa a ser atualizada na lista
62                 val position = newList.indexOfFirst { it.id == updateTask.id }
63
64                 // Envia a lista atualizada para o adapter
65                 taskAdapter.submitList(newList)
66
67                 //Atualiza a tarefa pela posição do adapter
68                 taskAdapter.notifyItemChanged(position)
69             }
70         }
71     }
72 }
```

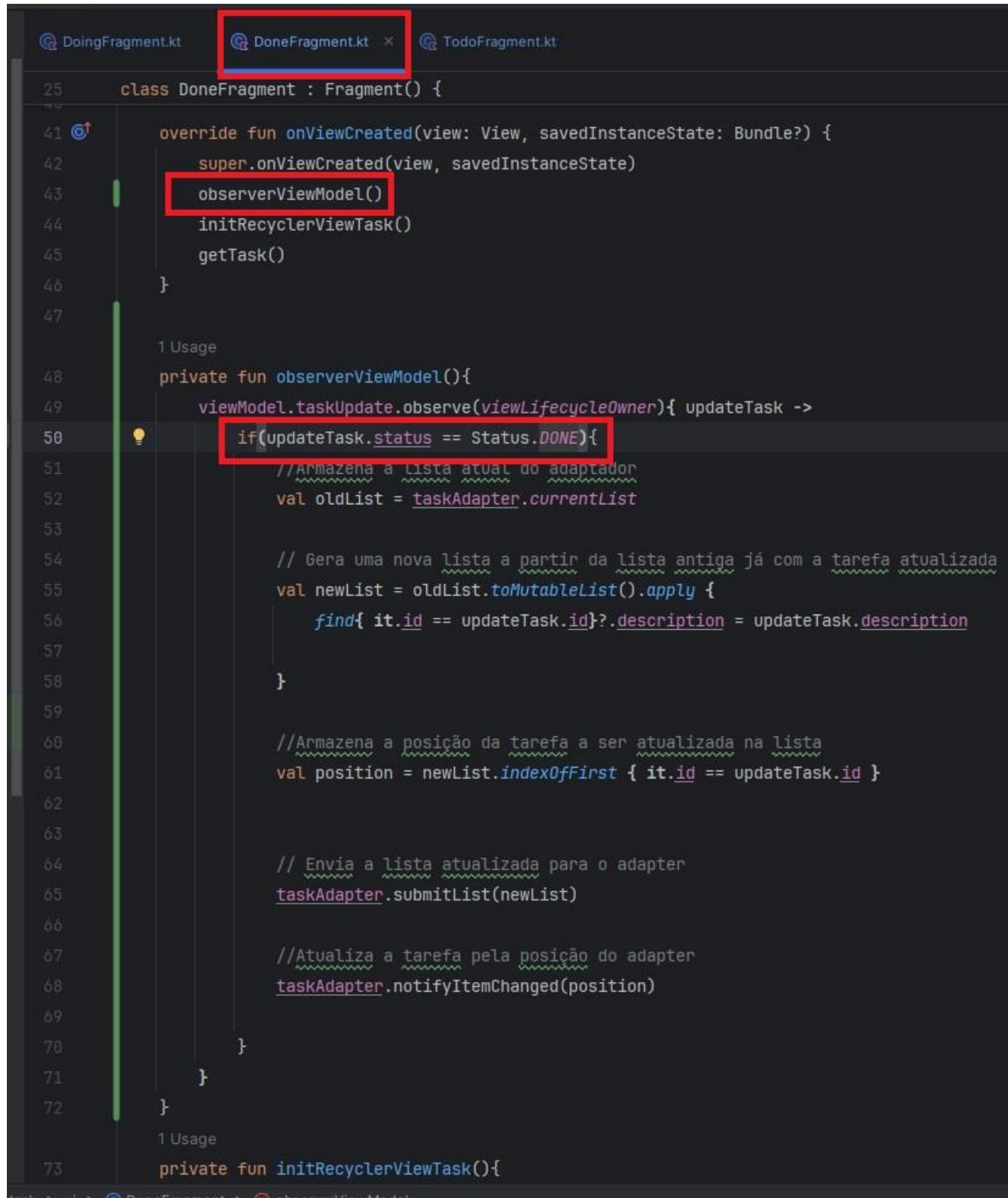
Figura 215

224. Agora é necessário replicar todo o código também para o DoneFragment.kt de forma que a tela funcione os botões de setinha e o de exclusão. Você pode copiar os métodos getTask, listEmpty , updateTask, deleteTask e observerViewModel da classe DoingFragment para DoneFragment. Depois ajustar o método optionSelected e o evento onViewCreated conforme Figuras 216, 217 e 218.



```
25  class DoneFragment : Fragment() {
72      private fun initRecyclerViewTask(){
73      },
74      },
79
80
81  }
82
83  private fun optionSelected(task:Task, option:Int){
84      when (option){
85          TaskAdapter.SELECT_REMOVER -> {
86              showBottomSheet(titleDialog = R.string.text_title_dialog_delete,
87                  message= getString( resId = R.string.text_message_dialog_delete),
88                  titleButton = R.string.text_button_dialog_confirm,
89                  onClick = {
90                      deleteTask(task)
91                  }
92              )
93          }
94      }
95      TaskAdapter.SELECT_EDIT -> {
96          val action = HomeFragmentDirections.actionHomeFragmentToFormTaskFragment(task)
97          findNavController().navigate( directions = action)
98      }
99      TaskAdapter.SELECT_DETAILS -> {
100          Toast.makeText( context = requireContext(), text = "Detalhes ${task.description}", duration = Toast.LENGTH_SHORT).show()
101      }
102      TaskAdapter.SELECT_BACK -> {
103          task.status = Status.DOING
104          updateTask(task)
105      }
106  }
107
108  }
109  > private fun getTask() {...}
133  > private fun listEmpty(taskList: List<Task>){...}
140
141
```

Figura 216



```
25 class DoneFragment : Fragment() {
41     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
42         super.onViewCreated(view, savedInstanceState)
43         observerViewModel()
44         initRecyclerViewTask()
45         getTask()
46     }
47
48     1 Usage
49     private fun observerViewModel(){
50         viewModel.taskUpdate.observe(viewLifecycleOwner){ updateTask ->
51             if(updateTask.status == Status.DONE){
52                 //Armazena a lista atual do adaptador
53                 val oldList = taskAdapter.currentList
54
55                 // Gera uma nova lista a partir da lista antiga já com a tarefa atualizada
56                 val newList = oldList.toMutableList().apply {
57                     find{ it.id == updateTask.id)?.description = updateTask.description
58                 }
59
60                 //Armazena a posição da tarefa a ser atualizada na lista
61                 val position = newList.indexOfFirst { it.id == updateTask.id }
62
63                 // Envia a lista atualizada para o adapter
64                 taskAdapter.submitList(newList)
65
66                 //Atualiza a tarefa pela posição do adapter
67                 taskAdapter.notifyItemChanged(position)
68             }
69         }
70     }
71 }
72
73     1 Usage
74     private fun initRecyclerViewTask(){
```

Figura 217

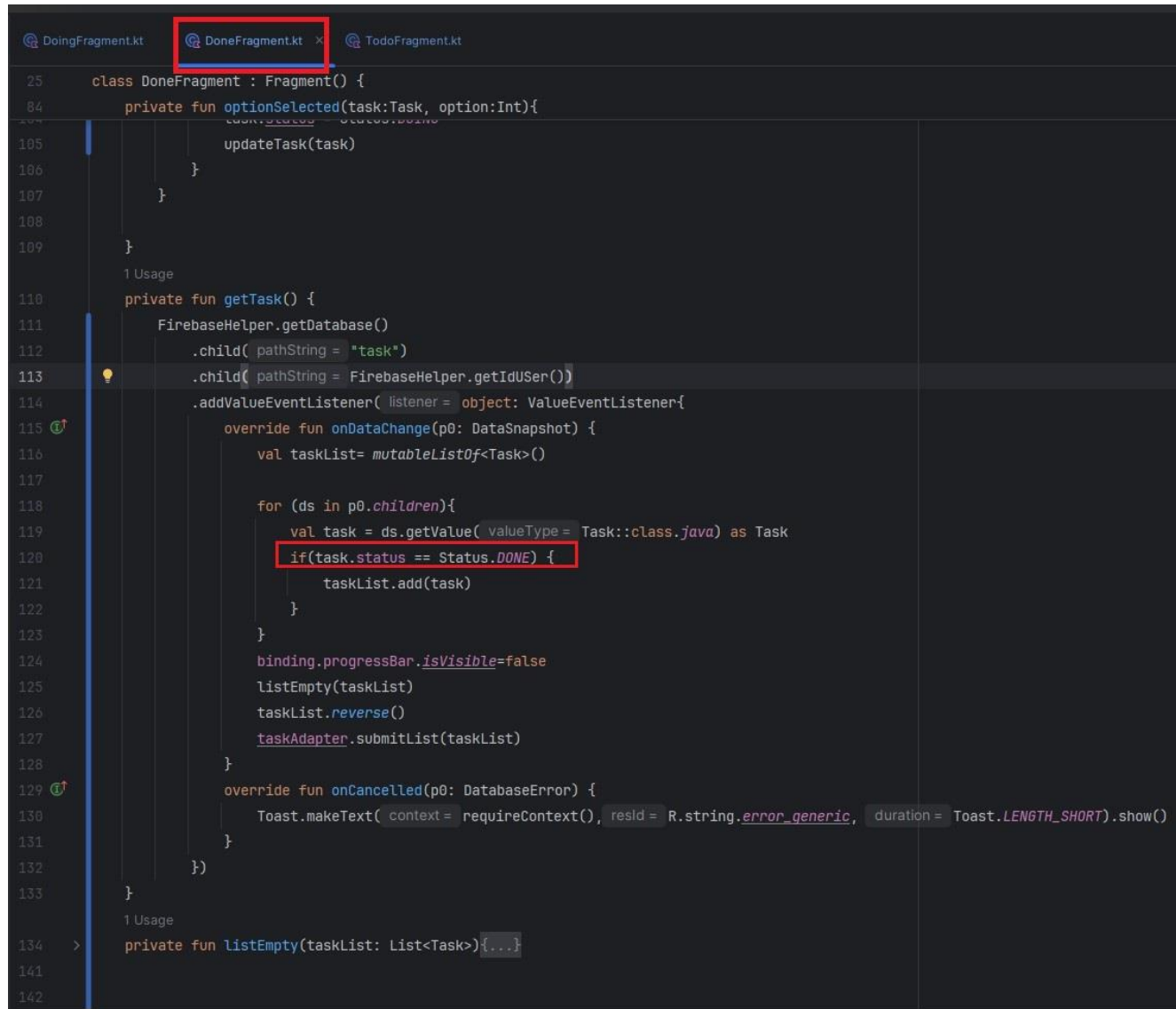
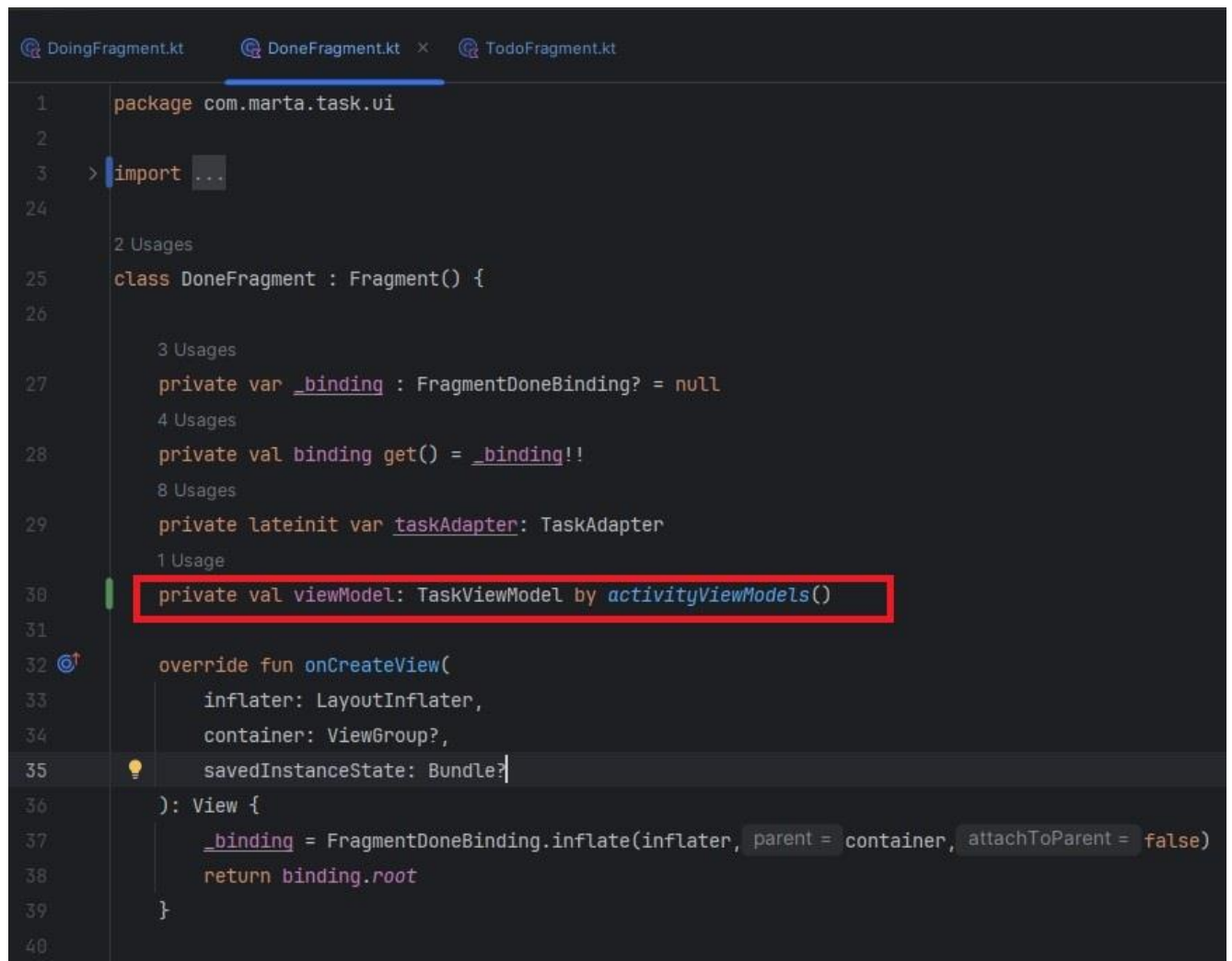


Figura 218

225. Não se esqueça de incluir a variável viewModel, conforme ilustrado na Figura 219. O ViewModel é responsável por armazenar e gerenciar os dados relacionados à interface do usuário (UI), garantindo sua persistência durante mudanças de configuração — como a rotação da tela — e promovendo a atualização automática da interface sempre que houver alterações nos dados provenientes do banco de dados.



```
1 package com.marta.task.ui
2
3 > import ...
4
5 2 Usages
6
7 class DoneFragment : Fragment() {
8
9     3 Usages
10     private var _binding : FragmentDoneBinding? = null
11     4 Usages
12     private val binding get() = _binding!!
13     8 Usages
14     private lateinit var taskAdapter: TaskAdapter
15     1 Usage
16     private val viewModel: TaskViewModel by activityViewModels()
17
18     override fun onCreateView(
19         inflater: LayoutInflater,
20         container: ViewGroup?,
21         savedInstanceState: Bundle?
22     ): View {
23         _binding = FragmentDoneBinding.inflate(inflater, parent = container, attachToParent = false)
24         return binding.root
25     }
26 }
```

226. Agora é testar e verificar se todas ações da tela estão funcionando.

Tabela de Avaliação

Item	Descrição
Cores	As cores devem ser distintas daquelas indicadas no passo a passo.
Ícones	Os ícones devem ser diferentes dos indicados no passo a passo.
Tela com o nome do aluno	Escolha 2 telas e inclua seu nome no TextView

Lista de Tarefas e demais dados fictícios	Todas as tarefas e demais dados fictícios devem ser customizados diferentes do especificado pelo passo a passo
Navegabilidade entre as telas	A navegabilidade entre todas as telas está funcionando corretamente.
Navegabilidade entre as abas	A navegabilidade entre todas as abas está funcionando corretamente.
Erros	Não há erros no código Kotlin e XML.